WEBSITE BOOK EDITION

Pixels → Perception

從像素到理解：視覺、語言與學習

基礎・影像・語言・學習・理論

第 4 冊（共 4 冊）

最佳化、資訊理論與部落格文章

繁體中文版・第 4 冊

2026-11-02

https://blog.shyandram.dev/

本書說明

本冊是 Pixels → Perception 網站完整課程分成的第 4 冊（共 4 冊），主題為「最佳化、資訊理論與部落格文章」，收錄 16 篇已發布教學與 2 篇部落格文章，依網站的學習單元順序編排。這是網站內容的書籍排版版本，並非另經同儕審查的出版品。

正文、程式碼、公式與參考文獻沿用網站內容。練習提示在本書中直接展開；靜態影像保留，互動示範則提供網站連結。原本的影像比較器改列為輸入與輸出影像。篇末參考文獻的編號僅適用於該篇教學。

本站內容仍在審閱中。製作過程使用 AI 協助，內容可能仍有錯誤或疏漏；請搭配原始文獻閱讀，並以網站的後續修訂為準。

兩種語言、各冊分別提供 PDF。連到同冊教學的連結會在本 PDF 內跳轉；連到其他冊的教學、特定小節、互動操作、筆記本與外部資料仍連至網站。內容快照: 2026-11-02

來源版本: `fde8eaa`

https://blog.shyandram.dev/

目錄

I　最佳化 1

OPT 01・決策建模與凸集合 2

OPT 02・凸函數、組合規則與共軛 14

OPT 03・線性、二次、錐與幾何規劃 26

OPT 04・對偶性、KKT 條件與證明憑證 38

OPT 05・近似、統計估計與實驗設計 51

OPT 06・梯度下降、牛頓步驟與數值可靠性 64

OPT 07・等式限制與內點法 77

OPT 08・幾何應用、分裂法與現代方法 90

II　資訊理論 102

IT 01・熵、相對熵與互資訊 103

IT 02・典型序列、AEP 與熵率 116

IT 03・無損編碼與通用壓縮 128

IT 04・通道容量與可靠通訊 142

IT 05・微分熵與高斯通道 155

IT 06・率失真與有損表示 167

IT 07・統計證據、最大熵與成長 180

IT 08・網路、資訊不等式與學習 193

III 部落格文章 205

介紹 Pixels → Perception 206

歡迎來到 Pixels → Perception 208

篇 I

最佳化

OPT 01・決策建模與凸集合

把工程決策轉成變數、目標與限制，理解凸幾何、投影、分離，以及數值解的保證範圍。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/modeling-convex-sets/

你將學到

- 把決策轉成變數、目標、限制與固定資料。
- 區分可行、最佳、不可行、無界與未達到下確界的問題。
- 理解仿射集合、凸組合、凸包與錐。
- 辨認半空間、多面體、範數球、橢球與半正定錐。
- 使用保留凸性的運算、投影與分離超平面。
- 檢查數值解對模型與實際應用各自提供什麼證據。

先看全貌

影像復原系統選擇輸出像素，排程系統選擇資源分配，擬合程序選擇係數。三者都有許多可選答案、比較答案好壞的方式，以及允許答案的規則。最佳化把這三個要素寫清楚。

本軌以 Stephen Boyd 與 Lieven Vandenberghe 的《Convex Optimization》[1] 為主軸，學習辨認、建立與求解具有全域保證的問題。前幾章建立集合、函數、標準問題與對偶；應用連到擬合、統計估計與幾何設計；最後討論數值演算法與現代延伸。

> **用最簡單的話說**
>
> 想像你在調配果昔，希望好喝，又有固定預算，不能用已經沒庫存的材料。材料用量是可以選的變數，口味是想改善的目標，預算與庫存是規則。若混合任意兩種合規配方仍合規，這些配方就形成凸集合。

比喻有其限制：口味不一定是凸函數，材料也可能只能整包購買。第一個能力是辨認實際決策的哪些部分適合凸模型、哪些不適合。合理建模，比對錯誤方程式使用快速求解器更重要。

凸最佳化也能出現在非凸學習流程中，例如在固定特徵上擬合線性分類器、計算受限預測、擬合校準曲線，或替困難問題建立下界。但它不會讓任意神經網路訓練自動變成凸問題。保證只屬於實際求解的那個數學模型。

從一句話到最佳化模型

白話版。先決定可以改什麼、已知什麼、什麼答案比較好，以及必須遵守哪些規則，再選演算法。

一般純量最佳化可寫成

$$\begin{aligned} \text{minimize}_{x\in\mathbb{R}^n} \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1,\dots,m, \\ & h_j(x) = 0, \quad j = 1,\dots,p. \end{aligned}$$

x 是含 n 個決策變數的向量；f_0 是要最小化的純量目標；f_i 是 m 個不等式限制函數；h_j 是 p 個等式限制函數。固定量測與設計參數出現在函數中，但除非模型明確指定，否則不是決策變數。

例如兩種作業的資源用量為 x_1, x_2，容量限制可以是 $2x_1 + x_2 \le 5$，且 $x_1, x_2 \ge 0$。若固定目標配置為 b，$\|x - b\|_2^2$ 偏好接近目標。這些數字定義合成模型，不是產線量測。

單位也是模型的一部分。把金錢成本與影像重建誤差相加，需要權重慣例。換算尺度的係數應有可解釋含意，並經過檢查。影像從零到 255 改成零到一，其他項不變時，正則化的相對強度也可能改變。

建立簡短的建模紀錄：每個變數的意義與單位、固定輸入的來源、每個限制的理由，以及目標的解釋。這能避免把方便的公式誤認成唯一合理的任務描述。

限制與懲罰代表不同承諾

硬限制表示答案不被允許；懲罰表示某種偏差不理想，但可以換取其他好處。即使某次答案相似，兩者的含意仍不同。

例如強制 $x \ge 0$，所有可行影像都非負；對負值加平方懲罰，仍可能容許少量負值，以換取較好的資料項。增加懲罰係數不自動等於精確限制，有限精度下尤其如此。

預算限制設定明確上限，成本懲罰則表示與係數有關的偏好。係數選擇是應用決策，凸性不會替你決定。對偶章會推導乘子如何描述放寬硬限制的敏感度。

算例：容量限制下的目標配置

目標配置未必符合現有容量。最佳化找最近的容許配置，而不是把每個座標減少相同數量。

取合成目標 $b = (3, 3)$，在 $x \ge 0$ 與 $2x_1 + x_2 \le 5$ 下最小化 $\frac{1}{2}\|x - b\|_2^2$。目標消耗九單位容量，因此不可行。最佳解的容量邊界為作用中限制，其法向量為 $a = (2, 1)$，所以最短修正沿著 a：

$$x^\ast = b - \alpha a, \qquad \alpha = \frac{a^\top b - 5}{a^\top a} = \frac{4}{5}, \qquad x^\ast = \left(\frac{7}{5}, \frac{11}{5}\right).$$

其中 $x^\ast$ 是候選配置、α 是修正尺度、a 是容量限制的法向量。代入可得 $2x_1^\ast + x_2^\ast = 5$，而且兩座標都為正，所以非負限制不會改變這個投影。

為什麼它是最近的點？對任何可行 x，都有 $a^\top(x - x^\ast) \le 0$。展開目標差：

$$\frac{1}{2}\|x - b\|_2^2 - \frac{1}{2}\|x^\ast - b\|_2^2 = \frac{1}{2}\|x - x^\ast\|_2^2 - \alpha a^\top(x - x^\ast) \ge 0.$$

其中 x 是任意可行競爭點，其餘符號如上定義。兩項都非負，因此候選點為全域最小解，目標值為 8/5。這個計算在使用求解器前，就連起變數、作用中規則、修正方向與最佳值 [1]。

可行性與最佳值

白話版。被允許的答案不一定是最佳答案。有時沒有任何答案符合規則，有時目標喜歡的是越跑越遠的選擇。

$$\mathcal{F} = \{x : f_i(x) \le 0 \text{ for every } i,\ h_j(x) = 0 \text{ for every } j,\ x \text{ in all required domains}\}.$$

$\mathcal{F}$ 是同時滿足所有限制與函數定義域的可行集合。集合中的點稱為可行點。最佳值為

$$p^* = \inf_{x \in \mathcal{F}} f_0(x).$$

p^* 是所有可達目標值的最大下界。最佳解 x^* 是達到 $f_0(x^*) = p^*$ 的可行點。下確界存在，不代表存在達到它的點。

要區分四種情況：可行且達到有限最佳值；可行集合為空；目標向負無限延伸；有限下確界卻沒有點達到。

例如最小化 x 且要求 $x > 0$，最佳值為零，但零不允許。最小化 $-x$ 且 $x \ge 0$，問題向下無界。同時要求 $x \ge 1$ 與 $x \le 0$，則不可行。求解器都可能沒有回傳有限最佳解，但原因完全不同。

非空緊緻可行集合上的連續目標會達到最小值。閉而無界的集合，若目標在無限遠成長得足夠，也可能存在最小值。解是否存在，是開始執行演算法前就屬於模型的問題。

數值狀態是有限精度計算的證據

求解器通常把限制殘差、最佳條件或原始—對偶 gap 與容忍值比較。成功狀態不表示實數意義下精確可行，更不保證量測、目標與應用限制都正確。

若應用必須真正守住安全上限，需說明數值容忍值如何對應該上限。$x \le 1$ 與 $10^6 x \le 10^6$ 數學等價，但未正規化殘差的尺度不同。

記錄殘差單位、尺度與容忍值。不同目標定義的數值不要直接比較。若停止原因是迭代上限，就如實報告；不能把點幾乎不動當成最佳性的證明。

仿射集合與凸集合

白話版。仿射集合包含任意兩點之間的整條直線；凸集合只需要包含它們之間的線段。

對 $u, v \in \mathbb{R}^n$：

$$x = \theta u + (1 - \theta) v.$$

u, v 為端點向量，x 為組合，純量 θ 控制直線位置。允許所有實數 θ，得到整條線；限制 $0 \le \theta \le 1$，得到線段。

包含所有仿射組合的集合是仿射集合；包含所有非負且係數總和一的凸組合，則是凸集合。所以仿射集合必為凸，但圓盤是凸而不是仿射。

有限多點的凸組合：

$$x = \sum_{i=1}^{r} \theta_i v_i, \qquad \theta_i \geq 0, \qquad \sum_{i=1}^{r} \theta_i = 1.$$

v_i 是 r 個端點，θ_i 是權重，組合是加權平均。非負很重要；外插可以離開凸集合。

凸性本身不要求平滑邊界、有界或閉。開球是凸，無界直線也是凸，單點也是凸。空集合慣例上也算凸，因為沒有違反定義的端點。

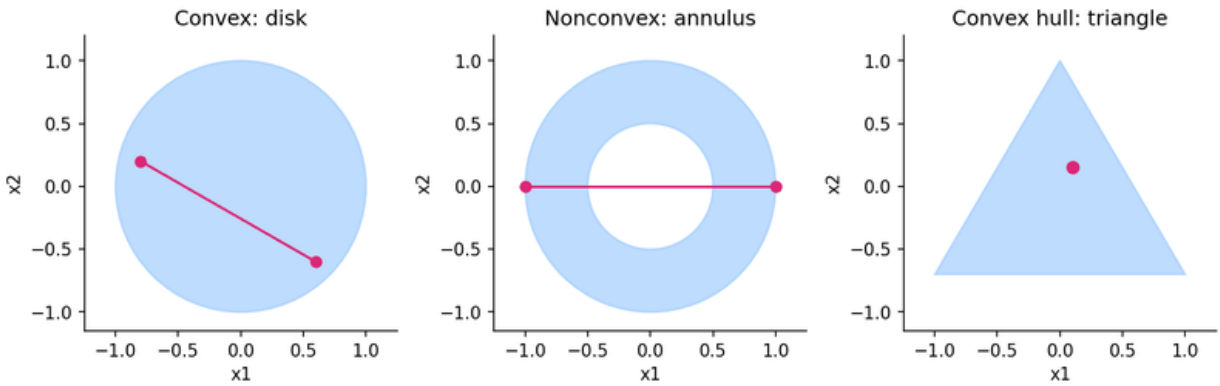


Figure 1 — 原創合成幾何。圓盤與填滿的三角形是凸集合；圓環不是，因為兩個允許端點的連線穿過被排除的孔洞。

反例能否證，取樣不能證明

要否證凸性，只需找到集合內兩點，使其中點或線段上其他點落在集合外。圓環可用相反端點平均成中心。

要證明凸性，使用定義或保留規則。隨機測很多點對能揭露實作錯誤，卻不能保證所有端點都成立。很小的非凸缺口可能完全沒有被抽到。

這與數值梯度檢查相似：它們是實作診斷，而數學證明處理全稱敘述。兩者用途不同，應標示清楚。

程式碼：凸組合與非凸反例

以下自成一體，驗證特定點。圓盤凸性的證明來自三角不等式，不是只有 assertion。

```
import numpy as np

u = np.array([0.8, 0.1])
v = np.array([-0.2, 0.7])
theta = np.linspace(0, 1, 101)
segment = theta[:, None] * u + (1 - theta[:, None]) * v
assert np.linalg.norm(segment, axis=1).max() <= 1 + 1e-12
ring_u, ring_v = np.array([1.0, 0.0]), np.array([-1.0, 0.0])
midpoint = (ring_u + ring_v) / 2
assert not (0.5 <= np.linalg.norm(midpoint) <= 1.2)
```

```
print("disk sampled-segment maximum norm:", np.linalg.norm(segment, axis=1).max())
print("ring midpoint:", midpoint, "norm:", np.linalg.norm(midpoint))
```

凸包與鬆弛

白話版。凸包填入原有選擇的所有加權平均。它能簡化離散決策，但新加入的點未必真的可執行。

$$\operatorname{conv}(S) = \left\{ \sum_{i=1}^{r} \theta_i v_i : r \text{ finite}, \; v_i \in S, \; \theta_i \geq 0, \; \sum_i \theta_i = 1 \right\}.$$

S 是原集合，r 是有限組合中的點數；$\operatorname{conv}(S)$ 是包含 S 的最小凸集合。

若必須在三個配置中選一個，凸包也容許混合。若混合代表時間分配，可能是實際可操作的選擇；若必須買一台不可分割的機器，分數配置只是鬆弛，不是原問題可行答案。

最小化問題擴大可行集合，不會提高最佳值，所以凸鬆弛可給離散問題下界。把鬆弛解變成真實離散選擇，還需額外步驟；捨入可能違反限制或傷害目標。

不可把鬆弛問題的精確最佳解說成原問題的精確最佳解。原可行集、鬆弛集合、下界與實際達成目標應分開。應用章會再討論。

常用凸集合詞彙

白話版。多數有用模型由幾種可重用幾何形狀組成。學會辨認後，陌生方程式也比較容易理解。

超平面、半空間與多面體

$$H = \{x : a^\top x = b\}, \qquad P = \{x : a^\top x \leq b\}.$$

$x \in \mathbb{R}^n$ 是點，$a \neq 0$ 是法向量，b 是純量偏移。超平面是仿射；閉半空間包含邊界與其中一側。

多面體是有限個半空間與超平面的交集，常寫 $Ax \leq b,\ Ex = d$。A, E 放係數，b, d 放界限，不等式逐座標比較。多面體可有界、無界、低維或空。

盒狀集合只限制各座標上下界；機率單純形再加入非負與總和一。這說明機率向量與有界像素常適合凸限制。

範數球與橢球

$$B = \{x : \|x - c\| \leq r\}.$$

c 是中心，$r \geq 0$ 是半徑，$\|\cdot\|$ 是範數。三角不等式給出凸性。二範數球是圓形，無限範數球是盒，一範數球帶有角點。

$$\mathcal{E} = \{c + Au : \|u\|_2 \leq 1\}.$$

A 把單位二範數球映射成橢球，u 是輔助座標，c 是中心。A 可逆時可用正定形狀矩陣寫二次描述；A 奇異時得到低維影像，仍然凸。

錐與矩陣不等式

凸錐對非負組合封閉：

$$\alpha u + \beta v \in K \quad \text{for } u, v \in K,\ \alpha, \beta \geq 0.$$

K 是錐，u, v 是其中的點，α, β 是非負純量；權重不需要總和一。非負 orthant 是例子。

二階錐包含 $\|z\|_2 \leq t$ 的 (t, z)，其中 t 是純量，z 是向量，可表達許多範數限制。

半正定錐包含滿足下式的對稱矩陣：

$$v^\top X v \geq 0 \quad \text{for every } v \in \mathbb{R}^n.$$

$X \in \mathbb{R}^{n \times n}$ 對稱，v 是任意測試向量。記為 $X \succeq 0$，不等於每個矩陣元素非負。共變異數矩陣自然落在此錐中。

廣義不等式用錐定義順序：$x \preceq_K y$ 表示 $y - x \in K$。Orthant 對應逐座標順序，對稱矩陣則對應半正定順序，必須說明使用哪個錐。

保留凸性的運算

白話版。已知一個集合凸後，可以透過特定運算得到另一個凸集合。這些規則比一直畫圖更可靠。

凸集合的交集仍凸：兩端點的線段會同時留在每個集合內。因此加入更多凸限制很直接。

仿射映射與仿射逆像保留凸性。C 凸時，$\{Ax + b : x \in C\}$ 與 $\{x : Ax + b \in C\}$ 都凸；A, b 是固定矩陣與向量。一般非線性轉換沒有此保證。

Cartesian product 與集合和保留凸性。取部分座標的投影是仿射映射，因此消去輔助變數後仍凸。這是 extended formulation 的基礎：增加變數反而可能更容易描述。

聯集通常不凸，兩個分離的球就示範失敗。限制敘述中的「且」改成「或」，即使每個分支都凸，也可能根本改變問題。

矩陣變數也一樣。X, Y 半正定時，$0 \leq \theta \leq 1$ 下的 $\theta X + (1 - \theta) Y$ 半正定，對每個向量檢查二次型即可。

兩個可重用的短證明

凸性證明通常追蹤一條線段如何通過集合構造。假設 $u, v \in C_1 \cap C_2$，且 C_1, C_2 都凸。對 $0 \leq \theta \leq 1$，混合點 $\theta u + (1 - \theta) v$ 同時屬於兩集合，因此屬於交集。任意多個凸集合的交集也可用相同推理。

對仿射映射，令 $y_1 = Au + b$、$y_2 = Av + b$，其中 $u, v \in C$。兩映射點的混合為

$$\theta y_1 + (1-\theta)y_2 = A[\theta u + (1-\theta)v] + b.$$

其中 C 是凸集合、A, b 是固定轉換資料、y_1, y_2 是映射後的點。中括號內的點屬於 C，所以混合點也在映射集合內。這就是消去輔助座標仍保留凸可行集合的原因 [1，第 2.3 節]。

聯集的對應證明會失敗，因為兩端點可能來自不同集合。例如 $-1 \in [-2,-1]$、$1 \in [1,2]$，但中點零不屬於任何一個區間。兩區間各自凸，聯集卻不凸。對建模而言，同時滿足凸規則保留凸性；二選一規則則可能破壞它。

投影與分離超平面

白話版。投影找離任意點最近的允許點。從投影指向原點的方向，也能描述可行集合位於邊界哪一側。

非空閉凸集合 C 的二範數投影為

$$\Pi_C(y) = \arg\min_{x \in C} \frac{1}{2}\|x - y\|_2^2.$$

y 是查詢點，x 是可行候選，$\Pi_C(y)$ 是唯一最小點。平方二範數的嚴格凸性確保唯一；其他距離或非凸集合未必如此。

令 $z = \Pi_C(y)$：

$$(y - z)^\top (x - z) \le 0 \quad \text{for every } x \in C.$$

z 是投影，x 遍歷所有可行點。從 z 朝任意可行 x 移動一小步，平方距離不能下降，就得到此條件。

若 $y \notin C$，$y - z \neq 0$，它定義通過 z 的超平面，使可行集合與 y 分處兩側，給出具體分離構造。

更一般的分離定理比較不相交凸集合 [1，第 2.5 節]。沒有附加假設時，分離未必有正 margin；閉性、緊緻性與集合距離都影響嚴格分離。

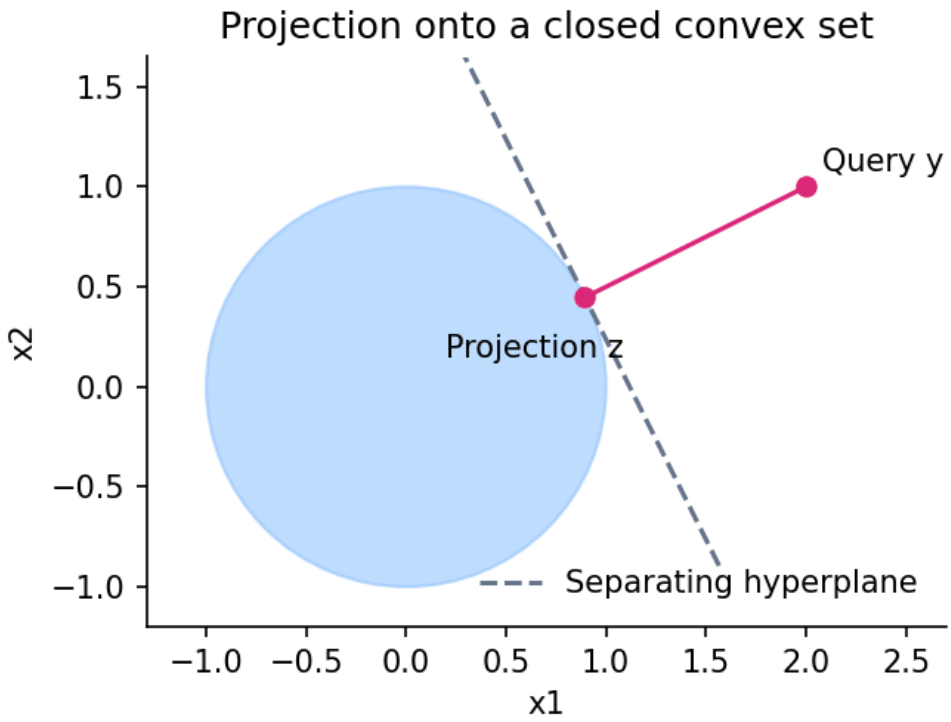


Figure 2 — 原創解析示意：外部點投影至單位圓盤，投影方向提供分離超平面的法向量。

程式碼：盒與球的投影

盒投影逐座標裁切；球投影只對外部點縮放。[1，第 8.1 節] 討論這些幾何應用。

```
import numpy as np

y = np.array([2.0, -1.5])
lower, upper = np.array([0.0, -1.0]), np.array([1.0, 1.0])
box_projection = np.clip(y, lower, upper)
radius = 1.0
ball_projection = y * min(1.0, radius / np.linalg.norm(y))
rng = np.random.default_rng(2)
points = rng.normal(size=(1000, 2))
points /= np.maximum(1.0, np.linalg.norm(points, axis=1))[:, None]
inequality = (points - ball_projection) @ (y - ball_projection)
assert inequality.max() <= 1e-12
assert np.all(box_projection >= lower) and np.all(box_projection <= upper)
print("box projection:", box_projection)
print("ball projection:", ball_projection)
```

取樣只展示投影不等式，不能替代所有可行點的證明。複雜集合的投影本身可能需要最佳化，所以「投影梯度」不代表投影便宜。

推導投影檢查條件

投影不等式來自檢查每個可行方向，而不是畫一條看起來合理的切線。令 $z = \Pi_C(y)$，任取 $x \in C$。凸性使 $z + \theta(x - z)$ 在 $0 \le \theta \le 1$ 時可行。因為 z 最小化平方距離，其單側方向導數必須非負：

$$\frac{d}{d\theta}\frac{1}{2}\|z + \theta(x - z) - y\|_2^2\Big|_{\theta=0} = (z - y)^\top (x - z) \ge 0.$$

其中 y 是查詢點、z 是其投影、x 是可行比較點、θ 是線段參數。這便得到前面的分離不等式。它也充分：把 x 處距離展開，就得到 z 處距離，加上一個非負內積與 $\frac{1}{2}\|x - z\|_2^2$。

唯一性也很直接。若不同的 z_1, z_2 都最小化平方距離，可行中點的值會嚴格更小。封閉性則確保存在：最小化序列留在有界的距離次水平集合內，因此有收斂子序列，而極限仍在 C。非空、封閉與凸性各自扮演明確角色 [1]。

內部、相對內部與隱藏等式

白話版。集合可能在自己的平面內有活動空間，卻在完整周圍空間沒有內部。

機率向量座標總和一，位於超平面，完整向量空間中的內部是空。但座標嚴格正的點，仍可位於此超平面中的相對內部。

仿射包是包含 C 的最小仿射集合；相對內部是在仿射包中量測的內部。這與可行性條件和對偶有關，不能要求完整維度內部而排除有用的等式模型。

不等式也可能隱藏等式：同時要求 $a^\top x \le b$ 與 $a^\top x \ge b$，就強制等號。辨認它能改善解釋與數值處理。

嚴格可行不是「算出很小違反」。它要求相關不等式嚴格成立，同時滿足等式。對偶與內點章會說明何時需要此條件。

凸問題需要集合與函數都合適

白話版。允許選擇是凸，只完成一半；目標形狀與等式也必須符合條件。

凸最小化使用凸目標、凸不等式函數與仿射等式 [1]。下一章定義凸函數並推導辨認規則。

凸可行集合本身不夠：在盒上最小化有多個谷地的函數仍非凸。凸目標加不連通離散限制，也非凸。

整體問題凸時，每個局部最小點都是全域最小點。但解未必唯一，線性目標可能在整個面上相同。可行集合上的嚴格凸性，可在存在最小值時保證唯一。

全域性與唯一性不保證所有尺度都容易。巨大密集問題可能超過記憶體，病態問題的緊容忍值可能難求。凸性提供結構優勢，但計算量與實作仍重要。

為什麼局部最小解必須是全域最小解

凸性把較好的遠處答案連到目前點，排除局部陷阱。假設 x 是凸可行集合 C 上凸函數 f 的局部最小解，但某個可行 y 有 $f(y) < f(x)$。對任意 $0 < \theta \le 1$，定義 $z_\theta = (1-\theta)x + \theta y$，則

$$f(z_\theta) \le (1-\theta)f(x) + \theta f(y) < f(x).$$

其中 z_θ 是可行線段點、θ 是混合權重。任意小的正 θ 都能讓該點任意靠近 x，而且改善目標，與局部最小性矛盾。證明不需要可微性 [1]。

它給全域性，卻不給唯一性。例如在 $0 \le x_1, x_2 \le 1$ 上最小化 x_1，每個 $(0, x_2)$ 都是全域最小解。嚴格凸目標會排除不同的同值最小解，因為可行中點的成本會更低。存在性仍獨立：在 $x > 0$ 下最小化 x 是凸問題，卻沒有最小解。

這三個問題決定對求解器的期待。凸性使局部最小解全域；嚴格曲率可選出唯一答案；封閉性與目標成長可確保答案存在。數值程序還需要適當收斂論證，不能只靠以上三點。

現代觀點

白話版。現代凸最佳化仍從建模與幾何開始；新方法改變如何存取可行集合，以及花多少成本取得準確證書。

Grant、Boyd 與 Ye 的 disciplined convex programming [2] 以已驗證的組合規則組織模型。這是建模方法，不保證任意方便公式都凸。代數上凸的式子，也可能需要改寫，讓規則系統看得出凸性。

Pokutta 的 2024 Frank–Wolfe 綜述 [3] 區分投影、barrier 與線性最小化 oracle 等集合存取方式。容易描述的集合，投影仍可能很貴；線性最小化有時較便宜，凸組合則維持可行。後續章節會推導更新與條件。

Applegate 等人的 PDLP 報告 [4] 於 2025 首次公開、2026 修訂，研究實用的一階線性規劃求解器，連結原始—對偶模型、尺度調整、restart 與可行性修整。這是原創求解器研究，不是綜述；它說明即使熟悉的凸問題，實作與需求精度仍重要。

目前挑戰包含以不過度膨脹的輔助變數表達複雜限制、利用稀疏矩陣、讓尺度下的容忍值有意義，以及把求解器嵌入較大流程時維持證書。學習式特徵可能非凸，後續分配問題卻凸，各階段保證應分開描述。

這是經挑選的閱讀路線，不宣稱檢閱所有最佳化綜述。核心理論依 [1]，現代觀點明確指出每個來源支持哪種延伸。

重點整理

- 變數、固定資料、目標、限制與單位必須明確。
- 可行、有界、達到最佳值與數值停止，是不同問題。
- 凸集合包含所有非負且權重總和一的平均。
- 仿射集合、多面體、範數球與錐是可重用積木。
- 交集與仿射轉換保留凸性，聯集通常不會。

- 閉凸集合投影唯一，並提供幾何證書。
- 凸最佳化保證針對建立的問題及其假設。

練習

1. 建立三種非負資源的預算配置模型，以平方距離接近固定目標，說明單位、變數與資料。

   提示
   向量 x、非負、總和限制與固定目標。

2. 直接證明兩個凸集合的交集凸，並提出聯集反例。

   提示
   線段留在兩者中；分離區間否證聯集。

3. 三種配置擇一時，何時凸混合能實際操作，何時只是鬆弛？

   提示
   時間分配與不可分硬體。

4. 推導中心在原點的二範數球投影，說明內部點的情況。

   提示
   外部點沿徑向落在邊界。

5. 提出最佳值有限卻未達到的凸問題，說明缺少哪個存在條件。

   提示
   開區間排除極限點。

6. 限制殘差為 10^{-6}，為何沒有尺度與應用單位就不能判斷是否足夠？

   提示
   同一不等式縮放後，原始殘差會改變。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004。**閱讀：第 1–2 章；第 4.1–4.2 與 8.1 節補充問題術語及投影**。官方書籍頁、官方免費 PDF。
2. Michael Grant, Stephen Boyd and Yinyu Ye, “Disciplined Convex Programming,” in *Global Optimization: From Theory to Implementation*, Springer, 2006。作者出版頁。
3. Sebastian Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024。開放綜述。
4. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O’Donoghue and Warren Schudy, “PDLP: A Practical First-Order Method for Large-Scale Linear Programming,” arXiv preprint, 2025，2026 修訂。已驗證論文記錄。

OPT 02・凸函數、組合規則與共軛

以弦線、支撐平面與曲率辨認凸目標，建立有效組合，理解 perspective、共軛與擬凸性。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/convex-functions/

你將學到

- 用定義域、弦線與上圖集合定義凸性。
- 在不假設處處可微時使用一階與二階條件。
- 辨認重要純量、範數與矩陣函數。
- 以加總、仿射代入、最大值、組合與 perspective 建模。
- 推導共軛，連結線性下界。
- 區分凸、強凸、擬凸與對數凹結構。

先看全貌

上一章描述允許的選擇，本章描述分數如何改變。凸目標有一種一致性：混合兩個選擇後，分數不高於兩端分數的同樣混合；局部支撐平面提供全域下界。

但熟悉的損失不一定對每個變數都凸。平方誤差對預測值與線性係數凸，對多層神經網路的全部參數通常不凸。必須說明正在最佳化的變數與定義域。

> **用最簡單的話說**
> 想像一個碗，用直棒連接碗上兩點，碗保持在棒下方。一塊從下方碰到碗的小平板，也一直在碗下方。這就是凸弦線與支撐平面的圖像。

Boyd 與 Vandenberghe 第 3 章 [1] 提供多種辨認方式。目標不是背大量公式，而是用少量基本函數與有效轉換規則，讓複雜目標有可檢查的凸性理由。

凸性包括定義域。某公式在所有可微處曲率都正，若完整定義域不連通，仍不是該集合上的凸函數。定義域需要和導數同樣仔細檢查。

弦線、定義域與上圖集合

白話版。連接兩個函數值的直線，在凸函數的所有中間輸入，都不低於真正函數值。

凸定義域 C 上的 $f : C \to \mathbb{R}$：

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta)f(y), \qquad 0 \le \theta \le 1.$$

$x, y \in C$ 是任意輸入，θ 是混合權重，所有選擇都成立時 f 為凸。反向不等式定義凹性；仿射函數等號成立，同時凸與凹。

此條件同時比較輸入與輸出。定義域凸，確保混合輸入可用；不能只對方便的部分端點驗證。

上圖集合（epigraph）：

$$\mathrm{epi} f = \{(x, t) : x \in C,\, f(x) \le t\}.$$

t 是純量高度，(x, t) 位於函數圖形上方或圖形上。f 凸，若且唯若其 epigraph 是凸集合，連結前章幾何與本章目標。

引入 t，最小化它且限制 $f(x) \le t$，就是 epigraph formulation。精確改寫不改變最佳值，也可把非平滑目標變成較容易表達的限制。

Jensen 不等式與期望

$$f\left(\sum_i \theta_i x_i\right) \le \sum_i \theta_i f(x_i), \qquad \theta_i \ge 0, \quad \sum_i \theta_i = 1.$$

x_i 是輸入，θ_i 是權重。隨機向量 X 若符合可積分與定義域條件，則 $f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]$。

因此先平均再算凸成本，可能比個別成本平均小。但這不表示平均任意模型參數都改善預測，還需要預測損失對那些參數凸。

非線性轉換後的期望，通常不等於期望後的轉換。偏向哪一側取決於凸或凹，不是「平均一定有代表性」的直覺。

支撐平面：一階條件

白話版。可微凸函數的切線近似是全域下界，不只是附近的草圖。

開凸定義域上，可微凸性等價於

$$f(y) \ge f(x) + \nabla f(x)^\top (y - x).$$

x 是近似位置，y 是任意定義域點，$\nabla f(x)$ 是對最佳化變數的梯度。右側仿射式從下方支撐函數。

對 $x + \theta(y - x)$ 使用弦線條件，整理、除以正的 θ，再令其趨近零，可得到此方向導數界。

若 $\nabla f(x) = 0$，不等式給所有 y 都有 $f(y) \ge f(x)$，因此無限制可微凸問題的駐點全域最佳，但未必唯一。

有限制時，非零梯度也可能最佳，因為改善方向被禁止。對偶與受限演算法章會明確表示此事；小梯度不是每種受限問題的合適停止規則。

非平滑支撐向量

角點不破壞凸性。次梯度（subgradient）g 在 x 滿足

$$f(y) \geq f(x) + g^{\top}(y - x) \quad \text{for every } y.$$

g 是提供全域仿射下界的向量，所有這類向量形成次微分 $\partial f(x)$。可微點只有梯度；絕對值在零的次梯度為 $[-1, 1]$。

絕對殘差、最大值與稀疏懲罰會用到此概念。次梯度不是在角點任選導數，必須滿足全域支撐條件。後面會連到 proximal 方法。

無限制凸函數的 $0 \in \partial f(x)$ 保證全域最佳。零梯度與零屬於一組次梯度，都是同一支撐觀念。

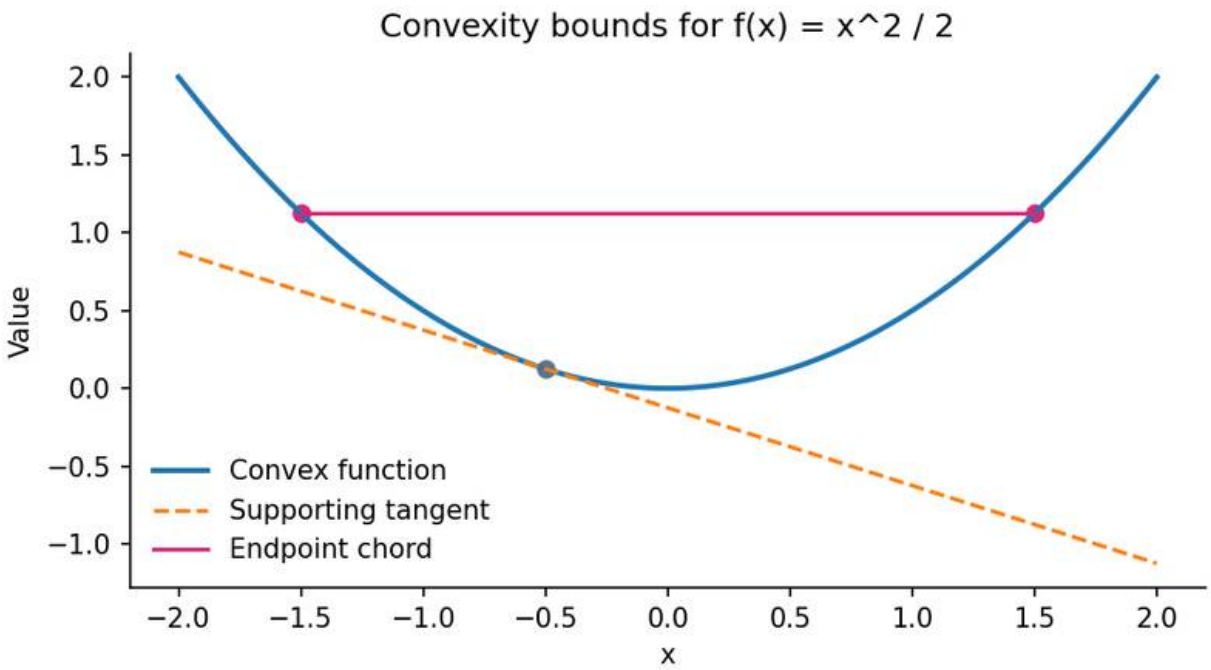


Figure 1 — 原創解析示意。凸函數位於端點弦線下方，指定位置的切線則提供全域仿射下界。

曲率：二階條件

白話版。平滑函數必須在所有方向都向上彎，才是凸；只看各座標可能漏掉向下彎的斜向方向。

開凸定義域上，二次連續可微 f 的凸性等價於

$$\nabla^2 f(x) \succeq 0 \quad \text{for every } x.$$

$\nabla^2 f(x)$ 是對稱 Hessian，半正定表示每個 $v^{\top}\nabla^2 f(x) v$ 都非負。

只有對角元素非負不夠。$f(x_1, x_2) = x_1 x_2$ 的座標二階導數為零，但 Hessian 不定，兩條對角方向分別向上與向下彎。

這是完整定義域上的全域條件，抽查十個位置的特徵值只是診斷。解析 Hessian 的半正定分解才可對所有位置建立證明。

嚴格凸與強凸

嚴格凸要求不同端點與嚴格介於零一的權重下，弦線不等式嚴格成立。凸集合上最小值存在時，嚴格凸可保證唯一。

二範數下的強凸要求某 $\mu > 0$：

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|_2^2.$$

μ 是統一曲率下界。二次可微時對應 $\nabla^2 f(x) \succeq \mu I$，$I$ 為單位矩陣。強凸比嚴格凸更強；嚴格凸未必有正的統一下界。

正二次正則化常在受懲罰座標提供強凸。未懲罰截距或限制定義域仍須分別分析，不能由一個正則化項推論全部參數強凸。

Hessian 條件實際量測什麼

多變數曲率測試可以化成每條直線上的純量測試。固定點 x 與方向 v，在仍屬於定義域的區間令 $q(t) = f(x + tv)$。微分兩次得到

$$q''(t) = v^\top \nabla^2 f(x + tv) v.$$

其中 t 是直線座標、q 是限制在直線上的純量函數、v 是任意方向。半正定 Hessian 讓每個直線限制都凸。每條線段都屬於某條直線，因此可推出弦線定義。反過來，每條直線都凸，就要求其二階導數非負。這說明測試為何檢查所有方向的二次型，而不是個別矩陣元素 [1]。

嚴格凸與強凸可以用具體例子區分。純量函數 $f(x) = x^4$ 在 $\mathbb{R}$ 上嚴格凸，因為導數 $4x^3$ 嚴格遞增。但 $f''(0) = 0$，所以不存在正的全域 μ 滿足強凸 Hessian 下界。其最小解唯一，卻不能使用 OPT 06 的強凸停止上界。加入 $\frac{\mu}{2}x^2$，其中選定 $\mu > 0$，可補足曲率下界，同時也改變目標。

有用函數的基本目錄

白話版。少數基本函數涵蓋很多實務目標，使用方式取決於其定義域與正負號。

仿射 $a^\top x + b$ 同時凸與凹。範數由齊次性與三角不等式建立凸性；平方二範數平滑，二範數本身在原點不平滑。

$$f(x) = \frac{1}{2} x^\top Q x + c^\top x + d.$$

Q 是對稱曲率矩陣，c 是線性係數向量，d 是純量。此函數凸若且唯若 $Q \succeq 0$；正定 Q 的最小特徵值提供強凸界。

指數處處凸，負對數在正輸入凸。冪次必須說清楚指數與定義域，「所有冪次都凸」是錯的；平方根在非負輸入凹，其負值則凸。

最大座標或仿射分數的最大值凸且通常非平滑；仿射函數的最小值凹，不普遍凸。正負號不能忽略。

矩陣定義域

對正定對稱 X，$-\log\det X$ 是凸函數 [1，第 3.1 節]。變數是矩陣，定義域排除奇異矩陣。

對稱擾動 H 的二階方向導數：

$$\operatorname{tr}(X^{-1}HX^{-1}H) \geq 0.$$

H 是對稱矩陣方向，tr 為 trace。Congruence 轉換後成為平方 Frobenius 範數，說明曲率非負。

對稱矩陣最大特徵值凸：

$$\lambda_{\max}(X) = \sup_{\|v\|_2=1} v^\top X v.$$

$\lambda_{\max}$ 表示最大特徵值，v 遍歷單位向量。固定 v 時二次型對 X 元素仿射，逐點上確界因此凸，展示「誰是變數」的重要性。

程式碼：二次函數與反例的 Hessian

以下 Hessian 固定，因此特徵值檢查適用所有輸入，並驗證一次支撐平面不等式。

```
import numpy as np

Q = np.array([[3.0, 1.0], [1.0, 2.0]])
bad = np.array([[0.0, 1.0], [1.0, 0.0]])
assert np.linalg.eigvalsh(Q).min() > 0
assert np.linalg.eigvalsh(bad).min() < 0
x, y = np.array([0.4, -0.6]), np.array([-1.0, 0.8])
f = lambda z: 0.5 * z @ Q @ z
support = f(x) + (Q @ x) @ (y - x)
gap = f(y) - support
assert np.isclose(gap, 0.5 * (y - x) @ Q @ (y - x))
print("quadratic Hessian eigenvalues:", np.linalg.eigvalsh(Q))
print("product Hessian eigenvalues:", np.linalg.eigvalsh(bad))
print("supporting-plane gap:", gap)
```

加總、代入與逐點最大值

白話版。以保留凸性的運算組合已知函數，運算規則就是證明。

凸函數的非負加權和凸，負係數通常翻轉或破壞性質。固定、與決策變數無關的分布下，定義良好的積分或期望也保留凸性。

仿射代入：

$$g(x) = f(Ax + b).$$

f 凸，A, b 固定。g 定義域要求變換後落在 f 的定義域。此規則建立平方殘差、範數殘差與線性估計正則化的凸性。

f_i 凸時，共同定義域上的 $\max_i f_i(x)$ 凸。引入 t 並限制所有 $f_i(x) \le t$，可建模最壞殘差、最壞情境與最大座標。

共同凸的輔助變數部分最小化也保留凸性：

$$g(x) = \inf_y F(x, y).$$

F 必須對 (x, y) 共同凸，y 是被消去變數；分別對各區塊凸不夠。這連結 epigraph 的座標投影。

哪些運算不自動保留凸性

凸函數相乘、相除，通常不保證凸；取最小值可能形成多個谷地。

x^2y^2 不會因為是兩個非負凸式的乘積就共同凸。Reciprocal 或指數外層需要正確單調性的組合論證。

固定參數乘變數是仿射，兩個決策變數相乘是 bilinear，通常非凸。把待學或未知量誤當固定資料，是常見建模錯誤。

組合與單調性

白話版。外層凸函數若以適當方向回應內層參數，就可保留凸性。

$$F(x) = h(g_1(x), \dots, g_k(x)).$$

h 是外層，g_i 是純量內層。充分規則是：h 凸，各參數上若內層凸則非遞減，若內層凹則非遞增，或內層為仿射。還須納入定義域與 extended-value 條件 [1，第 3.2 節]。

指數凸且遞增，所以指數作用在凸內層保留凸。負對數凸且遞減，作用在正的凹內層得到凸。

平方不能隨意作用在凸式：負輸入時它遞減，正輸入時遞增。非負凸內層平方是有效組合，任意正負則需要另證。

這些是充分辨認規則，不是判定所有代數式的完整程序。例如 $\sqrt{1 + x^2}$ 改寫成二範數，就能直接看出凸性。

從導數看組合規則

對純量平滑函數，鏈鎖律能看清單調性要求從哪裡出現。令 $F(x) = h(g(x))$，則

$$F''(x) = h''(g(x))[g'(x)]^2 + h'(g(x))g''(x).$$

其中撇號表示純量導數、g 是內層函數、h 是外層函數。若 h 凸且非遞減、g 凸，兩項都非負。若 h 凸且非遞增、g 凹，第二項仍非負。組合規則也適用非平滑函數，但這個計算揭露了正負號來源 [1]。

取凸內層式 $g(x) = x^2 - 1$ 與凸外層函數 $h(t) = t^2$。組合為 $(x^2 - 1)^2$，二階導數為 $12x^2 - 4$，在零點為負，所以組合不凸。內層輸出在零附近為負，平方函數在該區域遞減，因此所需符號配合失敗。

若定義域為凸，且凸內層式在整個定義域非負，平方就合法，因為外層在該區域非遞減。因此平方歐氏殘差範數凸，但「任意凸損失式都可再平方」不是一般有效規則。

Log-sum-exp 與穩定計算

白話版。 Log-sum-exp 平滑地從上方近似最大分數，也產生正規化權重。

$$f(x) = \log\left(\sum_{i=1}^{n} e^{x_i}\right), \qquad p_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

x_i 為純量座標，p_i 是正且總和一的權重。梯度為 p，Hessian 為

$$\nabla^2 f(x) = \operatorname{diag}(p) - pp^\top.$$

$\operatorname{diag}(p)$ 把權重放在對角。任意方向 v 的二次型是 v_i 的加權變異數，因此非負，證明凸。

所有座標加相同常數，函數也只平移該常數。因此全一方向曲率零，在完整空間不強凸。

溫度 $\tau > 0$ 下，$f_\tau(x) = \tau \log \sum_i e^{x_i/\tau}$：

$$\max_i x_i \le f_\tau(x) \le \max_i x_i + \tau \log n.$$

τ 控制平滑與近似差；越小越接近最大值，但可能使數值最佳化更病態。先扣除最大縮放座標，再做指數，能穩定計算。

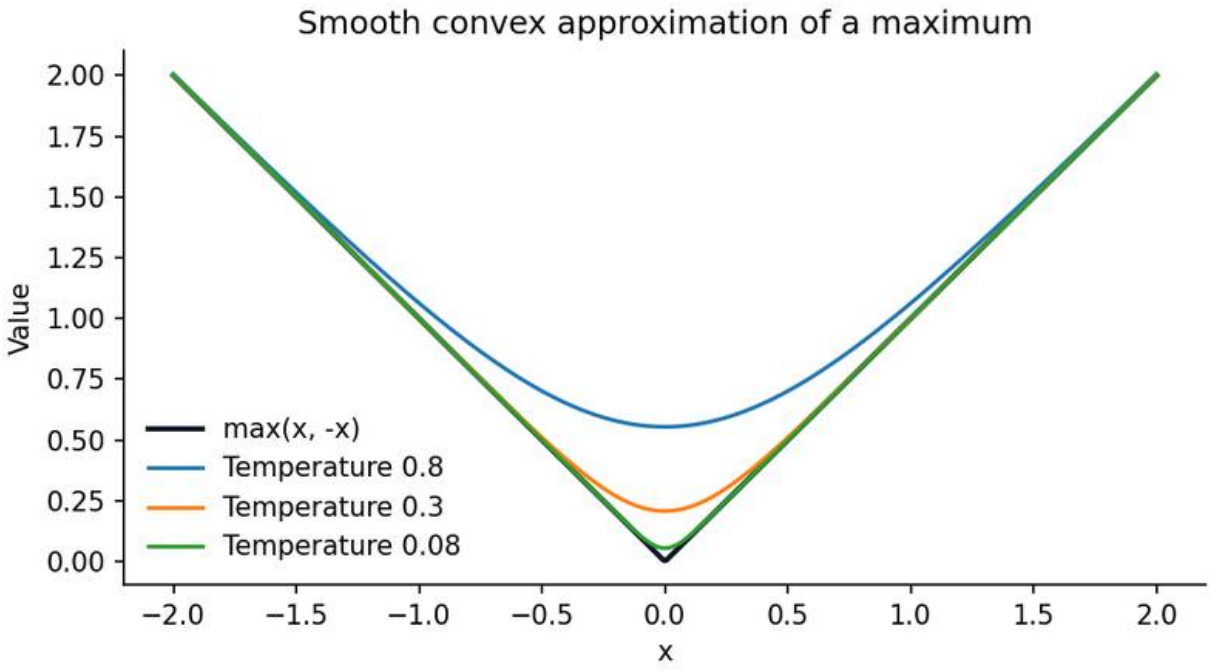


Figure 2 — 原創解析示意。溫度縮放 *log-sum-exp* 位於最大分數上方；較小溫度更接近，但轉折更尖。

程式碼：穩定計算與 Jensen 檢查

完整範例避免大輸入溢位，驗證 Hessian 並展示一次有限 Jensen 比較。

```
import numpy as np
from scipy.special import logsumexp

x = np.array([1000.0, 999.0, 997.0])
value = logsumexp(x)
p = np.exp(x - value)
H = np.diag(p) - np.outer(p, p)
assert np.linalg.eigvalsh(H).min() >= -1e-12
assert np.isclose(p.sum(), 1)
u, v = np.array([1., -1., 0.]), np.array([-2., 2., 1.])
theta = 0.3
left = logsumexp(theta * u + (1 - theta) * v)
right = theta * logsumexp(u) + (1 - theta) * logsumexp(v)
assert left <= right + 1e-12
print("stable value:", value, "weights:", p)
print("Jensen gap:", right - left)
```

Perspective：正規化變數並保留凸性

白話版。 Perspective 把正規化輸入的成本，和使用的尺度相乘，產生看似比值的有用凸式。

$$g(x,t) = tf(x/t), \qquad t > 0, \quad x/t \in \operatorname{dom} f.$$

f 凸，x 是向量，t 是正尺度，g 對 (x,t) 共同凸。正尺度定義域不可漏。

$f(u) = \|u\|_2^2$ 給 $g(x,t) = \|x\|_2^2/t$，稱 quadratic-over-linear。任意相除不保證凸，但此特定結構凸。

混合 (x_1,t_1) 與 (x_2,t_2) 後，正規化輸入的權重與 θt_1、$(1-\theta)t_2$ 成比例。對 f 使用凸性，再乘混合尺度，即可證明。

此形式用於正規化速率、分配時間或正質量，也可揭露精確錐表示。未仔細定義閉延伸前，不能任意允許零或負尺度。

Perspective 證明：正規化後權重會改變

普通平均與除以變動尺度不能直接交換。Perspective 仍然凸，是因為正規化產生另一組仍合法的權重。

取兩個定義域點 $(x_1,t_1),(x_2,t_2)$，其中 $t_1,t_2>0$，混合權重為 $\theta \in [0,1]$。定義

$$\bar{x} = \theta x_1 + (1-\theta)x_2, \qquad \bar{t} = \theta t_1 + (1-\theta)t_2,$$
$$\beta = \frac{\theta t_1}{\bar{t}}, \qquad \frac{\bar{x}}{\bar{t}} = \beta\frac{x_1}{t_1} + (1-\beta)\frac{x_2}{t_2}.$$

其中 $\bar{x}, \bar{t}$ 是混合後的分子與尺度；$\beta \in [0,1]$ 是正規化權重。對最後一行套用 f 的凸性，再乘上 $\bar{t} > 0$。係數變成 $\bar{t}\beta = \theta t_1$ 與 $\bar{t}(1-\beta) = (1-\theta)t_2$，恰好得到 $g(x,t) = tf(x/t)$ 的凸弦線界 [1]。

對二次除線性例子，取 $(x_1, t_1) = (2,1)$、$(x_2, t_2) = (2,3)$，且 $\theta = 1/2$。兩成本為 4 與 4/3，混合點 $(2,2)$ 的成本為 2，低於平均 8/3。其正規化輸入為 1：第一個正規化輸入 2 的權重為 $\beta = 1/4$，第二個 2/3 的權重為 3/4。依尺度調整權重是關鍵；直接平均正規化輸入會得到不同結果。

共軛與線性下界

白話版。共軛問某個線性分數最多能超過函數多少，答案決定該斜率下支撐線的最佳截距。

$$f^*(y) = \sup_{x \in \operatorname{dom} f} \{y^\top x - f(x)\}.$$

y 是斜率向量，x 是原變數，$f^*(y)$ 是差值上確界。有限值給下界：

$$f(x) \geq y^\top x - f^*(y).$$

這直接來自定義，是 Fenchel 不等式 [1，第 3.3 節]，用於對偶與 splitting 演算法。

$f(x) = \frac{1}{2} x^\top Q x$ 且 Q 正定時，最大差的點滿足 $Qx = y$：

$$f^*(y) = \frac{1}{2} y^\top Q^{-1} y.$$

Q 是曲率矩陣。程式用求解 $Qx = y$，不用明算反矩陣。

集合 indicator 的共軛是 support function。Extended-value indicator 在允許集合為零、其他地方正無限，可把限制併入目標，但不是有限懲罰。

Proper closed convex 函數取兩次共軛可回到原函數。缺少條件時，double conjugate 對應適當的閉凸包絡，不能任意假設重現原公式。

程式碼：共軛證書

二次 Fenchel 界在 $y = Qx$ 等號成立；範例檢查等號與其他斜率的非負 gap。

```
import numpy as np

Q = np.array([[4., 1.], [1., 2.]])
x = np.array([0.5, -0.8])
y = np.array([1.2, 0.3])
f = lambda z: 0.5 * z @ Q @ z
conjugate = lambda slope: 0.5 * slope @ np.linalg.solve(Q, slope)
gap = f(x) + conjugate(y) - x @ y
matching = Q @ x
assert gap >= -1e-12
assert np.isclose(f(x) + conjugate(matching), x @ matching)
print("Fenchel gap:", gap)
print("matching slope:", matching)
```

具有明確定義域的非平滑共軛

共軛能把非平滑懲罰轉成簡單的斜率限制。令 $f(x) = \lambda\|x\|_1$，其中固定 $\lambda > 0$。在定義中分開各座標，得到

$$f^*(y) = \sum_i \sup_{x_i \in \mathbb{R}} (y_i x_i - \lambda|x_i|) = \begin{cases} 0, & \|y\|_\infty \le \lambda, \\ +\infty, & \|y\|_\infty > \lambda. \end{cases}$$

其中 x_i, y_i 是原變數與斜率的座標；$\|y\|_\infty = \max_i |y_i|$。當 $|y_i| \le \lambda$，每個座標的式子至多為零，在 $x_i = 0$ 達到。只要某個斜率量值超過門檻，讓 x_i 同號並增加量值，就能使式子無界。

因此共軛的定義域是無限範數球。對應下界 $y^\top x \le \lambda\|x\|_1$ 在非零座標具有匹配斜率 $y_i = \lambda\,\mathrm{sign}(x_i)$ 時取等號。一個計算便連起範數幾何、次梯度與對偶可行性 [1，第 3.3 節]。OPT 08 會用同樣的門檻結構推導近端更新。

擬凸性與對數結構

白話版。有些函數不凸，但低於任何門檻的輸入集合都凸；另一些函數取對數後凸或凹。各自有不同保證。

擬凸（quasiconvex）要求所有下水平集合凸：

$$C_\alpha = \{x : f(x) \le \alpha\}.$$

α 是任意實數門檻，C_α 是其下水平集合。等價地，混合值不超過兩端較大值，比弦線條件弱。

凸函數都擬凸。區間上的單調純量函數，即使曲率非向上，也擬凸。適當形式下，可對門檻二分，每步解凸可行問題 [1，第 3.4、4.2 節]。

正函數 f 的 log-convex 表示 $\log f$ 凸，log-concave 表示 $\log f$ 凹。正性與定義域都是定義的一部分。密度的 log density 凹，不代表所有參數化下的估計問題都凸。

某些 geometric program 可對正變數取對數，精確轉成凸問題。這只適用指定類型，不是對任意目標和限制取對數的許可。下一章會推導。

公式的凸性與實作的凸性

數學凸表示式仍可能算得不好。指數溢位、矩陣邊界附近捨入，或意外陣列廣播，都可能使算出的量無效，原本函數卻依然凸。

應把符號曲率論證與數值評估檢查，作為不同證據。少數點的 Hessian 取樣可能找到反例，卻不能證明處處非負曲率。穩定實作也不會證明指定公式凸。

求解時參數固定，凸性宣稱針對的是該參數值下的決策變數。係數改變符號可能使組合規則失效。應記錄哪些參數範圍保留保證，不要無條件稱整個參數化族為凸。

學得函數也一樣。對後來當成決策變數的輸入為凸，不表示對訓練中的權重為凸。曲率規則一次描述一個最佳化問題。

現代觀點

白話版。資料學習函數時，仍可限制它對決策輸入凸，但訓練參數本身未必凸。

Disciplined convex programming [2] 將組合規則形式化，區分仿射、凸與凹，追蹤單調性與定義域。語法規則失敗可能只需要改寫，不一定否證數學凸性。

Schaller、Bemporad 與 Boyd 的 2025 稿件 [3] 擬合參數化凸函數，要求對有效參數值，函數在決策變數上仍凸且符合 disciplined programming。它連結資料學習與保持凸模型，是原創研究，不是綜述，也不保證任意神經損失。

Pokutta 的 2024 綜述 [4] 依幾何存取組織受限平滑方法，使用凸、平滑與強凸分析更新。全域凸性、統一下曲率與上平滑界有不同角色，應分別檢查。

開放問題包含結構函數類別的表達能力、雜訊參數下的定義域證明、不可微解映射，以及學習模型的泛化。最佳化證書證明擬合目標的性質，不證明它符合未來觀測。

重點整理

- 凸性同時包含凸定義域與弦線不等式。
- 切線與次梯度提供全域下界。
- 半正定 Hessian 建立所有方向的平滑凸性。
- 嚴格凸、強凸與平滑是不同假設。
- 組合規則需要單調性、正負號與定義域。
- Perspective 與共軛揭露比值和限制的結構。
- 擬凸與對數凸形式需要自己的推理。

練習

1. 從可微函數的弦線定義推導支撐平面不等式。

   提示

   整理弦線、除權重，再令其趨零。

2. 用二變數例子說明 Hessian 對角非負不保證凸。

   提示

   用 $x_1 x_2$ 的對角方向。

3. 把 $\sqrt{1 + x^2}$ 改寫成可辨認凸性的範數。

   提示

   向量 $(1, x)$ 的二範數。

4. 推導平方二範數的 perspective，說明定義域。

提示

代入 x/t 再乘正 t。

5. 推導 $\frac{1}{2}\|x\|_2^2$ 的共軛與 Fenchel 等號條件。

提示

微分二次差。

6. 提出區間上擬凸卻非凸的函數，驗證下水平集合。

提示

正區間上的平方根單調且凹，下水平集合仍為區間。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004。**閱讀：第 3 章；第 4.2 節的擬凸可行性**。官方書籍頁、官方免費 PDF。
2. Michael Grant, Stephen Boyd and Yinyu Ye, “Disciplined Convex Programming,” in *Global Optimization: From Theory to Implementation*, Springer, 2006。作者出版頁。
3. Michael Schaller, Alberto Bemporad and Stephen Boyd, “Learning Parametric Convex Functions,” manuscript, June 2025。已驗證稿件頁。
4. Sebastian Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024。開放綜述。

OPT 03・線性、二次、錐與幾何規劃

辨認標準凸問題，推導精確 epigraph 與錐改寫，轉換幾何規劃，檢查尺度與多目標權衡。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/convex-programs/

你將學到

- 辨認線性、二次與凸二次限制問題。
- 精確改寫絕對值、最大值與範數限制。
- 以正確矩陣順序表達二階錐與半正定限制。
- 對正變數取對數，把幾何規劃轉成凸問題。
- 區分精確改寫、凸鬆弛與近似。
- 選求解器前，說明尺度、可行檢查與多目標權衡。

先看全貌

模型可能凸，寫法卻把凸性藏起來。絕對殘差和看似非平滑，加入輔助變數就成為線性規劃；範數界變成錐限制；正變數設計可在對數座標下凸。

Boyd 與 Vandenberghe 第 4 章 [1] 整理這些問題家族，提供可重用模式，也幫助選擇利用結構的數值方法。類型由方程式與定義域決定，不由應用名稱決定。

> **用最簡單的話說**
>
> 不同拼圖有不同積木盒。有時拼圖看起來陌生，只是積木轉了方向，改寫規則就能認出熟悉類型。但增加新積木會改變拼圖，所以要分清楚改寫與鬆弛。

本章用合成小問題展示精確數學關係與數值檢查，不是 benchmark。程式使用既有數值函式庫，不新增建模系統依賴。

凸標準形式

白話版。目標、不等式與等式必須一起符合條件，只看目標凸不夠。

$$\begin{aligned} \text{minimize}_x \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1, \dots, m, \\ & Ax = b. \end{aligned}$$

$x \in \mathbb{R}^n$ 是決策向量；$f_0, \dots, f_m$ 在共同有效定義域凸；A, b 是固定等式係數與目標。凸下水平集合與仿射集合相交，得到凸可行集合。

凹函數可放另一側：g 凹時，$g(x) \geq a$ 等價於凸不等式 $a - g(x) \leq 0$。非線性凸函數等號通常不凸；$\|x\|_2^2 = 1$ 是球面，$\|x\|_2^2 \leq 1$ 才是凸球。

把等式拆成兩個不等式也不能修好，因為其中一側翻轉曲率。若物理問題真的要求球面，改成球就是鬆弛，可能改變答案。

精確改寫與改變問題

精確改寫在消去輔助變數或反轉一對一變換後，保留原決策與目標。鬆弛擴大允許集合，近似則修改函數或限制。

最小化 t 且 $f(x) \leq t$ 精確，因為目標偏好最小允許 t。但把相同不等式放進別的模型，若目標不偏好較小 t，未必達到等號。

增加變數時應證明兩方向：原可行點可延伸成新可行點，新點也映回原可行決策，且目標關係正確。

線性規劃

白話版。線性規劃的成本與規則都是直線，可行多面體具有角、面與可能延伸至無限的方向。

$$\begin{aligned} \text{minimize}_x \quad & c^\top x \\ \text{subject to} \quad & Gx \leq h, \qquad Ax = b. \end{aligned}$$

c 是固定成本向量，G, h 定義不等式，A, b 定義等式。座標界可另列或加入 G。

線性目標同時凸與凹，解未必唯一；若平行某可行面，整個面都可能最佳。

非負標準形式為 $Ax = b,\ x \geq 0$。加入非負 slack 將不等式轉成等式，自由純量可拆成兩個非負純量差。

這些改寫精確，但會增大問題與改變 conditioning。求解器若直接支援界與不等式，就不必為了形式手動展開。

絕對殘差變成線性限制

$$\min_x \|Ax - b\|_1.$$

A 是固定設計矩陣，x 是係數，b 是量測。每個殘差加入 t_i：

$$-t \leq Ax - b \leq t, \qquad \min_{x,t} \mathbf{1}^\top t.$$

t 限制殘差幅度，$\mathbf{1}$ 是全一向量。可行 t_i 不小於絕對殘差，目標加總使界緊。成對不等式已推得非負，也可明列。

Minimax 只加一個純量 t，限制 $-t\mathbf{1} \leq Ax - b \leq t\mathbf{1}$ 並最小化 t，控制最大絕對殘差，與總和代表不同偏好。

程式碼：小型配置 LP

SciPy 介面見 [2]。先檢查狀態，再獨立檢查可行性。

```
import numpy as np
from scipy.optimize import linprog

c = np.array([-3.0, -2.0])
G = np.array([[1.0, 1.0], [2.0, 1.0]])
h = np.array([4.0, 5.0])
result = linprog(c, A_ub=G, b_ub=h, bounds=[(0, None), (0, None)])
assert result.success, result.message
x = result.x
assert np.all(G @ x <= h + 1e-8)
assert np.all(x >= -1e-8)
assert np.allclose(x, [1, 3], atol=1e-7)
print("allocation:", x, "objective:", c @ x)
print("slacks:", h - G @ x)
```

負成本把最大化正收益轉為最小化。容量限制使非負選擇有界；移除關鍵界可能無界，是建模結果而非求解器缺陷。

二次規劃與凸 QCQP

白話版。二次規劃把直線成本換成碗狀成本，規則仍線性。向上彎的二次限制也能加入。

凸 QP 目標為

$$f_0(x) = \frac{1}{2} x^\top Q x + c^\top x + d, \qquad Q \succeq 0.$$

Q 對稱半正定，c 是線性係數，d 是不影響最佳解的常數，限制為仿射。

最小平方展開後 $Q = A^\top A$, $c = -A^\top b$。二次正則化加正對角曲率。設計矩陣 rank 不足時，未正則化解未必唯一。

凸 QCQP 允許

$$\frac{1}{2} x^\top Q_i x + a_i^\top x + r_i \le 0, \qquad Q_i \succeq 0.$$

Q_i 是限制曲率，a_i 是線性係數，r_i 是偏移。任意二次限制不保證凸；不定二次限制即使搭配強凸目標，也可能非凸。

數值結構很重要

無限制正定 QP 解 $Qx = -c$。受限最佳點可能在邊界，不能通常先解無限制再裁切。

一般二次函數耦合座標，調一個座標會改變其他座標最佳值。只有特定幾何，如普通二範數的盒投影，逐座標裁切才正確。

Q 大小與稀疏性影響計算。明算 $A^\top A$ 可能破壞稀疏性與放大病態，實作可保留殘差算子。演算法章會討論分解與矩陣向量乘法。

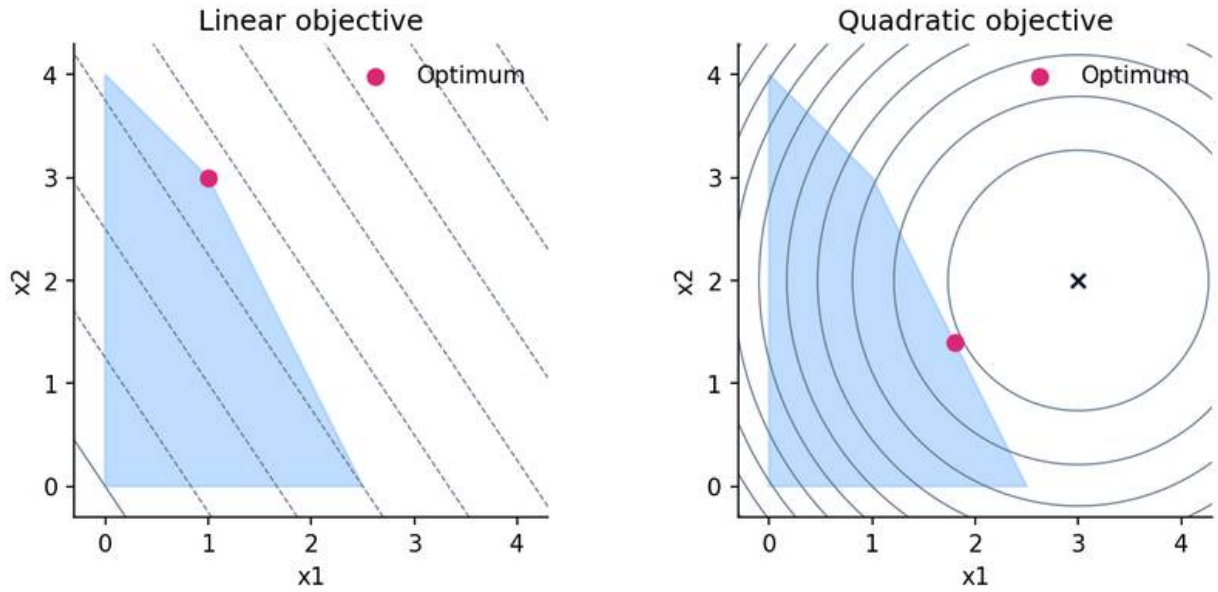


Figure 1 — 原創解析比較。本例線性成本偏好角點，二次成本選離目標最近的可行點；兩者使用相同凸可行集合。

反例算解：為什麼 QP 不能直接裁切

耦合曲率讓一個座標改變時，另一個座標的最佳值也跟著移動。考慮凸二次函數

$$f(x) = \frac{1}{2}(x-a)^\top Q(x-a), \qquad Q = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad a = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad x_1 \le 1.$$

其中 $x \in \mathbb{R}^2$ 是變數、a 是無限制目標、Q 是正定曲率矩陣。直接裁切 a 得到 $(1,0)$，目標值為一。但固定 $x_1 = 1$ 後，不受限制的第二座標之駐點條件給出 $(x_1 - 2) + 2x_2 = 0$，因此 $x_2 = 1/2$。

候選點 $(1, 1/2)$ 的目標值為 $3/4$，梯度為 $(-3/2, 0)$，由 $x_1 - 1 \le 0$ 的乘子 $3/2$ 平衡。可行性、乘子符號、駐點條件與互補鬆弛因此透過 OPT 04 的條件驗證最佳解。座標裁切忽略了 Q 的非對角元素；歐氏盒投影之所以可裁切，是因為其二次函數可按座標分離。

二階錐規劃

白話版。二階錐以範數小於純量表示弧形限制，形成標準幾何形式。

$$\|A_i x + b_i\|_2 \le c_i^\top x + d_i.$$

A_i, b_i 定義範數中的向量，c_i, d_i 定義仿射純量界，x 是決策。右側自動必須非負。

不能直接平方後漏掉右側正負條件，否則可能新增右側負值的點。錐表示正確保留正負。

常見 rotated second-order cone 要求 $u, v \geq 0$, $2uv \geq \|z\|_2^2$。係數二是慣例，需配合介面。令 $u = t$, $v = 1/2$, $z = Ax - b$，即可表達平方殘差不超過 t。

不同縮放數學等價，數值行為卻可能不同。

Robust 線性限制

係數不確定時，令 $a = \bar{a} + Eu$, $\|u\|_2 \leq 1$。要求所有允許 u 都滿足 $a^\top x \leq b$，等價於

$$\bar{a}^\top x + \|E^\top x\|_2 \leq b.$$

$\bar{a}$ 是名義係數，E 映射球狀不確定性，b 是固定界。範數來自單位球上 $u^\top E^\top x$ 的最大值。

此為指定不確定集合的精確 robust counterpart，不保護集合以外所有實際擾動。集合較大可能保守，集合不合理可能漏掉重要變異。

盒狀不確定性導向一範數項，橢球導向轉換後二範數，連到上一章的 support function [1]。

推導穩健對應式，而不是猜測懲罰

穩健範數項是容許擾動的精確最壞結果。展開不確定線性限制：

$$\sup_{\|u\|_2 \leq 1} (\bar{a} + Eu)^\top x = \bar{a}^\top x + \sup_{\|u\|_2 \leq 1} u^\top E^\top x = \bar{a}^\top x + \|E^\top x\|_2.$$

其中 x 是決策、$\bar{a}$ 是名目係數、E 是不確定性映射、u 是容許擾動。最後的等號來自內積範數上界；分母非零時，$u = E^\top x / \|E^\top x\|_2$ 可取等號。分母為零時，所有擾動的貢獻都為零。

對合成限制 $(1 + 0.2u_1)x_1 + (2 + 0.1u_2)x_2 \leq 4$，其中 $\|u\|_2 \leq 1$，精確穩健規則為 $x_1 + 2x_2 + \sqrt{0.04x_1^2 + 0.01x_2^2} \leq 4$。在 $x = (1, 1)$，左側最壞值為 $3 + \sqrt{0.05}$，因此穩健可行。最大化擾動與 $(0.2, 0.1)$ 成比例。這裡沒有憑空選調整權重；新增項直接來自所指定的不確定性幾何 [1]。

半正定規劃

白話版。 SDP 限制完整矩陣在每個向量方向都非負，不能只檢查元素。

$$F(x) = F_0 + \sum_{i=1}^{n} x_i F_i \succeq 0.$$

$F_0, \ldots, F_n$ 是固定對稱矩陣，x_i 是決策純量，$F(x)$ 必須半正定。搭配線性目標與仿射等式得到 SDP。

若矩陣全對角，退化為線性不等式；一般矩陣則編碼元素之間的關係。

$$X = \begin{bmatrix} 1 & r \\ r & 1 \end{bmatrix}.$$

r 是 off-diagonal correlation，此矩陣半正定若且唯若 $-1 \leq r \leq 1$。負元素可允許，過大正元素反而不定。

Schur complement 揭露結構

正定 P 下：

$$\begin{bmatrix} P & q \\ q^\top & t \end{bmatrix} \succeq 0 \quad \Longleftrightarrow \quad t - q^\top P^{-1} q \geq 0.$$

P 是對稱正定區塊，q 是向量，t 是純量。右側是 Schur complement。奇異 P 還需要 range 條件或廣義論證。

區塊對決策仿射時，可把部分反矩陣式轉成矩陣限制。凸性依賴仿射結構，不是只要寫成區塊矩陣就成立。

用配方證明 Schur 條件

區塊矩陣半正定，代表其二次型對每個向量都非負。對 $P \succ 0$、向量 v 與純量 α，

$$\begin{aligned} \begin{bmatrix} v \\ \alpha \end{bmatrix}^\top \begin{bmatrix} P & q \\ q^\top & t \end{bmatrix} \begin{bmatrix} v \\ \alpha \end{bmatrix} &= v^\top P v + 2\alpha q^\top v + \alpha^2 t \\ &= (v + \alpha P^{-1} q)^\top P (v + \alpha P^{-1} q) + \alpha^2 (t - q^\top P^{-1} q). \end{aligned}$$

其中 P 是正定區塊、q 是相容向量、t 是純量區塊、v, α 是任意測試座標。若 $t \geq q^\top P^{-1} q$，兩項都非負。反過來，選 $\alpha = 1$、$v = -P^{-1} q$，第一項消失，強制該純量不等式成立。如此證明等價性的兩個方向 [1]。

它也是建模方式。固定 P 且 $q(x)$ 為仿射向量時，上界 $q(x)^\top P^{-1} q(x) \leq t$ 可化成對 (x, t) 仿射的矩陣不等式。若 P 是矩陣決策變數，在正定定義域上，同樣的區塊表示仍可對其元素仿射。因此原反矩陣式雖把結構藏起來，非線性項仍可能具有凸上圖集合。

假設 $P \succ 0$ 才能配方並使用反矩陣。P 奇異時不能默默沿用同一公式；向量的值域條件會成為等價性的一部分。

程式碼：矩陣順序不是逐元素順序

此例檢查已知特徵值，並驗證 Schur 邊界。

```
import numpy as np

for r in [-0.8, 0.8, 1.2]:
    X = np.array([[1., r], [r, 1.]])
    eigenvalues = np.linalg.eigvalsh(X)
    print("r:", r, "eigenvalues:", eigenvalues)
    assert (eigenvalues.min() >= -1e-12) == (abs(r) <= 1)
P = np.array([[2., .3], [.3, 1.]])
q = np.array([.4, -.2])
t = q @ np.linalg.solve(P, q)
block = np.block([[P, q[:, None]], [q[None, :], np.array([[t]])]])
assert np.linalg.eigvalsh(block).min() >= -1e-12
print("Schur boundary t:", t)
```

這是 spectral 檢查，不是完整 SDP 求解器。小型直接計算說明模型，不暗示通用非線性 optimizer 提供專用錐方法的全部證書。

幾何規劃

白話版。正變數模型在原座標可能非凸，取對數使乘法變加法後，卻可能凸。

Geometric programming 的 monomial：

$$m(x) = c \prod_{i=1}^{n} x_i^{a_i}, \qquad c > 0, \quad x_i > 0.$$

c 正，a_i 可為任意實數，不同於基礎代數中的非負整數指數慣例。

Posynomial 是 monomial 加總：

$$f(x) = \sum_{k=1}^{K} c_k \prod_{i} x_i^{a_{ik}}, \qquad c_k > 0.$$

K 是項數，c_k 正，a_{ik} 是指數，不能有負係數。

GP 最小化 posynomial，限制 posynomial 不超過一、monomial 等於一，且變數正 [1，第 4.5 節]。任意加總、正負或等式不屬於此類。

推導對數轉換

令 $y_i = \log x_i$，則 $x_i = e^{y_i}$。Monomial 的對數仿射，posynomial 的對數：

$$\log f(e^y) = \log \sum_{k=1}^{K} \exp(a_k^\top y + \log c_k).$$

a_k 是第 k 項指數向量，右側為仿射式的 log-sum-exp，因此凸。

對數嚴格遞增，所以原目標與 log 目標有相同最佳選擇；回轉變數即可。Posynomial 不等式轉為凸不等式，monomial 等式轉為仿射等式。

目標數值已改變，報告應用成本時要轉回。這是正定義域的一對一變換；刪負項或把非 monomial 等式改不等式，則可能改變問題。

程式碼：對數座標的正配置

最小化 $1/x + 1/y$，且 $x + y \le 4$, $x, y > 0$。對稱性與凸目標給 $x = y = 2$，以下數值驗證。

```
import numpy as np
from scipy.optimize import minimize
from scipy.special import logsumexp
```

```
objective = lambda z: logsumexp(-z)
constraint = {"type": "ineq",
              "fun": lambda z: np.log(4.) - logsumexp(z)}
result = minimize(objective, np.log([1., 1.]),
                  constraints=[constraint], method="SLSQP",
                  options={"ftol": 1e-12, "maxiter": 100})
assert result.success, result.message
x = np.exp(result.x)
assert x.sum() <= 4 + 1e-7
assert np.allclose(x, [2., 2.], atol=1e-5)
print("positive variables:", x)
print("original objective:", np.sum(1 / x))
print("transformed objective:", result.fun)
```

介面見 [3]。這個小例與解析解比較，不宣稱通用方法在所有尺度可替代專用 GP 求解器。

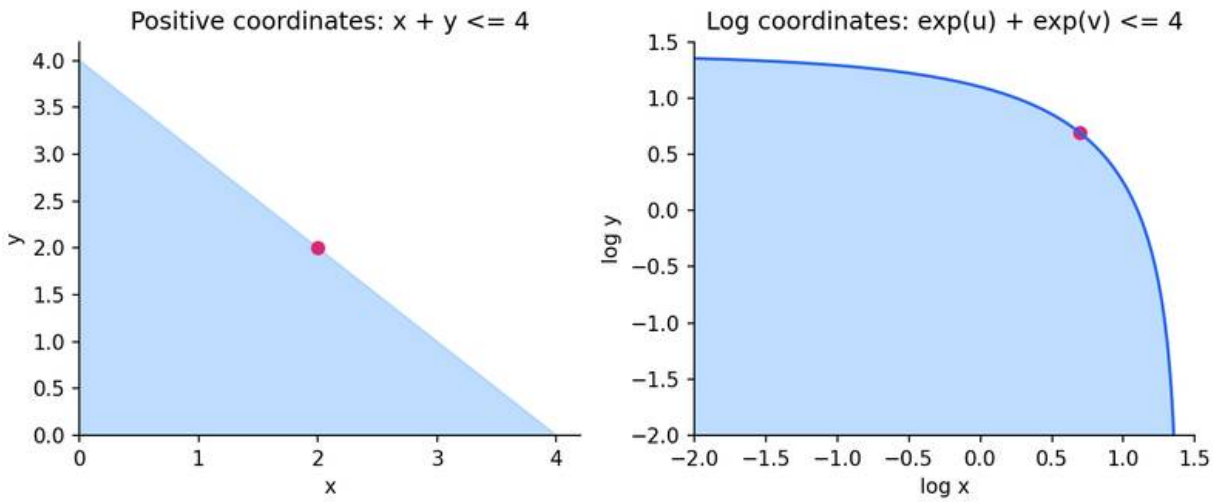


Figure 2 — 原創解析幾何。正變數下 $x + y \le 4$，對數座標變成 *log-sum-exp* $\le \log 4$。兩張圖的軸代表不同變數。

設計算例：非對稱的正配置

即使不能靠對稱性找到答案，對數轉換仍保留最佳解。在 $x+y \le 6$、$x,y > 0$ 下最小化 $4/x+1/y$。兩項都隨其變數增大而下降，因此最佳解會用完預算。

令 $y = 6 - x$。微分剩下的純量目標，得到

$$-\frac{4}{x^2} + \frac{1}{(6-x)^2} = 0 \quad \Longrightarrow \quad x = 2(6-x) \quad \Longrightarrow \quad (x^*, y^*) = (4, 2).$$

其中 x, y 是正配置量，星號表示最佳解。在 $0 < x < 6$ 上二階導數為正，所以駐點是唯一最小解，原成本為 $3/2$。

換成對數變數 $s = \log x$、$t = \log y$，目標變為 $\log(4e^{-s} + e^{-t})$，限制變為 $\log(e^s + e^t) \le \log 6$，兩者都是 log-sum-exp。轉換後解為 $(s^*, t^*) = (\log 4, \log 2)$，轉換成本為 $\log(3/2)$。若把 $\log(3/2)$ 回報成原成本，就混淆兩套座標。這個例子提供整個轉換的解析檢查 [1，第 4.5 節]。

以可行性求解擬凸問題

白話版。有些目標非凸，但「此分數能否達到」是凸可行問題。反覆詢問門檻，可找到最佳分數。

若 $\{x : f_0(x) \le \alpha\}$ 對每個固定門檻 α 都能用凸限制描述，可維持最佳分數上下界，檢查中間門檻：可行就降低上界，不可行就提高下界。

每個子問題的門檻是固定資料。若把它也當自由變數，可能產生非凸乘積；各子問題凸，不代表門檻與決策共同凸。

例如正分母下的仿射比值 $(a^\top x + b)/(c^\top x + d)$，固定 α 時限制變成 $a^\top x + b \le \alpha(c^\top x + d)$，對 x 仿射。分母嚴格正是必要條件。

接近最佳門檻時，可行區域可能很窄。近似求解失敗不等於不可行證書，需適當可行程序，報告門檻區間與容忍值。[1] 的擬凸討論擴展非凸目標，但保證仍精確限於凸子問題與外層二分。

求解前的建模稽核

逐條把原需求轉成變數，檢查定義域、正負號、單位，並追蹤輔助變數消去方式。

範數的輔助上界若不在目標中，可能保持鬆；若只要求限制，仍可正確，但不可直接當成實際範數。應自行計算真正範數。

Lifted 矩陣半正定凸，但要求等於向量外積會引入 rank 限制。刪除此限制是鬆弛，矩陣可能提供下界卻沒有對應原向量。

也要檢查邊界。GP 變數不能任意設零，log-determinant 矩陣不能在有限目標下奇異。允許這些點的改寫，需要明確延伸論證。

這些是數學驗證。求解器可能正確最佳化錯誤鬆弛或錯誤正負的模型；與小型解析案例比較，能檢查從文字到陣列再到解釋的整條流程。

多目標與 Pareto 權衡

白話版。兩目標衝突時，最佳化不會替你決定偏好；它提供指定偏好下的答案與權衡。

$F(x) = (f_1(x), \dots, f_k(x))$ 中，k 是目標數。若沒有其他可行點能改善一項而不傷害其他項，則為 Pareto 最佳。

$$\min_x \sum_{i=1}^{k} w_i f_i(x), \qquad w_i > 0.$$

w_i 編碼偏好與單位；達到的最佳解是 Pareto 最佳。零權重可忽略某項，只提供較弱效率敘述。

也可最小化一項，限制其他項不超過指定值。例如在固定記憶體預算下最小化重建誤差，比虛構 bytes 到誤差的通用換算更自然。

Scalarization 仍需符合凸規則。變動權重得到的是此模型的權衡曲線，不涵蓋被排除的離散選擇或上游系統行為。

建模與數值檢查

白話版。文字公式與實際陣列必須描述相同決策，微小錯誤可能悄悄改變限制。

檢查變數預設界。有的介面預設非負，有的自由；忽略預設可把原本自由迴歸係數變成非負。

檢查不等式方向。數學 $g(x) \le 0$ 與介面 $g(x) \ge 0$ 可能同時出現，必須刻意翻譯。

檢查形狀與對稱。非對稱陣列不符合對稱半正定記號；broadcasting 可能把純量限制變成不同向量限制。

縮放需保留與原變數的關係，解與殘差都應轉回原單位檢查。轉換形式與原形式都驗證。

狀態與證書也分開。小例有解析答案時，通用求解器成功提供數值支持；正式凸模型還應依類型檢查原始可行、對偶可行與 gap。

求解器轉換後，寫出的模型仍應可以審閱。應在同一份回報保留變數單位、定義域限制、轉換與所有鬆弛。若回傳轉換解，先映回原座標，再檢查物理限制與應用目標。

對數座標、輔助上圖變數與提升矩陣模型尤其需要這樣做。沒有映射的轉換可行點可能被誤解；若定義限制未準確滿足，輔助目標也可能與原表示式不同。

現代觀點

白話版。標準類型仍適用現代研究，新實作常利用結構改善成本，而非改變數學類型。

Disciplined convex programming [4] 透過曲率與單調性規則，將高層描述精確轉成標準形式。先讓凸性論證明確，再選求解器。

PDLP [5] 於 2025 公開、2026 修訂，以實用一階原始—對偶方法處理大型 LP，加入 preconditioning、restart 與可行修整。結論針對測試問題與精度，並非所有 LP 的通用冠軍。

Schaller、Bemporad 與 Boyd [6] 擬合符合 disciplined programming 的參數化凸函數，讓學習式目標或限制進入凸建模流程，但須維持有效參數與決策變數凸性。

開放問題包括減少不必要展開、利用錐稀疏性、尺度下可靠證明不可行，以及 active-set 變化附近的微分。凸類型是分析基礎，完整系統仍需資料、容忍值與工程驗證。

重點整理

- 凸標準形式需要凸目標與不等式、仿射等式。
- 輔助變數的精確性應證明兩方向。
- LP、QP、SOCP 與 SDP 編碼不同幾何。

- 矩陣順序不同於逐座標順序。
- GP 要求正變數與受限 monomial ／ posynomial。
- 多目標需要明確偏好或界限。
- 狀態、原變數可行性與最佳證書應分別檢查。

練習

1. 將 minimax 絕對殘差擬合改為 LP，證明等價。

提示

一個純量界限制所有正負殘差。

2. 範數右側可能負時，為何平方會新增點？

提示

範數非負。

3. 推導二乘二、對角一的半正定矩陣中 r 範圍。

提示

特徵值 $1+r, 1-r$。

4. 把 $x^{-1}+y^{-1}$ 且 $x+y\le 4$ 轉為 log 座標，再回復原目標。

提示

代入指數並對正式取 log。

5. 提出凸函數的非凸等號水平集合。

提示

正水平的平方範數。

6. 比較誤差—成本加權和，與固定成本限制下最小誤差的含意。

提示

一個是交換率，另一個是硬承諾。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004。**閱讀：第 4 章，特別 4.3–4.7；第 6.1 節範數擬合**。官方書籍、免費 PDF。
2. SciPy developers, “linprog,” *SciPy API documentation*, accessed 2026。已驗證介面。
3. SciPy developers, “minimize,” *SciPy API documentation*, accessed 2026。最佳化介面。
4. Michael Grant, Stephen Boyd and Yinyu Ye, “Disciplined Convex Programming,” in *Global Optimization: From Theory to Implementation*, Springer, 2006。作者出版頁。
5. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O’Donoghue and Warren Schudy, “PDLP: A Practical First-Order Method for Large-Scale Linear Programming,” arXiv preprint, 2025，2026 修訂。論文記錄。
6. Michael Schaller, Alberto Bemporad and Stephen Boyd, “Learning Parametric Convex Functions,” manuscript, June 2025。已驗證稿件。

OPT 04・對偶性、KKT 條件與證明憑證

推導拉格朗日下界，區分弱對偶與強對偶，以 KKT 驗證最佳性，並解讀乘子、敏感度與不可行性憑證。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/duality-kkt/

你將學到

- 從拉格朗日函數推導受限制最小化問題的下界。
- 區分對偶可行性、弱對偶、強對偶與最佳值是否達到。
- 寫出 KKT 條件，辨認它何時能證明全域最佳性。
- 以正確的乘子符號解讀互補鬆弛與敏感度。
- 建立簡單的不可行性憑證，並把對偶序關係延伸至錐。
- 回報殘差與間隙，同時保留數值證據真正支持的範圍。

先看全貌

找到好的可行解，能給最佳成本一個上界。但是，我們怎麼知道其他地方沒有藏著更好的答案？對偶問題會產生下界。當上界與下界相遇，兩者就共同證明最佳值。

Boyd 與 Vandenberghe 的第 5 章 [1] 把這個想法連到幾何、支撐平面、乘子與數值演算法。即使已經會解原問題，對偶仍有用：它能驗證答案、說明有限資源的價值，並揭露另一種計算結構。

> **用最簡單的話說**
>
> 你找到一份花十枚硬幣的食譜。另一個有力的論證指出，每份合乎規定的食譜至少都要十枚硬幣。兩者合在一起，就證明你的食譜最便宜。找到食譜與證明下界，是兩件不同的工作。

保證必須遵守符號與假設。任意選乘子算出的數值，不一定是下界。單靠很小的駐點殘差，也不是完整的證明。並非每個凸問題表示法都自動具備強對偶。

本章保留明確的定義域，使用能檢查精確答案的小例子推導。這套推理可以延伸至大問題，但實作細節與容許誤差仍是數值判斷的一部分。

從限制式到拉格朗日函數

白話說。替每個不等式與等式附上一個價格，再最小化加上價格後的目標，而不直接強制這些限制。

考慮

$$\min_{x\in D} f_0(x) \quad \text{subject to} \quad f_i(x)\le 0,\quad h_j(x)=0.$$

x 是決策向量，D 是最小化時保留的共同定義域，f_0 是目標，f_i 是不等式，h_j 是等式。

拉格朗日函數為

$$L(x,\lambda,\nu)=f_0(x)+\sum_{i=1}^{m}\lambda_i f_i(x)+\sum_{j=1}^{p}\nu_j h_j(x).$$

λ_i 是不等式乘子，ν_j 是等式乘子。若採用 $f_i\le 0$ 的慣例，就要求 $\lambda_i\ge 0$。等式乘子不限制符號，因為任何符號乘上可行點的等式殘差，結果都是零。

在可行點，不等式的貢獻非正，等式的貢獻為零。因此，當 λ 非負時，$L(x,\lambda,\nu)\le f_0(x)$。

把乘子稱為「懲罰」可能造成誤解。這裡的拉格朗日函數，並不是在無限制最小化時替違規加上一個很大的正成本。它的用途是建立一族下界，符號限制也由此而來。

定義對偶函數

$$g(\lambda,\nu)=\inf_{x\in D} L(x,\lambda,\nu).$$

g 是拉格朗日對偶函數，輸入為乘子對。它對 D 中的所有點取下確界，而非只看可行點。

對任一可行的 x 與非負的 λ，

$$g(\lambda,\nu)\le L(x,\lambda,\nu)\le f_0(x).$$

第一個不等式來自下確界，第二個來自原問題可行性與乘子符號。因此，g 是原問題最佳值的下界。

若下確界為負無窮，這仍是正確、但沒有提供資訊的下界。想取得有用的憑證，必須選擇能產生適當有限對偶值的乘子。

為什麼對偶問題是凹的

白話說。下界會隨價格改變。我們選擇能給最強下界的價格；這個最大化問題的目標是凹函數。

固定每個 x 時，L 對乘子都是仿射函數。仿射函數逐點取下確界，會得到凹函數。因此，即使原問題不是凸的，g 仍是凹的。

對偶問題是

$$\begin{aligned}\text{maximize}_{\lambda,\nu}\quad & g(\lambda,\nu)\\ \text{subject to}\quad & \lambda\ge 0.\end{aligned}$$

以 $d^\star$ 表示對偶最佳值。弱對偶（weak duality）指出 $d^\star\le p^\star$，其中 $p^\star$ 是原問題最佳值。這個結果不要求凸性，只需要符號正確的下界構造。

最大化凹函數，符合一般凸最佳化問題的意義。但是，求對偶函數值本身可能很困難，因為它也許要求解一個難解的非凸最小化。「對偶是凹的」不等於自動擁有容易計算的查詢工具。

對偶與原問題的維度也不必相同。每個明確對偶化的限制式各產生一個變數。把部分限制留在 D 中，而不加乘子，會改變寫出的對偶表示，但下界邏輯依然一致。

三行看懂凹性證明

對偶為凹，是因為取下確界後，混合價格的下界不會低於個別下確界的相應混合。令 $z = (\lambda, \nu)$ 收集乘子，取兩組容許價格 z_1, z_2 與 $\theta \in [0,1]$。拉格朗日函數對價格為仿射，因此

$$\begin{aligned} g(\theta z_1 + (1-\theta) z_2) &= \inf_{x \in D} [\theta L(x, z_1) + (1-\theta) L(x, z_2)] \\ &\geq \theta \inf_{x \in D} L(x, z_1) + (1-\theta) \inf_{x \in D} L(x, z_2) \\ &= \theta g(z_1) + (1-\theta) g(z_2). \end{aligned}$$

其中 D 是保留的原問題定義域、g 是對偶函數；此比較考慮有限的對偶值。在同一 x 處，每項都至少等於自己的下確界，因此得到中間不等式。這個步驟完全沒有假設原目標凸 [1]。

計算上必須區分：原問題非凸時，只找到 L 的局部最小值，並不等於計算其下確界。該局部值可能高於 $g(z)$，因而失去合法下界的意義。對偶凹性描述的是精確對偶函數，而不是任何自稱「對偶評估器」的近似程序。

完整的一維推導

白話說。一個小例子就能看清價格、無限制最小化解與下界之間的關係。

在 $x \leq 1$ 下最小化 $\frac{1}{2}(x-2)^2$。x 是純量。受限制最佳解是 $x^* = 1$，成本 $p^* = 1/2$。

使用乘子 $\lambda \geq 0$，

$$L(x, \lambda) = \frac{1}{2}(x-2)^2 + \lambda(x-1).$$

拉格朗日函數的無限制最小化解滿足 $x - 2 + \lambda = 0$，所以 $x_\lambda = 2 - \lambda$。代回後得到

$$g(\lambda) = \lambda - \frac{1}{2}\lambda^2.$$

x_λ 隨選擇的乘子而變，不一定是原問題可行點。對偶是一條凹拋物線，在 $\lambda^* = 1$ 達到最大值 $g(\lambda^*) = 1/2$。上界與下界於此相遇。

最佳乘子平衡了目標想增加 x 的力量，以及作用中的上界限制。若 λ 太小，拉格朗日最小化解會落在允許區域之外；若太大，即使價格更高，下界反而更弱。

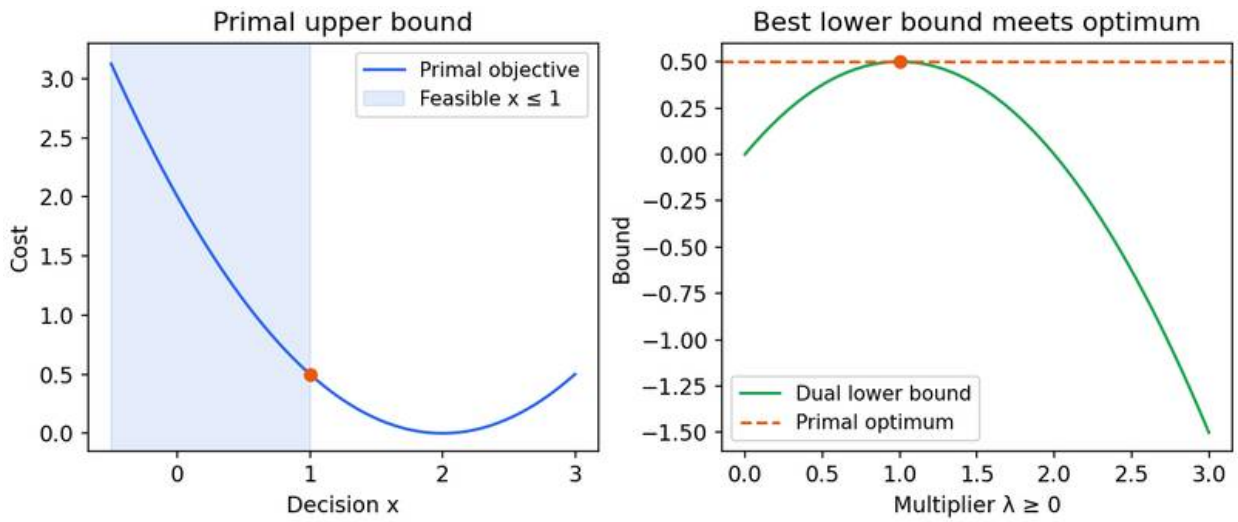


圖 *1* — 在 $x \le 1$ 下最小化 $(x-2)^2 / 2$ 的原創解析推導。可行原問題最佳解與最好的對偶下界，數值同為 $1/2$。

程式：驗證間隙與完整 KKT 條件

以下獨立計算會檢查原問題可行性、乘子符號、駐點條件、互補鬆弛，以及兩個數值相等。

```
import numpy as np

x, multiplier = 1.0, 1.0
objective = 0.5 * (x - 2)**2
dual_value = multiplier - 0.5 * multiplier**2
constraint = x - 1
stationarity = x - 2 + multiplier
complementarity = multiplier * constraint
assert constraint <= 0 and multiplier >= 0
assert abs(stationarity) < 1e-12
assert abs(complementarity) < 1e-12
assert np.isclose(objective, dual_value)
print("primal value:", objective, "dual value:", dual_value)
print("stationarity:", stationarity, "complementarity:", complementarity)
```

強對偶與限制資格條件

白話說。弱對偶永遠給出上下界次序。強對偶則說，最好的下界恰好等於最佳值；這需要額外條件。

強對偶（strong duality）表示 $d^* = p^*$。它描述最佳值，不會自動保證最佳解存在。問題可能只有一串解逼近某個數值，卻沒有任何點真正達到它。

對具有仿射等式的凸原問題，一個標準充分條件是 Slater 條件 [1，第 5.2 節]：

$$\exists x \in \operatorname{relint} D : f_i(x) < 0 \text{ for all } i, \qquad Ax = b.$$

D 是共同函數定義域，$\operatorname{relint} D$ 是它的相對內部，A, b 定義仿射等式限制。意思是存在一個點，位於適當內部、嚴格滿足不等式，並且滿足等式。

在這些假設下，強對偶成立。若最佳值有限，對偶最佳值也會達到。原問題是否達到最佳值，仍可能需要另外討論存在性。

仿射不等式有較弱的精細條件，所以要求它們全部嚴格成立，可能比必要條件更強。沒找到 Slater 點，本身不會推翻強對偶；它只表示這個充分條件尚未證明。

並非每種表示法中，凸性本身就已經足夠。定義域可能退化，限制資格條件也可能不成立。與其說「凸問題的間隙永遠是零」，應說明採用哪個條件，或直接驗證一對有效且相等的憑證。

定義域限制也是對偶的一部分

目標若包含對數或倒數，它的定義域仍要保留在拉格朗日最小化中。把它當成所有實數上都有定義的函數，會悄悄改變問題。

另一種方式是使用延伸值函數，在定義域外給正無窮。數學上這很方便，但數值程式不應把意外產生的無效浮點值當成模型慣例。

錐表示法可以一致地選擇對偶化或保留定義域與錐限制。這會影響對偶公式與符號。應從實際原問題推導下界，不要直接搬用符號不相容的公式。

零間隙也可能沒有最佳乘子

限制資格條件失敗不一定造成正對偶間隙，卻可能使乘子最佳值無法達到。考慮在 $x^2 \le 0$ 下最小化 x，其中純量 $x \in \mathbb{R}$。唯一可行點是零，所以 $p^\star = 0$，而且沒有嚴格可行點。

拉格朗日函數為 $L(x, \lambda) = x + \lambda x^2$。對 x 最小化得到

$$g(\lambda) = \begin{cases} -1/(4\lambda), & \lambda > 0, \\ -\infty, & \lambda = 0. \end{cases}$$

其中 $\lambda \ge 0$ 是二次限制的價格，g 是其對偶下界。對正 λ，最小化點為 $-1/(2\lambda)$。當 λ 無界增大，$g(\lambda)$ 趨近零。因此 $d^\star = p^\star = 0$，但沒有任何有限乘子達到對偶最佳值。

在原問題解處，駐點條件要求 $1 + 2\lambda x = 0$，卻變成 $1 = 0$。這種表示沒有 KKT 乘子。把等價限制改寫成等式 $x = 0$，就有乘子 $\nu = -1$。可行決策不變，正則性與乘子描述卻改善。這正是 OPT 01 的隱藏等式與 OPT 07 的初始化值得重視的原因 [1]。

KKT 最佳性條件

白話說。 KKT 結合允許的原問題選擇、允許的價格、力的平衡，以及「有餘裕的限制不能帶正價格」這條規則。

對可微凸函數 f_0, f_i 與仿射等式 $Ax = b$，條件為

$$
\begin{aligned}
& f_i(x) \le 0, \qquad Ax = b, \\
& \lambda_i \ge 0, \\
& \lambda_i f_i(x) = 0, \\
& \nabla f_0(x) + \sum_i \lambda_i \nabla f_i(x) + A^\top \nu = 0.
\end{aligned}
$$

x 是原問題候選點，λ 是不等式乘子，ν 是等式乘子。最後一列是拉格朗日駐點條件。整組稱為 Karush–Kuhn–Tucker 條件。

對這個凸且可微的問題，全部條件成立，就足以證明原問題與對偶最佳性。非負乘子使拉格朗日函數對 x 為凸；駐點條件使 x 成為它的全域最小化解；可行性與互補鬆弛則使拉格朗日值等於目標值。

在適當資格條件，例如 Slater 條件下，最佳點也會存在滿足 KKT 的乘子。這提供必要性，以及前述充分性。應把兩個邏輯方向分清楚。

非凸問題的拉格朗日駐點，可能是對 x 的局部最大值或鞍點。KKT 在正則條件下可能必要，通常卻不能證明全域最佳性。凹二次函數的內部駐點就是簡單反例。

互補鬆弛提供單向資訊

若 $f_i(x) < 0$，互補鬆弛會迫使 $\lambda_i = 0$。若 $\lambda_i > 0$，限制必須作用中：$f_i(x) = 0$。

作用中的限制仍可能有零乘子。退化或冗餘限制可能使價格不唯一。因此，「作用中」與「正價格」不能互換。

在平滑受限制最佳點，負目標梯度會落在作用中限制法向量與等式法向量、搭配適當符號形成的組合裡。這解釋了駐點條件的幾何意義：可行下降方向無法降低目標。

數值上的接近作用中，依賴容許誤差。很小的餘裕搭配很小的乘子，可能難以做離散分類。若區別重要，應回報實際數值，避免硬給確定的作用中或非作用中標籤。

等式限制二次問題

白話說。二次目標搭配線性等式時，KKT 條件形成一個線性系統，揭露後續受限制演算法需要的代數。

在 $Ax = b$ 下，目標為 $\frac{1}{2}x^\top Qx + c^\top x$，則

$$
\begin{bmatrix} Q & A^\top \\ A & 0 \end{bmatrix}
\begin{bmatrix} x \\ \nu \end{bmatrix}
=
\begin{bmatrix} -c \\ b \end{bmatrix}.
$$

Q 是對稱半正定矩陣，c 是目標係數向量，A 是等式矩陣，b 是目標值，ν 是等式乘子。

即使最佳化問題是凸的，這個區塊矩陣通常仍是不定矩陣。要求正定矩陣的求解器，不能直接套用到完整 KKT 系統。

若 Q 正定，且 A 的列獨立，系統就非奇異。更一般的情況，需要考慮等式零空間上的曲率，以及冗餘限制。偽逆可能給出一個解，但不應掩蓋背後的秩問題。

等式乘子會在可行仿射集合的法向方向，平衡目標梯度。它的符號不受限制。移除限制會同時改變可行集合與乘子的意義。

謹慎推導 LP 對偶

白話說。對偶公式沒有一套通用符號。它的正負號來自實際使用的原問題慣例。

取 LP：在 $Ax = b,\ x \geq 0$ 下最小化 $c^\top x$。把非負限制保留在定義域。拉格朗日函數為 $c^\top x + \nu^\top (Ax - b)$。

對非負 x 取下確界，恰在 $c + A^\top \nu \geq 0$ 時有限。任一係數為負時，該座標可以無限增大，使下確界變成負無窮。

令 $y = -\nu$，對偶變成

$$\max_y b^\top y \quad \text{subject to} \quad A^\top y \leq c.$$

y 是符號不受限制的等式價格向量。對任何可行的原問題與對偶解，

$$c^\top x - b^\top y = x^\top (c - A^\top y) \geq 0.$$

間隙是非負原問題座標與非負對偶餘裕的內積。這是特別直觀的弱對偶表示。

改用原問題最大化，或反轉不等式，就會改變對偶符號。推導拉格朗日函數並檢查下界，比記憶不同表格更能防止錯誤。

從範數共軛建立擬合對偶

對偶可把參數懲罰變成殘差限制。考慮稀疏線性擬合，目標為 $\frac{1}{2}\|Ax - b\|_2^2 + \lambda \|x\|_1$，其中 A, b 固定、$\lambda > 0$。引入殘差變數 $r = Ax - b$ 與不受符號限制的等式乘子 y：

$$L(x, r, y) = \frac{1}{2}\|r\|_2^2 + \lambda \|x\|_1 + y^\top (Ax - b - r).$$

其中 x 是參數向量，y 的每個座標對應一筆觀測。對 r 最小化得到 $r = y$，殘差相關項的值為 $-\|y\|_2^2/2$。依 OPT 02 的範數共軛計算，對 x 最小化恰好在 $\|A^\top y\|_\infty \leq \lambda$ 時有限。因此對偶為

$$\text{maximize}_y \quad -\frac{1}{2}\|y\|_2^2 - b^\top y \qquad \text{subject to} \quad \|A^\top y\|_\infty \leq \lambda.$$

其中 y 是對偶變數，範數限制是對偶可行性。在最佳解處，它等於擬合殘差，而殘差與設計欄向量的相關量必須留在懲罰門檻內。這連起下界構造與稀疏擬合幾何 [1，第 5–6 章]。

對合成 $A = I$、$b = (2, -0.3, 0.1)$ 與 $\lambda = 0.4$，原問題解為 $x = (1.6, 0, 0)$，對偶候選為 $y = x - b = (-0.4, 0.3, -0.1)$。兩者分別可行，目標值皆為 0.77。上下界相等才證明擬合最佳；只看到零座標並不足夠。

敏感度與限制的價值

白話說。在指定正則條件下，乘子衡量最佳成本如何隨限制變動。它的單位取決於限制的單位。

把不等式改為 $f_i(x) \le u_i$，等式改為 $Ax - b = v$。以 $p^\star(u, v)$ 表示最佳成本。若零擾動時強對偶成立且最佳乘子存在，

$$p^\star(u, v) \ge p^\star(0, 0) - \lambda^{\star\top} u - \nu^{\star\top} v.$$

u, v 是右側擾動；$\lambda^\star, \nu^\star$ 是原問題的最佳乘子。依此慣例，正 u_i 會放寬不等式。

此式給擾動最佳值一個全域仿射下界。若值函數在原問題處可微，它的導數就是 $-\lambda^\star$ 與 $-\nu^\star$。

若不可微，乘子可能描述支撐斜率，而非唯一導數。大的乘子表示模型在這個局部方向敏感，不是普遍適用的物理重要性分數。

上界改變的純量例子

在 $x \le b$ 下最小化 $\tfrac{1}{2}(x - 2)^2$。最佳點是 $\min(2, b)$，最佳值為

$$p^\star(b) = \frac{1}{2}\max(2 - b, 0)^2, \qquad \lambda^\star(b) = \max(2 - b, 0).$$

b 是允許的上界。在可微處，$dp^\star/db = -\lambda^\star(b)$。一旦上界超過無限制目標，再放寬已沒有幫助，乘子就是零。

限制縮放會使乘子反向縮放。把 $x - b \le 0$ 乘上正數，不會改變可行點，但會改變價格。比較敏感度時，應先處理單位與正規化。

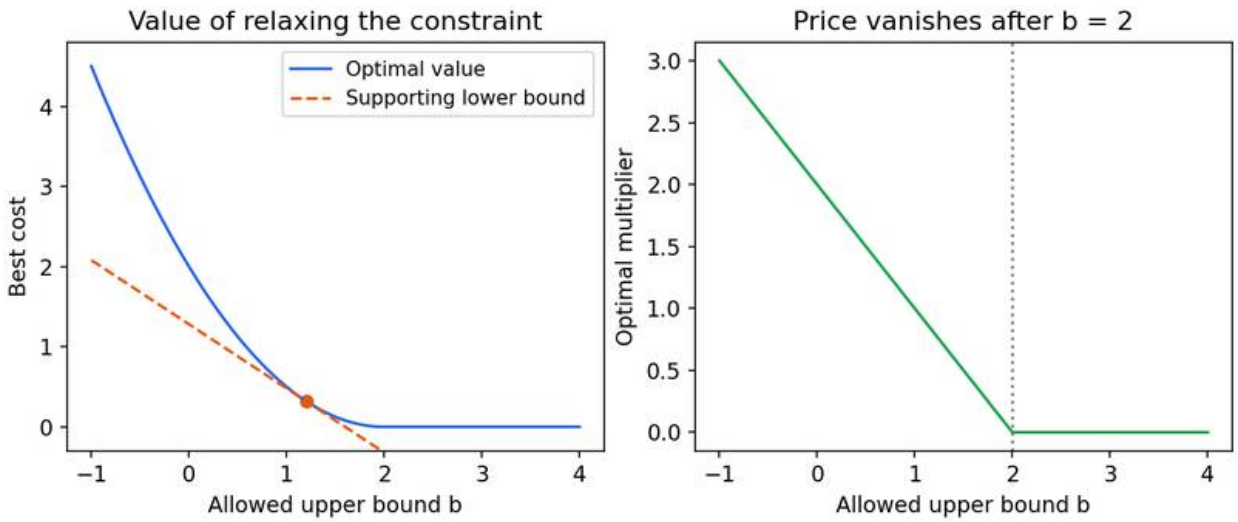


圖 2 — 原創解析敏感度例子。限制仍有效時，放寬上界會降低成本；允許無限制目標之後，最佳值與乘子都為零。

程式：有限差分敏感度檢查

```
import numpy as np

value = lambda b: 0.5 * max(2 - b, 0)**2
b, eps = 1.2, 1e-5
multiplier = max(2 - b, 0)
derivative = (value(b + eps) - value(b - eps)) / (2 * eps)
assert np.isclose(derivative, -multiplier, atol=1e-8)
test_bounds = np.linspace(-1, 4, 50)
lower_bound = value(b) - multiplier * (test_bounds - b)
assert np.all([value(t) for t in test_bounds] >= lower_bound - 1e-12)
print("finite-difference derivative:", derivative)
print("negative multiplier:", -multiplier)
```

從同一個下界推導敏感度

敏感度不等式不需要新原理。把原問題的最佳乘子放進擾動後的拉格朗日函數。對任何滿足 $f_i(x) \le u_i$ 與 $h_j(x) = v_j$ 的點，

$$f_0(x) \ge g(\lambda^*, \nu^*) - {\lambda^*}^{\top} u - {\nu^*}^{\top} v.$$

其中 u, v 是擾動向量，星號乘子對未擾動問題最佳。對擾動可行點取下確界，再使用原問題的強對偶，就得到前述值函數下界。負號來自在拉格朗日函數中扣除擾動後的右側。

對純量上界例子，在 $b = 1$ 時值為 $1/2$、$\lambda^* = 1$。把上界增加 δ，其中 $0 < \delta < 1$，實際值變成 $(1-\delta)^2/2$。實際下降為 $\delta - \delta^2/2$，乘子則預測一階下降 δ。下界預測 $1/2 - \delta$ 保持在實際值下方，差為 $\delta^2/2$。

若限制改寫為 $10(x - b) \le 0$，乘子變成 $1/10$。限制對 b 的導數為 -10，與乘子相乘仍預測斜率 -1。價格依賴表示方式，原成本變動的預測則一致 [1，第 5.6 節]。

不可行性憑證

白話說。合法加權組合限制式，可以證明沒有任何選擇滿足它們。這比演算法沒找到可行點更有力。

對不等式 $Gx \le h$，假設向量 y 滿足

$$y \ge 0, \qquad G^{\top} y = 0, \qquad h^{\top} y < 0.$$

G, h 定義不等式系統，y 是非負憑證向量。若可行 x 存在，加權求和就會得到 $0 = y^{\top} G x \le y^{\top} h < 0$，產生矛盾。

這是 [1，第 5.8 節] 討論的擇一定理形式背後的幾何。它區分數學憑證與找不到解的搜尋。

在浮點數下，等式與嚴格負號需要相對尺度判斷。很小的轉置殘差，加上極小的負憑證值，可能受累積誤差影響而無法下結論。數值認證需要適當容許誤差分析。

程式：互不相容的純量不等式

限制為 $x \le 0$ 與 $x \ge 1$。非負相加就能證明不可能。

```
import numpy as np

G = np.array([[1.], [-1.]])
h = np.array([0., -1.])
certificate = np.array([1., 1.])
assert np.all(certificate >= 0)
assert np.allclose(G.T @ certificate, 0)
assert h @ certificate < 0
print("transpose residual:", G.T @ certificate)
print("certificate contradiction:", h @ certificate)
```

錐、鞍點與數值間隙

白話說。從相應對偶錐選擇價格，就能把下界推理延伸到錐不等式。原問題與對偶解也可視為鞍點。

錐 K 的對偶錐定義為

$$K^* = \{y : y^\top z \ge 0 \text{ for every } z \in K\}.$$

z 遍歷原錐，y 是價格向量。若限制寫成屬於 K，應選擇讓對偶錐內積產生正確下界的符號。正交象限、二階錐與半正定錐，在常用內積下都有熟悉的自對偶結構。

矩陣變數的內積使用跡：對對稱矩陣，$\langle X, Y\rangle = \operatorname{tr}(XY)$。互補鬆弛因此是跡條件，不是逐元素相乘。

在適當原問題與對偶假設下，鞍點滿足

$$L(x^*, \lambda, \nu) \le L(x^*, \lambda^*, \nu^*) \le L(x, \lambda^*, \nu^*).$$

x^* 與 (λ^*, ν^*) 是最佳原問題與對偶點；x 遍歷定義域，$\lambda \ge 0$。左側在可行點最大化價格，右側在最佳價格下最小化決策。

實務上，只有兩點適當可行，且正確計算對偶值時，目標差才是有證明力的間隙。近似可行可能需要修正上下界。相對間隙應使用明確記錄的分母，在接近零或負目標值時仍有意義。

完整 LP 憑證與無效近似解

考慮在 $x_1 + x_2 = 1$、$x \ge 0$ 下最小化 $x_1 + 3x_2$。原問題點 $(1, 0)$ 成本為一。依前面推導的 LP 對偶慣例，純量對偶變數 $y = 1$ 可行，因為 $(y, y) \le (1, 3)$，對偶值也是一。上下界相等，不需枚舉所有可行點就能證明最佳性。

對偶餘裕為 $(0, 2)$，與原問題點的內積為零，直接呈現互補鬆弛。第二座標相對等式價格較昂貴，因此配置為零。此解釋來自係數與指定等式，不是一般特徵重要性結論。

現在取近似點 $(1.01, -0.01)$。它精確滿足等式，目標為看似更好的 0.98，卻違反非負性。與對偶值相減得到負數，不會推翻弱對偶，因為候選點在可行集合外。

只顯示等式殘差與目標的程式，可能因此回報誤導結果。應檢查所有邊界與錐限制，包括透過求解器變數邊界參數編碼、沒有放進明確矩陣列的限制。

裁切負座標得到 $(1.01, 0)$，隨即違反等式。在這個特定例子，再正規化會得到 $(1, 0)$；但裁切與正規化都不是耦合限制的通用有效修補。

近似殘差需要修正推理

對近似可行 LP 點對，代數目標差為

$$c^\top x - b^\top y = x^\top s + y^\top r, \qquad s = c - A^\top y, \quad r = Ax - b.$$

x, y 是原問題與對偶候選向量，s 是對偶餘裕，r 是原問題等式殘差。恆等式是精確代數；要得到非負且有證明力的間隙，還需要必要可行性。

若 $x \geq 0$、$s \geq 0$，但 $r \neq 0$，殘差項可能正或負。小殘差乘上大價格，仍可能重要。因此，單靠絕對等式容許誤差，不是通用目標準確度保證。

若對偶餘裕逐分量只保證不小於 $-\varepsilon$，而 $x \geq 0$，就有 $x^\top s \geq -\varepsilon \|x\|_1$。$\varepsilon \geq 0$ 衡量最壞負餘裕。大原問題向量可能放大微小對偶違反。

除非另外有所有可行原問題點的有效界，這個不等式不會把近似對偶值變成全域下界。無界可行集合尤其難以修正。不要把任意近似乘子描述為精確全域憑證。

應一起回報原問題殘差、負餘裕大小、乘子範數與目標差。若需要嚴格憑證，應依問題結構採有效修補、已驗證誤差上界，或更高精度檢查。

現代觀點

白話說。現代大規模方法直接使用原問題與對偶結構，應用也越來越常對受限制解做微分。兩者都依賴同樣的憑證與正則性觀念。

Applegate 等人的 PDLP 報告 [2] 在 2025 年首次發布，2026 年修訂，討論實用原問題與對偶步驟及可行性修整的大規模 LP 計算。它在此的意義是：有用的系統要同時監測可行性與成本改善。若目標誤差小，卻帶著不可接受的限制違反，仍是另一種結果。

Pokutta 的 2024 年綜述 [3] 說明 Frank–Wolfe 方法的原問題與對偶收斂。對可行集合做線性最小化，能從支撐平面不等式建立一階間隙。這與下界憑證有關，但假設與計算方式不同於一般拉格朗日乘子對偶。

Agrawal 等人的可微凸最佳化層 [4]，把結構化表示法與解的微分連起來。原論文發表於 2019 年，這裡把它作為現今最佳化層研究的基礎，不把它標成 2026 年新成果。退化或作用中集合改變附近，可微性與良好條件數可能失效；正確凸解不會自動保證良好的梯度。

目前問題包括超大規模的可靠憑證、限制由學習取得時的不確定性解讀、非唯一解的微分，以及精度和應用延遲的平衡。最佳化精度、模型有效性與學習泛化，應分別評估。

重點整理

- 可行原問題點給上界，可行對偶點給下界。
- 乘子符號取決於實際不等式慣例。
- 即使原問題非凸，對偶函數仍是凹的。
- 強對偶需要適當條件，或直接驗證相等的有效憑證。
- KKT 對可微凸問題是充分條件，必要性則需要正則性。
- 互補鬆弛與敏感度有明確的單向規則及符號。
- 數值殘差與間隙必須結合尺度、可行性解讀。

練習

1. 在 $x \le b$ 下最小化 $\frac{1}{2}(x-a)^2$，把 a, b 視為固定資料，推導對偶。

   提示
   先對純量最小化拉格朗日函數，再最大化乘子。

2. 給一個在最佳點作用中、乘子卻為零的不等式。

   提示
   無限制最小化點的冗餘邊界即可。

3. 保留非負變數在定義域中，推導 LP 對偶。

   提示
   要讓下確界有限，線性係數必須非負。

4. 說明在 $[-1, 1]$ 最小化 $-x^2$ 時，原點 KKT 駐點條件為何不能證明最佳性。

   提示
   原點是目標最大值。

5. 建立一組非負加權矛盾，證明兩個純量不等式不可行。

   提示
   把低於必要下界的上界加起來。

6. 把不等式乘上十，推導乘子的縮放，並說明原問題敏感度為何保持一致。

   提示
   保留拉格朗日函數中的乘積。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **閱讀：第 5 章，涵蓋對偶、KKT、擾動、擇一定理與廣義不等式**。官方書頁，免費 PDF。
2. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue and Warren Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. 已核對論文。
3. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. 開放綜述。
4. Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond and J. Zico Kolter, "Differentiable Convex Optimization Layers," *NeurIPS*, 2019. 官方出版頁。

OPT 05・近似、統計估計與實驗設計

選擇凸殘差損失、正則化與形狀限制，建立概似和偵測器問題，並以最佳實驗設計配置量測資源。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/fitting-estimation/

你將學到

- 依重要的誤差選擇殘差範數，而非只沿用習慣。
- 區分參數正則化、穩健殘差損失與不確定量測模型。
- 使用線性不等式，讓擬合函數保持單調或凸。
- 辨認凸的概似與機率估計表示法。
- 設計隨機化偵測器，理解以動差建立的風險上界。
- 使用資訊矩陣與明確準則，配置量測資源。

先看全貌

最佳化模型可以替觀測擬合曲線、估計分布，或決定接下來收集哪些觀測。它們有共同模式：選擇表示方式、說明如何衡量不一致，再指定允許哪些選擇。

Boyd 與 Vandenberghe 的第 6–7 章 [1] 連結近似與統計估計。適合一起閱讀：同一個平方殘差，可以是幾何近似誤差、雜訊模型下的概似，或受限制設計目標的一部分。即使數值公式相同，解釋仍帶有不同假設。

> **用最簡單的話說**
> 你想找一把靠近許多點的尺。一條規則在意平均不準多少；另一條在意最差的點；第三條不允許尺彎得太厲害。找最佳尺以前，要先決定哪種「最好」符合用途。

成功求解器回答的是你寫下的最佳化問題。它不會證明量測假設正確、曲線能預測新資料，或感測器配置可以實際執行。本章把這些問題與數學一起保留。

例子使用合成觀測與小型量測模型，展示各部分如何運作，不在程式裡隱藏下載、預訓練系統或特定應用假設。

殘差範數表達優先順序

白話說。殘差是預測與觀測的差。選擇哪種範數，會決定哪些誤差模式特別昂貴。

線性模型為

$$r = Ax - b, \qquad \min_x \|Ax - b\|.$$

$A \in \mathbb{R}^{m\times n}$ 是固定設計矩陣，$b \in \mathbb{R}^m$ 是觀測，$x \in \mathbb{R}^n$ 是參數，$r \in \mathbb{R}^m$ 是殘差向量。所選範數作用於殘差，不是直接作用於參數。

歐幾里得範數或其平方，透過平方貢獻強調大殘差。絕對值範數加總殘差大小。最大值範數則最小化最大的絕對殘差，適合要求每個量測誤差都不超過一致容許值的場合 [1，第 6.1 節]。

$$\|r\|_2^2 = \sum_i r_i^2, \qquad \|r\|_1 = \sum_i |r_i|, \qquad \|r\|_\infty = \max_i |r_i|.$$

r_i 是第 i 個殘差座標。當 r 對 x 仿射，三者都是凸目標。把非負歐幾里得範數平方不會改變最小化解，但會改變目標值、導數，以及與其他懲罰項的取捨。

絕對值殘差可以引入 $t_i \geq r_i$、$t_i \geq -r_i$，再最小化 $\sum_i t_i$。一致容許值則引入一個純量 t，要求 $-t \leq r_i \leq t$。若額外限制只有仿射式，兩者都是 LP 表示法。

最佳常數的三種意思

只擬合一個數字，就能看出殘差目標的差別。取合成觀測 $b = (0, 1, 8)$，對三者都預測同一個純量 c。對應設計矩陣是一欄全為一的向量。

對平方損失，微分 $c^2 + (c-1)^2 + (c-8)^2$ 得到 $6c - 18 = 0$，所以 $c = 3$。對絕對損失，離開資料點時，斜率等於低於 c 的觀測數減去高於 c 的觀測數。斜率在中位數由負轉正，因此 $c = 1$ 最小化總和。對最大絕對損失，平衡兩端誤差：$c = 8 - c$，因此 $c = 4$。

其中 c 是擬合常數、b 是固定觀測。在各自最佳解處，平方殘差和為 38、絕對殘差和為 8、最大殘差為 4。它們是不同單位的不同量，直接比較數值大小沒有意義。真正要比較的是回傳預測：平均擬合選三、中位數擬合選一、均勻誤差擬合選四。觀測八大幅拉動平均數，卻不會使中位數超過中間觀測 [1]。

實際任務須先問哪種誤差模式昂貴。儀器的統一容許誤差適合最大誤差；指定高斯量測模型可導向平方誤差；受污染的反應值可促使選用影響力有界的殘差損失。這些選擇都不能只由求解器方便與否決定。

單位與加權誤差

不同單位的量測，沒有指定縮放慣例就不能合理相加。一像素殘差與一秒殘差，不會自動具有可比性。

加權最小平方目標可以寫為

$$(Ax - b)^\top W (Ax - b), \qquad W \succeq 0.$$

W 是固定半正定權重矩陣。若 W 是已知正定雜訊共變異數的反矩陣，目標就有統計意義。若權重代表優先順序，則應明確說明那個意義。

非對角元素可以描述相關量測誤差。把完整共變異數換成對角權重，是模型近似，不只是實作細節。以矩陣因子白化殘差，再擬合時，應保留原本指定的二次形式。

大殘差不會自動代表離群值。它可能揭露錯誤模型、縮放不良的特徵、真正罕見的現象或損壞觀測。在降低某些誤差的影響以前，先看資料生成過程。

最小範數解與正則化

白話說。符合觀測可能仍留下許多參數選擇。參數範數能從中選解，或在擬合與複雜度間取捨。

最小範數問題為

$$\min_x \|x\| \quad \text{subject to} \quad Ax = b.$$

x 是參數向量，A, b 固定。這不同於殘差範數問題：此處要求精確一致，目標則選擇小的參數向量 [1，第 6.2 節]。

等式不一致時，問題不可行。把它換成軟殘差項會改變問題。在欠定且一致的系統中，歐幾里得最小範數解可由適當偽逆表示，但仍須關心數值秩與奇異值。

正則化近似則平衡兩個目標：

$$\min_x \frac{1}{2}\|Ax - b\|_2^2 + \lambda R(x), \qquad \lambda \geq 0.$$

R 是凸懲罰，λ 是非負權重。二分之一讓導數較簡潔。改變這個因子卻保持 λ 不動，會改變取捨。

取 $R(x) = \frac{1}{2}\|x\|_2^2$，就是 ridge 或二次正則化模型 [1，第 6.3 節]。正 λ 使 Hessian $A^\top A + \lambda I$ 正定。取 $R(x) = \|x\|_1$，非光滑幾何會鼓勵部分座標精確為零。

截距通常另行處理，因為懲罰它與懲罰特徵效果的意義不同。特徵單位也會改變參數範數。標準化矩陣欄，或採明確縮放的懲罰，可以讓這項選擇可見。

逐方向推導二次收縮

二次正則化減少量測薄弱方向造成的不穩定。對前面定義的 ridge 目標，令梯度為零得到

$$(A^\top A + \lambda I)x^* = A^\top b.$$

其中 x^* 是最佳解、A, b 是固定設計與觀測、I 是單位矩陣、$\lambda > 0$ 是懲罰權重。令 $A = U\Sigma V^\top$ 為奇異值分解，U, V 包含正交奇異向量，Σ 包含奇異值 σ_i。在方向 v_i 上，係數為

$$v_i^\top x^* = \frac{\sigma_i}{\sigma_i^2 + \lambda}\, u_i^\top b.$$

其中 u_i, v_i 是配對奇異向量。沒有正則化時，非零奇異值會把觀測係數除以 σ_i；很小的值可能放大量測誤差。加入正則化後，分母不會低於 λ。在這個所有座標都受懲罰的模型中，零空間方向的係數為零。

對 $A = \mathrm{diag}(1, 0.1)$ 與 $b = (1, 0.1)$，未正則化解為 $(1, 1)$。取 $\lambda = 0.1$ 後，解變為 $(10/11, 1/11)$。量測薄弱的第二方向被收縮得更多。這同時解釋數值穩定性與引入的偏差：正則化沒有學到第二個真實係數很小，而是在觀測對該方向限制較弱時，施加一種偏好 [1，第 6.3 節]。

限制與懲罰相關，但不同

在 $R(x) \le \rho$ 下最小化 $\|Ax - b\|_2^2$，會直接指定可接受的複雜度預算。在適當條件下，它的拉格朗日乘子可以連到懲罰表示。

這不表示特定數值 λ 事先就對應明顯的 ρ。非唯一解與未作用的限制，可能讓對應更複雜。應選擇最能表達真正需求的表示法。

正則化可能改善條件數，但數值優勢與預測改善是不同的事。條件數良好卻有偏差的模型，仍可能預測很差。若目標是預測，應用有效評估切分選擇調整參數，並把擬合目標與最後評估指標分開。

穩健損失與穩健量測模型

白話說。「穩健」可能指減少異常大殘差的影響，也可能指防範量測運算子的變動。兩者是不同模型。

一個凸殘差損失是 Huber 函數：

$$\phi_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta|r| - \frac{1}{2}\delta^2, & |r| > \delta. \end{cases}$$

r 是純量殘差，$\delta > 0$ 是轉換尺度。導數在零附近等於 r，外部大小則飽和於 δ。對仿射殘差加總此損失，就得到凸問題 [1，第 6.1 節]。

轉換尺度具有殘差的單位。把所有觀測乘上一個尺度、卻不改 δ，會改變哪些點減少影響。此損失不能抵抗所有污染；設計矩陣中的高槓桿誤差需要另外處理。

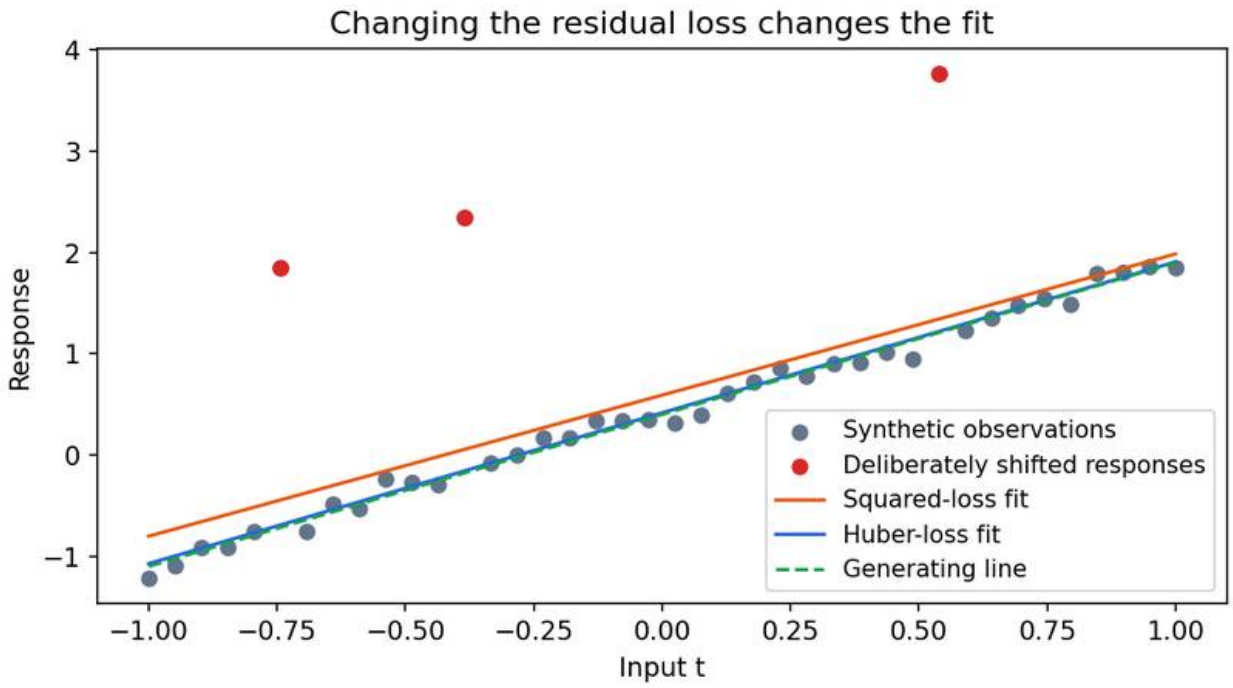


圖 *1* — 原創合成觀測與擬合解。部分反應值刻意向上移動。本示意比較殘差損失，不是基準測試，也不是一般穩健性保證。

程式：用兩種損失擬合直線

資料與偏移在本機生成。平滑最佳化呼叫遵循 SciPy 官方介面 [2]。我們比較解，並驗證穩健目標的梯度，而不宣稱某估計器永遠較好。

```
import numpy as np
from scipy.optimize import minimize

rng = np.random.default_rng(15)
t = np.linspace(-1, 1, 40)
A = np.column_stack([np.ones_like(t), t])
b = 0.4 + 1.5*t + rng.normal(0, 0.08, len(t))
b[[5, 12, 30]] += 2.5
least_squares = np.linalg.lstsq(A, b, rcond=None)[0]
delta = 0.2

def huber_objective(x):
    r = A @ x - b
    loss = np.where(abs(r) <= delta, 0.5*r*r,
                    delta*abs(r) - 0.5*delta**2)
    return loss.sum(), A.T @ np.clip(r, -delta, delta)

fit = minimize(huber_objective, least_squares, jac=True,
               method="BFGS", options={"gtol": 1e-9})
assert np.linalg.norm(huber_objective(fit.x)[1]) < 1e-6
print("squared-loss coefficients:", np.round(least_squares, 4))
print("Huber-loss coefficients:", np.round(fit.x, 4))
print("robust objective:", round(fit.fun, 6))
```

不確定運算子

假設矩陣本身會在指定集合 $\mathcal{A}$ 中變動，最壞情況近似模型是

$$\min_{x} \sup_{A \in \mathcal{A}} \|Ax - b\|.$$

x 是決策，b 是固定目標，$\mathcal{A}$ 是允許的量測矩陣。每個固定矩陣範數目標對 x 為凸，所以逐點上確界仍凸 [1，第 6.4 節]。

凸性不保證容易計算上確界。有限不確定集合可以產生有限個上圖限制。結構化橢球不確定性可能允許錐重寫。模糊描述的「小擾動」，不足以選擇正確對應模型。

期望平方誤差是另一個模型。若 $A = \bar{A} + U$ 且 $\mathbb{E}[U] = 0$，

$$\mathbb{E}\|Ax - b\|_2^2 = \|\bar{A}x - b\|_2^2 + x^\top P x, \qquad P = \mathbb{E}[U^\top U].$$

$\bar{A}$ 是平均運算子，U 是隨機變動，P 是半正定二階動差矩陣。零均值假設使交叉項消失。此處正則化來自指定的不確定模型，而非任意複雜度偏好。

最壞情況與期望保護回答不同問題。不確定集合不是機率分布；共變異數模型也不保證控制每個可能實現。

具有形狀限制的函數擬合

白話說。我們可以要求擬合函數遞增或凸，同時讓係數最佳化保持凸。

對固定基底函數 $\psi_j(t)$，

$$f(t) = \sum_{j=1}^{n} x_j \psi_j(t).$$

t 是輸入，ψ_j 是已知基底，x_j 是待擬合係數。在固定樣本輸入評估時，預測對 x 仿射，即使基底對 t 非線性。

這個區別很重要：對最佳化變數的凸性，不等於對函數輸入的凸性。多項式迴歸可以是凸係數擬合問題，擬合出的多項式卻既不遞增，也不凸。

在有序節點 $t_1 < \cdots < t_m$，令 $z_i = f(t_i)$ 為擬合值。單調性可要求 $z_{i+1} \geq z_i$。分段線性內插函數的凸性要求

$$\frac{z_{i+1} - z_i}{t_{i+1} - t_i} \leq \frac{z_{i+2} - z_{i+1}}{t_{i+2} - t_{i+1}}.$$

z_i 是變數，節點間距是固定正數。這些斜率順序不等式對擬合值為線性。與取樣任意彈性函數導數不同，它們會保證這個特定內插函數在整段節點區間的形狀。

程式：以 LP 做單調絕對誤差擬合

我們擬合五個有序量測，輔助變數表示絕對殘差上界。SciPy 的 LP 邊界全部明確指定 [3]，包括符號不受限的擬合值。

```
import numpy as np
from scipy.optimize import linprog

observed = np.array([0., 2., 1., 3., 4.])
n = len(observed)
I = np.eye(n)
order = np.zeros((n-1, n))
for i in range(n-1):
    order[i, i], order[i, i+1] = 1, -1
G = np.vstack([np.hstack([I, -I]),
               np.hstack([-I, -I]),
               np.hstack([order, np.zeros((n-1, n))])])
h = np.r_[observed, -observed, np.zeros(n-1)]
fit = linprog(np.r_[np.zeros(n), np.ones(n)], A_ub=G, b_ub=h,
              bounds=[(None, None)]*n + [(0, None)]*n,
              method="highs")
assert fit.success
```

```
z = fit.x[:n]
assert np.all(np.diff(z) >= -1e-9)
assert np.isclose(abs(z-observed).sum(), fit.fun)
print("monotone fitted values:", z)
print("total absolute residual:", fit.fun)
```

絕對誤差解可能不唯一。程式驗證性質與目標，不要求某個特定平台值。加入嚴格凸的次要目標可以選定一解，但那會是另一個指定問題。

觀測區間外的限制需要小心。分段線性內插提供明確區間內函數；外插則是額外規則。形狀限制可以增加合理性，但若真實過程違反它們，也會引入偏差。

概似與凸參數化

白話說。概似模型可以產生凸目標，但是否為凸，取決於所選參數化與定義域。

對自然參數為 θ 的分布，獨立觀測 y_i 的模型為

$$p(y \mid \theta) = \exp(\theta^{\top} T(y) - A(\theta) + c(y)).$$

$T(y)$ 是充分統計量向量，$A(\theta)$ 是對數配分函數，$c(y)$ 是與參數無關的基底項。指數族模型的 A 在自然定義域上為凸。

忽略與 θ 無關的項，負對數概似為

$$\ell(\theta) = N A(\theta) - \theta^{\top} \sum_{i=1}^{N} T(y_i).$$

N 是觀測數。此式對自然參數為凸。任意非線性參數轉換，不一定保留凸性 [1，第 7.1 節]。

對成功機率 $p \in (0,1)$ 的 Bernoulli 觀測，N 次有 k 次成功，負對數概似為 $-k \log p - (N-k)\log(1-p)$。若兩種結果都有出現，內部最佳點為 $p = k/N$。

若所有觀測都成功或都失敗，開放定義域的最佳值可能只在邊界逼近。允許邊界分布或加入先驗，會改變存在性問題。程式若裁切機率，應把裁切記錄成定義域選擇。

最大後驗估計會加入負對數先驗。若負對數先驗在所選座標為凸，整體凸性就保留。「Bayesian」本身不保證凸最佳化，點估計也不代表完整後驗不確定性。

非參數機率

對離散結果，在單純形上估計機率向量 q：

$$q_j \geq 0, \qquad \sum_j q_j = 1.$$

q_j 是結果 j 的機率。給定計數 n_j，負對數概似為 $-\sum_j n_j \log q_j$，正計數搭配零機率時，要採適當延伸值慣例。

已知動差形成仿射限制，例如固定結果特徵 a_j 與已知平均值 μ 的 $\sum_j q_j a_j = \mu$。最大化熵，等價於最小化 $\sum_j q_j \log q_j$，可在這些限制下選擇分布 [1，第 7.2 節]。

動差限制應來自與資料相同的母體、使用相同單位。不一致的經驗限制可能使單純形模型不可行。放軟限制，是決定接受多少不確定性的模型選擇，不是修補代數上的麻煩。

以線性規劃設計偵測器

白話說。看見離散證據後的決策，可以直接最佳化。允許隨機決策，會讓可行集合凸化。

假設 p_{kj} 是假說 j 下觀測 k 的已知機率。令 T_{ik} 表示看見 k 後決定假說 i 的機率。

$$T_{ik} \geq 0, \qquad \sum_i T_{ik} = 1, \qquad D_{ij} = \sum_k T_{ik} p_{kj}.$$

D_{ij} 是 j 為真時決定 i 的機率。T 是決策變數，條件機率 p_{kj} 固定。D 的每個元素對 T 都是線性的。

加權錯誤成本、正確偵測下界與錯誤決策上界，都能寫成線性目標和限制 [1，第 7.3 節]。假說先驗可以加權整體期望錯誤，但不應悄悄拿它代替條件錯誤保證。

二元假說的適當最佳偵測器表示法，會產生概似比閾值規則。邊界觀測可能需要隨機化，才能符合精確錯誤預算。確定性規則是受限子集；強制使用它們會引入離散選擇。

最佳偵測器的有效性，取決於條件分布。分布偏移、估計概似與校準不確定性，都應納入判斷。指定機率表下的數值最佳性，不會驗證機率表本身。

偵測器算例：隨機化為什麼有用

偽陽性預算可能落在兩個確定性規則之間。假設觀測為 a 或 b。假設 H_0 下的機率為 $(0.8, 0.2)$，H_1 下為 $(0.2, 0.8)$。這是合成條件機率表，不是量測頻率。

令 $t_a, t_b \in [0, 1]$ 為各觀測後判定 H_1 的機率。在偽陽性預算 0.1 下最大化真陽性機率，形成 LP：

$$\begin{aligned} \text{maximize}_{t_a, t_b} \quad & 0.2 t_a + 0.8 t_b \\ \text{subject to} \quad & 0.8 t_a + 0.2 t_b \leq 0.1, \\ & 0 \leq t_a, t_b \leq 1. \end{aligned}$$

其中目標為 H_1 下的偵測機率，第一限制為 H_0 下的錯誤機率。在 b 花一單位錯誤預算，可得四單位偵測機率；在 a 花同樣預算，只得四分之一單位。因此把預算給 b：$t_a = 0$、$t_b = 1/2$，真陽性機率為 0.4。

在 b 做確定性判定會消耗 0.2，超過預算。在這個離散觀測集合中，任何非平凡的確定性規則都違反預算，因此唯一可行確定性規則是永遠選 H_0，完全不偵測。隨機化填入確定性規則之間的凸空間，達到需要的操作點 [1，第 7.3 節]。這是在已知觀測後隨機決策，不是把未知機率當成決策變數。

動差與指數風險上界

白話說。有時我們知道動差，卻不知道整個分布。明確列出假設後，上界可以保守控制尾端事件。

對均值 μ、有限變異數 σ^2 的純量隨機變數 Y，Chebyshev 不等式為

$$\Pr(|Y-\mu| \ge a) \le \frac{\sigma^2}{a^2}, \qquad a > 0.$$

a 是偏離閾值。這個上界只使用前兩階動差，可能很鬆。大於一的上界可以換成一；它不是估計的事件機率 [1，第 7.4 節]。

對閾值 b，指數或 Chernoff 上界使用

$$\Pr(Y \ge b) \le \inf_{s>0} \exp(-sb)\, \mathbb{E}[\exp(sY)].$$

s 是正輔助參數，使用處的期望必須有限。最佳化其對數，得到 $\inf_{s>0}\{\log \mathbb{E}[\exp(sY)] - sb\}$。

對數動差生成函數在定義域上對 s 為凸。但這不會自動使所有設計與風險的聯合問題，對全部變數為凸。應檢查隨機變數如何依賴決策，以及保留哪些分布假設。

尾端上界可以支援保守設計限制，卻不能代替檢查動差估計是否可靠。厚尾、相依性與有限樣本不確定性，可能讓假定的動差生成函數不適合。

選擇下一批量測

白話說。估計品質不只取決於怎麼擬合，也取決於量測什麼。凸實驗設計先配置連續量測資源，再處理離散取整。

對候選量測向量 $a_j \in \mathbb{R}^n$、獨立單位變異數雜訊與正規化權重 w_j，定義

$$M(w) = \sum_j w_j a_j a_j^{\top}, \qquad w_j \ge 0, \quad \sum_j w_j = 1.$$

M 是每單位正規化量測資源的資訊矩陣。矩陣正定、量測模型正確且計入總資源時，相應線性估計器的共變異數會與其反矩陣成比例。

D-optimal 設計最小化 $-\log\det M$，針對不確定性體積。A-optimal 設計最小化 $\operatorname{tr}(M^{-1})$，針對總座標變異數。E-optimal 設計最大化 M 的最小特徵值，針對最弱量測方向 [1，第 7.5 節]。

它們是不同的純量摘要。單位與座標轉換對它們解釋的影響可能不同。小的總變異數仍可能藏著量測不良的方向；小不確定體積，也不保證每個座標的界。

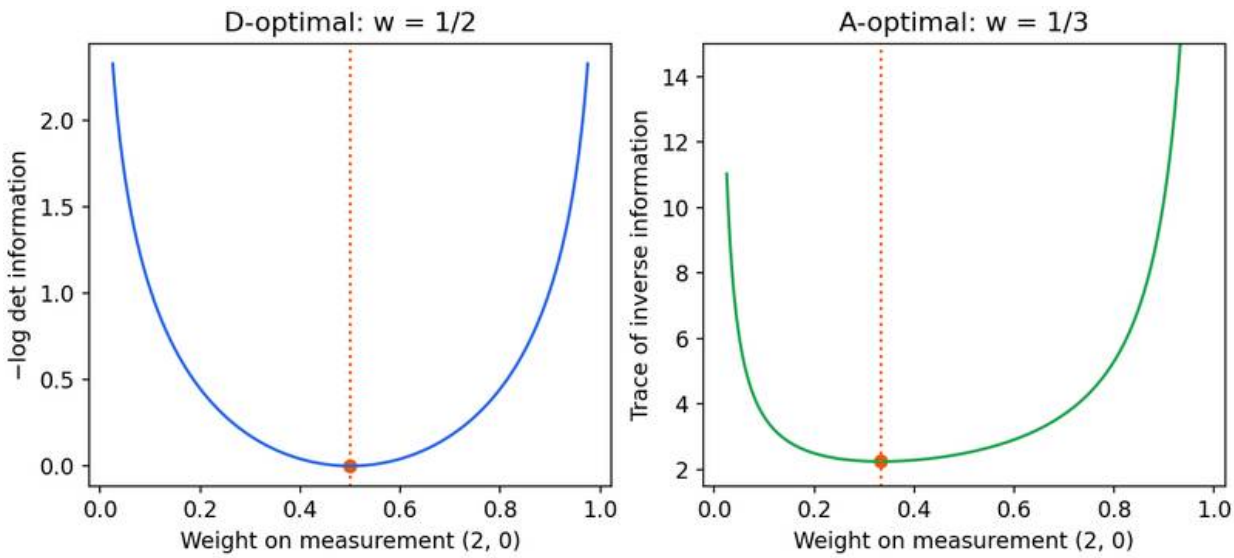


圖 2 — 原創兩方向解析設計。*D-optimal* 與 *A-optimal* 配置不同，因為體積和總座標變異數是不同目標。曲線假設獨立單位變異數雜訊，以及兩方向皆有正資源。

程式：兩種設計準則與精確檢查

候選向量為 $(2,0)$ 與 $(0,1)$，第一個權重 w，第二個 $1-w$。資訊矩陣為對角矩陣 $M = \mathrm{diag}(4w, 1-w)$。任一方向未量測時，兩種準則都發散。

```
import numpy as np
from scipy.optimize import minimize_scalar

def d_loss(w):
    return -np.log(4*w) - np.log(1-w)

def a_loss(w):
    return 1/(4*w) + 1/(1-w)

d = minimize_scalar(d_loss, bounds=(1e-6, 1-1e-6), method="bounded")
a = minimize_scalar(a_loss, bounds=(1e-6, 1-1e-6), method="bounded")
assert np.isclose(d.x, 0.5, atol=1e-5)
assert np.isclose(a.x, 1/3, atol=1e-5)
print("D-optimal first weight:", round(d.x, 6))
print("A-optimal first weight:", round(a.x, 6))
```

實驗總數固定時，實際次數是整數。連續權重是鬆弛，或隨機取樣策略。取整可能破壞預算，或讓某方向完全未量測；轉成排程後應重新檢查矩陣。

若候選向量無法張成參數空間，任何權重都不能使資訊矩陣正定。加入先驗矩陣可以正則化設計，但意義變成先驗加量測資訊。

推導兩種設計準則為什麼配置不同

兩方向實驗在執行數值程式前就可解出。候選向量為 $(2,0)$ 與 $(0,1)$，權重為 $w, 1-w$，資訊矩陣為 $M = \mathrm{diag}(4w, 1-w)$。令 $0 < w < 1$，確保兩方向都有量測。

D-optimal 目標與導數為

$$f_D(w) = -\log(4w) - \log(1-w), \qquad f_D'(w) = -\frac{1}{w} + \frac{1}{1-w}.$$

其中 f_D 是負對數行列式準則。駐點條件給出 $w = 1/2$。強度因子四只讓目標改變一個常數，不影響這個體積準則的配置。

對 A-optimal 設計，

$$f_A(w) = \frac{1}{4w} + \frac{1}{1-w}, \qquad f_A'(w) = -\frac{1}{4w^2} + \frac{1}{(1-w)^2}.$$

其中 f_A 是反資訊矩陣跡準則。分母為正，因此駐點方程化為 $1 - w = 2w$，得到 $w = 1/3$。兩個純量目標的二階導數都為正，而且在端點發散，因此這些駐點為唯一全域最小解 [1，第 7.5 節]。

解讀是：第一種量測本來就更強，因此最小化總變異數時，會把更多資源送往較弱的第二方向。最小化不確定性體積則平衡方向資訊的乘積，因此給相同資源。若量測成本或雜訊變異數不同，須先在資訊矩陣與預算中納入改變，再重新推導。沒有哪種配置永遠最好；每種都回答自己選定的不確定性問題。

擬合與設計的檢核

先從必須準確的應用量開始，決定它是預測、參數分量、最壞殘差或分布機率。選擇數值損失以前，先寫出表示法與單位。

對每個限制，問它是物理必要、統計估計，還是為計算方便引入。這些角色代表不同的不可行性處理方式。物理限制可能要求修正設計，經驗動差可能需要不確定性，冗餘代數可能需要移除。

擬合後驗證定義域、形狀限制與求解器殘差，再以適當獨立程序評估泛化或不確定性。訓練目標最佳解與可靠未來預測，是不同宣稱。

設計時檢查資訊譜、預算、雜訊假設與排程轉換。連同鬆弛目標一起回報最後可執行排程，讓最重要的近似保持可見。

不確定性橢球真正說明什麼

資訊矩陣的反矩陣，只有在相應量測與估計器假設下才能解讀為共變異數。若要轉成具有指定涵蓋機率的區域，還需要分布論證與正確閾值。只由矩陣畫出橢圓，不會建立信賴水準。

對正定共變異數 C，橢球可以寫成 $(x-\hat{x})^\top C^{-1}(x-\hat{x}) \le c$。$\hat{x}$ 是中心，$c > 0$ 是閾值。矩陣指定形狀，閾值指定大小；涵蓋率還取決於估計誤差的實際分布。

先驗共變異數、抽樣共變異數與數值 Hessian 反矩陣，不一定具有相同解釋。特定模型下可能一致，但交換使用前應檢查模型。估計器的最佳化容許誤差與抽樣不確定性，也具有不同單位與來源。

重複量測時，獨立等變異數假設可支持簡單的資訊相加。相關重複可能提供比次數所暗示更少的新資訊。忽略相關性的設計，可能把資源自信地配置到冗餘量測。

這也影響停止精度。求解精度遠低於輸入係數的不確定性，可能實際幫助有限；看似微小的硬物理限制違反，卻仍可能不可接受。應依應用尺度設定最佳化容許誤差，而非只看浮點數能提供幾位數。

最後，不確定性視覺化應說明畫的是解析模型、估計共變異數，或重複合成實驗。本章圖形展示示意擬合與目標曲線，沒有呈現經驗校準的涵蓋區域，也沒有宣稱特定信賴機率。

現代觀點

白話說。現今研究把結構化凸估計、大規模一階計算與學得的凸元件結合。凸性保證只屬於明確寫出的變數與定義域。

Pokutta 的 2024 年綜述 [4] 以凸可行集合的線性最小化、收斂保證與實用變體，組織 Frank–Wolfe 方法。單純形限制的實驗設計，是這個觀點的自然場合：極端點搜尋可以避免完整投影。但目標仍須滿足要求的平滑性與正定定義域控制。

Schaller、Bemporad 與 Boyd 的 2025 年手稿 [5] 討論學習參數化凸函數，再用於後續最佳化。它以有紀律的曲率規則連結擬合：對合法固定參數，學得元件可以保持對決策變數的凸性。訓練元件是另一個問題，本身未必凸。

兩來源涵蓋互補問題：如何在結構化可行集合中移動，以及如何建立能放入集合內使用的函數。它們都不會代替不確定性建模、保留資料評估或設計的物理可行性。

目前挑戰包括模型失配下的可靠不確定性、資源限制下有資訊的量測、樣本區間外仍合適的形狀限制，以及觀測逐步到來時的可擴充更新。應區分可證明凸的估計器與完整驗證的統計流程。

重點整理

- 殘差範數、參數範數與評估指標回答不同問題。
- 正則化權重依賴目標正規化與特徵單位。
- 穩健殘差損失與不確定運算子模型防範不同誤差。
- 固定非線性基底仍能產生凸係數擬合。
- 概似凸性依賴參數化與定義域。
- 離散證據的隨機偵測器可以寫成 LP。
- 實驗設計準則不同，連續權重也需要執行方案。

練習

1. 將最大絕對殘差擬合寫成 LP，包含符號不受限的參數。

提示

使用一個非負殘差上界。

2. 把觀測與預測乘以十。求 Huber 轉換尺度與二次正則化權重應如何改變，才能保留指定等價目標。

提示

寫出縮放後目標；答案取決於參數與懲罰單位是否也縮放。

3. 在五節點單調擬合例子加入凸性限制，使用不相等的正節點間距。

提示

排序相鄰斜率。

4. 推導 Bernoulli 最大概似估計，並解釋開放定義域的邊界情況。

提示

令概似導數為零，再看端點。

5. 建立具有明確偽陽性預算的雙假說隨機化偵測器 LP。

提示

使用每個觀測的決策機率，把條件錯誤寫成線性式。

6. 推導程式中兩方向 D-optimal 與 A-optimal 權重，再說明小量測預算取整後為何需要重新檢查。

提示

對純量準則微分；檢查每個取整方向是否有量測。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **閱讀：第 6 章第 6.1–6.5 節；第 7 章第 7.1–7.5 節**。 官方書頁，免費 PDF。
2. SciPy developers, “scipy.optimize.minimize” and “scipy.optimize.minimize_scalar,” *SciPy API documentation*, accessed 2026. 多變數介面，純量介面。
3. SciPy developers, “scipy.optimize.linprog,” *SciPy API documentation*, accessed 2026. 官方文件。
4. Sebastian Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. 開放綜述。
5. Michael Schaller, Alberto Bemporad and Stephen Boyd, “Learning Parametric Convex Functions,” manuscript, 2025. 官方手稿頁。

OPT 06・梯度下降、牛頓步驟與數值可靠性

推導下降方向與線搜尋，比較梯度和牛頓方法，解讀收斂假設，並實作穩定的凸邏輯斯求解器。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/gradient-newton/

你將學到

- 區分搜尋方向與沿方向選擇的步長。
- 在 Lipschitz 梯度假設下推導安全梯度步驟。
- 實作具有定義域與迭代保護的 Armijo 回溯。
- 解釋條件數、依範數而異的最陡下降，以及牛頓曲率。
- 解讀牛頓減量與快速局部收斂背後的假設。
- 以 CPU 例子驗證導數、線性求解與停止證據。

先看全貌

凸性告訴我們哪些答案為全域最佳；演算法則告訴我們，如何利用可計算資訊逐步靠近答案。兩者互補：凸問題仍可能尺度不良、昂貴或數值敏感。

Boyd 與 Vandenberghe 的第 9 章 [1] 發展無限制下降方法、梯度下降、最陡下降、牛頓方法、自協調性與實作。本章依這個順序前進，同時保留每步成本與停止判準的意義。

> **用最簡單的話說**
>
> 你正在下山。斜坡箭頭指出下降方向；描繪山坡彎曲的地圖，可以建議更聰明的路線。無論使用哪個方法，都還要檢查下一步安全落地，而且確實降低高度。

例子求解確定性的凸目標。不能把保證悄悄轉移到神經網路的非凸訓練目標、含雜訊導數或非光滑懲罰。實用想法可能能轉移，但證明假設需要重新檢查。

我們先用有精確解的二次函數，再從頭實作平滑正則化邏輯斯目標。兩者使用本機生成資料，在 CPU 執行，除 NumPy 外不需要求解器套件。

一個方向與一段距離

白話說。下降迭代先選往哪裡指，再選走多遠。太早把兩者混在一起，會隱藏步驟為何安全。

迭代寫成

$$x_{k+1} = x_k + t_k d_k.$$

x_k 是目前決策向量，d_k 是搜尋方向，$t_k > 0$ 是步長。對可微目標 f，嚴格下降方向滿足 $\nabla f(x_k)^\top d_k < 0$。

此內積是零步長處的方向導數。可微性表示，在目前點周圍的開放定義域，足夠小的正步長會降低目標，並保留定義域。大步長仍可能提高目標或離開定義域。

方向可能依賴梯度、Hessian、度量或其他結構；線搜尋評估候選距離。只計算接受的迭代，會隱藏許多拒絕的目標評估。

若可微凸函數在開凸定義域的梯度為零，支撐平面不等式就能證明全域最小化解。有約束時，零無限制梯度通常不是正確最佳性條件；下一章會用受限制系統。

精確與非精確線搜尋

精確線搜尋會在允許的正距離上最小化 $f(x + td)$。這個輔助一維問題可能昂貴，也可能不必要。

非精確搜尋接受足夠下降，不精確求解線問題。它也應保留目標定義域。對數項的引數無效時，候選點只是未通過定義域檢查，不是提供了目標證據。

浮點數又帶來另一個區別：數學上下降的步驟，在機器精度下可能與目前值無法區分。不斷縮小到比較偶然通過，不是穩健停止規則。

梯度下降與假設

白話說。梯度下降往梯度反方向走。曲率上界告訴我們，固定步長必須多小。

更新為

$$x_{k+1} = x_k - \eta \nabla f(x_k).$$

$\eta > 0$ 是步長。假設相關區域的梯度具有 Lipschitz 常數 L：

$$\|\nabla f(y) - \nabla f(x)\|_2 \le L\|y - x\|_2.$$

x, y 是連接線段留在該區域的點。L 衡量梯度改變多快，不是正則化係數，也不是損失值。

平滑性不等式為

$$f(x + d) \le f(x) + \nabla f(x)^\top d + \frac{L}{2}\|d\|_2^2.$$

d 是嘗試的位移。代入 $d = -\nabla f(x)/L$ 得到

$$f\left(x - \frac{1}{L}\nabla f(x)\right) \le f(x) - \frac{1}{2L}\|\nabla f(x)\|_2^2.$$

因此，假設與定義域要求成立時，$1/L$ 步長會下降。對二次可微目標，Hessian 最大特徵值上界可以建立此 L [1，第 9.3 節]。

只有凸性與平滑性時，標準固定步長分析通常給次線性目標收斂。強凸性加入正的曲率下界，讓線性幾何收斂成為可能。這些字描述數學速率，不保證每個實作都很快。

從單步下降到收斂速率

下降證明目標改善；收斂速率量化達到指定誤差需要多少工作。設 $\mathbb{R}^n$ 上的可微凸函數具有 L-Lipschitz 梯度，而且最小解 x^* 存在。使用 $x^+ = x - \nabla f(x)/L$。梯度支撐下界給出

$$f(x) - f(x^*) \le \nabla f(x)^\top (x - x^*).$$

其中 x 是目前點、x^+ 是下一迭代點、x^* 是任意最小解。展開更新後的平方距離：

$$\|x^+ - x^*\|_2^2 = \|x - x^*\|_2^2 - \frac{2}{L}\nabla f(x)^\top (x - x^*) + \frac{1}{L^2}\|\nabla f(x)\|_2^2.$$

所有符號保留上述定義。把第一個不等式與前面的平滑下降界結合，得到

$$f(x^+) - f(x^*) \le \frac{L}{2}\left(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2\right).$$

對前 k 次更新加總，中間距離逐項抵消。因為目標值下降，最後誤差不大於這 k 個誤差的平均：

$$f(x_k) - f(x^*) \le \frac{L\|x_0 - x^*\|_2^2}{2k}, \qquad k \ge 1.$$

其中 x_0 是起點、x_k 是 k 次更新後的點。這就是 $O(1/k)$ 目標速率的具體意思：依此界把保證誤差減半，通常要把迭代次數加倍。它是這些假設下的最壞情況保證，不是所有實測軌跡都如此 [1]。

強凸性與停止上界

若 f 是 μ 強凸，支撐模型加入 $\frac{\mu}{2}\|y - x\|_2^2$。最小化此下模型得到

$$f(x) - f^* \le \frac{\|\nabla f(x)\|_2^2}{2\mu}.$$

f^* 是全域最小值，$\mu > 0$ 是有效強凸常數。小梯度因此給出單位明確的目標間隙上界。

沒有驗證正 μ 時，同一公式不是憑證。很平坦的目標可能同時有小梯度與很大的解距離；小參數更新也可能只是步長過小。

不要只因數次印出的目標值看似相同就停止。應檢查適當殘差、模型尺度、進度趨勢，以及迭代或時間限制。

條件數改變路徑

白話說。拉長的碗有陡方向與平方向。對陡方向安全的步長，可能讓平方向進展非常慢。

正定二次函數為

$$f(x) = \frac{1}{2}x^\top Q x + c^\top x, \qquad \nabla f(x) = Qx + c.$$

Q 對稱正定，c 固定。Hessian 就是 Q，最小和最大特徵值給 μ 與 L。

比例 $\kappa = L/\mu$ 描述歐幾里得曲率條件數。比例很大時，單一純量步長要照顧非常不同的方向尺度。因此，座標單位會大幅改變梯度下降行為。

標準化變數或選適當度量可以改善路徑，但限制、係數與停止範數都要一致轉換。不能用未轉換的參數距離閾值，公平評估縮放後演算法。

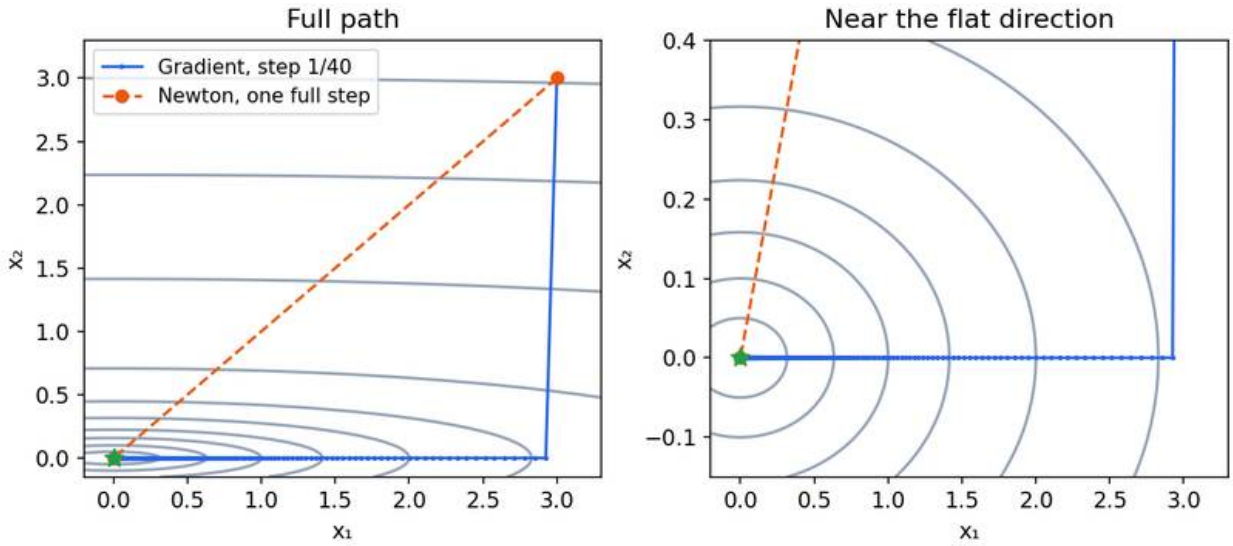


圖 1 — 原創解析二次函數 $f(x) = (x_1^2 + 40x_2^2)/2$。梯度路徑用 $1/40$ 步長；牛頓在一個完整步驟達到精確二次最小值。本例隔離條件數影響，沒有比較一般問題的總執行時間。

程式：比較固定梯度步與牛頓步

目標在零最小。我們比較目標下降，並驗證牛頓線性系統，不使用明確反矩陣。

```
import numpy as np

Q = np.diag([1., 40.])
x = np.array([3., 3.])
start = x.copy()
values = [0.5*x @ Q @ x]
for _ in range(200):
    x = x - (Q @ x)/40
    values.append(0.5*x @ Q @ x)
direction = np.linalg.solve(Q, -(Q @ start))
newton = start + direction
assert np.all(np.diff(values) <= 1e-12)
assert np.allclose(Q @ direction + Q @ start, 0)
assert np.linalg.norm(newton) < 1e-12
print("gradient objective after 200 steps:", round(values[-1], 8))
print("Newton objective after one step:", 0.5*newton @ Q @ newton)
```

一步牛頓解，是精確正定二次函數與準確線性求解的特殊結果。一般平滑函數移動時曲率會變，完整一步可能失敗。

逐特徵向量讀懂二次路徑

二次函數能精確呈現步長影響。對曲率為 Q 的正定二次函數，令 $e_k = x_k - x^*$ 為誤差。梯度下降給出 $e_{k+1} = (I - \eta Q)e_k$。在 Q 的特徵值為 q_i 的特徵向量方向上，誤差係數乘以 $1 - \eta q_i$。

其中 η 是固定步長、I 是單位矩陣、$q_i > 0$ 是方向曲率。所有係數收縮的充分必要條件為 $0 < \eta < 2/L$，其中 L 是最大特徵值。若 $\eta > 2/L$，即使目標凸，最陡方向的誤差量值仍會增大。

對本章的 $Q = \mathrm{diag}(1, 40)$ 與 $\eta = 1/40$，陡座標在一次更新後消失，平坦座標則每次乘以 $39/40$。從 $(3, 3)$ 開始，k 次更新後第一座標誤差為 $3(39/40)^k$。緩慢路徑是方向曲率的結果，而非隨機數值失敗。

對最小特徵值為 μ 的二次函數，取 $\eta = 2/(L + \mu)$ 可平衡極端收縮因子的量值，其最壞值為 $(L - \mu)/(L + \mu)$。相較保守步長，這能改善上界，但需要已知頻譜端點，不能取代任意非二次函數的線搜尋 [1]。

Armijo 回溯

白話說。從候選距離開始，縮小到相對局部斜率有足夠下降。

足夠下降條件為

$$f(x + td) \le f(x) + \alpha t \nabla f(x)^\top d.$$

d 是下降方向，$t > 0$ 是測試距離，$\alpha \in (0, 1)$ 是固定下降比例。回溯反覆把 t 乘上 $\beta \in (0, 1)$，直到定義域與下降測試通過 [1，第 9.2 節]。

牛頓方法常選 $\alpha < 1/2$，讓標準局部分析下，正則最佳點附近能接受完整步。這是考慮假設的選擇，不是通用最佳參數。

實作需要最多試驗次數、最小有意義步長、有限值檢查，以及清楚失敗回報。否則無效方向或錯誤梯度，可能導致無限縮小迴圈。

拒絕步驟告訴我們什麼

拒絕可能表示初始距離太大、遠離解時曲率模型不準、跨越定義域或導數錯誤。單次拒絕本身不能分辨原因。

先檢查方向導數。若非負，一般下降回溯就沒有原本的正當性。可能需要修改方向，也可能候選點已滿足最佳性測試。

最佳點附近反覆拒絕，可能來自浮點抵消。要求低於可表示精度的改善以前，應比較目標與導數尺度。停止時應說明符合哪個數值條件。

最陡下降依賴範數

白話說。「最陡」需要定義一單位移動。不同範數會選出不同方向。

正規化最陡方向求解

$$\min_{\|d\|\le 1} \nabla f(x)^\top d.$$

d 是方向，$\|\cdot\|$ 是選定移動範數，x 是目前點。歐幾里得範數下，非零梯度給方向 $-\nabla f(x)/\|\nabla f(x)\|_2$。

對正定度量矩陣 P，定義 $\|d\|_P = \sqrt{d^\top P d}$。相應未正規化下降方向是

$$d = -P^{-1}\nabla f(x).$$

P 決定方向成本。方向導數為 $-\nabla f(x)^\top P^{-1}\nabla f(x)$，梯度非零時嚴格為負。

選擇 P 類似預條件化：納入已知尺度或曲率，不一定建立真正 Hessian。好處取決於度量系統求解多便宜，以及它多能代表問題。

採座標型移動範數時，最陡方向可能選單一座標，或以不同方式分配移動。書中 [1，第 9.4 節] 說明，最陡下降是一族方法，不是單一與座標無關的程序。

牛頓的局部二次模型

白話說。牛頓以二次函數近似目標，再求該近似的最小化解。

目前點的模型為

$$m(d) = f(x) + g^\top d + \frac{1}{2} d^\top H d, \qquad g = \nabla f(x),\ H = \nabla^2 f(x).$$

d 是位移，g 是梯度，H 是 Hessian。若 H 正定，模型唯一最小化位移滿足

$$Hd = -g.$$

這就是牛頓方向。以線性求解或分解計算，不要建立反矩陣。正定性保證 $g \neq 0$ 時有下降方向。

凸性只給半正定 Hessian，仍可能奇異。因此，完整牛頓步不會自動有定義。正則化 Hessian、受限子空間或其他方向可能合適，但每項都會改變指定數值程序。

牛頓減量

定義

$$\lambda_N(x)^2 = g^\top H^{-1} g = -g^\top d.$$

λ_N 是牛頓減量（Newton decrement），假設 H 正定。二次模型預測，最佳步驟會下降 $\lambda_N^2/2$。

對精確二次函數，預測等於目標間隙。對一般函數，它是局部模型量；嚴格間隙解釋需要額外假設。不要把預測下降標成無條件全域誤差憑證。

導數與求解精確時，在可逆仿射座標轉換下，方向與減量會一致轉換。這是牛頓與歐幾里得梯度下降不同的一個原因，但浮點條件數與實作仍重要。

局部收斂有條件

在適當平滑性、最小化點附近非奇異正定 Hessian，以及合適線搜尋下，牛頓最終會接受完整步，並可二次收斂 [1，第 9.5 節]。

二次收斂表示，在足夠小鄰域中，誤差約以先前誤差平方縮小。它不表示任何初始點的前幾步都如此。

梯度、Hessian 與線性求解誤差可能限制速率。遠離解時，保護措施可能必要；接近解時，要求低於導數準確度的殘差，可能浪費計算或引發誤導的失敗。

牛頓法為什麼尊重座標轉換

牛頓法利用曲率調整方向，所以改變變數單位不會改變其理想的實際步驟。令 $x = Tz + b$，其中 T 可逆、b 為固定位移，定義 $\tilde{f}(z) = f(Tz + b)$。在對應點，鏈鎖律給出

$$\tilde{g} = T^{\top} g, \qquad \widetilde{H} = T^{\top} H T.$$

其中 g, H 是 x 座標下的梯度與正定 Hessian，波浪號表示 z 座標下的對應導數。轉換後的牛頓方程為

$$T^{\top} H T\, \Delta z = -T^{\top} g \quad \Longrightarrow \quad H(T\Delta z) = -g.$$

其中 Δz 是新座標的位移。線性解唯一，因此 $T\Delta z = \Delta x$，等於原牛頓位移。預測下降量也不變：$\tilde{g}^{\top} \widetilde{H}^{-1} \tilde{g} = g^{\top} H^{-1} g$。

普通梯度下降的轉換步驟映回後，卻是 $-\eta T T^{\top} g$，通常不同於 $-\eta g$。這連起座標縮放與最陡下降中的度量。牛頓不變性假設精確導數與求解；條件數不良的座標轉換仍可能傷害浮點實作 [1，第 9.5 節]。

自協調性與受控曲率

白話說。某些凸函數透過本身局部曲率，控制曲率改變速度，提供分析牛頓步的特殊途徑。

標準一維自協調函數滿足

$$|f'''(t)| \le 2\big(f''(t)\big)^{3/2}.$$

t 是純量輸入，撇號表示導數，二階導數非負。多維時，沿定義域中的直線套用此條件 [1，第 9.6 節]。

$t > 0$ 上的對數障礙 $-\log t$ 滿足這個純量關係，連結無限制牛頓分析與下一章內點方法。

自協調性比一般平滑凸性更具體。它允許局部範數控制變動，不必只靠單一全域歐幾里得 Hessian-Lipschitz 常數。理論包含精確的減量區域與複雜度陳述；它們需要正確正規化與障礙假設。

包含對數的函數，不會自動成為每種限制表示的有效障礙。定義域、邊界行為與縮放都要符合問題。即使兩者都有對數，也應區分平滑概似項與限制障礙。

穩定的凸邏輯斯求解器

白話說。完整實作應穩定評估目標、驗證導數，並以最佳性量停止。

對固定列 $a_i^\top$ 與二元結果 $y_i \in \{0,1\}$，使用

$$f(w) = \frac{1}{N}\sum_{i=1}^{N}[\log(1+\exp(a_i^\top w)) - y_i a_i^\top w] + \frac{\rho}{2}\|w\|_2^2.$$

w 是參數向量，N 是觀測數，$\rho > 0$ 是正則化係數。本例正則化每個座標。固定設計邏輯斯概似，遵循 [1，第 7 章] 的凸估計推理。

令 $p_i = 1/(1+\exp(-a_i^\top w))$，導數為

$$g = \frac{1}{N}A^\top(p-y) + \rho w, \qquad H = \frac{1}{N}A^\top \operatorname{diag}(p_i(1-p_i))A + \rho I.$$

A 堆疊量測列，p, y 是觀測向量，I 是單位矩陣。正 ρI 提供有效全域強凸常數 $\mu = \rho$。

使用穩定 softplus 評估，不要先算可能溢位的指數再取對數。下列合成分數大小受控，允許直接 sigmoid；對任意極端分數，也應使用數值穩定的 sigmoid 實作。

推導邏輯斯曲率界限

知道曲率常數，能讓穩定目標更容易最佳化。對單一分數 $s = a_i^\top w$，$\log(1+e^s)$ 的導數為 p_i，二階導數為 $p_i(1-p_i)$。因此扣除線性項 $y_i s$ 後，梯度為 $(p_i - y_i)a_i$，Hessian 為 $p_i(1-p_i)a_i a_i^\top$。

其中 a_i 是固定特徵向量、w 是參數向量、$y_i \in \{0,1\}$ 是觀測標籤、p_i 是 sigmoid 機率。對觀測取平均，再加二次懲罰，即得上面列出的導數。因為 $0 \le p_i(1-p_i) \le 1/4$，

$$\rho I \preceq \nabla^2 f(w) \preceq \left(\rho + \frac{\|A\|_2^2}{4N}\right) I.$$

其中 f 是正則化平均邏輯斯目標、A 是特徵矩陣、N 是觀測數、$\rho > 0$ 是懲罰權重，$\|A\|_2$ 是最大奇異值。下界來自所有座標的正則化；上界可對任意方向 v 檢查 $v^\top A^\top \operatorname{diag}(p_i(1-p_i))Av$ 得出。

因此 $\mu = \rho$ 與 $L = \rho + \|A\|_2^2/(4N)$ 是合法全域常數，能對這個特定模型提供安全梯度步長與目標間隙停止界。若截距不受懲罰，就無法直接推出每個座標都有同樣正下界。數學公式、懲罰實作與停止測試必須一致 [1]。

程式：具有導數與停止檢查的牛頓方法

這是最佳化示範，不是分類基準。求解前先檢查方向導數，最後驗證梯度、下降序列與強凸間隙上界。

```
import numpy as np

rng = np.random.default_rng(16)
A = rng.normal(size=(160, 5))
```

```
truth = np.array([0.8, -0.5, 0.3, 0., 0.4])
prob = 1/(1+np.exp(-(A @ truth)))
y = (rng.random(len(A)) < prob).astype(float)
rho = 0.1

def evaluate(w):
    z = A @ w
    p = 1/(1+np.exp(-z))
    value = np.mean(np.logaddexp(0, z)-y*z) + rho*(w @ w)/2
    gradient = A.T @ (p-y)/len(y) + rho*w
    Hessian = (A.T * (p*(1-p))) @ A/len(y) + rho*np.eye(A.shape[1])
    return value, gradient, Hessian

w = np.zeros(A.shape[1])
direction = rng.normal(size=len(w))
eps = 1e-5
finite_difference = (evaluate(w+eps*direction)[0] -
                     evaluate(w-eps*direction)[0])/(2*eps)
assert np.isclose(finite_difference, evaluate(w)[1] @ direction,
                  atol=1e-8)
history = []
for iteration in range(30):
    value, g, H = evaluate(w)
    history.append(value)
    if np.linalg.norm(g) < 1e-9:
        break
    step_direction = np.linalg.solve(H, -g)
    slope = g @ step_direction
    assert slope < 0
    step = 1.0
    for trial in range(50):
        candidate = w + step*step_direction
        candidate_value = evaluate(candidate)[0]
        if np.isfinite(candidate_value) and candidate_value <= value+0.1*step*slope:
            w = candidate
            break
        step *= 0.5
    else:
        raise RuntimeError("backtracking did not accept a step")
else:
    raise RuntimeError("iteration limit reached")

value, g, H = evaluate(w)
assert np.linalg.norm(g) < 1e-9
assert np.all(np.diff(history) <= 1e-12)
assert np.linalg.eigvalsh(H).min() >= rho-1e-12
print("accepted Newton steps:", len(history)-1)
print("objective:", round(value, 8))
print("gradient norm:", np.linalg.norm(g))
print("objective-gap upper bound:", (g @ g)/(2*rho))
```

導數檢查只測一個方向與一個點，不能證明全部導數正確。修改模型時，應在代表性非零點加入檢查。Hessian 對稱性、特徵值預期與線性求解殘差，提供其他獨立證據。

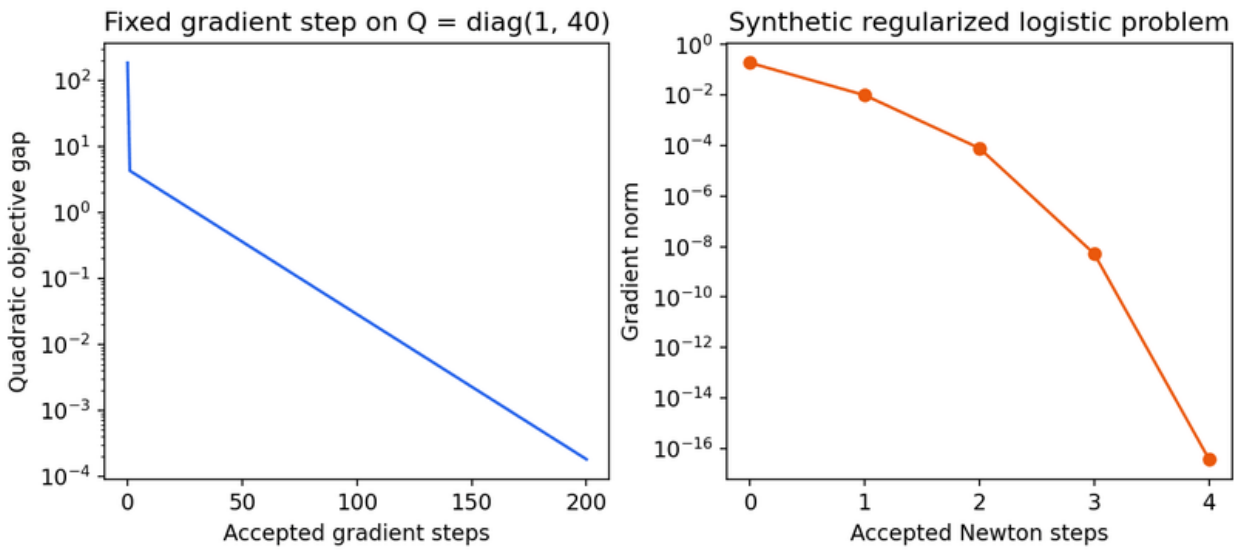


圖 2 — 原創計算收斂曲線。左圖有已知零二次最佳值；右圖是合成邏輯斯目標的梯度範數。接受步數不是執行時間，也不是跨問題基準。

實作與停止回報

白話說。求解器結果應描述為何停止、驗證什麼，以及做了多少計算。

稠密牛頓儲存的矩陣大小，隨變數數量平方增加，分解可能主導成本。梯度方法常只需矩陣向量乘法，因此很多便宜迭代對大問題有吸引力。

稀疏矩陣、低秩項與區塊結構，可能大幅改變比較。利用結構前，要先確認轉換系統等價，且分解填入量可接受 [1，第 9.7 節與附錄 C]。

Hessian 向量乘積可以支援迭代線性求解，不需儲存完整 Hessian。內層求解也有自身殘差與容許誤差。粗略解出的牛頓系統，不一定重現精確牛頓方向與收斂率。

應回報目標值、適當梯度或駐點殘差、定義域檢查、迭代次數，以及指定停止條件。若時間重要，要計入導數評估與線搜尋嘗試，而不只看接受步數。

縮放目標會改變梯度大小。固定絕對梯度容許誤差，不能在任意損失正規化間直接比較。相對判準需要明確尺度，在零附近也應有意義。

迭代限制是資源停止，不是最佳性憑證；線搜尋失敗是演算法診斷，不是不存在解的證明。應保留最後候選點與殘差證據，讓結果可以評估。

精確二次線搜尋揭露取捨

對前述正定二次函數與非零下降方向 d，線上目標為純量二次函數。對距離微分得到

$$t^\ast = -\frac{g^\top d}{d^\top Q d}.$$

$g = Qx + c$ 是目前梯度，Q 是正定曲率，$t^* > 0$ 是最小化線距離。此公式只對這個特定二次函數給精確線搜尋，不適用於所有平滑目標。

對負梯度方向，分子變成 $g^\top g$，分母為 $g^\top Qg$。距離納入該方向的曲率，可能比保守固定步快，卻不會修正背後拉長的幾何。

連續方向仍可能鋸齒移動。沿一條線精確最小化，不表示線直接指向全域最小值。這清楚區分方向與距離：更好的距離選擇，不能完全取代更好的方向幾何。

牛頓方向使用完整二次曲率系統，在精確算術下以距離一達到最小值。度量近似可能降低系統成本，卻給較不準路線。公平比較須計入度量如何建立與重用，不只看可見路徑長度。

矩陣分解若能對許多右側重用，初始成本就可攤提；若每次迭代矩陣都變，優勢可能縮小。小教材目標評估很便宜，但內含模擬的應用可能很昂貴。宣稱速度前，應說明真正計算原語。

導數檢查需要尺度

中央有限差分把兩個鄰近目標值的差，除以兩倍擾動。縮小擾動，一開始會減少理想截斷誤差，最後卻放大浮點抵消。

應試幾種擾動大小，同時比較相對與絕對差。合理縮放方向、避開定義域邊界，並檢查結果是否在有用區間穩定。一次極小步比較失敗，可能表示數值解析度，而非導數錯誤。

對 Hessian，可比較梯度差與 Hessian 向量乘積。這比只看目標差，測到實作不同層次。目標二次連續可微且預期 Hessian 對稱時，也應檢查對稱性。

自動微分可以減少手動代數，卻仍是對實作表示式微分，而不是對意圖中的模型。錯誤廣播、加總位置不對或意外正則化截距，都可能產生錯誤目標的完全一致導數。

應把檢查與表示法一起記錄。目的是連結符號與陣列，不只是取得通過標記。確認實作符合模型後，數值容許誤差與演算法假設才有意義。

可重現比較也需要起點、目標正規化、停止規則與數值精度。任一項改變，都可能改變表面收斂。合成實驗應保留參數與隨機種子，讓之後的圖能從相同指定問題重新生成。

即使演算法確定性，平台小差異仍可能影響最後幾位數或邊界線搜尋比較。應以有理由的容許誤差驗證性質，不要求完全相同的小數軌跡。本章檢查下降、線性系統一致性與梯度證據，因為它們直接連到表示法。

現代觀點

白話說。現代凸演算法在曲率資訊、投影、近端運算與通訊成本間取捨。適合的選擇取決於結構與要求精度。

Dvurechensky、Shtern 與 Staudigl 的一階綜述 [2] 提供廣泛基礎，包括非歐幾里得近端方法、加速與免投影方法。它發表於 2021 年，是背景地圖，不是 2026 年新結果。

Condat 等人的 2023 年 SIAM Review 綜述 [3]，以原問題與對偶乘積空間和選擇的度量，統一高維非光滑凸問題的近端分裂。它說明加入絕對值懲罰，為何會改變合適計算原語：不能直接讓一般平滑牛頓公式穿過轉折點微分。

Pokutta 的 2024 年 Frank–Wolfe 綜述 [4] 專注受限制集合的線性最小化，以及收斂與憑證結構。它透過詢問能否避開投影或昂貴系統求解，補充梯度與牛頓比較。

這些綜述共同提供實用分類：平滑與非光滑目標、便宜與昂貴可行集合運算、歐幾里得與結構化幾何，以及局部曲率與便宜一階步驟。近期變體不會消除模型差異。

目前議題包括有可靠憑證的自適應方法、大線性系統的精度限制、分散式通訊，以及對求解器輸出微分。應讓演算法符合模型、硬體與要求誤差，並在可比精度下比較總工作量。

重點整理

- 下降方向與步長是兩種不同選擇。
- 固定梯度步需要有效曲率與定義域論證。
- 不良條件數會拖慢純量步長梯度方法。
- 回溯需要保護措施與正確導數符號。
- 牛頓使用線性求解，局部收斂有條件。
- 減量是模型量，除非已建立間隙假設。
- 可靠回報包含停止證據與拒絕工作成本。

練習

1. 推導 $1/L$ 步長的平滑下降不等式。

提示

把負縮放梯度代入二次上模型。

2. 把二次特徵值 40 換成 400，預測並測試固定步長梯度進度如何改變。

提示

平方向收縮取決於特徵值比例。

3. 在 $x > 0$ 上替 $-\log x + x^2$ 實作回溯，每次評估目標前先檢查定義域。

提示

立即拒絕非正候選點。

4. 在非零參數向量，以三個隨機方向驗證邏輯斯梯度。

提示

使用幾種有限差分尺度，比較中央差分與內積。

5. 說明奇異半正定 Hessian 為何不一定定義唯一牛頓方向。

提示

檢查線性系統零空間與一致性。

6. 把邏輯斯目標乘上一百，求如何改變停止量與容許誤差，保留原本要求的目標準確度。

提示

一起轉換梯度、強凸常數與要求間隙。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **閱讀：第 9 章第 9.1–9.7 節；線性系統背景看附錄 C；邏輯斯概似連結看第 7 章。** 官方書頁，免費 PDF。
2. Pavel Dvurechensky, Shimrit Shtern and Mathias Staudigl, “First-Order Methods for Convex Optimization,” *EURO Journal on Computational Optimization*, 9, 100015, 2021. 已核對開放手稿，出版文章。
3. Laurent Condat, Daichi Kitahara, Andrés Contreras and Akira Hirabayashi, “Proximal Splitting Algorithms for Convex Optimization: A Tour of Recent Advances, with New Twists,” *SIAM Review*, 65(2), 375–435, 2023. 出版綜述，開放手稿。
4. Sebastian Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. 開放綜述。

OPT 07・等式限制與內點法

求解受限牛頓系統、追蹤對數中心路徑、區分可行與嚴格可行，並查核原始／對偶憑證。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/constrained-interior-point/

你將學到

- 把牛頓法延伸到含仿射等式的凸目標函數。
- 推導對數障礙函數、中心路徑及其對偶間隙。
- 實作具有明確線搜尋的小型可行障礙法求解器。
- 區分可行、嚴格可行與不可行的限制系統。
- 解讀原始／對偶殘差、互補性與不可行性憑證。
- 依數值需求比較現代內點法與一階求解器。

先看全貌

限制式說明解可以做什麼。求解器改善目標時不能忽略它們。等式把每個容許步驟限制在較低維度的空間；不等式則產生必須處理的邊界。內點法（interior-point method）透過一系列問題逼近有用的邊界解，而迭代點保持在邊界內側。

> **用最簡單的話說**
>
> 想像要在有圍籬的花園找到最低的位置。你得留在花園內，地上的線也可能要求你沿特定路徑走。先從離圍籬有一段距離的地方開始；靠近圍籬時，一個想像中的軟墊把你推回來。逐漸減弱軟墊，就能接近圍籬旁的低點而不走出去。

本章搭配 Boyd 與 Vandenberghe 第 10–11 章 [1]：第 10 章處理等式限制下的最小化，第 11 章發展障礙函數、中心路徑、初始化與原始／對偶方法。先讀梯度與牛頓法；憑證看不懂時，回到對偶性與最適條件。Bubeck 的演算法專論 [2] 提供複雜度與求解器家族的另一種觀點。

範例使用低維度合成問題，讓每項主張都能獨立查核。程式是附有斷言、假設透明的教學實作。正式錐規劃求解器還需要尺度調整、稀疏矩陣分解、數值正則化、停止政策與不可行性處理；理解小問題有助於理解這些工程選擇。

1. 等式限制改變容許的方向

目標下降時，仿射等式仍須成立。目前點若滿足 $Ax = b$，步驟 Δx 保持可行的充分必要條件是 $A\Delta x = 0$。先看幾何再看矩陣：容許的步驟位於等式矩陣的零空間。

考慮凸問題：

$$\begin{aligned} \text{minimize}_x \quad & f(x) \\ \text{subject to} \quad & Ax = b. \end{aligned}$$

其中 $x \in \mathbb{R}^n$ 是變數，f 是二次可微的凸目標函數，$A \in \mathbb{R}^{p\times n}$ 與 $b \in \mathbb{R}^p$ 是固定資料。先假設 A 各列獨立，且等式系統有解。拉格朗日乘子 $\nu \in \mathbb{R}^p$ 表示等式限制的影響。

在可行迭代點，沿可行方向最小化二次近似，得到牛頓系統：

$$\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ w \end{bmatrix} = - \begin{bmatrix} \nabla f(x) \\ 0 \end{bmatrix}.$$

其中 $H = \nabla^2 f(x)$ 是 Hessian 矩陣，Δx 是原始變數的候選步驟，w 是局部二次子問題的等式乘子，零區塊大小為 $p \times p$。第二個區塊方程強制 $A\Delta x = 0$。w 解的是局部模型，不一定等於另一個持續維護的乘子估計之增量。

Hessian 只須沿非零可行方向具有正曲率，受限二次模型就能表現良好；被等式排除的方向不必有正曲率。因此完整 Hessian 可能奇異，但受限最小解仍唯一。判斷奇異性是否代表多解前，先檢查真正的可行子空間。

區塊矩陣稱為 Karush–Kuhn–Tucker（KKT）矩陣 [1]。即使 H 正定，整個矩陣通常仍是不定矩陣。不要因原目標凸，就對整個矩陣使用只適合正定矩陣的分解。其結構同時表達原始與對偶方程；小型稠密範例使用一般線性求解即可。

另一種方法是消去等式。取一個可行點 x_0，以及欄向量張成 A 零空間的矩陣 F，寫成 $x = x_0 + Fz$，其中 $z \in \mathbb{R}^{n-p}$ 是降維變數，再解 z 的無限制問題。消去法能簡化小問題，卻可能破壞大問題的稀疏性。兩者幾何相同，計算表示決定成本。

限制子空間曲率與區塊消去

Hessian 奇異不代表受限步驟不明確。在 $x_2 = 3$ 下最小化 $f(x) = x_1^2/2$。Hessian 為 $\operatorname{diag}(1, 0)$，但可行方向只能改變 x_1。唯一最小解為 $(0, 3)$：等式恰好移除平坦方向。

一般而言，令 F 的獨立欄向量張成 A 的零空間。縮減 Hessian 為 $F^\top H F$。這個較小矩陣正定，就能讓可行方向上的局部二次模型有唯一最小解，即使 H 奇異也一樣。其中 H 是目前 Hessian、A 是滿列秩的等式矩陣 [1，第 10 章]。

若 H 本身正定，也可消去原問題方向來推導 KKT 步驟。寫 $g = \nabla f(x)$，利用第一個區塊方程：

$$\begin{aligned} \Delta x &= -H^{-1}(g + A^\top w), \\ (AH^{-1}A^\top)w &= -AH^{-1}g. \end{aligned}$$

其中 Δx 是步驟、w 是局部等式乘子。把第一行代入 $A\Delta x = 0$ 即得第二行。A 各列獨立時，這個較小矩陣為正定：對非零 v，二次型為 $(A^\top v)^\top H^{-1}(A^\top v) > 0$。

這些方程解釋區塊求解，但反矩陣符號只是代數表示。實作應透過 H 的線性求解計算乘積，再比較消去法與直接稀疏 KKT 分解的成本。前面的奇異例子仍能使用零空間表示，卻不能使用這種完整 Hessian 消去。

2. 帶有可查核憑證的受限二次問題

二次目標讓 KKT 方程成為精確條件而非近似。區塊系統產生候選解；檢查等式可行性與駐點條件，才能判斷候選解是否構成這個凸問題的最適性憑證。

對於：

$$f(x) = \frac{1}{2}x^\top Qx + c^\top x,$$

$Q \in \mathbb{R}^{n\times n}$ 是對稱半正定矩陣，$c \in \mathbb{R}^n$ 是固定向量，x 是變數。最適配對 (x^*, ν^*) 滿足 $Qx^* + c + A^\top \nu^* = 0$ 及 $Ax^* = b$；星號代表最適值。對可行凸二次問題，這些方程足以保證最適；唯一性還需要可行方向的額外曲率。

```
import numpy as np

Q = np.diag([2.0, 4.0])
c = np.array([-2.0, -1.0])
A = np.array([[1.0, 1.0]])
b = np.array([1.0])
K = np.block([[Q, A.T], [A, np.zeros((1, 1))]])
solution = np.linalg.solve(K, np.concatenate([-c, b]))
x, nu = solution[:2], solution[2:]
primal = A @ x - b
dual = Q @ x + c + A.T @ nu
print("Primal solution:", x)
print("Equality multiplier:", nu)
print("Residual norms:", np.linalg.norm(primal), np.linalg.norm(dual))
assert np.allclose(x, [5/6, 1/6])
assert np.linalg.norm(primal) < 1e-12
assert np.linalg.norm(dual) < 1e-12
```

兩個座標相加為一。無限制最小解的座標則為一與四分之一，違反總量限制。乘子修正駐點方程以反映限制。目標值小不足以證明這項修正；殘差檢查把它明確表達出來。

殘差依賴單位。把等式乘上很大的常數，不改變可行集合，卻改變原始殘差與乘子的尺度。回報原限制的單位及尺度，並用有意義的容許誤差比較殘差。求解器內部的正規化停止條件須轉回應用的量測單位。

不要為了求解而明確計算矩陣反矩陣。線性求解較直接，通常也較穩定。重複求解時，矩陣結構允許才能重用分解。更新線性項與更新 Hessian 或限制矩陣不同；規劃暖啟動與快取時要區分。

非線性目標需要重新計算局部曲率，並採可行線搜尋。完整牛頓步驟即使保持等式，也可能離開目標函數定義域。對數與正定矩陣變數不是處處有定義；等式方程本身無法保護這些定義域。

3. 不等式轉成平滑的內部問題

障礙函數接近邊界時無限增大，只在嚴格可行點有限。如此平滑的受限牛頓法就能把不等式透過目標處理，同時保留明確的仿射等式。

考慮凸不等式 $f_i(x) \le 0$，$i = 1, \dots, m$，以及 $Ax = b$，定義：

$$\phi(x) = -\sum_{i=1}^{m} \log(-f_i(x)), \qquad F_t(x) = tf_0(x) + \phi(x).$$

其中 f_0 是原凸目標，f_i 是各凸不等式函數，m 是純量不等式數目，ϕ 是對數障礙函數，$t > 0$ 控制原目標的相對重要性。定義域要求每個 $f_i(x) < 0$；障礙問題在 $Ax = b$ 下最小化 F_t。

限制函數應以一致單位表示，或取對數前除以正的參考尺度。正數縮放只增加障礙函數的常數項，不改變最小解，但有限精度計算仍可能受影響。數學上的不變性不能當成忽略數值尺度的理由。

二次可微不等式的障礙項，梯度為 $\nabla f_i(x)/(-f_i(x))$，Hessian 為：

$$\frac{\nabla f_i(x)\nabla f_i(x)^\top}{f_i(x)^2} + \frac{\nabla^2 f_i(x)}{-f_i(x)}.$$

其中 ∇f_i 是梯度欄向量，$\nabla^2 f_i$ 是 Hessian。因 f_i 凸，嚴格可行定義域內兩項皆半正定。式子也顯示接近活躍邊界、$-f_i(x)$ 變小時，曲率迅速增加。

限制的表示影響路徑。重複同一限制會重複其障礙項，因此有限 t 的近似問題不同，雖然精確可行集合相同。等價限制集合不一定產生相同中間迭代點；標準化與尺度調整是能觀察效果的演算法選擇。

障礙法不能從邊界開始。非負變數起點若為零，$-\log x$ 無定義。自由變數可以用嚴格正值初始化；若等式強制變數為零，則需要重寫問題。任意給限制加一個正偏移會改變問題，不能當作無害的實作修正。

4. 中心路徑連接內點與對偶界限

對每個障礙權重準確求解內部問題，其最小解形成中心路徑（central path）。增加權重會減弱障礙相對原目標的影響，讓解接近可能位於邊界的受限最小解。

令 $x(t)$ 在 $Ax = b$ 下最小化 F_t，定義：

$$\lambda_i(t) = \frac{1}{t[-f_i(x(t))]}, \qquad \nu(t) = \frac{w(t)}{t}.$$

其中 $\lambda_i(t) > 0$ 是不等式乘子，$w(t)$ 是縮放障礙問題的等式乘子，$\nu(t)$ 則是原目標尺度的乘子。t、f_i 與 $x(t)$ 的定義同前。

障礙駐點條件成為原拉格朗日函數的駐點條件；互補性改為 $\lambda_i(t)[-f_i(x(t))] = 1/t$。各純量不等式貢獻相同的正乘積。真正最適點的互補鬆弛要求這些乘積為零；t 增加時，障礙路徑逐漸逼近這項條件。

在完全置中且滿足所述假設的點，其原始／對偶間隙為：

$$f_0(x(t)) - g(\lambda(t), \nu(t)) = \frac{m}{t}.$$

其中 g 是拉格朗日對偶函數，對對偶可行乘子提供最適目標的下界。原始值是上界，對偶值是下界，間隙因此限制目標次最適誤差。這是關於所建立凸問題的敘述，不保證模型符合應用。

等式需要精確置中及有效的對偶構造。駐點殘差很大的數值迭代點，不能只印出 m/t 就宣稱得到精確間隙。以理論值規劃障礙排程，再檢查真實殘差，能計算時也求出真正的對偶下界。如此才不會把外迴圈計數當作憑證。

外層障礙法交替進行兩件事：解固定 t 的內部問題，再增加 t。以前一解作下一起點通常有幫助，因最小解往往相近；但 t 一次改太多會使置中昂貴。依置中實際成本選擇增幅，而不是假設外層迭代越少，總計算就越少。

間隙 m/t 真正從哪裡來

中心路徑間隙是明確價格乘以明確鬆弛量的總和。在精確置中的點，把障礙問題駐點方程除以 t：

$$\nabla f_0(x(t)) + \sum_i \frac{\nabla f_i(x(t))}{t[-f_i(x(t))]} + A^\top \frac{w(t)}{t} = 0.$$

其中 $x(t)$ 最小化障礙問題、$w(t)$ 是其等式乘子、f_0, f_i 是原目標與不等式、$t > 0$ 是障礙權重。代入已定義的 $\lambda_i(t) = 1/[t(-f_i(x(t)))]$ 與 $\nu(t) = w(t)/t$，便恰好得到原拉格朗日函數的駐點方程。

原函數凸且不等式乘子非負，因此拉格朗日函數對 x 凸。駐點條件使 $x(t)$ 成為其全域最小解，所以在此點評估 L 確實算出對偶函數。等式可行性消去等式項，得到

$$g(\lambda(t), \nu(t)) = f_0(x(t)) + \sum_i \lambda_i(t) f_i(x(t)) = f_0(x(t)) - \frac{m}{t}.$$

其中 g 是對偶函數、m 是純量不等式數量。從可行原問題目標減去這個下界，即得間隙。中心點通常不是原最佳解：它在目標改善與正鬆弛量間取捨，但其乘子已提供憑證，而且誤差隨障礙減弱而下降 [1，第 11.2 節]。

若希望理論間隙至多為 $\varepsilon > 0$，選 $t \geq m/\varepsilon$，再把置中問題解得足夠準確。選擇 t 提供目標；駐點求解才是使用公式的依據。只增加 t 卻不置中，就漏掉證明的關鍵等式。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

5. 完整的一維障礙法實驗

使用真正最小解容易查核的二次問題：在 $0 \leq x \leq 1$ 下最小化 $\frac{1}{2}(x-2)^2$。無限制最小點是二，最近的容許點是一，目標值為二分之一。障礙解嚴格位於零與一之間，逐漸接近上端點。

此例為：

$$F_t(x) = \tfrac{t}{2}(x-2)^2 - \log x - \log(1-x).$$

其中 $x \in (0,1)$ 是純量變數，$t > 0$ 是目標權重。一、二階導數為 $F_t'(x) = t(x-2) - 1/x + 1/(1-x)$ 與 $F_t''(x) = t + 1/x^2 + 1/(1-x)^2$；障礙定義域內所有分母皆正。

```
import numpy as np
```

```
def center(t, x):
    def value(z):
        return 0.5*t*(z-2)**2 - np.log(z) - np.log1p(-z)
    for _ in range(100):
        grad = t*(x-2) - 1/x + 1/(1-x)
        hess = t + 1/x**2 + 1/(1-x)**2
        step = -grad/hess
        if grad*grad/hess < 1e-16:
            break
        alpha = 1.0
        for _ in range(100):
            trial = x + alpha*step
            if 0 < trial < 1 and value(trial) <= value(x) + 0.01*alpha*grad*step:
                break
            alpha *= 0.5
        else:
            raise RuntimeError("Line search failed")
        x = trial
    else:
        raise RuntimeError("Centering iteration limit reached")
    return x
x = 0.5
for t in [1., 4., 16., 64., 256., 1024.]:
    x = center(t, x)
    lower, upper = 1/(t*x), 1/(t*(1-x))
    stationarity = x-2-lower+upper
    primal = 0.5*(x-2)**2
    dual = 2-upper-0.5*(-2-lower+upper)**2
    gap = primal-dual
    print(int(t), round(x, 6), round(gap, 7), round(2/t, 7))
    assert 0 < x < 1
    assert abs(stationarity) < 1e-5
    assert abs(gap-2/t) < 1e-8
    assert primal-0.5 <= gap + 1e-8
```

可行回溯檢查在目標比較前執行，避免於定義域外計算對數。充分下降條件使用方向導數，牛頓遞減量則用於置中停止條件。迭代上限讓意外失敗變成明確錯誤，而非無窮迴圈。

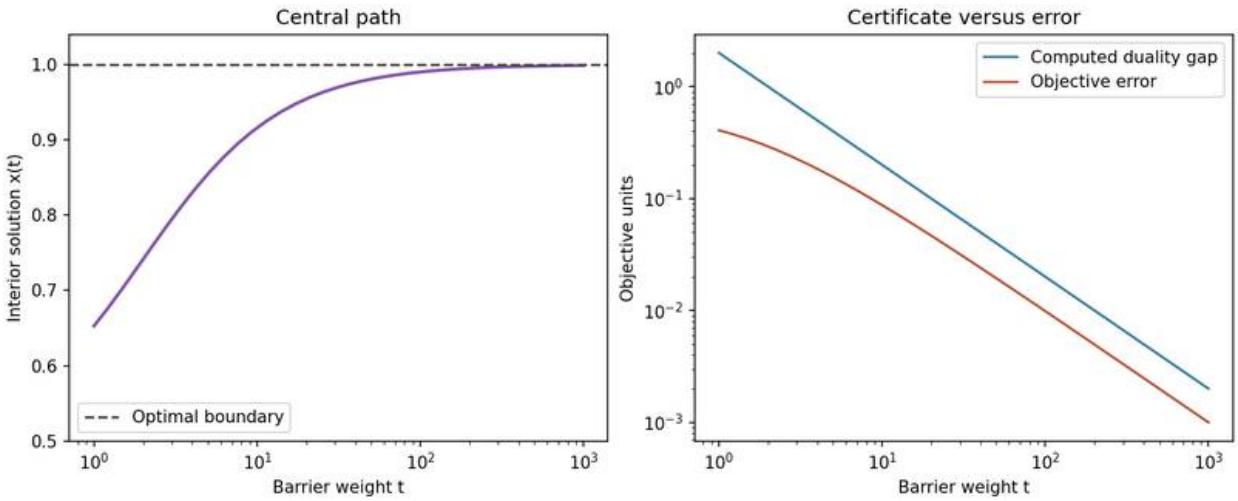


圖 *1* — 合成受限二次問題讓中心路徑憑證可見：解從內部接近一，計算出的對偶間隙限制其目標誤差。

實驗透過純量拉格朗日函數的無限制最小值計算真正對偶值，因此能獨立比較間隙與指定的 $2/t$。大型非線性問題中，對偶函數本身可能需要另一個最佳化問題才能求值。原始／對偶殘差檢查此時有用，但必須說明它在何種正則條件下證明什麼。

接近端點時，1-x 很小，障礙 Hessian 很大。即使數學問題不變，更小的誤差目標仍可能使數值系統更難解。達到應用單位下足夠的準確度即可停止；要求每個可表示位數都準確，可能浪費工作，或暴露應用根本不需處理的條件數問題。

6. 可行性、嚴格可行性與 Phase I

可行解可能存在，卻沒有任何嚴格可行解。原不等式表示的障礙函數此時沒有定義域。這與不可行不同，求解器必須先區分，再嘗試追蹤中心路徑。

簡單的 Phase I（第一階段）構造增加純量 s：

$$\begin{aligned} \text{minimize}_{x,s} \quad & s \\ \text{subject to} \quad & f_i(x) \le s \quad (i = 1, \dots, m), \\ & Ax = b. \end{aligned}$$

其中 s 是限制最大不等式違反量的新增變數。$s < 0$ 的點對原不等式嚴格可行；最適 s 有經證明的正下界，就能證明不可行。最適值零可能代表只有邊界可行；未達到下確界與數值不確定性須另外處理。有限迭代失敗本身不能證明任何一種結果。

矛盾系統 $x \le 0$ 與 $x \ge 1$ 的違反量為 $\max(x, 1 - x)$，在 $x = 1/2$ 達到最小值二分之一。對兩條不等式取相同非負權重，也能產生明確的不可行性憑證。

```
import numpy as np

A = np.array([[1.0], [-1.0]])
```

```
b = np.array([0.0, -1.0])
y = np.array([0.5, 0.5])
print("A transpose y:", A.T @ y)
print("b transpose y:", b @ y)
assert np.all(y >= 0)
assert np.allclose(A.T @ y, 0)
assert b @ y < 0
# If A x <= b existed, multiplying by y would imply 0 <= b@y < 0.
grid = np.linspace(-1, 2, 301)
violation = np.maximum(grid, 1-grid)
assert np.isclose(violation.min(), 0.5)
# A different system is feasible only on its boundary.
A_boundary = np.array([[1.0], [-1.0]])
b_boundary = np.zeros(2)
assert np.all(A_boundary @ np.array([0.0]) <= b_boundary)
```

網格只是已推導最小值的示意，不是任意最佳化問題全域求解的證明。憑證才是關鍵論證：若 $Ax \le b$，乘以 $y \ge 0$ 應得到 $0 = y^\top Ax \le y^\top b < 0$，矛盾。可直接查核資料與乘子。

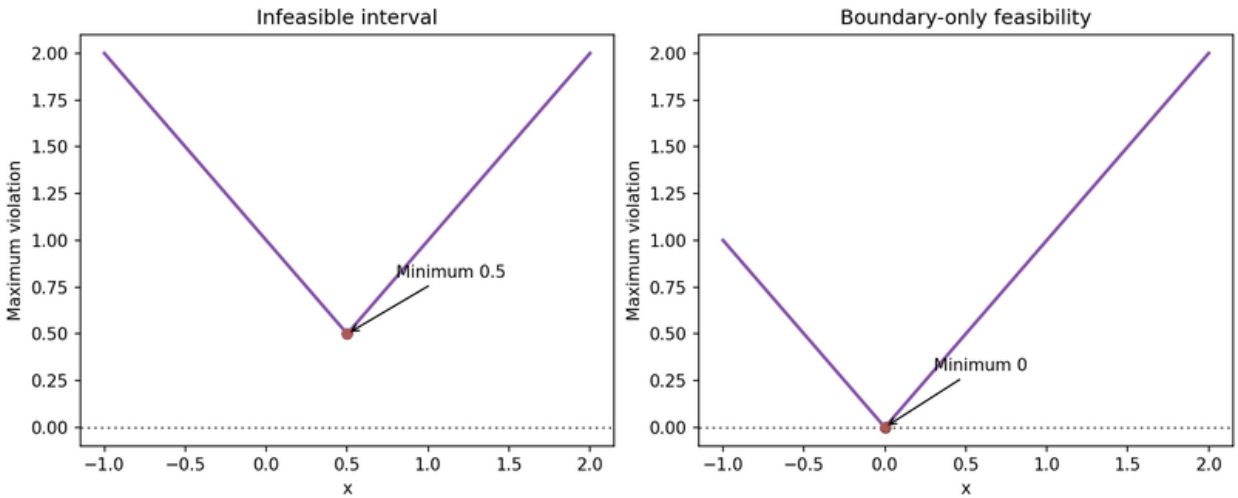


圖 2 — 正的最小違反量證明矛盾區間不可行；第二例的零最小值則指出一個沒有嚴格內部的可行點。

只有邊界可行的系統 $x \le 0$、$-x \le 0$，唯一可行點是零。消去被固定的變數，或把條件寫成等式。起點稍微移動沒有用，因往任一方向移都會違反其中一條。相關內部是把強制等式處理後的相對內部。

Phase I 也依賴 f_i 定義域。放寬不等式值不會自動讓對數、矩陣反矩陣或其他運算有定義。須在函數定義域內初始化，或採合適改寫。函數定義域可行性與仿射、不等式殘差應分開檢查。

Phase I 的對偶價格會正規化

最大違反量形式把初始化連到對偶憑證。對 $f_i(x) - s \le 0$ 附乘子 $\lambda_i \ge 0$，對 $Ax - b = 0$ 附乘子 ν。其拉格朗日函數為

$$L_I(x, s, \lambda, \nu) = \left(1 - \sum_i \lambda_i\right) s + \sum_i \lambda_i f_i(x) + \nu^\top (Ax - b).$$

其中 x, s 是 Phase I 變數、λ, ν 是其乘子、f_i, A, b 是原限制資料。因為 s 不受符號限制，其下確界只在 $\sum_i \lambda_i = 1$ 時有限。因此不等式價格形成正規化非負權重。剩下加權函數的有限下界，會界定原限制的最大違反量 [1，第 11.4 節]。

對 $f_1(x) = x$、$f_2(x) = 1 - x$，取 $\lambda_1 = \lambda_2 = 1/2$。加權和對每個 x 都恰為 $1/2$。因此 Phase I 最佳值至少為 $1/2$，而 $x = 1/2, s = 1/2$ 達到該值。原問題與對偶值相等，證明原不等式不可行。

對只有邊界可行的 $f_1(x) = x$、$f_2(x) = -x$，相同正規化加權和為零。可行 Phase I 點 $x = 0, s = 0$ 匹配它，卻不可能有負的 s。零違反量可證明普通可行性，卻無法提供對數障礙需要的嚴格內部。消去被強制的等式，是適當的下一個建模步驟。

7. 原始／對偶步驟與可靠停止

原始／對偶方法直接處理近似最適方程，不必完全解完每個獨立障礙問題。它維護正的不等式乘子與鬆弛量，再解線性化系統，同時降低駐點、可行性與互補性誤差。

令不等式 i 的鬆弛量為 $s_i = -f_i(x) > 0$，乘子 $\lambda_i > 0$。原問題的殘差包括：

$$r_{\text{dual}} = \nabla f_0(x) + \sum_i \lambda_i \nabla f_i(x) + A^\top \nu,$$

$$r_{\text{pri}} = Ax - b,$$

$$\eta = \sum_i \lambda_i s_i.$$

其中 r_{dual} 是駐點殘差，r_{pri} 是等式殘差，η 是互補乘積總和。函數、矩陣、乘子與鬆弛量定義同前。仿射不等式可以明確加入獨立的鬆弛量方程；非線性表示則可透過 $f_i(x)$ 表達。

η 很小但可行性殘差很大，不是有效的近似解。無限制座標中的梯度很小，也不能證明受限最適。所有相關殘差、乘子符號與定義域條件都要通過。原始點與乘子可行且滿足駐點時，互補乘積總和對應對偶間隙；否則需要限定解讀。

保留邊界距離的線搜尋（fraction-to-boundary）限制步幅，保持鬆弛量與乘子為正。它是演算法定義域的數值保護，不是新增的解限制。不可行起點法可以一開始違反仿射等式，同時保留公式所需的內部條件；其評價函數須減少殘差，而不能只在不可行點比較原目標。

稀疏線性代數往往占主要成本。非零樣式決定分解填入、記憶體及可否重用。少量迭代也可能每一步都很昂貴。比較方法時，除迭代次數外也記錄分解時間與尖峰記憶體。稠密教學實作隱藏這些成本，不宜預測大型問題效能。

標準化求解後，須回到原問題查核解。求解器可能引入輔助變數、錐限制，正規化資料或移除冗餘列。這些操作有助計算，但應用需要原單位限制與可解讀的目標。記錄狀態、殘差、容許誤差、模型版本，以及任何改動回傳點的後處理。

線性化原始／對偶方程

原始／對偶牛頓步驟同時處理最適方程。對 $Q \succeq 0$ 的二次目標、仿射不等式 $Gx + s = h$ 與等式 $Ax = b$，使用正鬆弛向量 s 和正乘子向量 λ。令 c 為目標線性係數，ν 為等式乘子。

定義 $r_d = Qx + c + G^\top \lambda + A^\top \nu$、$r_g = Gx + s - h$、$r_a = Ax - b$。選定互補性目標 $\tau > 0$ 後，線性化得到

$$\begin{aligned} Q\Delta x + G^\top \Delta\lambda + A^\top \Delta\nu &= -r_d, \\ G\Delta x + \Delta s &= -r_g, \\ A\Delta x &= -r_a, \\ \Lambda \Delta s + S\Delta\lambda &= \tau \mathbf{1} - S\lambda. \end{aligned}$$

其中 G, h 指定仿射不等式係數與界限；A, b 指定等式；r_d, r_g, r_a 分別為駐點、不等式可行性與等式可行性殘差，τ 是互補性目標。$\Delta x, \Delta s, \Delta\lambda, \Delta\nu$ 是增量；$S = \operatorname{diag}(s)$、$\Lambda = \operatorname{diag}(\lambda)$，$\mathbf{1}$ 是全一向量。最後一列線性化乘積 $s_i \lambda_i = \tau$，只省略增量的二階乘積。相較完整障礙置中求解，一步就同時降低可行性、駐點與互補性誤差 [1，第 11.7 節]。

若某個鬆弛增量為負，該座標要求步長 $\alpha < -s_i / \Delta s_i$ 才能保持正值。負乘子增量也套用相同規則，取最小容許界限並略留在內側。這便推導出朝邊界比例限制，而非任意裁切規則。仍須加上殘差下降測試：只保持正值並不保證收斂。

8. 在應用邊界解讀憑證

最適性憑證回答指定模型的數學問題。應用決策還取決於模型是否納入正確的限制、量與不確定性。最終報告分開查核兩者：數值正確不能補救遺漏的需求。

假設資源分配模型限制總功率，卻漏掉個別裝置上限。求解器可能得到全域最適且精確證明的分配，但某台裝置無法執行。補上遺漏限制再解，不要把先前結果叫作求解器失敗。相反，限制都已納入，回傳點卻超出容許誤差，問題屬於數值驗收或求解本身。

看到答案前先選容許誤差。溫度上界、預算等式與無因次機率限制需要不同解讀。先用原單位寫出可接受偏差，再連接求解器誤差與尺度設定。後續四捨五入若改變變數，須再次查核可行性；未四捨五入向量的憑證，不會自動證明顯示或實際執行的決策。

符合對偶章節假設時，乘子可解釋敏感度。緊限制的大乘子表示局部小幅放寬可能明顯影響目標；它不是任意大幅變動的預測，尤其活躍限制改變時。小幅擾動、重解問題，並比較實際目標變化與局部預測。

近退化問題須謹慎解讀。幾條幾乎冗餘的限制可能使乘子難以穩定決定，原始決策卻很穩定。乘子變化此時可能誇大決策本身的不確定性。原始與對偶敏感度都要看，影子價格的精度不可超過資料與條件數能支持的程度。

暖啟動也屬於這個邊界。昨天的解可能靠近今天的最小解，卻違反更新後的限制。可行起點障礙法須先找新的內點；原始／對偶或齊次嵌入求解器則可能採其他初始化策略。只有方法支援該可行性狀態才傳入暖啟動；耦合等式存在時，不要逐座標裁切。

有用的結果組合包含模型與資料版本、原目標、原始變數、相關乘子、原單位最大違反量、有效

的間隙或下界，以及停止狀態。效能重要時再附實測時間與記憶體。讀者才能區分可信的近似最小解、不可行性憑證、尚未解決的數值結果，以及需要修正的模型。

現代觀點

內點理論仍有用，現代求解器則在精度、記憶體與硬體上採不同選擇。以下資料提供演算法分類與截至 2026 年 10 月 4 日查核的現代證據。

Bubeck 專論 [2] 依 oracle 提供的資訊、幾何與結構整理一階、結構式及內點方法。比較應同時考慮每步成本，以及 oracle 與假設附帶的保證。本章綜合觀點是依問題中便宜的運算選方法：矩陣向量乘法、投影、近端映射或矩陣分解。

Parikh 與 Boyd 的近端綜述 [3] 用小型凸子問題整理非平滑、受限與分散式演算法，補足障礙觀點：投影或近端運算可以不透過對數內部表示就處理限制。其成本依集合不同而變；數學凸性不決定哪個運算便宜。

Goulart 與 Chen 的 2026 年 Clarabel 論文 [4] 描述使用齊次嵌入、支援多種錐及二次目標的內點求解器，把教科書原理連接到處理不可行性與計算結構的軟體。比較結果屬於所測問題與設定，不代表每個凸模型都有同一個全面贏家。

Applegate 等人的 PDLP 手稿 [5] 於 2025 年首次發布、2026 年修訂，以矩陣向量運算為基礎，加入前處理、尺度調整與自適應策略，發展大型線性規劃的一階方法。它是預印本，效能主張是所報工作負載上的實驗證據。與內點法比較時，納入要求精度、記憶體預算與原單位殘差。

求解器家族	昂貴運算	結構問題	須驗證什麼
等式限制牛頓法	KKT 線性求解	能否重用稀疏結構與分解？	可行性與駐點條件
障礙／內點法	改變的曲率與受限求解	是否有合適內部與錐表示？	定義域、殘差、間隙或憑證
投影／近端方法	投影或近端子問題	限制或非平滑項是否易處理？	迭代殘差與可行輸出
大型一階線性規劃	矩陣向量乘法	其他方法是否受記憶體或分解限制？	指定準確度下的縮放與原始殘差

目前方法論的重點不限於偏好一階或二階。模型表示、前處理、尺度、線性代數與停止政策都是實際演算法行為的一部分。只比較命名的更新公式，會漏掉決定可靠度的許多系統工作。

未解問題包括尺度不良模型的準確憑證、幾乎低維可行區域的穩健性、大型分解的記憶體限制，以及限制改變時的有效暖啟動。學習式預測可以提供起點，但仍須查核可行性與最適性。近似解的誤差若能用應用可容忍的量表示，就有價值。

求解器實驗可建立只改變一項性質的問題族：條件數、活躍限制數、稀疏性或要求誤差。盡可能保留已知最小解或獨立查核的界限。除了時間，也回報失敗與不準確狀態。這類控制問題族比一次預設成功求解更有教學價值。

重點整理

- 等式限制牛頓步驟保持在等式矩陣的零空間。

- 即使目標凸，KKT 矩陣通常仍不定。
- 對數障礙需要相關定義域中的嚴格可行性。
- 完全置中點在所述假設下產生間隙為 m/t 的對偶構造。
- 數值迭代點須檢查真實可行性、駐點條件與互補性。
- 只有邊界可行與不可行不同；Phase I 須用憑證可靠區分。
- 在原問題單位下比較求解成本、精度與記憶體。

練習

1. **受限曲率**。建立完整 Hessian 奇異、但在仿射等式集合上有唯一最小解的半正定二次問題。

   提示

   讓零曲率方向被等式排除，只檢查滿足齊次等式的非零向量之曲率。

2. **等式縮放**。將二次範例的等式乘以 1000，比較解、乘子與殘差尺度，說明哪些量應保持不變。

   提示

   可行集合不變；乘子反向縮放，原始殘差隨限制列縮放。兩個表示都要查核駐點條件。

3. **重複障礙**。複製純量範例的上界不等式，比較有限 t 中心路徑與理論間隙使用的項數。

   提示

   可行集合相同，但上界對數出現兩次，現在有三個純量障礙項。

4. **初始化**。讓純量求解器從零開始，解釋失敗。對此問題及等式強制為零的變數，分別給出有原則的修正。

   提示

   自由非負變數能從嚴格正值開始；被固定為零的變數須消去或改用等式表示，不能只微調數值。

5. **不可行性憑證**。把矛盾區間推廣成 $x \le a$ 與 $x \ge b$，$a < b$。推導最大違反量的最小值及非負憑證。

   提示

   中點平衡兩項違反量。相同權重消掉變數，留下負的右側總和。

6. **停止政策**。求解器回報互補性很小，等式殘差卻非零。寫出接受結果前須加的檢查，解釋只回報目標為何不足。

   提示

   納入原單位等式與不等式殘差、定義域、乘子符號、駐點條件，以及能取得時的有效下界。

參考文獻

1. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, Chapters 10–11. Book and official PDF
2. S. Bubeck, “Convex Optimization: Algorithms and Complexity,” Foundations and Trends in Machine Learning, 2015. Monograph
3. N. Parikh and S. Boyd, “Proximal Algorithms,” Foundations and Trends in Optimization, 2014. Author page and article
4. P. J. Goulart and Y. Chen, “Clarabel: An Interior-Point Solver for Conic Programs with Quadratic Objectives,” Mathematical Programming Computation, 2026. Published paper · Author manuscript
5. D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O’Donoghue and W. Schudy, “PDLP: A Practical First-Order Method for Large-Scale Linear Programming,” arXiv preprint, 2025, revised 2026. Paper

OPT 08・幾何應用、分裂法與現代方法

把幾何與訊號模型轉成凸問題，選擇近端、ADMM 或條件梯度運算，並以明確憑證評估最佳化流程。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/applications-modern-methods/

你將學到

- 用凸幾何建立距離、中心、分離與配置模型。
- 區分凸應用模型與離散或非凸問題的鬆弛。
- 推導非平滑項與限制的近端更新。
- 實作合成訊號去雜訊的分裂法及單純形上的條件梯度法。
- 依昂貴運算選擇方法，查核相應收斂證據。
- 建立可重現的最佳化專題，連接現代綜述與方法。

先看全貌

問題結構同時告訴你如何建模、如何求解時，凸最佳化就有實際價值。幾何模型可能讓投影簡單；訊號模型可能把平滑資料項與非平滑先驗分開；分散式模型可能讓局部求解便宜，但通訊昂貴。合適的演算法運用這些結構。

> **用最簡單的話說**
>
> 想像安排一間工作室：每台機器要有空間、門口要保持暢通，也希望走路距離短。先決定哪些安排被允許，再決定「更好」是什麼。有些規則能讓配置平滑改善；有些則要求一台機器完全在另一台左邊或右邊。這類二選一規則須特別處理。圖畫起來平順，不代表所有配置問題都凸。

幾何閱讀搭配 Boyd 與 Vandenberghe 第 8 章，並連接第 4、6 章的建模與近似 [1]。演算法延伸搭配 Parikh 與 Boyd 的近端綜述 [2]、Boyd 等人的 ADMM 綜述 [3]、Condat 等人的分裂綜述 [4]，以及 Pokutta 的條件梯度導論 [5]。本章把這些運算連成完整的建模、求解與驗證流程，作為單元收尾。

全部實驗使用合成資料、在本機 CPU 執行，目的是讓目標、假設、殘差與失敗模式可查核。數值目標降低，只有在可行集合、資料尺度與評估規則都確認後才有意義。

1. 距離、投影與分離

距離問題問兩個點或集合能在各自限制下靠多近。投影找最近的容許點。這些幾何運算是應用與最佳化演算法的共同積木。

對非空閉凸集合 $C \subseteq \mathbb{R}^n$，定義：

$$\Pi_C(v) = \arg\min_{x \in C} \tfrac{1}{2}\|x - v\|_2^2.$$

其中 $v \in \mathbb{R}^n$ 是固定查詢點，x 是最佳化變數，$\|\cdot\|_2$ 是歐氏範數，$\Pi_C(v)$ 是投影。目標對 x 嚴格凸，因此最小解唯一；閉性確保能達到最小值，非空則確保有可行點。

盒狀集合投影逐座標裁切到區間；仿射集合投影解受限二次問題；半正定矩陣錐投影涉及對稱矩陣的譜分解。它們都是凸投影，成本卻很不同。「投影梯度」只有在理解投影成本後才是有用的演算法描述。

投影滿足 $(v - p)^\top (x - p) \le 0$，對每個 $x \in C$ 成立，其中 $p = \Pi_C(v)$。從投影指向查詢點的位移，背離所有可行方向。它提供最適性檢查，且當查詢點位於集合外時產生分離超平面。

兩個凸集合的距離可寫成最小化 $\|x - y\|_2$，限制為 $x \in C$、$y \in D$，其中 D 是第二個集合。範數對變數配對凸。但任意無界集合的有限下確界不一定能達到；解讀迭代點是否收斂至最近配對前，先查核存在性。緊緻集合較容易處理。

以分離進行分類也有幾何解釋。線性分數 $a^\top x + b$ 定義超平面，a 是法向量、b 是偏移。資料與標籤固定時，間隔限制可以凸；若表示與分類器全部參數共同學習，則可能成為另一個非凸問題。圖形識別單元說明其統計脈絡。

2. 中心、橢球與配置

「中心」有不同意思。最小化到觀測點最大距離的中心，不同於可行區域內最大球的中心；不等式表示的解析中心又是另一個物件。稱一個點為中心前，先說清楚幾何。

對 $a_i^\top x \le b_i$ 描述的多面體，最大內接歐氏球的中心 c 與半徑 r 由下式求得：

$$\begin{aligned} \text{maximize}_{c,r} \quad & r \\ \text{subject to} \quad & a_i^\top c + r\|a_i\|_2 \le b_i \quad \text{for every } i, \\ & r \ge 0. \end{aligned}$$

其中 $a_i \in \mathbb{R}^n$ 與 $b_i \in \mathbb{R}$ 是固定資料，$c \in \mathbb{R}^n$ 與 $r \in \mathbb{R}$ 是變數。長度至多 r 的位移在法向量方向上的最壞貢獻為 $r\|a_i\|_2$。法向量固定，因此每條限制對 (c, r) 都仿射。

此中心以歐氏單位平衡與邊界的距離。把不等式乘正數會同時縮放法向量範數與右側，不改變幾何條件。零法向量須另行檢查，其限制不是永遠成立就是不可能；不可直接除以其範數。

橢球允許各方向不同半徑。寫成 $E = \{c + Bu : \|u\|_2 \le 1\}$，其中 B 是對稱正定形狀矩陣，c 是中心，u 是單位球的輔助座標。包含於半空間的條件變成 $a_i^\top c + \|B^\top a_i\|_2 \le b_i$；最大化 $\log\det B$ 對應最大體積，差一個加法常數 [1]。$B \succ 0$ 定義域不可省略。

置中與包圍的限制不同。內接橢球須位於區域內，包圍橢球則須包含指定點；便利的凸參數化也不同。形狀矩陣看似合理，若不小心引入反矩陣與變數乘積，仍可能得到非凸模型。每項變換都要追溯定義域與凸性規則。

配置問題常最小化可移動點與固定錨點的距離總和。連接與限制固定時，歐氏範數給凸目標；選擇哪些連接存在則引入離散變數。矩形的相對次序先選定後，平面配置子問題可能凸；未知不重疊次序包含析取。原實體問題與凸子問題須分開標示。

中心算例：矩形不一定只有一個答案

最大內切球的中心未必唯一。取矩形 $0 \le x_1 \le 6$、$0 \le x_2 \le 2$。球的限制化為 $r \le c_1 \le 6 - r$ 與 $r \le c_2 \le 2 - r$，並要求 $r \ge 0$。

其中 $c = (c_1, c_2)$ 是中心、r 是半徑。短邊要求 $r \le 1$。最大半徑為一時，所有滿足 $1 \le c_1 \le 5$ 的中心 $(c_1, 1)$ 都最佳。問題凸，最佳解卻不唯一。對稱中心 $(3, 1)$ 只是有效答案之一，不是模型唯一指定的答案。

對內切橢球，對稱正定 B 把單位球映成所需形狀。其體積等於單位球體積乘以 $\det B$，因為線性映射以行列式縮放體積。因此最大化 $\log\det B$ 保留體積最大解。半空間包含規則來自在 $\|u\|_2 \le 1$ 上最大化 $a_i^\top (c + Bu)$，得到 $a_i^\top c + \|B^\top a_i\|_2$。這連起 OPT 03 的支撐函數計算與 OPT 02 的矩陣曲率 [1，第 8 章]。

凸形式是在包含限制下最小化 $-\log\det B$。半空間法向量固定，因此範數項對 B 凸。讓 B 奇異會離開有限對數行列式定義域，並使橢球體積塌縮。

3. 鬆弛需要明確的模型解讀

鬆弛（relaxation）擴大可行集合，讓困難問題較容易處理。它能提供有用界限，或產生稍後需取整的候選點。只有應用允許混合、共享或時間分配時，分數解才可執行。

假設二元配置變數 z_i 必須在 $\{0, 1\}$。改成 $0 \le z_i \le 1$，且其他部分凸時，是離散要求的凸鬆弛。最小化問題擴大可行集合只能降低最適值，因此鬆弛值是原最適值的下界，但不表示鬆弛點是有效的離散決策。

取整後要再次評估原目標與限制。逐座標取整可能違反總預算、連接要求或等式。修復後的可行解提供上界，鬆弛最適值提供下界；兩者差距只有在對應同一原目標，且下方數值是精確鬆弛最適值、有憑證的對偶界或經誤差修正的界時，才有意義。近似的鬆弛原始目標值通常是鬆弛最適值的上界，本身不一定是原離散最適值的下界。

視覺與學習的凸替代模型也一樣。範數懲罰可以鼓勵稀疏，不代表精確解出一般的非零項數最小化問題。凸資料項可能刻意近似物理量測過程。說明模型是統計估計器、計算鬆弛或精確改寫，三種目的支持不同主張。

參數選擇也不在數值憑證內。把正則化問題解得很準，不證明其正則化強度能預測新資料。以正確驗證規則選強度，再於保留資料測試一次。數值最佳化誤差、模型選擇誤差與泛化誤差分開處理。

4. 近端運算處理非平滑結構

非平滑項不必用陡峭平滑函數近似。近端運算（proximal operation）在該項與留在候選點附近之間取捨。許多懲罰與限制有簡單閉式解，適合一階方法 [2]。

對適當、閉、凸函數 g，步幅尺度 $\eta > 0$，定義：

$$\operatorname{prox}_{\eta g}(v) = \arg\min_{z} \left[g(z) + \frac{1}{2\eta}\|z - v\|_2^2 \right].$$

其中 $v \in \mathbb{R}^n$ 是候選點，$z \in \mathbb{R}^n$ 是子問題變數，g 可取正無窮以表示限制，η 控制二次接近項的尺度。在所述條件下，子問題有唯一最小解。

閉凸集合的指示函數之近端運算就是投影。對 $g(z) = \lambda\|z\|_1$、$\lambda \geq 0$，則是逐座標軟閾值：$\operatorname{sign}(v_i)\max(|v_i| - \eta\lambda, 0)$，其中 i 是座標索引。它能產生精確零；絕對值懲罰在零點不可微，不能直接用普通梯度下降。

最小化 $f(x) + g(x)$、且 f 可微凸時，近端梯度更新為：

$$x^{k+1} = \operatorname{prox}_{\eta g}\left(x^k - \eta\nabla f(x^k)\right).$$

其中 k 是迭代索引，∇f 是平滑梯度，η 依方法假設選擇。梯度若為 L-Lipschitz，保守固定選擇是 $0 < \eta \leq 1/L$。L 限制梯度變化，不是目標值大小。

受限最小平方法變成先梯度再投影；稀疏最小平方法變成先梯度再軟閾值。較複雜的組合，如非平滑函數作用在線性差分算子上，直接近端映射可能昂貴。分裂法引入輔助變數，把便宜運算分開。

有用的殘差是縮放更新 $G_\eta(x) = [x - \operatorname{prox}_{\eta g}(x - \eta\nabla f(x))]/\eta$，其中 G_η 稱近端梯度映射。映射為零時滿足凸組合問題的一階最適條件。其大小是收斂診斷，仍須搭配尺度、可行性及能取得時的目標界限解讀。

在轉角推導軟閾值

近端映射精確處理非平滑轉角。對單一座標，在 $\eta > 0$、$\lambda \geq 0$ 下最小化 $\lambda|z| + (z - v)^2/(2\eta)$。最適條件為

$$0 \in \lambda\partial|z| + \frac{z - v}{\eta}.$$

其中 v 是候選座標、z 是輸出、$\partial|z|$ 是絕對值的次微分集合。當 $z > 0$，次梯度為一，所以 $z = v - \eta\lambda$，只在 $v > \eta\lambda$ 時有效。當 $z < 0$，次梯度為負一，得到 $z = v + \eta\lambda$，只在 $v < -\eta\lambda$ 時有效。在零點，次微分為 $[-1, 1]$，因此零恰好在 $|v| \leq \eta\lambda$ 時最佳 [2]。

三種情況合起來就是閾值公式。對合成向量 $v = (2, -0.3, 0.1)$ 與閾值 $\eta\lambda = 0.4$，輸出為 $(1.6, 0, 0)$。非零輸入能變成精確零，因為零點具有整個區間的支撐斜率。

近端梯度更新來自最小化平滑項的局部上界模型：把 $f(z)$ 換成 $f(x) + \nabla f(x)^\top(z - x) + \|z - x\|_2^2/(2\eta)$，保留精確的 $g(z)$。配方後，模型中心變成 $v = x - \eta\nabla f(x)$。因此平滑成本走梯度步，非平滑成本則做精確小型求解。當 $1/\eta \geq L$，平滑性不等式使模型成為上界，解釋了保守步長條件。

5. 把訊號模型拆成簡單子問題

分段常數訊號可以在符合觀測與懲罰相鄰差異之間取捨估計。二次資料項避免修正太大，絕對差懲罰鼓勵許多鄰近值相同。這個凸估計器適合控制去雜訊實驗，其行為與均勻平滑不同。

令 $y \in \mathbb{R}^n$ 是含雜訊觀測，$D \in \mathbb{R}^{(n-1)\times n}$ 計算相鄰差分，求解：

$$\operatorname{minimize}_x \quad \tfrac{1}{2}\|x - y\|_2^2 + \lambda\|Dx\|_1.$$

其中 $x \in \mathbb{R}^n$ 是估計訊號，$\lambda \geq 0$ 是正則化強度。這是近端與分裂文獻的一維總變異正則化 [2–4]，描述局部變化，不宣稱所有真實訊號都分段常數。

引入 $z = Dx$，使用交替方向乘子法（alternating direction method of multipliers，ADMM）[3]。縮放形式為：

$$\begin{aligned} x^{k+1} &= (I + \rho D^{\top} D)^{-1} \left[y + \rho D^{\top} (z^k - u^k) \right], \\ z^{k+1} &= \operatorname{shrink}(D x^{k+1} + u^k, \lambda/\rho), \\ u^{k+1} &= u^k + D x^{k+1} - z^{k+1}. \end{aligned}$$

其中 $\rho > 0$ 是懲罰參數，I 是單位矩陣，u 是縮放對偶變數，shrink 是上述逐座標軟閾值。反矩陣記號只描述數學解，實作使用分解與線性求解。分裂等式在收斂時成立，不必每個中間迭代點都成立。

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

rng = np.random.default_rng(8)
n = 48
clean = np.r_[np.zeros(16), np.ones(16), np.full(16, 0.3)]
y = clean + rng.normal(0, 0.16, n)
D = np.diff(np.eye(n), axis=0)
lam, rho = 0.4, 4.0
factor = cho_factor(np.eye(n) + rho*D.T @ D)
x, z, u = y.copy(), np.zeros(n-1), np.zeros(n-1)
for k in range(2000):
    x = cho_solve(factor, y + rho*D.T @ (z-u))
    z_old = z.copy()
    v = D @ x + u
    z = np.sign(v) * np.maximum(np.abs(v)-lam/rho, 0)
    u += D @ x-z
    primal = np.linalg.norm(D @ x-z)
    dual = rho*np.linalg.norm(D.T @ (z-z_old))
    if max(primal, dual) < 1e-7:
        break
objective = lambda a: 0.5*np.sum((a-y)**2) + lam*np.abs(D @ a).sum()
print("Iterations and residuals:", k+1, primal, dual)
print("Objectives, raw and estimated:", objective(y), objective(x))
print("Synthetic MSE, raw and estimated:", np.mean((y-clean)**2), np.mean((x-clean)**2))
assert max(primal, dual) < 1e-7
assert objective(x) <= objective(y) + 1e-8
assert np.mean((x-clean)**2) < np.mean((y-clean)**2)
```

最後一項斷言只查核這個合成例子，不是正則化一定改善重建的定理。太強的正則化可能壓平真邊緣或移除窄峰。目標在模型內選取折衷；乾淨訊號誤差則需要真值，實際部署不一定有。

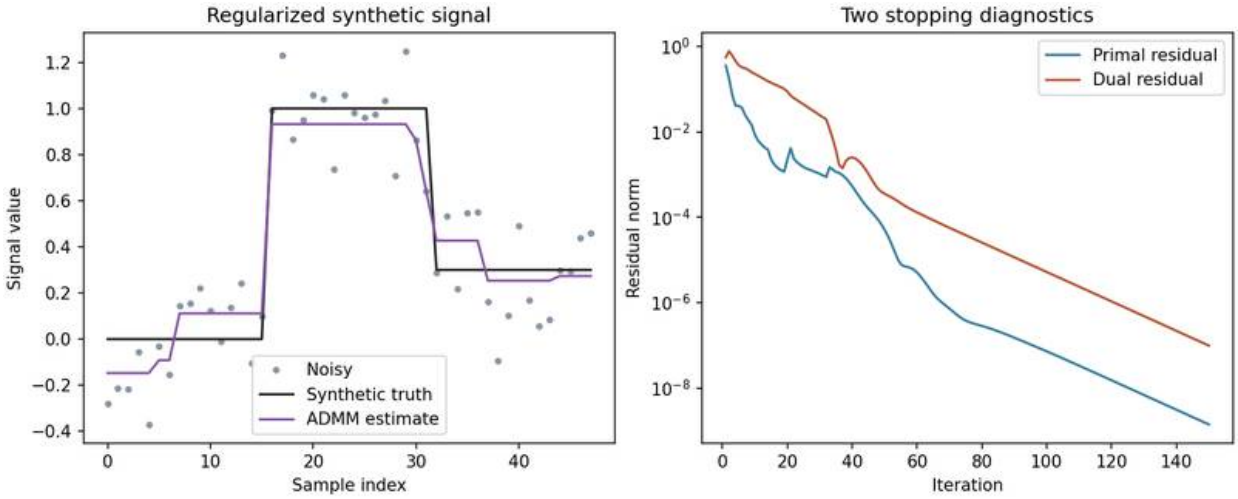


圖 1 — 分裂法分開二次求解與軟閾值。收斂時兩種殘差都趨近零，但不要求每一步單調下降；去雜訊改善屬於這個合成訊號與正則化選擇。

D 與 ρ 固定，所以分解能重用。改正則化強度只改閾值；改 ρ 會改線性系統，須重新分解。若改 ρ 且保留同一未縮放乘子，u 也須一致重縮放，否則更新偷偷改了對偶狀態。

影像中，相鄰差分改成水平與垂直算子，懲罰也有不同合併方式。邊界條件與像素尺度影響估計器。本章一維結果不能當影像復原基準。量測模型請看影像處理，學習式復原比較請看視覺深度學習。

推導拆分更新與殘差

ADMM 方程來自保留明確的差分變數，而不是直接計算困難的組合近端映射。以 $z = Dx$ 建立縮放增廣目標：

$$\frac{1}{2}\|x - y\|_2^2 + \lambda\|z\|_1 + \frac{\rho}{2}\|Dx - z + u\|_2^2 - \frac{\rho}{2}\|u\|_2^2.$$

其中 x 是訊號、y 是觀測、D 是差分矩陣、z 是拆分差分、u 是縮放乘子、$\rho > 0$ 是懲罰參數。最後一項在原問題子步驟中固定。對 x 微分得到 $(I + \rho D^\top D)x = y + \rho D^\top(z - u)$。對 z 最小化就是以 $Dx + u$ 為中心、閾值為 λ/ρ 的軟閾值問題。乘子更新接著累積拆分等式誤差 [3]。

為什麼監測兩個殘差？原問題殘差 $r_p = Dx - z$ 檢查耦合；對偶殘差 $r_d = \rho D^\top(z^{k+1} - z^k)$ 檢查循序更新在駐點條件中留下的不一致。其中 k 是迭代索引；改變正負號慣例不影響殘差範數。

資料項讓這個特定目標對 x 強凸，因此即使相鄰差分在非平滑轉角為零，去雜訊訊號仍唯一。殘差說明拆分演算法是否到達該解，卻不量測與未知乾淨訊號的吻合程度。合成實驗因為知道生成訊號，才能同時檢查兩者。

6. 條件梯度使用線性 oracle

投影可能昂貴，但在集合上最小化線性函數可能簡單。條件梯度法以線性最佳化 oracle 和凸組合更新取代投影；可行初始化後，迭代點保持可行 [5]。

對緊緻凸集合 C 上的可微凸函數 f，求解：

$$v^k \in \arg\min_{v \in C} \nabla f(x^k)^\top v, \qquad x^{k+1} = (1-\gamma_k)x^k + \gamma_k v^k.$$

其中 x^k 是目前可行點，v^k 是線性 oracle 解，$\gamma_k \in [0,1]$ 是步幅權重。新點依凸性可行。Frank–Wolfe 演算法 [5] 就是搭配適當步幅與收斂假設的基本條件梯度構造。

有用的憑證是：

$$g_{\mathrm{FW}}(x) = \nabla f(x)^\top (x - v(x)).$$

其中 $v(x)$ 是在 x 的線性 oracle 解。oracle 精確且目標可微凸時，支撐超平面不等式給 $f(x) - f(x^\ast) \le g_{\mathrm{FW}}(x)$，其中 $x^\ast$ 是 C 內最小解。間隙因此限制目前可行點的目標誤差。近似 oracle 需要額外誤差容許量。

在機率單純形 $C = \{x \ge 0 : \sum_i x_i = 1\}$，線性 oracle 選梯度最小的座標，回傳單純形頂點。混合模型若能便宜選出新成分，此法就有用。每次加入頂點可能形成稀疏迭代表示，但表示與精度折衷仍須檢查。

```
import numpy as np

target = np.array([-0.1, 0.6, 0.5])
def project_simplex(v):
    sorted_v = np.sort(v)[::-1]
    threshold = (np.cumsum(sorted_v)-1)/np.arange(1, len(v)+1)
    active = np.flatnonzero(sorted_v > threshold)[-1]
    return np.maximum(v-threshold[active], 0)
optimum = project_simplex(target)
x = np.full(3, 1/3)
for k in range(1000):
    grad = x-target
    vertex = np.eye(3)[int(np.argmin(grad))]
    gap = float(grad @ (x-vertex))
    error = 0.5*np.sum((x-target)**2)-0.5*np.sum((optimum-target)**2)
    assert error <= gap + 1e-12
    if gap < 1e-4:
        break
    gamma = 2/(k+2)
    x = (1-gamma)*x + gamma*vertex
# Recompute certificate at the returned iterate.
grad = x-target
vertex = np.eye(3)[int(np.argmin(grad))]
gap = float(grad @ (x-vertex))
print("Simplex iterate:", x)
print("Frank–Wolfe gap:", gap)
assert np.all(x >= 0) and np.isclose(x.sum(), 1)
assert gap < 0.01
```

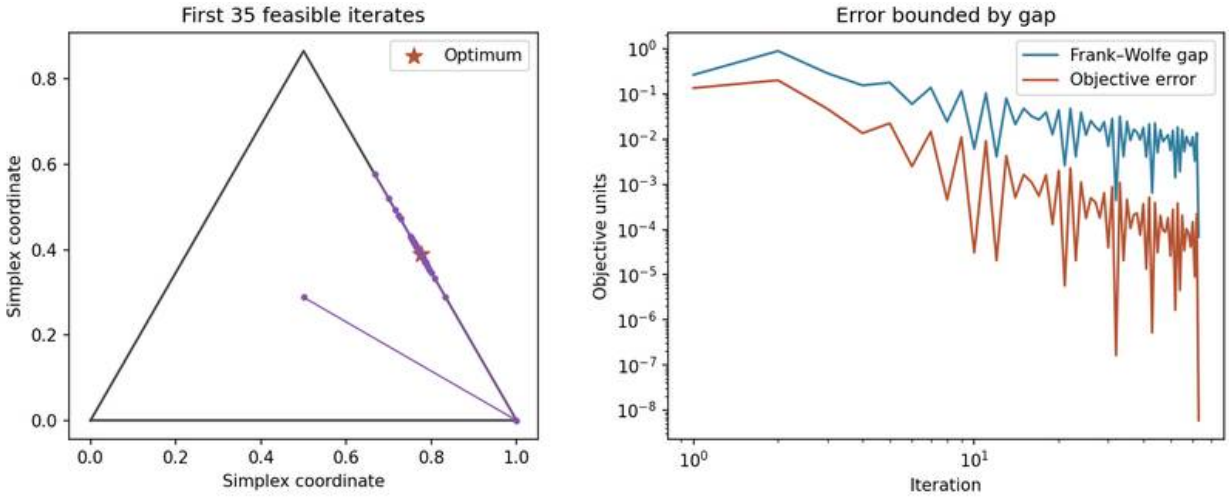


圖 2 — 線性 *oracle* 提供可行頂點與間隙憑證。目標誤差下降時，路徑仍可能來回鋸齒。

範例的參考投影只用於計算精確二次最小解，方便比較；條件梯度本身從不呼叫投影。如此可測間隙界限，而不只看曲線下降便推斷收斂。最後更新後重新計算間隙，讓它描述真正回傳的點。

緊緻可行集合與平滑目標支持通常的收斂分析。無界 oracle 或非平滑目標改變問題。變體能處理額外結構，但不可未檢查假設便沿用基本保證。綜述 [5] 說明選擇與幾何動機。

證明 oracle 間隙，並包含非精確 oracle

條件梯度間隙是最佳線性化可行方向允許的改善。凸性給出 $f(x^\ast) \ge f(x) + \nabla f(x)^\top (x^\ast - x)$。因為精確 oracle 在 C 上最小化該線性分數，

$$f(x) - f(x^\ast) \le \nabla f(x)^\top (x - x^\ast) \le \nabla f(x)^\top (x - v) = g_{\mathrm{FW}}(x).$$

其中 x 是目前可行點、$x^\ast$ 是最佳解、v 是精確線性 oracle 的解。第一個不等式使用目標凸性，第二個使用 oracle 與 $x^\ast$ 的比較。這兩個要素把可計算方向量變成全域誤差上界 [5]。

若可行的近似 oracle 回傳 $\tilde{v}$，滿足 $\nabla f(x)^\top \tilde{v} \le \min_{z \in C} \nabla f(x)^\top z + \delta$，其中已知 $\delta \ge 0$，有效上界就是計算出的間隙加上 δ。其中 δ 是 oracle 的線性目標誤差容許量。因為 $\tilde{v} \in C$，即使間隙須修正，更新仍保持可行。

對單純形梯度 $(2, -1, 3)$，精確 oracle 選第二頂點，分數為 -1。選第一頂點的分數為 2，需要容許量 $\delta = 3$。在 $x = (1/3, 1/3, 1/3)$，精確間隙為 $7/3$，近似 oracle 間隙卻為 $-2/3$。加上三後恢復 $7/3$。此處未修正負間隙代表 oracle 不佳，不代表目標誤差低於零。

7. 依結構、精度與資訊選擇方法

沒有全面最好的更新方程。建模決策決定可取得的資訊及便宜運算。比較名稱或迭代次數前，先比較真正運算成本。

中型平滑問題若能利用曲率，牛頓或內點法可透過較少昂貴步驟取得精確解。巨大稀疏模型可能受分解記憶體限制，一階法便宜的矩陣向量運算更合適。簡單非平滑懲罰可用便宜精確的近端運算；複雜集合若線性 oracle 簡單，條件梯度可避免昂貴投影。

分散式最佳化再加入通訊成本。局部目標可分離，共識限制卻連接變數。ADMM 能暴露局部子問題，但收斂時間須包括通訊輪次與局部求解。數學上分散式不代表實作平行；本章去雜訊範例完全循序執行。

原始／對偶混合梯度法（primal–dual hybrid gradient）以原始與對偶更新處理鞍點表示 [4]，步幅依算子範數與分裂項結構決定。PDLP [6] 等實際線性規劃系統在此結構外加入前處理與自適應技巧。短小理論遞迴與正式求解器是不同描述層次。

要求精度可能反轉比較。快速產生有用近似解的方法，可能要很久才達到極小憑證間隙；另一法前置成本大，但高精度收斂有效。畫出達到幾種指定誤差所需時間，並與間隙一起回報原單位可行性，讓折衷可查核。

記憶體與重複求解也重要。參數掃描在方法支援時能重用資料結構、分解或前一解；稀疏樣式改變可能阻止重用。首次量測納入前處理與設定，暖啟動重複求解另行回報。不要偷偷拿一法暖執行與另一法冷執行比較。

共識：局部求解與共享平均

分散式 ADMM 明確呈現通訊機制。假設 J 個參與者共享向量決策，但各有局部凸成本 f_j。引入局部副本 x_j 與共享變數 z，在所有 $x_j = z$ 下求解 $\min \sum_{j=1}^{J} f_j(x_j)$。

縮放形式的一次共識迭代為

$$x_j^{k+1} = \arg\min_x \left[f_j(x) + \frac{\rho}{2} \| x - z^k + u_j^k \|_2^2 \right],$$

$$z^{k+1} = \frac{1}{J} \sum_{j=1}^{J} (x_j^{k+1} + u_j^k),$$

$$u_j^{k+1} = u_j^k + x_j^{k+1} - z^{k+1}.$$

其中 u_j 是參與者 j 的縮放乘子、$\rho > 0$ 是懲罰參數、k 是迭代索引。第一列為獨立局部求解；第二列對 z 最小化二次共識項之和，其導數使 z 等於平均；第三列累積不一致 [3]。

對純量成本 $f_j(x) = \frac{1}{2}(x - a_j)^2$，局部解為 $x_j^{k+1} = [a_j + \rho(z^k - u_j^k)]/(1 + \rho)$，其中 a_j 是局部目標。兩參與者的目標若為零與四，必須在全域最佳值二達成共識。各自最適的零與四違反共識；平均與乘子更新協調折衷。

因此通訊屬於演算法本身：共享更新需要各個局部量，局部求解則需要最新共享值。平行實作除了局部算術，也須量測訊息成本與同步。若增加共享限制，z 子問題也會改變，常從簡單平均變成受限近端運算。

8. 完整凸最佳化專題

專題須讓模型與證據可重現。選合成訊號估計、幾何配置或資源模型，求解前先寫變數、單位、凸性論證、定義域與可行集合解讀。說明鬆弛及如何把結果轉成可執行決策。

建立一個有解析答案或獨立憑證的小實例，用來檢查符號、形狀、尺度與停止規則。接著一次改一項：雜訊、正則化、條件數、稀疏性、大小或容許誤差。同一實例比較演算法時固定資料種子。

在真正回傳點記錄目標與殘差。分裂法追蹤耦合與對偶殘差；條件梯度報目前間隙與 oracle 精度；障礙法區分外層目標與計算憑證；鬆弛則分開下界與可行取整上界。

數值收斂之外也評估應用。去雜訊器可能準確最小化一個會移除真事件的懲罰；配置求解器可能遵守全部已建模規則，卻漏了門口需求；資源最佳化可能對不確定需求敏感。這些是模型驗證發現，不應模糊數學憑證的意義。

學習流程在選正則化或學前處理前先切分資料。驗證集重複求解、選規則，再回報保留資料結果。ML/DL 評估章說明更廣的學習規則。數值收斂與預測評估回答不同問題，結果表應分不同欄。

現代觀點

閱讀地圖把整理演算法的綜述與改變實務建模／求解的新工作結合。以下來源查核至 2026 年 10 月 4 日，未出版手稿明確標示。

Parikh 與 Boyd [2] 以小凸子問題整理近端法；**Boyd 等人 [3]** 以可分離性、耦合與分散式統計應用整理 ADMM。本章綜合重點是：分裂後哪些子問題變簡單，新增哪些殘差需要控制。分裂能減少總工作才有價值，不是因目標有數項就值得做。

Condat 等人 [4] 透過原始／對偶乘積空間與單調包含統一近端分裂，重訪參數範圍與變體。不同遞迴可依算子與度量連接。用它查明真實假設，尤其平滑二次項可能允許與一般平滑函數不同的選擇。統一觀點避免把任意參數組合當成有保證的方法。

Pokutta 的 2024 年導論 [5] 以線性 oracle 與可行組合整理條件梯度，幫助比較投影法與 oracle 法，也解釋可行集合幾何如何影響行為。先估 oracle 與表示成本，再使用有效間隙。便宜 oracle 仍可能需很多迭代才達應用精度。

Shorinwa 等人的 2024 年多機器人綜述 [7] 區分分散式一階、循序凸規劃與 ADMM 方法，把通訊、協調及實體限制放在數學更新旁。部分機器人模型非凸，或透過一連串凸子問題求解；凸子問題收斂不證明原軌跡或協調問題全域最適。

Applegate 等人的 PDLP 手稿 [6] 於 2025 年發布、2026 年修訂，提供大型一階線性規劃的新實務證據。**Schaller、Bemporad 與 Boyd 的 2025 年手稿 [8]** 則配適相容於規範式程式設計的參數化凸函數。前者關於求解流程，後者關於對決策變數保持凸性的學習函數；兩者都是手稿，也都不會使任意神經網路訓練凸化。

問題	有用家族或閱讀	應蒐集的證據
限制投影便宜嗎？	投影／近端法 [2]	投影成本與最適性殘差
各項能拆成簡單運算嗎？	ADMM、原始／對偶分裂 [3,4]	局部求解成本與耦合殘差

問題	有用家族或閱讀	應蒐集的證據
線性最小化比投影容易嗎？	條件梯度 [5]	oracle 成本、可行點與間隙
通訊是否主導成本？	分散式分類 [7]	輪次、訊息、局部工作與實體可行性
線性模型太大而無法分解嗎？	實務一階線性規劃 [6]	記憶體、準確度與原單位殘差
學習函數能保留決策變數凸性嗎？	參數化凸建模 [8]	結構限制與保留資料上的模型適切性

綜合而言，建模與計算越來越透過可重用結構元件交會：凸子問題、便宜近端映射、學習式凸函數及配合硬體的求解運算。價值仍依明確模型、有效假設與端到端量測。求解器基準不證明泛化，預測基準也不證明可行性。

未解問題包含自動結構辨識、不良尺度下的穩健憑證、通訊效率、活躍限制改變時的可微性，以及學習預測與精確限制的可靠整合。有用研究問題隔離一項議題並保留獨立可查核結果，例如：「同一憑證誤差下，暖啟動是否降低總求解時間？」

本單元已提供完整路徑：凸集合與函數、問題建立、對偶性、估計、數值方法、受限求解器與結構式應用。接著可選一組教科書練習，圍繞假設建立小型原創實驗。核心習慣是寫清楚最佳化什麼、為何凸、演算法計算什麼，以及接受結果的證據。

重點整理

- 幾何建模包含投影、分離、中心、橢球與結構式配置。
- 未知離散選擇與析取可能使原應用非凸。
- 鬆弛下界與可行可執行解扮演不同角色。
- 近端運算直接處理簡單非平滑懲罰與限制。
- ADMM 把耦合拆成易處理運算，須查核原始及對偶殘差。
- 條件梯度透過線性 oracle 保持可行性，並在所述假設下提供間隙。
- 比較原單位準確度、設定、記憶體、重複求解與應用結果。

練習

1. **幾何中心**。推導軸對齊矩形的最大內接球，說明一邊較長時中心為何可能不唯一。

   提示

   較短邊限制半徑；中心可沿長邊移動而仍保留足夠邊界距離。

2. **平面配置**。先選定兩矩形哪個在左，寫凸不重疊限制，解釋次序未知時有何改變。

   提示

   固定次序給仿射分離不等式；未知次序須二選一，是析取而非單一凸半空間。

3. **正則化掃描**。以五種強度執行去雜訊，比較目標、合成重建誤差與近乎零的相鄰差分數。

提示

不同強度的目標是不同函數，不能用原始值公平比較收斂。分開模型品質與求解精度。

4. **懲罰更新**。在實驗中途改 ADMM 懲罰，保持分解與縮放對偶變數一致，再與固定懲罰比較殘差。

提示

未縮放乘子是懲罰乘縮放對偶變數。改懲罰時保持此乘積。

5. **非精確 oracle**。故意使用次最適單純形頂點，找出 oracle 目標誤差如何改憑證界限，以及迭代是否仍可行。

提示

頂點的凸組合保持可行；非精確 oracle 的間隙須加上其線性目標誤差上界。

6. **專題報告**。在一個合成凸應用比較兩種合適演算法，納入模型、凸性證明、停止政策、原單位殘差、時間、記憶體及一個模型失敗。

提示

資料與驗收誤差相同；設定成本與重複求解分開。未解決的數值執行須如實回報，不能算成功求解。

參考文獻

1. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, Chapter 8, with Chapters 4 and 6 for modeling and approximation. Book
2. N. Parikh and S. Boyd, “Proximal Algorithms,” Foundations and Trends in Optimization, 2014. Survey
3. S. Boyd, N. Parikh, E. Chu, B. Peleato and J. Eckstein, “Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers,” Foundations and Trends in Machine Learning, 2011. Review
4. L. Condat, D. Kitahara, A. Contreras and A. Hirabayashi, “Proximal Splitting Algorithms for Convex Optimization: A Tour of Recent Advances, with New Twists,” SIAM Review, 2023. Author manuscript
5. S. Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” Jahresbericht der Deutschen Mathematiker-Vereinigung, 2024. Survey
6. D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O’Donoghue and W. Schudy, “PDLP: A Practical First-Order Method for Large-Scale Linear Programming,” arXiv preprint, 2025, revised 2026. Paper
7. O. Shorinwa, T. Halsted, J. Yu and M. Schwager, “Distributed Optimization Methods for Multi-Robot Systems: Part 2—A Survey,” IEEE Robotics & Automation Magazine, 2024. Survey
8. M. Schaller, A. Bemporad and S. Boyd, “Learning Parametric Convex Functions,” manuscript, June 2025. Author page

篇 II

資訊理論

IT 01・熵、相對熵與互資訊

從機率建立資訊量，推導鏈式法則與資料處理不等式，並區分精確恆等式和從資料估計資訊量的不確定性。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/entropy-mutual-information/

你將學到

- 為什麼對數能把機率轉成可相加的驚訝程度。
- 計算熵、條件熵、相對熵與互資訊。
- 推導鏈式法則，辨認基本不等式何時取等號。
- 解釋處理能丟棄、保留或增加什麼，不把隨機性混同於資訊。
- 用有限機率表與完整 CPU 範例檢查資訊恆等式。
- 區分母體資訊量、經驗估計與學習目標。

先看全貌

感測器傳送一串量測，有些值可預測，有些出乎意料。壓縮器利用可預測性；接收者則需要能減少目標不確定性的量測。這兩種價值相關，但不相同。一串非常隨機的訊號，也可能幾乎沒有關於場景的有用證據。

> **用最簡單的話說**
>
> 想像你在猜盒子裡有什麼。如果每個盒子都裝玩具，「裡面是玩具」幫助不大；「裡面是唯一那台紅色火車」卻可能解決問題。資訊理論依可能結果的機率，量測訊息減少了多少不確定性。

本主線以 Thomas M. Cover 與 Joy A. Thomas 的 *Elements of Information Theory* 第 2 版為核心 [1]。這些資訊量有實際意義：熵連到壓縮，互資訊連到通訊限制，相對熵則把機率失配連到額外描述長度與統計辨別。我們先從有限字母集合開始，讓每個量都能逐項相加檢查。

Stanford 熵講義 [2] 提供易讀的配套說明。本章使用明確合成機率模型，讓定義與數值檢查可以直接逐項確認。

主線如何銜接

教學	核心問題	第 2 版主要章節
IT 01	資訊到底量測什麼？	1–2，以及 17 的基本工具
IT 02	為什麼長序列的整體行為較可預測？	3–4
IT 03	如何縮短描述而不丟失符號？	5、13–14

教學	核心問題	第 2 版主要章節
IT 04	有雜訊的通道能可靠傳多少資訊？	7
IT 05	連續與高斯變數有何不同？	8–9
IT 06	容許失真時需要多少位元率？	10
IT 07	資訊量如何支援推論與決策？	6、11–12、16
IT 08	多使用者、限制與學習帶來什麼改變？	15，以及 17 的進階工具

這種分組涵蓋全書主要主題，但不表示每個定理都能用短推導完整處理。先理解定義，再做數值檢查，最後閱讀證明。計算不清楚時，先明列機率表，再加入神經估計器或通訊系統。

隨機變數、結果與機率模型

隨機變數描述觀測前可能發生什麼。機率來自對實驗的模型。同一個觀測值，在某個模型下罕見，在另一模型下可能很普通。

令 X 的值來自有限字母集合 $\mathcal{X}$：

$$p_X(x) = \Pr(X = x), \qquad p_X(x) \geq 0, \qquad \sum_{x \in \mathcal{X}} p_X(x) = 1.$$

X 是隨機變數，x 是一個可能結果，$\mathcal{X}$ 是結果集合，p_X 是機率質量函數，Pr 表示機率。聯合分布 $p_{XY}(x,y)$ 為成對結果指定機率，邊際分布由加總取得：

$$p_X(x) = \sum_y p_{XY}(x,y), \qquad p_Y(y) = \sum_x p_{XY}(x,y).$$

Y 是另一有限隨機變數，y 是其結果，p_Y 是邊際機率質量函數。若 $p_Y(y) > 0$，條件機率為 $p_{X|Y}(x \mid y) = p_{XY}(x,y)/p_Y(y)$。零機率事件需要另外處理；它不會貢獻以下有限加總。

未特別說明時，對數以二為底，單位是 **bit（位元）**。自然對數對應 **nat**；把 nat 數值除以 $\ln 2$ 可換成 bit。bit 是資訊單位，不一定等於某種檔案格式實際儲存的一個二進位數字。

比較分布時，先固定實驗。一個模型描述灰階值，另一個描述整張影像，兩者熵不能直接比較。改變變數、字母集合、量化或已知條件，都是改變問題。

對數讓驚訝程度可以相加

越罕見的結果，驚訝程度越高。兩個獨立觀測應帶來各自驚訝程度的總和；對數讓加法與機率乘法一致。

定義某結果的自資訊（self-information）：

$$\imath_X(x) = -\log_2 p_X(x).$$

$\imath_X(x)$ 是模型 p_X 下，結果 x 的驚訝程度。獨立結果的機率若為 a 和 b，聯合機率是 ab，因此 $-\log_2(ab) = -\log_2 a - \log_2 b$。等可能結果數加倍，資訊量增加一 bit，而非變成兩倍。

機率一的結果沒有驚訝；機率 1/8 的結果有三 bit。這些描述的是觀測前的機率模型；罕見符號不一定具有較高的實際任務價值。

若模型對實際出現的結果給零機率，自資訊就是無限大。預測應用中，這可能表示支撐集合錯誤：模型宣稱實際可能發生的事件不可能。加入很小的數值常數會改變模型；它可能有用，但不是原分布的計算。

平均驚訝程度就是熵（entropy）：

$$H(X) = -\sum_{x} p_X(x) \log_2 p_X(x) = \mathbb{E}[\iota_X(X)].$$

$H(X)$ 是 X 的熵，$\mathbb{E}$ 依其分布取平均。約定 $0 \log 0 = 0$ 來自極限；從不出現的結果不貢獻平均。

因此熵是分布的性質，不是某個已實現樣本的性質。平均熵很小的分布，也可能偶爾產生高驚訝結果。影像具有顯眼圖案，也仍可能被合適的機率模型輕易預測。

熵的界限與二元例子

結果確定時，不確定性為零。可能結果數固定時，所有結果等機率會使不確定性最大。

若字母集合有 K 個值：

$$0 \le H(X) \le \log_2 K.$$

$K = |\mathcal{X}|$ 是集合大小。集中在單一結果的分布達到下界；均勻分布達到上界。若只有 K_+ 個結果具有正機率，更緊的上界是 $\log_2 K_+$。

二元變數的結果一有機率 p 時：

$$h_2(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

h_2 是二元熵函數，$p \in [0,1]$。曲線關於 $p = 1/2$ 對稱，最大值是一 bit，接近兩端時趨近零。偏斜來源比公平來源容易預測，即使兩者都只有兩種符號。

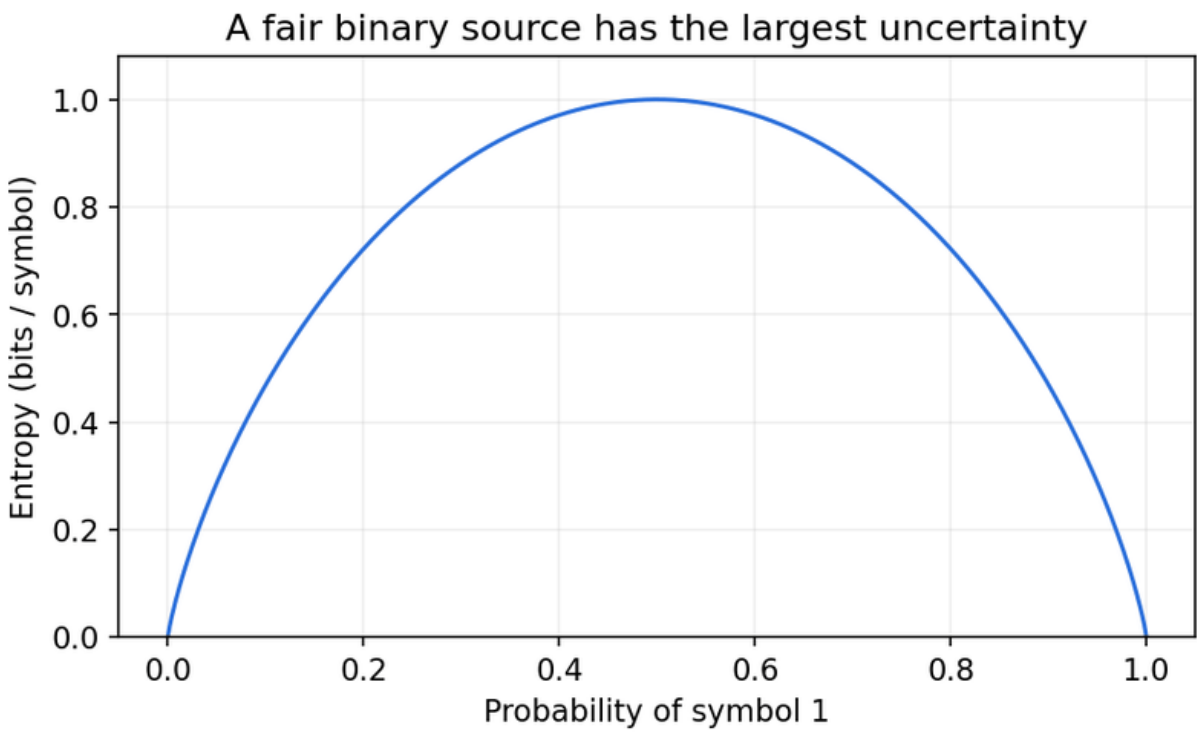


圖 1 — 二元熵的解析曲線。可能符號數不變，只有機率改變。

熵對分布是凹函數。混合兩個來源分布、卻隱藏選了哪個來源，熵不會低於個別熵的加權平均。揭露來源選擇可能消除部分增加的不確定性，因此混合來源常比個別來源更難預測。

最大熵界限也能用與均勻模型的相對熵證明。稍後同一非負性論證，還會說明為什麼條件降低平均熵，以及互資訊為什麼不能為負。

聯合熵與鏈式法則

一起描述兩個變數，可能比各自獨立描述便宜。知道第一個後，共享結構可能讓第二個更容易預測。

聯合熵為

$$H(X,Y) = -\sum_{x,y} p_{XY}(x,y)\log_2 p_{XY}(x,y).$$

$H(X,Y)$ 量測成對變數 (X,Y) 的不確定性。條件熵為

$$H(Y \mid X) = -\sum_{x,y} p_{XY}(x,y)\log_2 p_{Y|X}(y \mid x).$$

$H(Y \mid X)$ 是已知 X 後，對 Y 的平均剩餘不確定性；每個條件分布的熵，依對應 x 的機率加權。

利用 $p_{XY}(x,y) = p_X(x)p_{Y|X}(y \mid x)$，可得

$$H(X, Y) = H(X) + H(Y \mid X).$$

每一項必須來自同一聯合模型。乘積取對數產生兩項，第一項對 y 加總後留下 $p_X(x)$。這說明恆等式的來源，不只是要背的公式。

對序列 $X_1, \dots, X_n$：

$$H(X_1, \dots, X_n) = \sum_{i=1}^{n} H(X_i \mid X_1, \dots, X_{i-1}).$$

n 是變數數量，i 是序列索引；第一項沒有條件變數。此法則不需要獨立假設。若獨立，每個條件熵等於邊際熵，聯合熵就是總和。

影像中，直方圖只描述邊際像素頻率，丟掉空間相依性。兩張影像的直方圖相同，知道鄰近像素後的可預測性卻可能很不同。序列模型決定這些相依性是否能縮短描述。

條件平均有幫助，不代表每種結果都如此

知道另一變數，不會使最佳平均預測變差。但某個特定觀測，可能揭露你正處在特別不確定的情況。

$$H(X \mid Y) \le H(X), \qquad H(X, Y) \le H(X) + H(Y).$$

第一式比較平均條件不確定性與邊際不確定性；第二式由鏈式法則得到。X 和 Y 獨立時取等號。

「平均」很重要。假設大部分日子感測器固定輸出零，少數診斷日輸出公平二元結果。知道今天是診斷日，反而比邊際混合分布更不確定。但平均正常日與診斷日後，知道日子類型仍降低平均不確定性。

加入更多條件時：

$$H(X \mid Y, Z) \le H(X \mid Y).$$

Z 是額外資訊。這比較同一固定聯合分布的熵，並不是說有限神經網路多讀輸入就一定泛化更好。最佳化、資料不足與虛假相關，都可能使學得預測器無法達到最佳條件預測。

有限變數的條件熵非負。後續連續變數的微分熵有不同性質；不能不查假設，就把每個有限字母集合不等式套到連續熵。

用數值檢查條件化

讓診斷日例子更具體：$Y = 1$ 表示診斷日，機率 0.1。平常日 $X = 0$，診斷日的 X 是公平位元。因此 $X = 1$ 的邊際機率為 $0.1 \times 0.5 = 0.05$，得到 $H(X) = h_2(0.05) = 0.286397$ 位元。

知道今天是診斷日，會把不確定性提高至 $H(X \mid Y = 1) = 1$ 位元；但平均所有日子得到 $H(X \mid Y) = 0.9 \times 0 + 0.1 \times 1 = 0.1$ 位元。條件不等式比較的是 0.1 與 0.286397，不是每個個別條件熵與邊際熵。日子標籤平均移除 $I(X; Y) = 0.186397$ 位元。

相對熵量測機率失配

模型預期錯誤結果，可能浪費描述長度。相對熵以真正產生觀測的分布為方向，量測這種失配。

$$D(p\|q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}.$$

p 和 q 是同一字母集合上的機率質量函數，$D(p\|q)$ 是相對熵，也稱 Kullback–Leibler divergence [1]。$p(x) = 0$ 的項為零；若 $p(x) > 0$ 而 $q(x) = 0$，相對熵無限大。

非負性可由對數凹性得到：

$$\begin{aligned} -D(p\|q) &= \mathbb{E}_p\left[\log_2 \frac{q(X)}{p(X)}\right] \\ &\le \log_2 \sum_{x:p(x)>0} q(x) \le 0. \end{aligned}$$

$\mathbb{E}_p$ 是依 p 取期望值。第一個不等式是 Jensen 不等式，最後加總不超過一。兩分布一致時取等號。若模型在來源支撐集合外也分配機率，此證明仍成立。

方向重要：$D(p\|q)$ 一般不等於 $D(q\|p)$。它不是度量距離，因為不符合對稱及一般三角不等式等要求。口語稱為「距離」，不能據此套用需要真正距離的演算法。

例如公平來源與一個對結果一給 0.9 機率的模型。模型往某方向過度自信，會為常出現卻未預期的零付代價。反轉分布，描述的是不同實驗及不同平均懲罰。

對均勻分布 $u(x) = 1/K$，展開可得 $D(p\|u) = \log_2 K - H(p)$。非負性給出熵上界，也證明固定有限字母集合上，只有均勻來源達到最大熵。

交叉熵連結預測與編碼

交叉熵用提出的機率模型為觀測打分，包含來源不可避免的不確定性，以及模型不準確的代價。

$$H(p,q) = -\sum_x p(x) \log_2 q(x) = H(p) + D(p\|q).$$

$H(p,q)$ 是交叉熵，$H(p)$ 是真實來源熵，q 提供預測機率。理想機率編碼中，$-\log_2 q(x)$ 是描述長度目標；實際編碼還有整數長度、框架、模型描述與實作成本。

p 固定、參數改變 q 時，最小化母體交叉熵等同最小化相對熵。用資料估計時，經驗平均負對數機率是

$$\widehat{H}_N = -\frac{1}{N} \sum_{j=1}^{N} \log_2 q_\theta(x_j).$$

N 是觀測數，x_j 是第 j 個觀測，θ 是模型參數；帽號表示經驗量。它和母體平均的關係，依觀測取樣方式及是否重用來選模型而定。

分類器對已知特徵的標籤使用條件版本，生成模型則對觀測或序列使用這個觀點。低訓練值描述訓練觀測的擬合，不直接證明未來資料表現。這連到 ML/DL 評估規則。

訓練用自然對數時，loss 單位是 nat。乘正常數會改變梯度尺度，但不改變數學最佳解。比較論文中的數值，須統一單位、正規化方式及預測目標。

計算例：一張表，三種資訊問題

以下是母體模型，每格表示觀察到該點對的機率。它描述兩個公平位元，但兩者相同的機會高於不同的機會。

聯合機率	$Y=0$	$Y=1$	列總和
$X=0$	0.4	0.1	0.5
$X=1$	0.1	0.4	0.5
欄總和	0.5	0.5	1.0

各列、欄總和給出邊際機率。兩個邊際都公平，因此 $H(X)=H(Y)=1$ 位元。觀察到 $X=0$ 後，把該列除以總和 0.5，得到 Y 的條件機率 0.8 與 0.2。另一列把兩者對調，但熵相同。

$$H(Y\mid X)=0.5h_2(0.2)+0.5h_2(0.2)=0.721928,$$
$$H(X,Y)=1+0.721928=1.721928,$$
$$I(X;Y)=1-0.721928=0.278072.$$

h_2 是二元熵，$H(Y\mid X)$ 平均兩個條件列，$H(X,Y)$ 描述點對，$I(X;Y)$ 則是知道另一位元後，任一位元減少的不確定性。數值單位都是位元，取至小數第六位；下方程式使用未取整數值計算。

把聯合表與每格都是 0.25 的獨立表比較，也得到同樣互資訊。相同位元格的比值是 $0.4/0.25=1.6$，不同位元格是 $0.1/0.25=0.4$；加權對數比總和為 0.278072。單一格的對數比可以負，平均散度卻仍非負。

接著保留公平單位元來源，但用 0.1、0.9 預測。來源熵仍為一位元，交叉熵則成為 $-0.5\log_2(0.1)-0.5\log_2(0.9)=1.736966$ 位元，其中 0.736966 位元是模型失配。聯合表問題量測相依性，預測問題量測錯誤模型；兩者都用散度，卻比較不同分布。

這張表也能解釋分類。預測 $Y=X$ 的正確機率是 0.8，但條件分布仍有 0.721928 位元不確定性。正確率計算決策是否正確，熵評估整個預測分布，兩者不能互相取代。

程式碼：精確機率表檢查

完整 CPU 範例遮掉零機率項，不把它換成小常數。輔助函式也檢查陣列是否為正規化機率。

```
import numpy as np

def probability(a):
    a = np.asarray(a, dtype=float)
    assert np.all(a >= 0) and np.isclose(a.sum(), 1)
    return a

def entropy(a):
    a = probability(a)
```

```
    positive = a > 0
    return -np.sum(a[positive] * np.log2(a[positive]))

def divergence(p, q):
    p, q = probability(p), probability(q)
    assert p.shape == q.shape
    positive = p > 0
    if np.any(q[positive] == 0):
        return np.inf
    return np.sum(p[positive] * np.log2(p[positive] / q[positive]))

joint = probability([[0.4, 0.1], [0.1, 0.4]])
px, py = joint.sum(axis=1), joint.sum(axis=0)
independent = np.outer(px, py)
hx, hy, hxy = entropy(px), entropy(py), entropy(joint)
information = divergence(joint, independent)
print("H(X), H(Y), H(X,Y):", hx, hy, hxy)
print("H(Y|X), I(X;Y):", hxy - hx, information)
assert np.isclose(information, hx + hy - hxy)
assert np.isclose(divergence(independent, independent), 0)

p, q = probability([0.5, 0.5]), probability([0.1, 0.9])
cross = -np.sum(p * np.log2(q))
print("Source entropy / cross-entropy / mismatch:",
      entropy(p), cross, divergence(p, q))
assert np.isclose(cross, entropy(p) + divergence(p, q))
assert np.isinf(divergence(probability([0.5, 0.5]), probability([0, 1])))
```

這裡的恆等式是在指定母體機率表上檢查。若表來自有限樣本，還多了估計不確定性；代數能精確成立，數值估計卻仍可能不準。

互資訊量測相依性

知道一個變數會改變另一個的預期時，兩者共享資訊。這不只限於線性關係。

$$\begin{aligned} I(X;Y) &= D(p_{XY} \| p_X p_Y) \\ &= H(X) + H(Y) - H(X,Y) \\ &= H(X) - H(X \mid Y). \end{aligned}$$

$I(X;Y)$ 是互資訊（mutual information），$p_X p_Y$ 是邊際分布乘積。它非負、對稱，且只有獨立時為零。上界是兩個邊際熵中較小者。

相對熵形式問實際成對分布和獨立模型差多少；熵形式問一變數移除另一變數多少平均不確定性。兩觀點一致，但凸顯不同應用。

相關係數較狹窄：零相關一般不能證明獨立。非線性相依可能很強，特定線性摘要卻為零。另一方面，高維互資訊通常比相關係數更難估準。

條件互資訊為

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y,Z).$$

Z 是條件資訊。此量平均各條件分布內的相依性，非負，但可能比無條件 $I(X;Y)$ 大，也可能較小。

令 X 與 Y 是獨立公平二元變數，$Z = X \oplus Y$ 是同位元。無條件時，X 對 Y 沒幫助；已知 Z 後，X 就決定 Y。所以無條件互資訊為零，條件值為一 bit。條件能揭露相依性，不只是減去固定資訊量。

資訊鏈式法則為

$$I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y).$$

(Y,Z) 是聯合觀測。第二項是已有 Y 後，Z 額外提供多少資訊。直接相加兩個無條件互資訊，可能重複計入共享證據，或漏掉聯合效果。

互資訊是平均預測改善

利用 $p_{XY}(x,y) = p_X(x)p_{Y|X}(y \mid x)$ 展開聯合分布比值，邊際因子抵消，得到

$$I(X;Y) = \sum_x p_X(x) D(p_{Y|X=x} \| p_Y).$$

$p_{Y|X=x}$ 是觀察 $X = x$ 後的 Y 條件分布，p_Y 是尚未取得該觀測的邊際預測，散度單位為位元。此式把資訊表達成無條件到條件預測的平均改變。

它也證明等號對應獨立。每個條件散度非負，加權總和為零，必須每個正機率 x 都不改變預測；這正是有限表的獨立條件。計算例中，每個條件列與公平邊際的散度都相同，為 $1 - h_2(0.2)$。

對數預測損失下，最佳無條件預測器付出 $H(Y)$，最佳條件預測器付出 $H(Y \mid X)$，差就是互資訊。這是最佳母體分布的陳述；擬合預測器可能因資料、容量或最佳化不足，無法實現好處。推導辨認可得預測利益，也說明經驗估計仍需要另外驗證。

資料處理、可逆性與額外隨機性

如果處理只讀目前觀測，就不能創造關於原始來源的新證據。它能增加隨機變動，但變動不等於來源資訊。

Markov 關係 $X \to Y \to Z$ 表示

$$p(x,y,z) = p_X(x)p_{Y|X}(y \mid x)p_{Z|Y}(z \mid y).$$

Z 由 Y 產生，不額外讀取 X。這推出資料處理不等式（data-processing inequality）：

$$I(X;Z) \le I(X;Y).$$

兩邊都是同一 X 的資訊。這不表示任意隨機處理都有 $H(Z) \le H(Y)$；加入獨立雜訊可能增加輸出熵，同時降低輸入資訊。

一個有用證明是以兩種順序展開 $I(X;Y,Z)$。Markov 關係使 $I(X;Z \mid Y) = 0$；另一順序包含非負的 $I(X;Y \mid Z)$，所以 $I(X;Y) = I(X;Z) + I(X;Y \mid Z)$。取等號表示 Z 沒有丟掉 Y 原本提供的相關 X 資訊。

確定性轉換滿足 $H(f(X)) \le H(X)$，因為它合併可能結果。在支撐集合上一對一，就不損失不確定性。任一變數做可逆轉換，也保留互資訊；多對一量化則可能丟掉相依性。

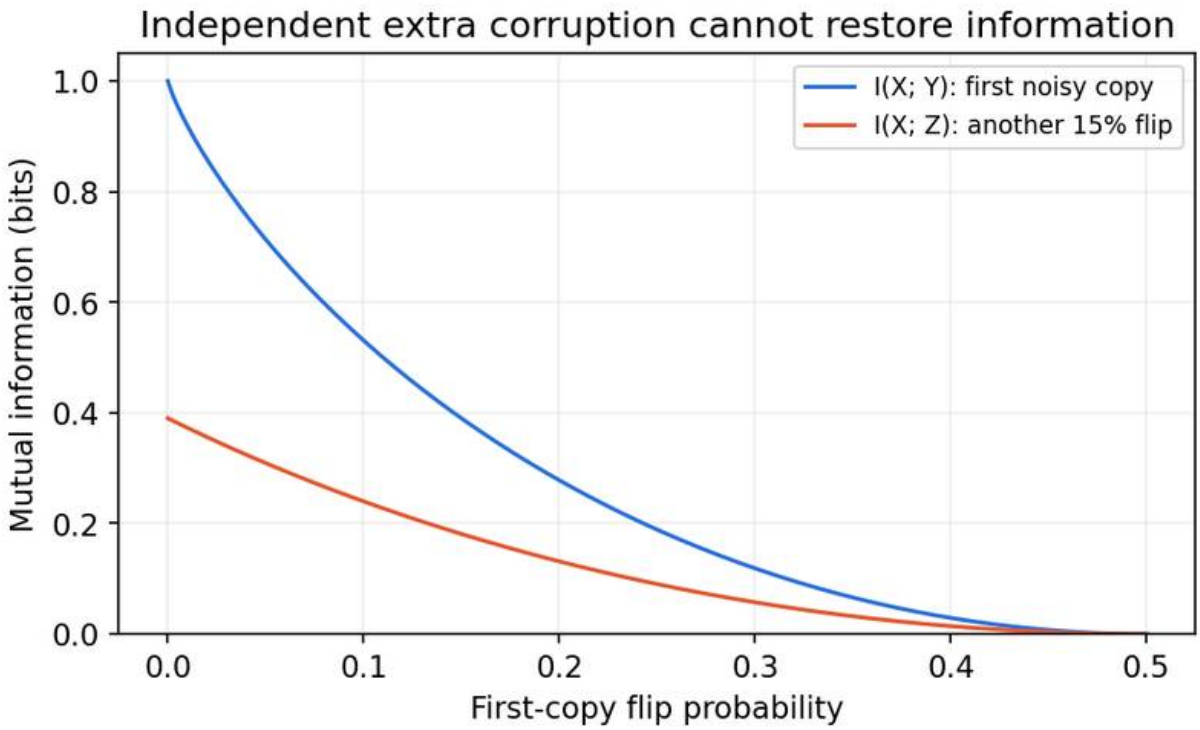


圖 2 — 資料處理的解析例子。第二通道以 *0.15* 機率獨立翻轉輸入；兩條曲線都量測同一原始來源的資訊。

程式碼：處理與同位元

第二個獨立範例建立完整三變數機率表，不使用學得估計器，直接檢查不等式與同位元例子。

```
import numpy as np

def entropy(p):
    p = np.asarray(p, dtype=float)
    assert np.all(p >= 0) and np.isclose(p.sum(), 1)
    q = p[p > 0]
    return -np.sum(q * np.log2(q))

def mutual(joint):
    return entropy(joint.sum(axis=1)) + entropy(joint.sum(axis=0)) - entropy(joint)

pxy = np.array([[0.4, 0.1], [0.1, 0.4]])
channel = np.array([[0.85, 0.15], [0.15, 0.85]])
pxyz = pxy[:, :, None] * channel[None, :, :]
pxz = pxyz.sum(axis=1)
```

```
print("Before / after processing:", mutual(pxy), mutual(pxz))
assert mutual(pxz) <= mutual(pxy) + 1e-12

parity = np.zeros((2, 2, 2))
for x in range(2):
    for y in range(2):
        parity[x, y, x ^ y] = 0.25
pxz, pyz = parity.sum(axis=1), parity.sum(axis=0)
pz = parity.sum(axis=(0, 1))
conditional = entropy(pxz) + entropy(pyz) - entropy(pz) - entropy(parity)
print("Parity I(X;Y) / I(X;Y|Z):",
      mutual(parity.sum(axis=2)), conditional)
assert np.isclose(conditional, 1)
```

估計互資訊違反資料處理時，可能原因包括取樣誤差、不同估計器偏差或 Markov 假設不成立；精確定理本身沒有失效。

估計不只是套恆等式

計算觀測次數容易，判斷未見結果還有多少機率卻不容易。大字母集合與強相依，會讓資訊估計脆弱。

N 個觀測中，結果 x 出現 N_x 次，經驗分布與 plug-in 熵為

$$\hat{p}_N(x) = \frac{N_x}{N}, \qquad \hat{H}_N = -\sum_x \hat{p}_N(x) \log_2 \hat{p}_N(x).$$

N_x 是該結果計數，N 是總數，帽號表示估計。未見結果的經驗機率為零，即使真正機率正值。

固定有限字母集合、獨立取樣且條件適當時，樣本增加會使估計收斂。樣本少時通常低估熵，因為漏掉可能結果，使分布看起來太集中。聯合分布要涵蓋的集合，又比任一邊際大。

量化也定義變數。把連續特徵分成 8 級或 256 級，是不同有限集合問題。更細分格可能提供解析度，也產生更多稀疏格。資訊曲線看起來平滑，不會消除這個取捨。

以下獨立模擬對已知均勻來源做重複合成取樣，檢查 plug-in 估計的平均，不以單次抽樣聲稱信賴程度。

```
import numpy as np

rng = np.random.default_rng(31)
alphabet = 20
truth = np.log2(alphabet)
for n in [40, 400, 4000]:
    estimates = []
    for _ in range(200):
        sample = rng.integers(alphabet, size=n)
        p = np.bincount(sample, minlength=alphabet) / n
        positive = p > 0
        estimates.append(-np.sum(p[positive] * np.log2(p[positive])))
```

```
    print("N:", n, "true entropy:", round(truth, 4),
          "mean estimate:", round(np.mean(estimates), 4))
assert abs(np.mean(estimates) - truth) < 0.02
```

實務上要指定取樣單位、特徵轉換、估計器類型、超參數選擇及不確定性評估。同一錄影的重複影格，不等於獨立場景。先用已知資訊量的分布測估計器，是有用起點。

現代觀點

基本資訊量沒有改變，但現代應用常使用高維學得表示。困難通常是依模型估計或最佳化資訊量，而非寫出定義。

Butakov 等人在 NeurIPS 2024 研究透過 normalizing flow 估互資訊 [3]。方法利用可逆轉換，在假設成立時保留目標資訊、讓估計較容易。換座標能幫助估計器，不等於創造資訊。這是原始方法，不是綜述，也不證明所有神經估計器都無偏。

Aubret、Matignon 與 Hassas 的 2023 綜述 [4] 以**驚訝、新穎性與技能學習**分類內在動機。不可預測觀測、新到訪狀態與可控制技能，是不同目標。本章的解讀是：「最大化熵」必須先指定變數與條件結構，才成為可用設計規則。

El Gamal 2026 的 *Information Theory and Statistical Learning* [5] 連結以散度為基礎的預測及生成學習。它是**建議納入未來第 3 版的草稿章節**，不是已出版的第 3 版，也不取代本主線使用的第 2 版章號。

開放問題包括稀疏高維估計、偏差不同的估計器比較、考慮相依性的評估，以及方便的資訊目標與真正任務之間的關係。方法改善估計下界，不直接證明下游決策改善。

讀這類主張時，可以逐項問：哪些隨機變數？哪個聯合分布？什麼單位？數值是精確量、估計、界限還是最佳化目標？條件獨立假設由什麼支持？這些問題把數學精確性連到有效實驗證據。

重點整理

- 熵是分布下的平均驚訝，不是單一觀測值的性質。
- 鏈式法則處理共享結構，不需要獨立假設。
- 條件降低平均不確定性，個別條件結果可能不同。
- 相對熵有方向、對支撐集合敏感，且非負。
- 交叉熵分開來源不確定性與模型失配。
- 互資訊量測相依，條件與無條件相依可能差很多。
- 資料處理限制固定來源的證據，輸出隨機性卻可能增加。
- 經驗資訊估計繼承取樣、量化與模型假設。

練習

1. 三符號來源機率為 $(1/2, 1/4, 1/4)$，計算各結果驚訝與熵，再比較均勻三符號來源。

提示
用機率加權對數驚訝。

2. 建立均勻邊際、互資訊零的二元聯合表，再建立互資訊一 bit 的表，解釋條件熵。

提示
比較外積與對角表。

3. 展開與 K 符號均勻模型的 $D(p\|u)$，用非負性證明熵上界，寫出取等號條件。

提示
展開為 $\log_2 K - H(p)$。

4. 建立一個具體模型，使某個 y 有 $H(X \mid Y = y) > H(X)$，平均條件不等式仍成立。

提示
用少數公平診斷日和多數確定日。

5. 處理範例再加第三個雜訊通道，逐階檢查互資訊，指出獨立假設。

提示
先乘轉移機率，再消去中間狀態。

6. 固定樣本數，把估計模擬的字母集合放大，區分估計器結論與真正熵結論。

提示
同時記錄未見符號數。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006，第 1–2、17 章。出版社與目錄。
2. T. Weissman, “Entropy, Relative Entropy, and Mutual Information,” Stanford EE276 Lecture 2 notes, 2024。一手講義。
3. I. D. Butakov, A. D. Tolmachev, S. V. Malanchuk, A. M. Neopryatnaya and A. A. Frolov, “Mutual Information Estimation via Normalizing Flows,” NeurIPS, 2024。會議論文。
4. A. Aubret, L. Matignon and S. Hassas, “An Information-Theoretic Perspective on Intrinsic Motivation in Reinforcement Learning: A Survey,” *Entropy*, 25(2), 327, 2023。已出版綜述。
5. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026。草稿與出版狀態。

IT 02・典型序列、AEP 與熵率

推導長來源區塊為何集中在熵附近，連結典型集與編碼，並區分平穩、獨立、相依與非遍歷來源。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/typicality-entropy-rates/

你將學到

- 推導獨立有限字母來源的漸近均分性質。
- 分別限制典型集的機率與大小，不混淆兩者。
- 說明最可能的單一序列為何不一定具有代表性。
- 推導平穩有限狀態 Markov 來源的熵率。
- 用具體反例區分平穩性、獨立性與遍歷性。
- 連結條件典型性、聯合典型性與壓縮及通訊證明。

先看全貌

單次取樣充滿不確定性，長序列的整體行為卻常相當穩定。具有偏差的二元來源，其下一個位元仍無法完全預測，但長區塊的平均描述長度往往接近固定值。典型性把這項觀察寫成精確敘述。

> **用最簡單的話說**
>
> 每次從袋子抽出什麼顏色的珠子都不確定。不過抽很多次之後，大致會有多少種顏色、整串珠子有多令人意外，反而可以預測。多數會遇到的珠串因此能放進比所有可能珠串更小的目錄。

本章依循 Cover 與 Thomas 的第 3–4 章 [1]。Stanford 的 AEP 原始講義 [2] 提供精簡的獨立來源證明。我們會推導結果、說明有限區塊的限制，再延伸至相依來源。後續章節會把存在性論證轉成實際編碼器與解碼器。

典型性永遠相對於指定的來源分布。它不代表外觀普通、語意正確，或對每個族群都有代表性。樣本可能在錯誤的模型下具有高機率，卻仍不符合應用需求。

區塊機率與平均驚訝量

不只問一個符號有多令人意外，而是計算整個區塊的驚訝量，再除以長度。對獨立符號，這就是各符號驚訝量的平均。

設 $X_1, \dots, X_n$ 在有限字母集 $\mathcal{X}$ 上，依機率質量函數 p 獨立同分布。定義區塊 $X^n = (X_1, \dots, X_n)$：

$$p(x^n) = \prod_{i=1}^{n} p(x_i), \qquad J_n = -\frac{1}{n} \log_2 p(X^n) = \frac{1}{n} \sum_{i=1}^{n} -\log_2 p(X_i).$$

其中 n 是區塊長度，x^n 是特定序列，i 是位置，J_n 是以每符號位元數表示的隨機平均驚訝量。等式利用獨立性；其期望為單一來源符號的熵 $H(X)$。

有限字母設定使正機率結果上的驚訝量可積分。字母集中可以包含不可能的符號，但實際取樣區塊不會出現它們。因此大數法則給出

$$J_n \xrightarrow{\Pr} H(X).$$

箭頭表示依機率收斂：對任何固定正容許誤差，J_n 與熵的差超過該誤差的機率會趨近零。這就是**漸近均分性質**（asymptotic equipartition property，AEP）[1,2]。

「均分」是近似且漸近的。它不表示每個區塊都具有相同機率，也不表示所有符號最後都會等頻出現。它表示多數機率質量集中於機率具有近似相同指數尺度的區塊。

對符號一機率為 p 的 Bernoulli 來源，含有 k 個一的區塊滿足

$$-\frac{1}{n}\log_2 p(x^n) = -\frac{k}{n}\log_2 p - \left(1-\frac{k}{n}\right)\log_2(1-p).$$

其中 k 是觀察到的個數，k/n 是經驗比例。當比例接近 p，平均驚訝量接近二元熵 $h_2(p)$。一的個數通常不會接近區塊長度的一半。

典型集：分清機率與計數

多數可能觀察到的結果能放進合理區塊的目錄。目錄內每個區塊的機率可能很低，但加總後涵蓋幾乎全部來源機率。

對容許誤差 $\epsilon > 0$，定義

$$A_\epsilon^{(n)} = \left\{ x^n : \left| -\frac{1}{n}\log_2 p(x^n) - H(X) \right| \le \epsilon \right\}.$$

其中 $A_\epsilon^{(n)}$ 是弱典型集，大括號內的 $|\cdot|$ 表示絕對值，ϵ 是以每符號位元數表示的容許誤差。其元素滿足

$$2^{-n(H(X)+\epsilon)} \le p(x^n) \le 2^{-n(H(X)-\epsilon)}.$$

這限制單一典型區塊的機率。AEP 另外指出 $\Pr(X^n \in A_\epsilon^{(n)})$ 趨近一。

當 n 足夠大，假設典型集機率至少為 $1-\delta$，其中 $\delta > 0$ 是指定的機率容許誤差，則

$$(1-\delta)2^{n(H(X)-\epsilon)} \le |A_\epsilon^{(n)}| \le 2^{n(H(X)+\epsilon)}.$$

其中 $|A_\epsilon^{(n)}|$ 表示典型序列的個數，而非其機率。上界來自每個典型區塊至少具有 $2^{-n(H+\epsilon)}$ 的機率，且總機率不能超過一。下界來自集合至少涵蓋 $1-\delta$ 的機率，而各區塊機率至多為 $2^{-n(H-\epsilon)}$。

把 ϵ 與 δ 分開，可以看清角色：前者限制描述長度的偏差，後者限制被排除的機率質量。教科書可能在選擇足夠大的區塊後，用同一個小參數表示兩者；有限長度計算仍必須區分。

總共有 $|\mathcal{X}|^n$ 種可能序列。若 $H(X) < \log_2 |\mathcal{X}|$，典型目錄的指數尺度就小得多。這是壓縮背後的計數事實：若允許很小的錯誤機率，就不必讓每個可想像的區塊都有短描述。

最可能的區塊不一定典型

單一機率最高的字串可能沒有代表性。許多單獨機率稍低的字串，加起來反而占大多數機率。

對 $p = 0.2$ 的獨立二元來源，全零區塊是單一機率最高的區塊。其平均驚訝量為

$$-\frac{1}{n}\log_2(0.8^n) = -\log_2(0.8).$$

其中 0.8 是零的機率。這個值與 $h_2(0.2)$ 不同。對較窄的容許誤差，全零字串一直在典型集外，即使沒有任何單一字串比它更可能。

它的總機率 0.8^n 趨近零；反之，約五分之一符號為一的區塊有非常多個，加總機率趨近一。機率質量與單一結果排名是不同的對象。

生成模型也有類似區別：最大機率序列不一定像模型的代表性樣本。典型性提供數學例子，卻不保證某個取樣演算法會產生有用語言或影像。

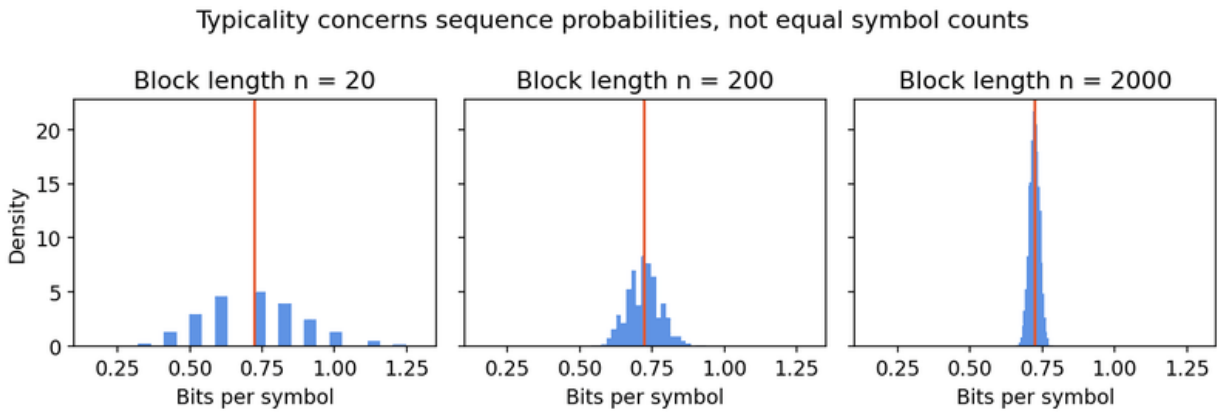


圖 *1* — 每個長度產生 *5,000* 個合成二項取樣區塊。橘線是精確來源熵；橫軸是驚訝量，不是一的比例。

程式：精確典型機率與有限個數

第一個完整例子利用二項計數分布，將典型集機率加總至浮點運算精度。另用小區塊枚舉檢查基數界限。

```
import itertools
import numpy as np
from scipy.stats import binom

p, epsilon = 0.2, 0.1
entropy = -p * np.log2(p) - (1 - p) * np.log2(1 - p)
for n in [20, 200, 2000]:
    k = np.arange(n + 1)
    per_symbol = -(k / n) * np.log2(p) - (1 - k / n) * np.log2(1 - p)
    typical = np.abs(per_symbol - entropy) <= epsilon
```

```
    probability = binom.pmf(k, n, p)[typical].sum()
    print("Length:", n, "typical probability:", probability)

n, epsilon = 10, 0.15
count, mass = 0, 0.0
for block in itertools.product([0, 1], repeat=n):
    k = sum(block)
    probability = p**k * (1 - p)**(n - k)
    surprise = -np.log2(probability) / n
    if abs(surprise - entropy) <= epsilon:
        count += 1
        mass += probability
assert count <= 2**(n * (entropy + epsilon))
assert count >= mass * 2**(n * (entropy - epsilon))
print("Small-block typical count / mass:", count, mass)
```

小區塊的典型機率不一定接近一。「漸近」描述固定容許誤差下，區塊長度持續增加的行為，不代表任何長度都能套用長區塊結論。

計算例：先數字串，再加總機率

使用上方小區塊枚舉的 $n = 10$、$p = 0.2$、$\epsilon = 0.15$。有 k 個一的區塊，其每符號平均驚訝量為 $0.321928 + 0.2k$ 位元，來源熵則為 0.721928。只有 $k = 2$ 符合容許誤差；我們先決定成員，再進行計數。

這些字串有 $\binom{10}{2} = 45$ 個。每個機率為 $0.2^2 \times 0.8^8 = 0.0067108864$，所以典型集機率是 $45 \times 0.0067108864 = 0.301989888$。相較之下，單一全零字串機率為 $0.8^{10} = 0.1073741824$。它比每個典型字串更可能，但比 45 個字串合計的機率低。

這分別回答三個問題：**成員資格**由驚訝量容許誤差決定，**基數**計算接受字串數，**機率質量**加總其機率。45 個字串的目錄需要六個定長索引位元，因為五位元只能區分 32 種選項。但該目錄只涵蓋這個短區塊來源約 30.2% 的機率。因此，即使碼率是每符號 0.6 位元，僅靠長度十目錄仍不是好的幾乎無損編碼。

熵界限沒有失效。固定此容許誤差時，高機率結論需要更長區塊。基數取整與排除機率，是極限論證不會從短實驗中消除的成本。

若改用公平位元模型評分，同一批區塊全部恰好每符號一位元。該模型無法用驚訝量區分全零與兩個一的字串。真正來源未變，改變的是評分模型。計算典型集時，必須指定由哪個分布提供機率。

經驗頻率與組合計數直覺

區塊的符號頻率定義一群具有相同來源機率的字串。計算這一群的大小，就能說明單獨不太可能的字串為何能共同占大多數機率。

對恰好有 k 個一的二元區塊，排列數與該計數的總機率為

$$N_{n,k} = \binom{n}{k}, \qquad \Pr(K_n = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

其中 $N_{n,k}$ 計算字串個數，K_n 是隨機的一的個數，$\binom{n}{k}$ 是二項係數，p 是來源產生一的機率。同一計數的每種排列機率相同，但排列數差異非常大。

兩個極端分別只有一個全零與一個全一字串；中間則有大量排列。對偏差來源，每種排列的機率與排列數朝不同方向變化，乘積在接近來源期望計數處達到高值。

這也說明經驗頻率典型性與驚訝量典型性的差別。公平二元來源的每個長度 n 字串機率都是 2^{-n}，平均驚訝量恰好一位元。因此任何正容許誤差下，所有可能字串都是弱典型，包括全零。

觀察到全零字串的機率仍趨近零。以頻率定義的典型集可要求一的比例接近一半，從而排除全零。它的機率也趨近一，但成員條件更嚴格。

兩種定義都沒有錯；它們用於不同證明。機率尺度敘述可能只需要弱典型性；需要匹配經驗條件分布的構造，可能必須控制頻率。務必確認證明實際使用哪個定義，尤其從邊際來源編碼轉向聯合輸入輸出關係時。

來源編碼論證與限制

將典型區塊編號，傳送其目錄索引。區塊夠長時，平均每符號使用接近熵的位元數，只在少量被排除的結果上失敗。

典型集至多有 $2^{n(H+\epsilon)}$ 個成員，因此索引大致需要不超過 $n(H+\epsilon)$ 位元，再加取整與區塊框定的成本。解碼器用相同目錄重建所選區塊。

這是**幾乎無損的定長**構造，並未正確編碼所有區塊。可以保留一個額外符號表示失敗，但旗標不能重建非典型區塊。要真正無損，就必須提供足以完整還原它的逸出表示。

簡單的反向論證可說明，遠低於熵的速率為何無法涵蓋多數機率。假設編碼最多正確表示 2^{nR} 個區塊，將它們與典型集取交集。每個典型區塊機率至多為 $2^{-n(H-\epsilon)}$，可接受機率質量至多為

$$2^{nR} 2^{-n(H(X)-\epsilon)} = 2^{-n(H(X)-\epsilon-R)}.$$

其中 R 是以每來源符號位元數表示的定長編碼速率。若 $R < H(X) - \epsilon$，典型集內可接受的機率趨近零。被排除的非典型機率也趨近零，因此編碼無法以高機率重建來源。

論證假設編碼器與解碼器只有有限個不同訊息。這是計數證明，不是有效率的實作；枚舉巨大目錄通常不實際。下一章研究前綴編碼、機率編碼與自適應模型，把理論速率轉成可計算程序。

加上逸出分支，讓目錄真正無損

有時失敗的目錄，可以變成還原每個允許有限區塊的碼。先傳一位元分支旗標；區塊典型時傳定長目錄索引，否則傳逐符號固定寬度的未壓縮表示。

令 $m = \lceil \log_2 |\mathcal{X}| \rceil$，非空典型集的索引寬度為 $\ell_n = \lceil \log_2 |A_\epsilon^{(n)}| \rceil$。期望長度滿足

$$\frac{\mathbb{E}[L_n]}{n} = \frac{1}{n} + \frac{P_T \ell_n}{n} + (1 - P_T) m \le \frac{2}{n} + P_T (H(X) + \epsilon) + (1 - P_T) m.$$

L_n 是區塊總位元長度，m 是原始符號固定寬度，ℓ_n 是典型索引寬度，$P_T = \Pr(X^n \in A_\epsilon^{(n)})$ 是典型機率，$n, \epsilon, H(X)$ 沿用前述定義。等式計入旗標與兩分支；不等式利用典型集基數與向上取整。

解碼器先讀旗標，便知道該讀哪個固定分支長度。典型分支回傳共享目錄項目，逸出分支還原原始符號，沒有區塊被默默省略。小有限區塊的典型集若為空，先只用原始分支，直到有可用目錄。

固定 ϵ 下，AEP 給 $P_T \to 1$，使逸出分支的每符號貢獻消失。旗標與取整成本也隨 n 增長而消失，期望速率因此漸近不超過 $H(X) + \epsilon$。先建立足夠長區塊，再選較小容許誤差，便能任意接近熵。

構造揭露定長幾乎無損與變長完全無損的差別。逸出分支在罕見區塊可能很長，後者結論因此針對**期望長度**，不是每個區塊都有短上界。兩端還須共享目錄與來源模型。這是速率可達證明，明確枚舉通常太昂貴，不是實用壓縮演算法。

有限區塊集中速度取決於來源

長區塊很有用，但需要多長，取決於來源與容許誤差。稀有符號可能造成很大的驚訝量波動。

對驚訝量變異數有限的獨立符號，

$$\operatorname{Var}(J_n) = \frac{V}{n}, \qquad V = \operatorname{Var}[-\log_2 p(X)].$$

其中 V 是來源驚訝量變異數，Var 表示變異數。結果來自獨立變異數相加再除以 n^2。一般變異數界限給出

$$\Pr(|J_n - H(X)| > \epsilon) \le \frac{V}{n\epsilon^2}.$$

其中 ϵ 是前述驚訝量容許誤差。界限可能寬鬆，但明確顯示依賴關係。很小的機率可能提高驚訝量變異數；要求更窄誤差也會增加所需長度。

跨區塊的經驗標準差有用，卻不是通用的有限區塊編碼保證。相依性改變變異數計算；小規模模擬可能漏掉稀有事件；模型不匹配會改變集中位置。

若來源 p 由模型 q 評分，在可積分條件下，獨立區塊平均驚訝量收斂至 $H(p, q)$，而不是 $H(p)$。兩者的差是相對熵。對模型典型與對真實來源典型，是分開的問題。

熵率考慮相依性

鄰近符號常能幫忙互相預測。因此，長序列每符號需要的資訊，可以小於單一邊際符號的熵。

對隨機程序 $X_1, X_2, \ldots$，若極限存在，定義熵率為

$$\bar{H}(X) = \lim_{n \to \infty} \frac{1}{n} H(X_1, \ldots, X_n).$$

其中 $\bar{H}(X)$ 是每符號平均新增的不確定性，n 是區塊長度。定義討論區塊熵，不是擬合模型在某一條已實現序列上的損失。

鏈式法則給出

$$\frac{1}{n}H(X^n) = \frac{1}{n}\sum_{i=1}^{n} H(X_i \mid X^{i-1}).$$

其中 $X^{i-1} = (X_1, \dots, X_{i-1})$ 是先前歷史，$i = 1$ 時為空。相依性使條件項不同於邊際熵。

當所有時間指標共同平移後，每個有限維聯合分布都不改變，程序稱為**平穩**。這比平均值或共變異數固定更強。有限字母的平穩來源，其給定更長過去的條件熵不增加，而且以零為下界，故極限存在；區塊平均熵有相同極限。

兩種極限相等，是因為對收斂的條件熵序列取平均。平穩性允許將固定長度歷史移到共同時間點，而增加過去資訊會降低條件熵。

獨立同分布是特例：每個條件熵都等於 $H(X_1)$。相依的平穩來源可以在每個時間擁有相同邊際分布，但每一步需要的新資訊少得多。

Markov 來源的明確計算

一階 Markov 模型只需目前狀態即可預測下一個狀態。長期不確定性是各轉移列不確定性的加權平均。

設有限 Markov 來源的轉移矩陣為 P：

$$P_{ij} = \Pr(X_{t+1} = j \mid X_t = i), \qquad \pi_j = \sum_i \pi_i P_{ij}.$$

其中 i, j 是來源狀態索引，t 是時間，P_{ij} 是轉移機率，π 是平穩狀態分布。P 每列總和為一；π 非負且總和為一。

若程序以 π 初始化，平穩性與 Markov 性質給出

$$\bar{H}(X) = H(X_{t+1} \mid X_t) = -\sum_i \pi_i \sum_j P_{ij} \log_2 P_{ij}.$$

內部總和是轉移列 i 的熵，外部總和按平穩占有率加權。有限區塊熵為

$$H(X^n) = H(\pi) + (n-1)\bar{H}(X).$$

其中 $H(\pi)$ 是初始狀態熵。鏈式法則貢獻一個初始項與 $n-1$ 個相同轉移項。

對翻轉機率為 a 的對稱二元來源，平穩分布均勻。邊際熵為一位元，熵率則是 $h_2(a)$。當 $a = 0.1$，下一符號多半重複目前符號，因而減少新增不確定性。

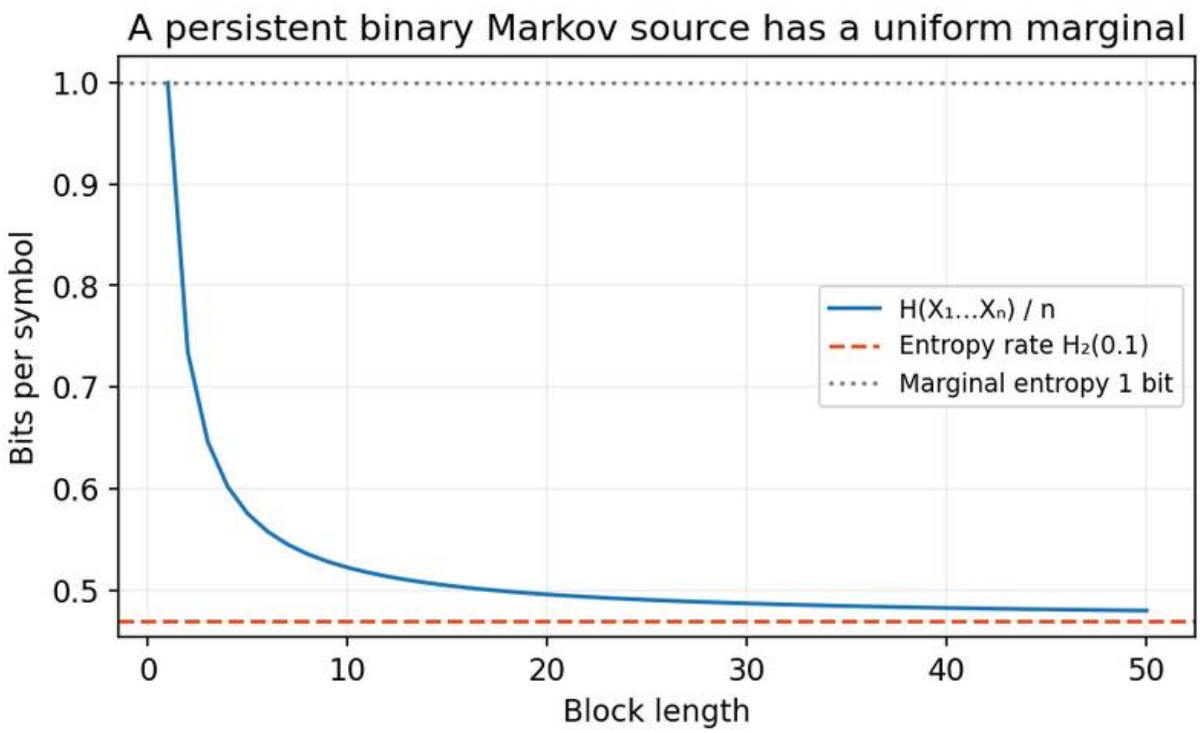


圖 2 — 平穩二元 *Markov* 來源的解析例子。單一符號頻率均勻，不代表來源獨立。

程式：邊際不變，預測性不同

獨立的翻轉事件產生平穩二元 Markov 區塊，再將觀察到的平均驚訝量與精確區塊熵公式比較。

```
import numpy as np

rng = np.random.default_rng(32)
a = 0.1
rate = -a * np.log2(a) - (1 - a) * np.log2(1 - a)
for n in [64, 256, 1024]:
    initial = rng.integers(2, size=1000)
    flips = rng.binomial(1, a, size=(1000, n - 1))
    states = np.column_stack([
        initial, initial[:, None] ^ np.bitwise_xor.accumulate(flips, axis=1)])
    surprise = 1 + (-flips * np.log2(a) - (1 - flips) * np.log2(1 - a)).sum(axis=1)
    expectation = (1 + (n - 1) * rate) / n
    print("Length:", n, "one-frequency:", states.mean(),
          "mean surprise rate:", surprise.mean() / n,
          "exact entropy per symbol:", expectation)
    assert abs(states.mean() - 0.5) < 0.08
    assert abs(surprise.mean() / n - expectation) < 0.02
print("Entropy rate:", rate, "marginal entropy:", 1.0)
```

初始狀態均勻取樣，因此來源平穩。每個區塊固定從零開始會改變有限區塊分布；適當轉移假設下漸近速率仍可能相同，但精確的初始熵項會改變。

邊際公平，為什麼仍容易預測？

翻轉機率 $a = 0.1$ 的對稱 Markov 例子，第一位元有一位元不確定性，後續每個位元貢獻 $h_2(0.1) = 0.468996$ 位元。長度十的區塊熵為 $1 + 9h_2(0.1) \approx 5.220960$ 位元，即每符號 0.522096 位元；十個獨立公平位元則有十位元區塊熵。

可以用初始位元加九個翻轉指示表示整段序列。這個映射可逆：從第一位元開始，依序與翻轉指示做互斥或，就恢復下一狀態。它揭露相依性，沒有捨棄來源資訊。在此特定模型下，指示是有偏的獨立位元，因此比十個邊際公平狀態更容易編碼。

長期速率趨近 0.468996，因為初始化的一位元成本被越來越長的區塊攤提。可預測性來自轉移關係，即使每個時間位置的邊際都公平。影像直方圖也無法揭露利用鄰近像素預測所能獲得的全部節省。

隱藏狀態與觀測對象

轉移模型可能描述內部狀態，接收端卻只看得到帶雜訊的測量。觀測序列因此有自己的預測問題。

設 S_t 是有限隱藏 Markov 狀態，Y_t 是發射觀測，模型可將區塊分解為

$$p(s^n, y^n) = p(s_1) \prod_{t=1}^{n-1} p(s_{t+1} \mid s_t) \prod_{t=1}^{n} p(y_t \mid s_t).$$

其中 s^n 是狀態序列，y^n 是觀測序列，各因子描述轉移與發射。對隱藏序列加總後得到觀測概似 $p(y^n)$。

Y 的熵率通常不只是按平穩分布平均的發射列熵。過去觀測能幫助推論隱藏狀態，剩餘狀態不確定性也影響預測。相關項是 $H(Y_t \mid Y^{t-1})$，而不只是 $H(Y_t \mid S_t)$。

對視覺而言，物件動態模型與相機測量模型描述不同的隨機程序。更多感測雜訊可能提高觀測熵，同時降低關於場景的有用資訊。速率計算必須說明究竟描述哪個程序。

共同典型值需要遍歷性

平穩表示分布不隨時間漂移，卻不保證一條長序列會探索混合族群的全部部分。

在適當的有限字母平穩遍歷假設下，長區塊正規化驚訝量收斂至共同的熵率值。遍歷性排除某個持續存在的隱藏選擇，把程序分成行為不同的平穩成分。

考慮只選一次成分 C：一半機率永遠產生零，一半機率產生獨立公平位元。每個成分平穩，混合也平穩。

條件區塊熵為 $H(X^n \mid C) = n/2$。恆等式

$$H(X^n) = H(X^n \mid C) + I(C; X^n)$$

使用成分標籤 C、區塊 X^n 與互資訊 I。因為 $I(C; X^n) \le H(C) = 1$，區塊平均熵趨近半位元。

然而，確定成分產生的已實現序列，其正規化驚訝量趨近零；公平成分的序列通常趨近一。多數實現不會共享接近一半的單一值。僅有熵率極限，並不足以套用遍歷 AEP 結論。

程式：平穩但非遍歷的反例

這個完整例子從一的個數解析計算混合區塊機率，避免先計算極小公平區塊機率，再取對數。

```
import numpy as np

rng = np.random.default_rng(33)
for n in [128, 1024]:
    component = rng.integers(2, size=2000)
    counts = rng.binomial(n, 0.5, size=2000) * component
    # A nonzero sequence can only come from the fair component:
    # its probability is 0.5 * 2**(-n), so surprise is n + 1.
    all_zero_surprise = -np.log2(0.5 + 0.5 * np.exp2(-float(n)))
    per_symbol = np.where(counts == 0, all_zero_surprise, n + 1) / n
    print("Length:", n, "deterministic mean:", per_symbol[component == 0].mean(),
          "fair mean:", per_symbol[component == 1].mean(),
          "fraction within 0.1 of 0.5:", np.mean(abs(per_symbol - 0.5) <= 0.1))
    assert np.mean(abs(per_symbol - 0.5) <= 0.1) == 0
```

例子描述持續的異質性，而不是時間漂移。資料集包含不同來源類型時，可能需要條件模型或明確混合，不能假設一段長紀錄代表整個族群。

聯合與條件典型性

兩條各自普通的序列，配在一起可能非常異常。通訊證明需要輸入與輸出的關係，不能只看各自的直方圖。

從聯合機率質量函數 p_{XY} 獨立取樣配對 (X_i, Y_i)，考慮聯合與條件平均驚訝量：

$$-\frac{1}{n}\log_2 p(X^n, Y^n) \to H(X, Y), \qquad -\frac{1}{n}\log_2 p(Y^n \mid X^n) \to H(Y \mid X).$$

其中箭頭表示在指定獨立配對模型下依機率收斂。X^n, Y^n 是配對區塊，每對內部允許相依。

聯合典型配對依照來源的聯合關係檢查，只有邊際典型性並不足夠。若 Y 是 X 的帶雜訊副本，兩條無關的公平位元序列可以各自有典型邊際計數，卻具有不符模型的差異頻率。

區塊資訊密度為

$$\imath(x^n; y^n) = \log_2 \frac{p(x^n, y^n)}{p(x^n)p(y^n)}.$$

其中 ι 衡量聯合配對相對於獨立配對模型的證據。在獨立配對取樣及可積分條件下，其正規化平均趨近 $I(X;Y)$。

這連結通道解碼：接收端尋找與收到區塊具有通道預期關係的碼字。它也連結配對與表徵學習，正配對必須仔細定義。從錯誤族群抽取負配對，會改變參照分布。

有限字母的強典型性另外限制各符號或配對的經驗頻率，並尊重不可能事件；弱典型性限制平均對數機率。若證明假設使用特定定義，不能在未說明下把兩者互換。

現代觀點

平均驚訝量與有意義輸出的區別，對現代機率模型尤其重要。長上下文與學習式預測器能描述固定 Markov 階數以外的相依性，損失仍是在指定觀測與詞元化模型下評分。

El Gamal 於 2026 年提出的學習章節 [3]，以散度連結預測與生成模型。它是草稿補充教材，不是已出版的第三版。從來源編碼觀點推論，序列概似應連同預測目標、單位與條件上下文一起報告。

Huang 與 Wu 的 2024 年綜述 [4] 依人類重建與機器分析的區別，整理學習式影像壓縮。這個分類凸顯壓縮表示是否有用，取決於接收者。熵率給出來源描述的限制，卻不單獨指定任務可接受的失真。

許多應用仍面對三個問題：實際區塊長度下，漸近行為多快才有用；擬合來源模型跨族群是否仍有效；以及取樣、詞元化、前處理造成的相依性，如何影響估計速率。

有限評估應比較獨立區塊或取得資料的群組，並保留非典型案例。只報告平均驚訝量，可能掩蓋速率高得多的稀有來源成分。在一個模型下測量概似，不能證明所有來源成分共享同一典型集。

下一章會實際搭起橋梁：編碼器必須定義機率模型、更新規則、終止慣例，以及未預期符號的處理。速率定理引導選擇，完整編碼再解碼的測試則檢查實作是否還原原始資料。

重點整理

- AEP 在指定來源假設下，讓正規化區塊驚訝量集中。
- 典型集機率與基數是不同量，具有不同界限。
- 單一機率最大的區塊，不一定占大多數總機率。
- 典型目錄說明幾乎無損編碼限制，不是有效率的實作。
- 熵率衡量利用相依性後，每一步新增的不確定性。
- 平穩性不保證遍歷性，也不保證共同的已實現驚訝量。
- 聯合典型性檢查邊際典型性可能漏掉的關係。
- 漸近主張必須伴隨有限區塊、模型不匹配與異質性的討論。

練習

1. 對 $p = 0.1$ 的 Bernoulli 來源，計算全零區塊的驚訝量率與來源熵，選一個能在所有長度排除全零的容許誤差。

提示

比較 $-\log_2(0.9)$ 與 $h_2(0.1)$。

2. 以容許誤差 0.05、0.1、0.2 重跑精確典型機率計算，說明較窄容許誤差為何可能需要更長區塊。

提示

平均驚訝量視窗縮窄。

3. 用不同的 ϵ 與 δ 推導典型集基數上下界，指出每一步使用哪個單一區塊機率界限。

提示

以區塊數乘最大或最小允許機率，限制總和。

4. 二元 Markov 鏈轉移矩陣為 $P = \begin{pmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{pmatrix}$。求平穩分布、邊際熵與熵率。

提示

解 $\pi = \pi P$，再加權轉移列熵。

5. 將非遍歷混合例子的成分機率改為四分之一確定來源、四分之三公平來源，求區塊熵率極限與各實現的驚訝量率極限。

提示

持續存在的成分仍阻止共同實現值。

6. 構造兩條邊際典型公平位元序列，使它們在 10% 翻轉通道下不是聯合典型。說明個別符號計數為何無法發現不匹配。

提示

使用差異過多或過少的配對。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 3–4. 出版社書籍與目錄.
2. T. Weissman, “Asymptotic Equipartition Property,” Stanford EE276 Lecture 3 notes, 2024. 原始課程講義.
3. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. 草稿與出版狀態.
4. C.-H. Huang and J.-L. Wu, “Unveiling the Future of Human and Machine Coding: A Survey of End-to-End Learned Image Compression,” *Entropy*, 26(5), 357, 2024. 已出版綜述.

IT 03・無損編碼與通用壓縮

建立可解碼的 Huffman 與算術編碼例子，推導熵限制，並連結自適應通用編碼、預測與演算法複雜度。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/lossless-universal-coding/

你將學到

- 區分單射、唯一可解碼與前綴自由描述。
- 利用 Kraft–McMillan 不等式推導熵下界。
- 建立 Huffman 編碼並測試精確重建。
- 解釋算術編碼區間，實作短訊息的精確算術往返。
- 連結自適應機率預測、通用編碼與模型額外成本。
- 區分可計算壓縮與理想的演算法複雜度。

先看全貌

壓縮用較短、接收者能解讀的表示取代訊息。關鍵是**能解讀**。若串接後的字串產生歧義，只替每個符號分配短字串還不夠。有用的編碼器必須指定訊息如何開始與結束、機率如何取得，以及接收端如何完全反轉表示。

> **用最簡單的話說**
>
> 你和朋友替玩具取綽號，常見玩具用短名字。但如果兩個綽號連在一起，看起來像另一個綽號，朋友就無法正確開箱。壓縮規則既需要短名字，也需要沒有歧義的閱讀方式。

閱讀對照是 Cover 與 Thomas 的第 5 章無損編碼、第 13 章通用來源編碼，以及第 14 章 Kolmogorov 複雜度 [1]。Stanford 的變長編碼與 Huffman 原始講義 [2,3] 提供易讀推導。本章使用原創例子，不重製書中表格或練習。

前章描述幾乎無損的典型集目錄；這裡要求精確重建每個允許訊息。成本包括未預期符號的逸出表示，以及辨識模型所需的資訊。後續率失真理論會明確放寬重建要求，不能將它混入無損主張。

無損編碼必須保留什麼

壓縮表示必須識別原始訊息，包括邊界。兩個不同訊息不能變成相同的傳送物件。

編碼器 f 與解碼器 g 滿足

$$g(f(x)) = x$$

對每個允許的來源訊息 x 成立。其中 $f(x)$ 是表示，g 是重建規則；這要求 f 在允許訊息集合上為單射。

逐符號編碼以串接各符號碼字表示來源字串。**唯一可解碼**的編碼讓不同來源序列的串接結果都不同。**前綴自由**編碼則沒有任何碼字以另一個完整碼字開頭；走到碼樹葉節點時，就能立即解碼。

前綴自由必然唯一可解碼，反向卻不一定成立。孤立符號的單射映射，甚至不保證串接後唯一可解碼。例如三個符號分別使用 0、1、01，第三個符號便無法與第一個接第二個區分。

提供分隔符可以修補歧義，但分隔符也占傳輸空間。提供訊息長度有助於判定終止，長度本身卻不會修補符號剖析歧義。解碼器使用的任何側資訊都屬於通訊協定，必須明確計入。

若來源只有一個已知符號且外部提供長度，空碼字可能成立；它不適用於下方一般多符號碼樹。例子為每個符號至少分配一位元，並假設兩端共享字母集、樹與訊息邊界。

Kraft–McMillan：碼樹的機率預算

很短的碼字占用二元碼樹的大部分空間。短碼字太多，就沒有空間放剩餘符號。

對碼長為整數 ℓ_i 的二元唯一可解碼編碼，

$$K = \sum_i 2^{-\ell_i} \le 1.$$

其中 i 是來源符號索引，ℓ_i 是符號 i 的碼字位元數，K 是 Kraft 總和。反之，滿足不等式的有限非負整數碼長配置，都能實現為前綴碼，單符號情況依通常慣例處理 [1,3]。

在前綴樹中，長度 ℓ 的葉節點，在任何足夠深的層級占全部碼樹的 $2^{-\ell}$。前綴自由使占用後代互不重疊，因此比例總和不能超過一。延伸到唯一可解碼情況，需要跨串接碼字的計數論證，不能直接把樹圖套在非前綴碼上。

不等式測試碼長是否存在，不給出唯一碼樹、位元串流格式，也不保證實際來源下的效率。許多合法碼樹的期望長度仍不必要地偏大。

熵下界

可逆二元描述的平均長度不能短於來源不確定性。看似更好的速率，通常表示成本或假設改變了。

設 p_i 是來源機率，並定義碼長誘導的正規化機率 $q_i = 2^{-\ell_i}/K$，則

$$\mathbb{E}[\ell(X)] = H(p) + D_{\mathrm{KL}}(p\|q) - \log_2 K \ge H(p).$$

其中 X 是隨機來源符號，$\mathbb{E}$ 表示期望，$H(p)$ 是熵，D_{KL} 是以位元表示的相對熵，K 是 Kraft 總和。兩個額外項都非負，因為散度非負且 $K \le 1$。

等式把低效率拆成分布不匹配與未使用的碼樹容量。完整樹滿足 $K = 1$；但 p 與整數碼長誘導的二元冪機率不匹配，仍會造成額外成本。機率符合適當完整二元樹時，才可能達到相等。

選擇 $\ell_i = \lceil -\log_2 p_i \rceil$ 滿足 Kraft 不等式，並給出

$$H(X) \le \mathbb{E}[\ell(X)] < H(X) + 1.$$

上取整符號表示向上取整數。嚴格上界適用於正機率的有限符號及通常非退化慣例。額外位元是每個編碼單位的成本；把長度 n 的來源區塊當單位，每符號取整成本就降至小於 $1/n$。

區塊編碼並非免費：區塊字母集可能指數成長，儲存表格昂貴，延遲也增加。算術編碼不用替每個可能長訊息指定獨立固定樹葉。

Huffman 編碼：合併最不可能的選項

不斷把兩個最不可能的符號合併成暫時符號。逆轉這些合併就得到二元樹，常見符號的路徑比較短。

Huffman 編碼將

$$L = \sum_i p_i \ell_i$$

在指定有限來源分布的二元前綴碼中最小化。其中 L 是期望長度，p_i 是符號機率，ℓ_i 是葉深度。最佳性只針對這個模型與逐符號編碼類別，不涵蓋所有區塊或上下文相依編碼器 [1,3]。

最佳碼樹可以讓兩個最低機率符號位於最深的兄弟葉節點。把較小機率交換到較長路徑，不會增加期望長度。合併這兩個節點後，最佳化轉成較小字母集；遞迴求解就得到貪婪演算法。

相同機率可能產生多棵同樣最佳的樹。若兩端獨立重建樹，確定性的平手規則很重要。實作用遞增計數器，讓堆積比較沒有歧義，也不用比較異質的樹物件。

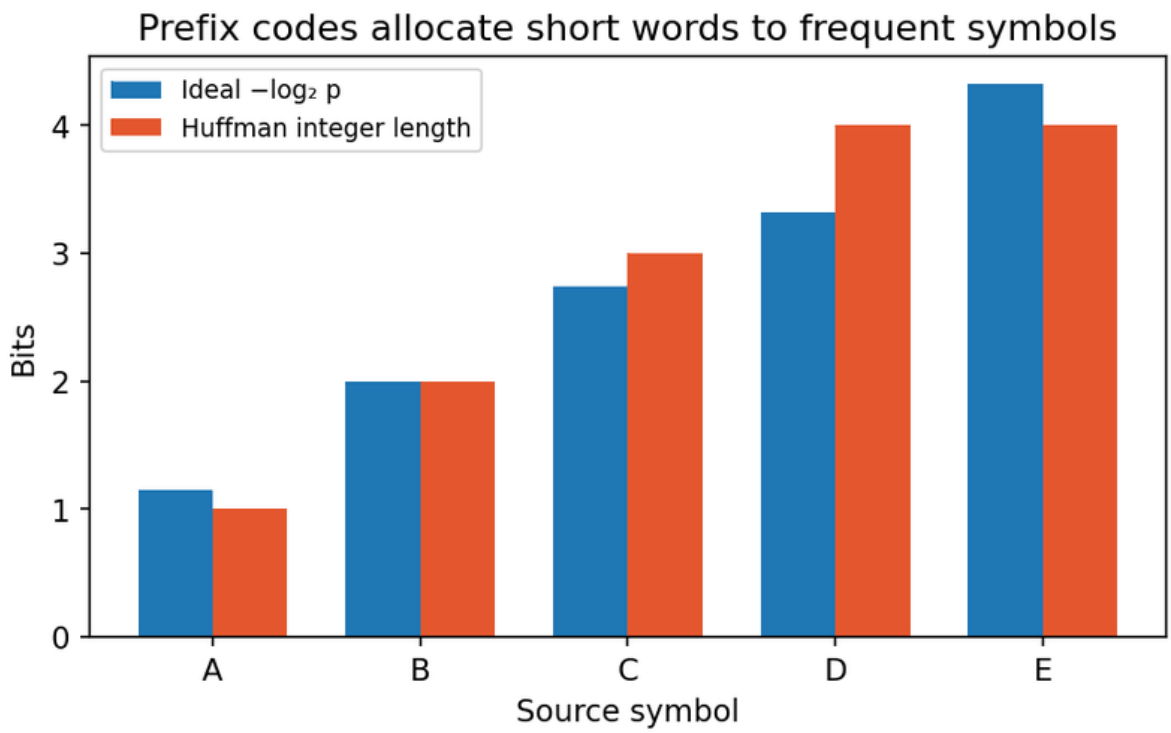


圖 *1* — 機率為 *0.45*、*0.25*、*0.15*、*0.10*、*0.05*。整數符號碼長近似理想驚訝量，但不必逐符號相等。

貪婪合併為什麼保留最佳性？

交換論證可以直接計算成本。若 $p_i \geq p_j$，但符號 i 比 j 深，交換葉標籤後的期望長度變化為

$$\Delta L = (p_i - p_j)(\ell_j - \ell_i) \leq 0.$$

p_i, p_j 是兩機率，$\ell_i > \ell_j$ 是原深度，ΔL 是交換後減交換前的成本。高機率符號放得比低機率深，從來不會有好處。

最佳二元樹可選最深葉為兄弟節點，否則多餘單分支能縮短。交換論證允許把兩個最低機率放到此兄弟葉，不增加成本。將它們替換為機率 $p_i + p_j$ 的暫時符號，深度比原葉少一，因此

$$L_{\text{original}} = L_{\text{merged}} + (p_i + p_j).$$

兩個 L 是合併前後期望長度。增加項固定，不依其餘縮小樹如何安排。因此最佳縮小樹給最佳原樹，假如原樹還有更好解，也會導出更好的縮小樹。反覆使用此論證，才是貪婪演算法的理由，不只是觀察輸出看起來合理 [1,3]。

手算五符號碼樹

機率為 0.45、0.25、0.15、0.10、0.05 時，合併順序容易手算。先合併 0.05 與 0.10，得到 0.15；再把暫時權重與原本的 0.15 合併為 0.30。接著合併 0.25 與 0.30 為 0.55，最後合併 0.45 與 0.55，得到一。

符號碼長等於葉節點到根之間經過的合併次數，結果是 1、2、3、4、4。以下是一個合法配置：

符號	機率	碼字	長度
A	0.45	`0`	1
B	0.25	`10`	2
C	0.15	`110`	3
D	0.10	`1110`	4
E	0.05	`1111`	4

這組碼字前綴自由：讀到 `0` 就確定是 A，讀到 `111` 則還需要一位元才能區分 D 與 E。交換左右子樹會改變碼字，但保留長度與期望成本。

Kraft 總和為 $1/2 + 1/4 + 1/8 + 1/16 + 1/16 = 1$。期望長度為 $0.45 \times 1 + 0.25 \times 2 + 0.15 \times 3 + 0.10 \times 4 + 0.05 \times 4 = 2$ 位元。來源熵約 1.977235 位元，因此逐符號碼每符號付出約 0.022765 位元的取整失配。五個符號也可使用定長三位元碼，但相較此期望長度，每符號多花一位元。

A 的一位元描述比理想驚訝量 $-\log_2(0.45)$ 短，其他符號會補償。定理限制機率加權平均，不要求每個碼字都至少等於自己的驚訝量。下方程式可能為平手節點選不同位元標記，但會得到同樣碼長與兩位元期望。

程式：建樹、編碼、解碼與長度計算

完整例子使用與圖相同的合成機率，檢查每對碼字的前綴自由性，並重建獨立取樣訊息。列印的負載長度排除傳送樹與長度標頭；這些排除明確說明。

```
import heapq
import itertools
import numpy as np

symbols = list("ABCDE")
p = np.array([0.45, 0.25, 0.15, 0.10, 0.05])
counter = itertools.count()
heap = [(float(w), next(counter), s) for s, w in zip(symbols, p)]
heapq.heapify(heap)
while len(heap) > 1:
    wa, _, a = heapq.heappop(heap)
    wb, _, b = heapq.heappop(heap)
    heapq.heappush(heap, (wa + wb, next(counter), (a, b)))
tree = heap[0][2]
code = {}
def visit(node, prefix=""):
    if isinstance(node, str):
        code[node] = prefix or "0"
```

```
    else:
        visit(node[0], prefix + "0")
        visit(node[1], prefix + "1")
visit(tree)
assert all(not b.startswith(a) for i, a in enumerate(code.values())
           for j, b in enumerate(code.values()) if i != j)

rng = np.random.default_rng(34)
message = "".join(rng.choice(symbols, size=2000, p=p))
payload = "".join(code[s] for s in message)
inverse = {v: k for k, v in code.items()}
decoded, prefix = [], ""
for bit in payload:
    prefix += bit
    if prefix in inverse:
        decoded.append(inverse[prefix])
        prefix = ""
assert not prefix and "".join(decoded) == message
entropy = -np.sum(p * np.log2(p))
expected = sum(w * len(code[s]) for s, w in zip(symbols, p))
assert entropy <= expected < entropy + 1
print("Code:", code)
print("Entropy / expected length:", entropy, expected)
print("Payload bits per symbol:", len(payload) / len(message))
```

熵界限討論來源下的期望長度，不是每個有限訊息。特定樣本可比期望速率更短或更長。小訊息的樹傳送成本可能占大部分；跨許多訊息共享穩定字母模型，可以攤提成本。

若由訊息本身估計機率，接收端不能直接使用該估計，除非它被傳送，或可由已解碼符號重現。先擬合直方圖、只報告負載，便隱藏真正的模型成本。

算術編碼：用區間識別整個訊息

從單位區間開始，每個符號挑選與機率成比例的子區間。反覆選擇後，剩下的小區間唯一描述整個訊息。

設 $F_t(s)$ 是符號 s 之前的累積模型機率，$q_t(s)$ 是條件機率，區間界限更新為

$$L_t = L_{t-1} + (U_{t-1} - L_{t-1})F_t(x_t),$$
$$U_t = L_{t-1} + (U_{t-1} - L_{t-1})[F_t(x_t) + q_t(x_t)].$$

其中 $[L_t, U_t)$ 是處理 t 個符號後的區間，x_t 是目前符號，F_t, q_t 在兩端使用相同已解碼歷史，初始區間是 $[0, 1)$。第二行使用舊下界，不是剛更新的下界。

最後寬度為

$$U_n - L_n = \prod_{t=1}^{n} q_t(x_t) = q(x^n).$$

其中 n 是訊息長度，$q(x^n)$ 是模型聯合機率。夠短的二元描述若能指定區間內的點或包含於其中

的二元冪區間，就能識別訊息。理想區間方法只為整個區塊付出有界的終止成本，而不逐符號取整。

實際實作使用有限整數暫存器、重新正規化與明確終止。實數不是可傳送物件，無限精度也不是產品編碼器。下方短例子刻意用精確有理數示範區間不變量，不宣稱建立完整檔案格式。

看懂前三次區間更新

使用例子中有順序的模型：A 機率 1/2，B 為 1/3，C 為 1/6，累積起點分別是 0、1/2、5/6。前綴 ABA 的更新為：

已讀符號	下界	上界	寬度
無	0	1	1
A	0	1/2	1/2
AB	1/4	5/12	1/6
ABA	1/4	1/3	1/12

讀 A 後，可用區間寬 1/2。B 選擇**該區間內**從二分之一到六分之五的部分，所以下界為 $0 + (1/2)(1/2) = 1/4$，上界為 $0 + (1/2)(5/6) = 5/12$。接著 A 選寬度 1/6 區間的下半部。最後寬度為 $1/2 \times 1/3 \times 1/2 = 1/12$，恰好等於 ABA 的模型機率。

解碼時，把標記在目前區間的位置正規化，依相同累積分割選 A、B 或 C，再更新區間並重複。模型與訊息長度讓程序可逆，小區間則解釋理想描述成本。精確有理數標記用來教機制，有限整數實作才決定實際輸出哪些位元。

程式：短訊息的精確區間往返

編碼器輸出有理數標記；解碼器另取得共享模型與訊息長度。分子與分母在此是概念側資訊，其序列化大小不宣稱等於最佳算術碼長。

```
from fractions import Fraction

model = {"A": Fraction(1, 2), "B": Fraction(1, 3), "C": Fraction(1, 6)}
message = "ABACAB"
cumulative = {}
running = Fraction(0)
for symbol, probability in model.items():
    cumulative[symbol] = running
    running += probability
assert running == 1
low, high = Fraction(0), Fraction(1)
joint = Fraction(1)
for symbol in message:
    width = high - low
    old_low = low
    low = old_low + width * cumulative[symbol]
    high = old_low + width * (cumulative[symbol] + model[symbol])
    joint *= model[symbol]
```

```
assert high - low == joint
tag = (low + high) / 2

low_d, high_d = Fraction(0), Fraction(1)
decoded = []
for _ in range(len(message)):
    position = (tag - low_d) / (high_d - low_d)
    symbol = next(s for s in model
                  if cumulative[s] <= position < cumulative[s] + model[s])
    width, old_low = high_d - low_d, low_d
    low_d = old_low + width * cumulative[symbol]
    high_d = old_low + width * (cumulative[symbol] + model[symbol])
    decoded.append(symbol)
assert "".join(decoded) == message
print("Final interval:", low, high, "width:", high - low)
print("Exact decoded message:", "".join(decoded))
```

真實符號的模型機率若為零，區間會消失。編碼器需要正機率逸出路徑，或涵蓋所有合法輸入的字母模型。只在編碼端截斷機率不夠，解碼端也必須使用相同正規化與整數量化。

模型成本是交叉熵，不只是熵

再好的位元封裝，也救不了差的機率模型。模型未預期的符號仍需長描述。

對實際區塊分布 p 與編碼模型 q，

$$\mathbb{E}_p[-\log_2 q(X^n)] = H_p(X^n) + D_{\mathrm{KL}}(p_{X^n} \| q_{X^n}).$$

其中 $\mathbb{E}_p$ 依真實來源取平均，$H_p(X^n)$ 是真實區塊熵，散度比較完整區塊分布。有限編碼器與框定成本還會增加位元。

使用先前符號的模型可以接近熵率，而不只是邊際熵。對持續的 Markov 序列使用獨立模型，就丟掉可預測的重複；更豐富上下文模型也可能過度擬合、需要過多記憶體，或對未見歷史分配脆弱機率。

評估應保留獨立紀錄或資料取得群組，報告實際序列化大小，包括標頭、模型選擇旗標與必要側串流。若預訓練權重永久共享，說明慣例；若每個訊息調適權重，計入更新成本。

有意義的比較，是共同假設下的完整編碼器與解碼器協定。某方法的理想對數機率與另一方法的實際位元組數可供診斷，但不是直接的編碼器比較。

訊息框定與不合法串流

完整實驗也指定訊息邊界如何處理。前綴樹告訴解碼器符號在哪裡結束，卻不一定告訴它整個串流何時結束。長度標頭、訊息結束符號或容器邊界，提供另外這項資訊。

填補到位元組邊界的位元，可能看起來像合法碼字。因此接收端需要有意義的位元數或原始符號數。樹例子的負載是教學用 Python 位元字串，不是省空間的位元組容器。

解碼器應拒絕最後不完整碼字與不可能轉移，不能默默回傳較短結果。完整性機制可能偵測損壞，但偵測檔案損壞與壓縮未損壞檔案是不同要求。若該機制屬於選用協定，傳送大小也必須計入。

這些細節對小訊息與公平比較最重要。漸近敘述可讓固定標頭的每符號成本忽略不計，實際短檔案仍必須支付每個標頭位元。

通用編碼邊解碼邊學習

不知道來源機率時，兩端可依已傳送符號更新相同預測器。更新不用透露未來符號。

序列模型分解為

$$q(x^n) = \prod_{t=1}^{n} q_t(x_t \mid x^{t-1}).$$

其中 x^{t-1} 是先前已解碼歷史，q_t 是正規化條件分布。編碼器觀察目前符號；解碼器從區間知道該符號後，再執行相同狀態更新。

對未知的獨立 Bernoulli 來源，Krichevsky–Trofimov 預測器使用

$$q_t(1 \mid x^{t-1}) = \frac{N_1(t-1) + 1/2}{t}, \qquad q_t(0 \mid x^{t-1}) = 1 - q_t(1 \mid x^{t-1}).$$

其中 $N_1(t-1)$ 計算先前的一，$t-1$ 是先前觀測數。分母為先前計數加一。半計數平滑使任何下一符號都具有正機率，包括第一次觀測 [1]。

這是使用 beta 半計數先驗的 Bernoulli 混合後驗預測分布。適當漸近編碼意義下，它對指定 Bernoulli 家族具有通用性，但不會自動對任意相依來源通用。

定義相對於知道真實來源的參照模型之序列遺憾：

$$\mathcal{R}_n(x^n) = -\log_2 q(x^n) + \log_2 p(x^n).$$

其中 p 是選定參照來源，q 是自適應模型。固定來源下的期望遺憾是區塊散度，非負；單一樣本的實現遺憾卻可為負，因為混合模型可能為該樣本分配比參照來源更高的機率。

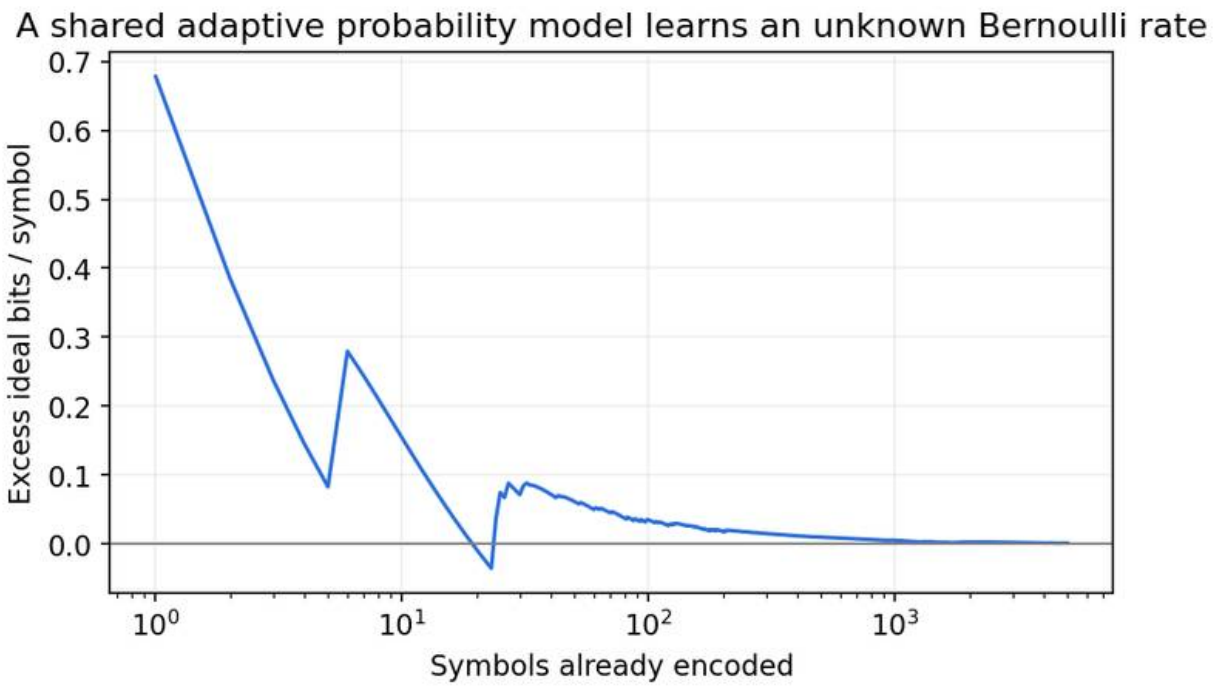


圖 2 — 固定種子的合成 *Bernoulli* 序列。曲線是每符號已實現對數損失差，不是序列化位元組數，也不是單調保證。

程式：調適與突然變動的來源

完整實驗測量理想序列長度，比較固定公平位元模型、匹配真實來源的參照模型，以及半計數調適。第二條序列在中途改變生成機率，顯示平穩計數模型的限制。

```
import numpy as np

rng = np.random.default_rng(23)
def adaptive_length(bits):
    ones, loss = 0, 0.0
    for t, bit in enumerate(bits):
        q = (ones + 0.5) / (t + 1)
        loss -= np.log2(q if bit else 1 - q)
        ones += int(bit)
    return loss

for n in [100, 1000, 5000]:
    bits = rng.binomial(1, 0.2, size=n)
    oracle = -np.sum(bits * np.log2(0.2) + (1 - bits) * np.log2(0.8))
    print("Length:", n, "oracle:", oracle / n,
          "adaptive:", adaptive_length(bits) / n, "fair model:", 1.0)
bits = np.r_[rng.binomial(1, 0.1, size=2500),
             rng.binomial(1, 0.9, size=2500)]
one_model = adaptive_length(bits)
```

```
two_models = adaptive_length(bits[:2500]) + adaptive_length(bits[2500:])
print("Changed source, whole / reset ideal rate:", one_model / 5000, two_models / 5000)
assert two_models < one_model
```

重設比較免費提供變動點。真實編碼器必須識別該位置，或使用兩端共享的預定重設時程。計入重設好處卻省略通知成本，會高估可達節省。

另有兩種不同基準：相對於固定真實機率的遺憾，以及相對於觀察序列事後最佳擬合參數的遺憾。兩者不能互換。即使來源模型正確，有限樣本也可能偏好不同於生成機率的參數。

看因果預測器逐符號學習

尚無觀測時，半計數模型替兩種位元各給 1/2 機率。若已解碼前綴是 `001`，下一個一的機率為 $(1+1/2)/(3+1) = 3/8$，零的機率為 5/8。兩端從相同前綴得到同一狀態，因此不需要傳直方圖。

模型一開始不知道來源偏差，必須支付學習成本。固定 Bernoulli 來源的計數增加時，初期成本被更多符號攤提。但交替字串的總計數公平，卻可由前一符號完全預測；純計數模型漏掉此關係，持續預測接近二分之一。Bernoulli 家族內的通用性指學未知偏差，不是學會所有序列結構。

字典、相依性與通用來源類別

字典編碼器學習重複片語，而不只符號頻率。長重複模式可以用先前片語的參照表示。

Lempel–Ziv 家族使用持續成長的已觀察字串集合建立描述 [1]。漸近通用性結果指定來源類別，例如平穩遍歷有限字母程序，也指定精確演算法；並不表示任何字典實作都在每個有限檔案上勝過每個機率模型。

字典項目、片語邊界與參照索引必須能由解碼器重現。字典增長時，索引需要更多位元。剖析出的片語個數不是壓縮長度，還需要整數編碼規則與終止格式。

不同來源結構適合不同機制。單符號預測器捕捉偏差，上下文預測器捕捉短相依性，片語字典利用重複子字串，學習模型則可利用訓練族群共享的結構。若宣稱無損，仍都需要可逆符號傳送。

通用性相對於基準與極限設定。未知平穩參數是一種問題，變動字母集或任意對抗來源則是另一種。記憶體限制、延遲、短訊息與重設規則可能主導漸近好處。

引用通用速率結果前，應問四件事：涵蓋什麼來源類別、使用何種參照長度、收斂是期望還是實現路徑，以及是否包含模型、字典與框定成本。

Kolmogorov 複雜度：描述的理想極限

想像能印出某字串並停止的最短電腦程式。有結構字串可用短程式，大多數長字串卻不行。

相對於固定通用描述機器 U，定義

$$K_U(x) = \min_{p:U(p)=x} |p|.$$

其中 x 是有限字串，p 是讓機器 U 輸出 x 並終止的程式，$|p|$ 是程式位元長度。某些版本使用前綴自由程式；機器與慣例必須說明 [1]。

不同適當通用機器之間，複雜度只差與機器相關的加法常數。但對短字串，這不表示實際數值與機器無關。更重要的是，一般情況下精確最短程式函數不可計算，不能從定義得到總能找出最佳壓縮的演算法。

把解碼器與框定計入後，無損壓縮器給出演算法描述複雜度的上界；輸出長度不是精確 Kolmogorov 複雜度。某壓縮器不能縮短字串，也不證明字串具演算法隨機性。

計數論證說明，沒有單射壓縮器能縮短每個固定長度輸入：長度 n 的字串，比所有短於 n 的二元描述更多。在誠實邊界慣例下，讓一些輸入更短的任何方法，必須使其他輸入維持原長或變長。

這不妨礙在有結構來源分布上取得優秀壓縮。壓縮利用非均勻機率或重複結構，沒有移除所有可能輸入的計數限制。

現代觀點

Yang、Mandt 與 Theis 的 2023 年已出版專論 [4]，區分無損機率編碼與有損表徵設計。其無損分類包括自迴歸、潛在變數與可逆模型，並強調模型分解與熵編碼器的相容性。我們用這個分類，追問機率如何到達可執行解碼器。

現代自迴歸網路可取代簡單計數預測器，同時保留鏈式協定。下一符號機率必須能由已解碼上下文取得。潛在變數模型需指定傳送潛在資訊的協定；低變分訓練目標不會自動等於最後位元串流長度。

El Gamal 的 2026 年擬議學習章節 [5]，以散度連結壓縮與統計學習。它仍是草稿。古典不匹配恆等式依然有用：更好的預測機率可以降低理想編碼成本，計算與同步則決定節省是否實際可用。

開放問題包括有效調適分布轉移、降低逐步解碼成本、公平比較完整格式，以及讓小訊息負擔得起模型更新。較大預測器可以節省負載，同時增加共享記憶體或計算需求。

具體評估應分開記錄三項：保留資料的負對數機率、包含必要側資訊的實際檔案大小，以及共同硬體假設下的編解碼時間。只有第二項能建立實際壓縮率，只有完整往返斷言能建立精確重建。

下一章轉換角色：來源編碼移除可預測冗餘，通道編碼加入精心設計的冗餘以抵抗雜訊。兩者都使用機率，但解決不同重建問題。

重點整理

- 無損性是具有明確邊界的精確編碼器與解碼器恆等式。
- Kraft–McMillan 限制碼長，熵限制其期望值下界。
- Huffman 對指定二元逐符號前綴問題最佳。
- 算術區間把取整成本攤到整個訊息。
- 模型不匹配先增加散度，再加有限實作額外成本。
- 自適應編碼需要同步因果更新與正機率支撐。
- 通用表現依賴指定來源類別與基準。
- 可計算壓縮長度是理想演算法描述的上界。

練習

1. 在碼字 0、1、01 下，找兩個產生相同串流的不同符號序列。說明其他配置中，提供符號數為何仍可能有歧義。

   提示

   從第三符號與前兩符號開始。

2. 驗證碼長 1、2、3、4、4 的 Kraft 總和，建立實現這些長度的前綴樹，並以例子分布比較期望長度。

   提示

   加總二的冪，配置互不為祖先的葉節點。

3. 將 Huffman 機率改成五符號均勻來源，說明平手為何改變樹而不改變最佳期望長度。

   提示

   保持確定性平手規則。

4. 將精確算術例子改成上下文相依的二元預測器，讓解碼器依已解碼符號更新，驗證最後區間寬度。

   提示

   使用精確半計數分數與相同歷史。

5. 比較半計數調適在常數、交替、獨立偏差二元字串上的結果，指出單一計數模型漏掉哪些來源結構。

   提示

   頻率沒有編碼順序。

6. 學習預測器每符號節省 0.1 理想位元，但需傳送 8,000 位元調適資訊。忽略其他變化，求抵銷額外成本所需訊息長度。

   提示

   總額外成本除以每符號節省。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 5, 13–14. 出版社書籍與目錄.
2. T. Weissman, “Variable Length Lossless Compression,” Stanford EE276 Lecture 4 notes, 2024. 原始課程講義.
3. T. Weissman, “Kraft-McMillan Inequality and Huffman Coding,” Stanford EE276 Lecture 5 notes, 2024. 原始課程講義.
4. Y. Yang, S. Mandt and L. Theis, “An Introduction to Neural Data Compression,” *Foundations and Trends in Computer Graphics and Vision*, 15(2), 113–200, 2023. 已出版專論作者版本.
5. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. 草稿與出版狀態.

IT 04・通道容量與可靠通訊

推導有限通道容量，說明編碼可達性與 Fano 反向界限，並用 Blahut–Arimoto 計算最佳訊號分布。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/channel-capacity/

你將學到

- 定義離散無記憶通道，區分訊息與通道符號。
- 推導二元對稱通道與二元抹除通道的容量。
- 透過聯合典型性與配置直覺理解隨機編碼可達性。
- 利用 Fano 不等式推導可靠速率的反向界限。
- 以 Blahut–Arimoto 計算容量與最佳性差距。
- 區分漸近限制、有限區塊可靠性與任務導向通訊。

先看全貌

雜訊通道不必正確還原每個傳送符號，接收者仍可能恢復訊息。編碼將每個訊息表示成較長且有結構的序列；接收端把帶雜訊觀測與可能序列比較，判斷送出哪個訊息。

> **用最簡單的話說**
>
> 朋友可能聽錯一些字母，傳送精心選擇的較長綽號就能幫忙。額外字母讓不同綽號較難混淆。但在同一個帶雜訊空間塞進太多不同綽號，最後還是會重疊。

來源編碼移除可預測冗餘，通道編碼增加有用冗餘；兩者目標不同。重複位元是簡單通道編碼，卻不一定有效利用通道。核心問題是每次通道使用能傳多少不同訊息，同時讓重建錯誤趨近零。

主要閱讀是 Cover 與 Thomas 第 7 章；書中的計算討論將容量最佳化與 Blahut–Arimoto 方法連結 [1]。Stanford 的通訊與反向定理講義 [2,3] 提供原始推導。我們先建立假設與數值實驗，再說明實際短訊息為何需要額外評估。

先定義通道，再談容量

通道指定輸入符號如何隨機轉成輸出符號。來源分布與通道轉換是不同的對象。

對有限輸入字母集 $\mathcal{X}$ 與輸出字母集 $\mathcal{Y}$，離散無記憶通道由列隨機矩陣表示：

$$W_{xy} = \Pr(Y = y \mid X = x), \qquad p(y^n \mid x^n) = \prod_{t=1}^{n} W_{x_t y_t}.$$

其中 x, y 是符號索引，$W_{xy} \geq 0$ 且每列總和為一。區塊 x^n, y^n 長度為 n，t 索引一次使用。分解定義無記憶性與不隨時間改變的轉移律。

無記憶性描述**給定整個輸入區塊後**的輸出。編碼輸入符號可能相依，因為它們都由同一訊息決定；未條件化的輸出也可能因此相依。證明反向界限時假設輸入獨立，會不必要地限制允許編碼。

單符號計算中，發送者選擇輸入分布 $q_x = \Pr(X = x)$，誘導輸出機率

$$r_y = \sum_x q_x W_{xy}.$$

其中 r_y 是輸出邊際。改變 q 時，通道 W 不變。更改傳播律、接收觀測、資源限制或可用回饋，便定義不同通訊問題。

相同的轉移列無法區分對應輸入。不同列也會因重疊程度而較易或較難區分。輸出字母大小給出粗略資訊上界，實際可利用的區別則由轉移機率決定。

速率與可靠性是操作量

通訊速率是訊息資訊除以通道使用次數。可靠性討論恢復整個訊息，不只是讓多數通道符號正確。

設訊息 M 在 N 個選項中均勻分布，長度 n 編碼把它映至 X^n，再從收到的 Y^n 估計 $\widehat{M}$。定義

$$R_n = \frac{\log_2 N}{n}, \qquad P_e^{(n)} = \Pr(\widehat{M} \neq M).$$

其中 N 是碼本大小，R_n 是每次使用的位元數，$P_e^{(n)}$ 是指定均勻訊息與通道下的平均訊息錯誤機率。它不同於符號錯誤率。

若存在一系列編碼，使漸近速率至少為 R 且訊息錯誤趨近零，則 R 可達。容量是可達速率的上確界：

$$C = \sup\{R : R \text{ is achievable}\} = \max_q I_q(X; Y).$$

其中 q 遍歷有限字母集上的所有輸入分布，I_q 是聯合分布 $q_x W_{xy}$ 下的互資訊。第二個等號是指定有限無記憶通道的通道編碼定理 [1,2]。

標準可達敘述討論嚴格低於容量的速率，不保證有限長度零錯誤，也不提供快速解碼器。容量也不同於**零錯誤容量**：後者要求允許輸出集合永遠不混淆訊息。某通道可以有正的一般容量，但每個有限區塊仍有非零混淆機率。

平均錯誤設定假設均勻訊息。所有訊息的最大錯誤是另一種操作準則；標準有限無記憶結果可移除差的碼字，僅付出漸近可忽略的速率損失，連結兩種容量設定。有限長度可靠性統計仍可能不同。

二元對稱通道

每個位元以相同機率獨立翻轉。公平輸入充分利用兩種輸出選項。

翻轉機率為 e 的二元對稱通道滿足

$$W = \begin{pmatrix} 1-e & e \\ e & 1-e \end{pmatrix}, \qquad C_{\mathrm{BSC}} = 1 - h_2(e).$$

其中 $e \in [0,1]$ 是翻轉機率，h_2 是二元熵，容量單位為每次使用的位元數。任一輸入的條件輸出熵為 $h_2(e)$。因為 $H(Y) \le 1$，互資訊至多 $1 - h_2(e)$；公平輸入達到此界。

$e = 0$ 時無雜訊傳送一位元；$e = 1/2$ 時輸出與輸入獨立，容量為零；$e = 1$ 時每個位元確定翻轉，接收端反轉即可，容量又為一。整個區間的雜訊程度不能只看原始翻轉比例。

圖將翻轉機率限制於零至二分之一，這段範圍自然對應不確定性增加；另一半分支可透過重新標記輸出得到。

二元抹除通道

接收者不是看到正確位元，就是看到明確的缺失標記。知道哪些觀測缺失，本身就有用。

抹除機率為 e 的二元抹除通道，其輸出是零、一與抹除符號：

$$W = \begin{pmatrix} 1-e & 0 & e \\ 0 & 1-e & e \end{pmatrix}, \qquad C_{\mathrm{BEC}} = 1 - e.$$

其中列對應輸入零與一，欄對應兩種正確符號與抹除。抹除事件與輸入獨立。

給定未抹除輸出，輸入已知；給定抹除，不確定性仍為 $H(X)$。因此 $I(X;Y) = (1-e)H(X)$，公平輸入達到每次使用 $1-e$ 位元。

相同數值的事件機率，對翻轉與可見抹除產生不同容量。抹除告訴接收者資訊在哪裡缺失，未標記翻轉卻可能暗示錯誤選項。差異來自不同輸出律，不是所有實體通訊系統的普遍排名。

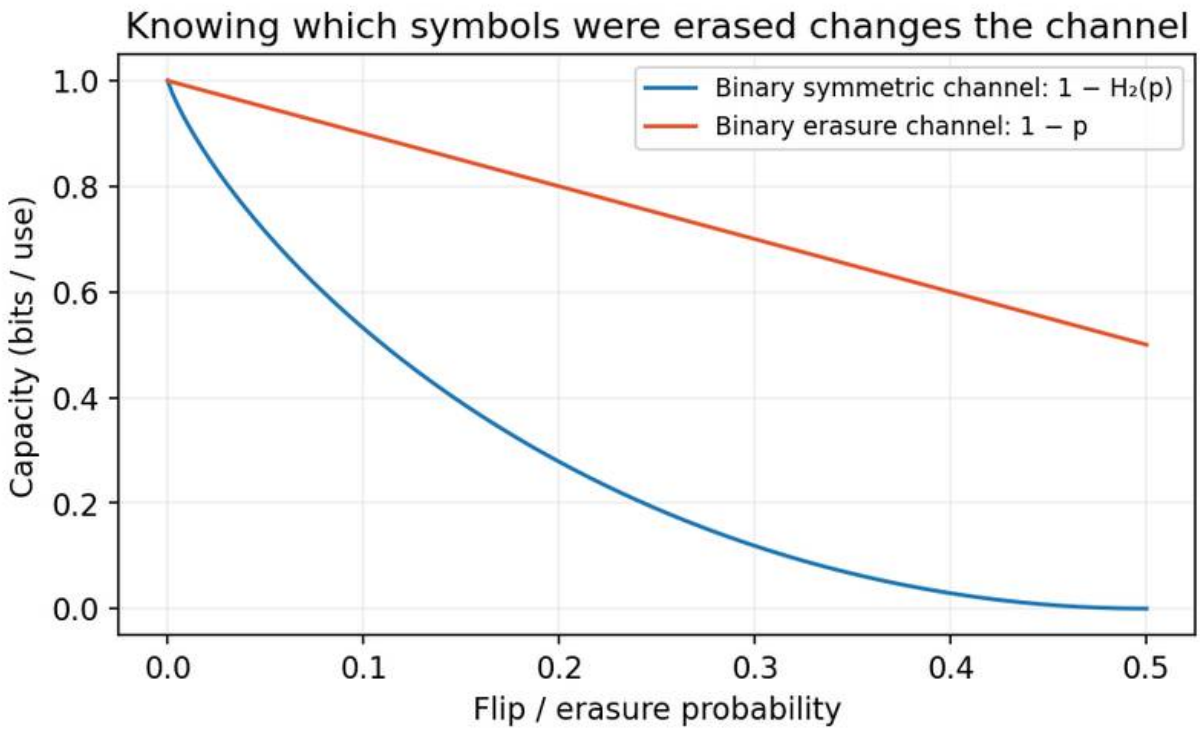


圖 *1* — 每次使用一個二元輸入的解析容量。可見抹除與未標記翻轉是不同通道模型。

不對稱通道需要不對稱訊號分布

一種輸入比另一種可靠時，均勻分布未必最佳。容量最佳化各輸入使用頻率。

考慮零正確到達，而一以機率 a 變成零的 Z 通道：

$$W = \begin{pmatrix} 1 & 0 \\ a & 1-a \end{pmatrix}, \qquad I(q) = h_2(q(1-a)) - q\,h_2(a).$$

其中 a 是一轉零機率，$q = \Pr(X = 1)$ 是純量輸入參數。輸出一機率為 $q(1-a)$，條件輸出熵為 $qh_2(a)$。

第一項鼓勵多樣輸出，第二項計入使用一時付出的雜訊成本。權衡可能使最大值偏離 $q = 1/2$。只傳可靠的零也沒有用：常數輸入不攜帶訊息資訊。

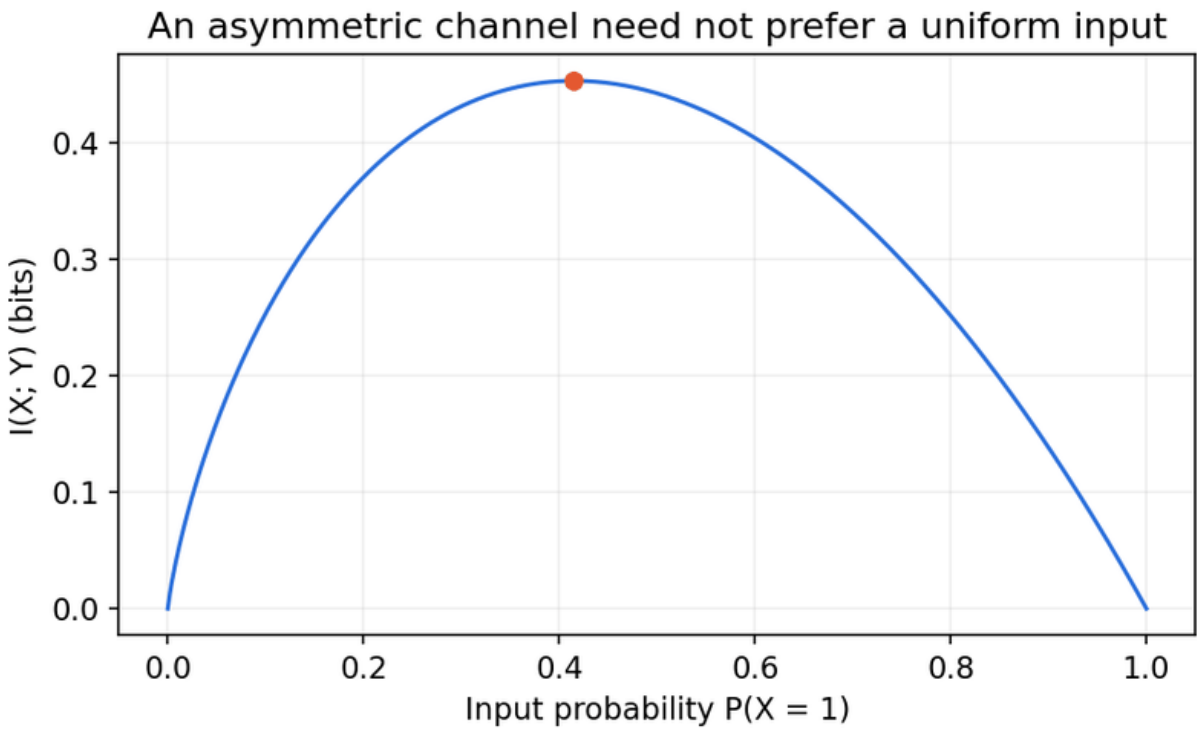


圖 2 — 輸入分布的解析最佳化。某符號較可靠，不代表應專門傳送它。

互資訊對輸入分布為凹函數，因為輸出熵是凹的，條件輸出熵是線性的，因此數值最佳化可行。不同輸入冗餘時，仍可能有平坦最大區域，最佳分布不必唯一。

為何低於容量的速率可達

選許多看起來像合法來源區塊的長碼字。收到的區塊通常符合真正碼字的通道關係，卻很少符合無關碼字。

固定輸入分布 q，從 q^n 獨立產生約 2^{nR} 個碼字。聯合典型解碼器依聯合律 $q_x W_{xy}$，尋找唯一符合收到輸出的碼字 [1]。

主要有兩種錯誤：真正輸入輸出配對未通過典型測試，或另一個碼字通過。第一種機率由聯合 AEP 趨近零；獨立競爭碼字的聯合典型機率，其指數尺度約為 $2^{-nI_q(X;Y)}$。

對競爭碼字使用聯集界限，得到示意尺度

$$P_{\text{competing}} \lesssim 2^{nR} 2^{-n(I_q(X;Y)-\eta)}.$$

其中 $P_{\text{competing}}$ 是出現競爭典型碼字的機率，$\eta > 0$ 吸收典型性容許誤差，$\lesssim$ 省略有限前因子與細節。這是證明直覺，不是精確有限長度效能公式。

若 $R < I_q(X;Y) - \eta$，競爭錯誤界限趨近零。選擇最大化輸入並令容許誤差夠小，便得到低於容量的速率。隨機碼本的平均錯誤很小，表示至少有一個確定碼本錯誤很小。

論證不需要每次訊息傳送隨機種子。碼本在通訊前已固定，兩端皆已知。若實際系統選擇或傳送新碼本，設定協定需另外計算。

隨機編碼證明存在性。逐一搜尋指數大小碼本通常不可行；典型性不是部署解碼器的直接配方。容量定理指出目標，實際結構化編碼另外解決複雜度問題。

Fano 不等式限制接收端推論

接收者經常猜錯，就仍有訊息不確定性。反之，低錯誤機率意味很少剩餘的條件訊息不確定性。

對 $N \geq 2$ 個可能訊息，以及任何根據 Y^n 的估計，

$$H(M \mid Y^n) \leq h_2(P_e) + P_e \log_2(N-1) \leq 1 + P_e \log_2 N.$$

其中 M 是真訊息，Y^n 是收到區塊，$P_e = \Pr(\widehat{M} \neq M)$，$h_2$ 是二元熵。這是 Fano 不等式 [1,3]。

引入錯誤指標便能理解邏輯：描述是否錯誤，至多需要其二元熵。沒有錯誤時，估計識別訊息；有錯誤時，至多剩 $N-1$ 個選項。平均後得到第一個界限。

均勻訊息滿足 $H(M) = \log_2 N$。資料處理與通道無記憶性限制

$$I(M; Y^n) \leq I(X^n; Y^n) \leq \sum_{t=1}^{n} I(X_t; Y_t) \leq nC.$$

其中 X^n 是傳送碼字。中間不等式允許碼符號相依：輸出熵至多為各邊際輸出熵總和，條件輸出熵則在無記憶通道下分離。

合併兩個界限得到

$$(1 - P_e)\log_2 N \leq nC + 1, \qquad P_e \geq 1 - \frac{C}{R_n} - \frac{1}{nR_n}.$$

其中 $R_n = \log_2 N / n$，只有右側為正時下界才有意義。固定速率嚴格超過容量時，錯誤不可能趨近零。

這個基本反向證明建立超過容量的正錯誤底限，單獨並未證明錯誤趨近一。強反向敘述需要額外定理與假設。應精確說明顯示的不等式到底證明什麼。

手算速率、錯誤與來源預算

翻轉機率 $e = 0.1$ 時，BSC 容量為 $1 - h_2(0.1) = 0.531004$ 位元／次使用。同樣抹除機率的 BEC 容量是 0.9。差異來自觀測模型：BEC 會揭露哪些符號遺失。

長度 100、碼率 0.8 的編碼共有 2^{80} 種可能訊息。把 BSC 容量代入基本逆定理：

$$P_e \geq 1 - \frac{0.5310044064}{0.8} - \frac{1}{100 \times 0.8} \approx 0.323744.$$

P_e 是平均整訊息錯誤，100 是通道使用數，0.8 是每次使用的訊息位元。代入容量保留較多小數位，再將答案取整。在指定通道下，無論解碼器如何，這個編碼都不可能達成 1% 訊息錯誤目標。但此界限不決定任何特定編碼的精確錯誤。

碼率改成 0.5 時，同一界限為負，因此只剩平凡下界零。它與低於容量的漸近可達性相容，卻不證明長度 100 已經有足夠可靠性。兩個問題需要不同證據：區塊持續增長的定理，以及有限編碼計算或實驗。

再取一個一的機率為 0.2 的獨立二元來源，每符號熵為 0.721928 位元。漸近分離設定下，要透過此 BSC 幾乎無損傳送，需要每來源符號超過 $0.721928/0.531004 \approx 1.359552$ 次通道使用。一次不足，兩次則有速率餘裕。若先用原始表示，每來源符號需要一位元，再加通道保護；這說明利用來源偏差與抵抗雜訊是不同設計步驟。

頻寬比例不必是整數。長區塊可以分配相應通道使用總數，實現有理數或近似比例；延遲與有限區塊額外成本仍是另外的限制。

Blahut–Arimoto 容量計算

從輸入混合開始，檢查每個通道列與其輸出混合有多能區分，再把機率移向較有資訊的列。重複直到上下界接近。

對目前正輸入機率 q_x 與誘導 r_y，以自然單位定義列散度：

$$d_x = \sum_{y:W_{xy}>0} W_{xy} \ln \frac{W_{xy}}{r_y}, \qquad q_x^{\mathrm{new}} = \frac{q_x e^{d_x}}{\sum_j q_j e^{d_j}}.$$

其中 d_x 是通道列 x 與目前輸出分布的散度，$\ln$ 是自然對數，j 索引輸入。分母使用同一目前分布，更新不會將它換成均勻先驗。自然單位除以 $\ln 2$ 轉成位元。

目前分布的上下界為

$$L(q) = \frac{\sum_x q_x d_x}{\ln 2}, \qquad U(q) = \frac{\max_x d_x}{\ln 2}, \qquad L(q) \le C \le U(q).$$

其中 $L(q)$ 是目前互資訊，$U(q)$ 是以目前輸出混合作候選參照分布得到的對偶形式界限。差距可作為數值停止準則。

這個無限制演算法適用於固定有限通道。輸入成本限制需要修改最佳化，不能先算無限制容量，再宣稱符合功率或符號使用預算。

數值上界為什麼成立？

停止間隙有用的前提，是兩端真的夾住最佳值。固定目前輸出混合 r，考慮**任意**候選輸入分布 u，其輸出混合為 $r_u = uW$。展開對數比得到

$$\sum_x u_x D_{\mathrm{nats}}(W_x \| r) = I_{u,\mathrm{nats}}(X;Y) + D_{\mathrm{nats}}(r_u \| r).$$

W_x 是通道列 x，u_x 是候選使用機率，r 是固定參照輸出，r_u 是候選輸出邊際，$I_{u,\mathrm{nats}}$ 是 u 下的互資訊。整個恆等式使用自然對數。

最後散度非負，因此候選互資訊不超過左側加權總和；該總和又不超過最大的列散度。因為對所有 u 都成立，目前最大列散度就是容量上界。目前輸入本身的互資訊，是單字母最佳化可行值，提供下界。

最佳點分配正機率的輸入，具有相同資訊列散度；未使用輸入也不能提供更高邊際增益。更新把機率移向目前散度較高的列，直到這些條件近乎平衡。因此，間隙衡量此矩陣還可能改善多少，不只是最新參數變動有多小。

程式：驗證解析容量與最佳化 Z 通道

完整實作移除所有輸入都不可能產生的輸出欄，遮罩零列元素，並用穩定對數正規化。回傳輸入分布與其對應下界速率。

```
import numpy as np
from scipy.special import logsumexp
from scipy.optimize import minimize_scalar

def binary_entropy(p):
    if p == 0 or p == 1:
        return 0.0
    return -p * np.log2(p) - (1 - p) * np.log2(1 - p)

def capacity(W, tolerance=1e-10, maximum=20000):
    W = np.asarray(W, dtype=float)
    assert W.ndim == 2 and np.all(W >= 0)
    assert np.allclose(W.sum(axis=1), 1)
    W = W[:, W.sum(axis=0) > 0]
    q = np.full(W.shape[0], 1 / W.shape[0])
    positive = W > 0
    for iteration in range(maximum):
        r = q @ W
        ratio = np.zeros_like(W)
        np.log(np.divide(W, r, out=np.ones_like(W), where=positive),
               out=ratio, where=positive)
        d = (W * ratio).sum(axis=1)
        lower, upper = q @ d / np.log(2), d.max() / np.log(2)
        if upper - lower < tolerance:
            return lower, q, upper - lower, iteration + 1
        log_q = np.log(q) + d
        q = np.exp(log_q - logsumexp(log_q))
    raise RuntimeError("Capacity iteration did not meet the requested gap")

e = 0.1
bsc = np.array([[1-e, e], [e, 1-e]])
bec = np.array([[1-e, 0, e], [0, 1-e, e]])
for W, analytic in [(bsc, 1-binary_entropy(e)), (bec, 1-e)]:
    value, q, gap, steps = capacity(W)
    print("Capacity / analytic / input / gap:", value, analytic, q, gap)
    assert abs(value - analytic) < 1e-8
a = 0.35
z = np.array([[1, 0], [a, 1-a]])
value, q, gap, steps = capacity(z)
```

```
objective = lambda p: binary_entropy(p*(1-a)) - p*binary_entropy(a)
reference = minimize_scalar(lambda p: -objective(p), bounds=(0, 1), method="bounded")
assert abs(value + reference.fun) < 1e-8
print("Z capacity / P(input one) / gap / iterations:", value, q[1], gap, steps)
```

與解析二元例子一致，能檢查單位與矩陣方向；與獨立純量最佳化一致，能檢查不對稱例子。這些測試可發現誤用欄隨機機率、漏掉目前 q 因子，或混用自然與二進位對數。

幾乎冗餘輸入與極端機率可能收斂慢或造成浮點下溢。小差距是計算精度內，對這個有限矩陣的數值證據，不證明矩陣準確描述真實通道。

重複實驗顯示有限區塊權衡

重複一位元能幫助多數決解碼，但每個訊息位元消耗更多使用次數。可靠性較高，不自動代表容量效率好。

位元在獨立 BSC 中重複奇數 k 次，使用多數決，則

$$P_e(k) = \sum_{j=(k+1)/2}^{k} \binom{k}{j} e^j (1-e)^{k-j}, \qquad R = \frac{1}{k}.$$

其中 j 是翻轉副本數，e 是翻轉機率，$P_e(k)$ 是單位元訊息錯誤，R 是碼率。這是精確有限編碼計算，不是容量公式。

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(35)
e = 0.1
for copies in [1, 3, 5, 9]:
    noise = rng.binomial(1, e, size=(50000, copies))
    observed_error = np.mean(noise.sum(axis=1) > copies // 2)
    exact_error = binom.sf(copies // 2, copies, e)
    print("Copies / rate / simulated error / exact error:",
          copies, 1 / copies, observed_error, exact_error)
    assert abs(observed_error - exact_error) < 0.005
```

模擬可假設訊息位元為零，因為對稱編碼對任一傳送位元有相同錯誤機率。更多副本降低此單位元問題的錯誤，同時使速率趨近零。容量則允許碼本以正漸近速率攜帶持續增加的位元數。

有限評估應指定區塊長度、速率、訊息錯誤準則、通道參數與解碼規則。只報告容量，無法描述短編碼的延遲、複雜度或可靠性。

資源限制與不同通道假設

容量計算回答一個精確問題。更改發送者預算、加入記憶或提供不同觀測，問題就改變了。

若輸入 x 的非負成本為 $c(x)$，每次使用的平均預算為 B，有限無記憶通道的限制問題為

$$C(B) = \max_{q:\sum_x q_x c(x) \le B} I_q(X;Y).$$

其中 $c(x)$ 是輸入使用成本，B 是允許平均，q 是滿足限制的機率分布。通道仍是相同矩陣。公式採平均成本慣例；每個符號的嚴格峰值限制定義不同可行集合。

對不對稱二元通道，輸入一的頻率限制可能排除無限制最佳分布。結果應是容量與成本函數，不能將功率標籤貼在無限制數字上。

每秒位元的吞吐量也需要指定每秒通道使用次數。每次使用位元數，不自動等於每個樣本、像素或詞元的位元數。比較調變或表示選擇時，說明每種選擇如何改變使用次數與資源。

回饋允許編碼器先觀察過去通道輸出，再選下一輸入。一般無記憶點對點模型中，無雜訊因果回饋不增加漸近容量 [1,3]，仍可改變有限編碼設計、停止規則與可達錯誤行為。不能未檢查就移用至有記憶或多使用者通道。

持續隱藏雜訊狀態可能造成群聚錯誤，而不是獨立翻轉。合併全部使用估計單一矩陣，會隱藏相依性。上述容量證明使用條件乘積律，不能直接把群聚錯誤當成平均翻轉機率的獨立取樣。

接收端側資訊也會造成類似差異。若側資訊揭露抹除位置、雜訊狀態或部分原訊息，就改變接收觀測。應比較完整觀測模型，不能讓解碼器利用基準沒有取得的資訊，卻把好處全部歸給解碼方法。

通道估計是另一個實驗

數值最佳化可能對錯誤矩陣非常準確。相信速率前，先判斷轉移機率是否描述操作條件。

對帶輸入標籤的輸入輸出觀測，簡單列估計為

$$\widehat{W}_{xy} = \frac{N_{xy}}{\sum_z N_{xz}}.$$

其中 N_{xy} 計算輸入 x 與輸出 y 的配對次數，z 遍歷輸出，$\widehat{W}$ 是經驗通道矩陣。每個輸入列都需觀測；經驗零次數不證明事件不可能。

有限樣本使估計容量與最佳分布都不確定。跨獨立取得資料重複實驗可揭露變異；依已知操作條件分組可揭露不同通道律。最佳化差距衡量一個矩陣的計算收斂，實驗不確定性則討論該最佳化哪個矩陣。

若從資料擬合解碼器或輸入策略，訓練與評估也應使用分開觀測。在同一個極小樣本選擇有利輸入分布並估計表現，容易得到樂觀速率。

有用報告因此包含取樣協定、各輸入次數、操作條件、經驗列機率與保留資料重建結果。稀有輸出需明確處理，因為把它們當成零機率，會扭曲解碼器與資訊估計。

來源資訊如何配合通道資訊

來源編碼把有用不確定性打包成位元，通道編碼可靠地運送位元。兩者速率必須使用相同來源與通道使用預算比較。

若有限無記憶來源每符號熵為 $H(S)$，每來源符號可使用 b 次通道，熟悉的漸近分離條件為

$$H(S) < bC.$$

其中 S 是來源符號，b 是使用比例，C 是點對點容量。在標準長區塊假設下，來源與通道編碼能各自以餘裕接近限制 [1]。

不等式未指定實際區塊長度，也不要求有限延遲下獨立設計最佳；更未宣稱所有網路都有一般來源通道分離。保留假設，才能避免把漂亮公式套至無關限制。

有損重建需要的來源速率改為率失真函數，而不是完整熵；連續振幅通道的容量由實體限制決定。這兩個延伸引出後續章節。

現代觀點

Nguyen 等人的 2025 年已出版綜述 [4]，將語意通訊分成接收者意圖模型、生成模型與深度聯合來源通道編碼。分類依系統想保留什麼、兩端如何協調區分，而非把所有學習系統當成相同編碼器。

這些目標可能改變所需重建。任務接收者也許只需要類別標籤，而非全部原始像素，因此需要的來源資訊可能減少；但在通道與限制不變時，Shannon 容量不會增加。

生成式重建也需要明確準則。合理輸出與原始內容的關係，可能不同於精確解碼訊息。明確評估任務準確度或感知失真很合理，但不能用它們取代位元可靠性主張中的訊息錯誤機率。

從古典模型推論，應分清三層：來源接收者需要什麼、實體通道傳遞什麼，以及編碼如何使用有限資源。聯合最佳化可改善實際匹配，無須推翻對應限制。

開放問題包含通道律變動時的穩健表現、短區塊可靠操作、共享模型同步，以及跨任務與接收者知識的評估。一種任務或通道的綜述結果，不建立通用語意速率。

下一章將字母從有限符號改成連續振幅，先建立微分熵與功率限制，再推導 Gaussian 容量。把連續密度直接代入離散熵公式，會漏掉重要的單位依賴。

重點整理

- 無記憶通道在給定輸入下分解輸出，碼符號不必獨立。
- 容量是可靠訊息速率，在指定假設下等於最佳化互資訊。
- 翻轉與可見抹除有不同通道容量。
- 不對稱通道可能偏好非均勻輸入。
- 隨機編碼以典型性與配置建立存在性。
- Fano 與資料處理限制訊息錯誤趨零的速率。
- Blahut–Arimoto 最佳化有限通道並提供數值差距。

- 有限可靠性與語意目標各需明確評估。

練習

1. 說明翻轉機率 0.9 與 0.1 的 BSC 容量為何相同，描述接收端如何重新標記輸出。

   提示

   反轉每個輸出。

2. 對任意二元輸入機率推導 BEC 互資訊，不預設公平輸入。

   提示

   抹除保留原輸入熵。

3. 將 Z 通道參數改成 0.1 與 0.7，比較最佳輸入機率與二分之一，解釋兩個熵項的競爭。

   提示

   最佳化輸出熵減加權列熵。

4. 長度 100、碼率 0.8 的編碼通過翻轉機率 0.1 的 BSC，計算基本 Fano 訊息錯誤下界，說明它證明與未證明的事。

   提示

   使用 $1 - C/R - 1/(nR)$，負值時保留零下界。

5. 複製通道矩陣某一列並重跑容量，解釋容量為何不變，最佳輸入混合卻可能不再唯一。

   提示

   相同列產生相同條件輸出。

6. 在相同總使用預算下比較重複長度 3 與 9，說明單位元錯誤較小，為何不能單獨判斷系統可靠傳送的資訊較多。

   提示

   同時計算訊息位元與錯誤。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapter 7 and the computational capacity/rate–distortion discussion. 出版社書籍與目錄.
2. T. Weissman, “Communication and Channel Capacity,” Stanford EE276 Lecture 7 notes, 2024. 原始課程講義.
3. T. Weissman, “Channel Coding Theorem: Converse Part,” Stanford EE276 Lecture 10 notes, 2024. 原始課程講義.
4. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, “A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding,” *IEEE Communications Surveys & Tutorials*, 2025, DOI: 10.1109/COMST.2025.3616973. 已出版論文資料與作者版本.

IT 05・微分熵與高斯通道

以明確單位處理連續資訊，推導功率預算下的高斯容量，並用注水法最佳化平行通道。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/continuous-gaussian-channels/

你將學到

- 定義微分熵，避免把密度當成機率。
- 追蹤單位與座標改變，同時保留互資訊。
- 從相對熵推導高斯最大熵性質。
- 求平均功率預算下的實數高斯通道容量。
- 以注水法配置平行通道功率。
- 區分解析資訊極限、估計與實作成本。

先看全貌

振幅可以在連續範圍取值，但有雜訊的接收器不能免費區分無限多訊息。更精細的量測需要資源，發送器通常也不能使用任意大的振幅。連續資訊理論會把這些限制明確保留。

> **用最簡單的話說**
>
> 你可以指向尺上的任何地方，但朋友模糊的視力無法分辨每個細小差異。把位置分遠一點有幫助，可是尺的大小有限。真正問題是：在可用空間內，能放入多少可靠區分的訊號？

本章依 Cover 與 Thomas 第 8–9 章 [1]，並補充 Stanford 的連續變數與高斯通道講義 [2,3]。順序很重要：先定義以密度為基礎的資訊，再證明分布上界，最後用它最佳化功率限制下的通訊。

連續公式也出現在視覺特徵、潛在表示與感測器雜訊模型。即使場景不變，數值仍可能隨單位改變。比較關係時，互資訊是更穩定的量。

密度不是點機率

連續密度描述每單位量測的機率。密度大於一完全可能，並不違反機率規則。

對具有密度 f_X 的實數隨機變數 X，

$$\Pr(X \in A) = \int_A f_X(x)\,dx, \qquad \int_{\mathbb{R}} f_X(x)\,dx = 1.$$

A 是可測區域，x 是所選單位的座標，$f_X(x)$ 是密度，不是精確點的機率。一般連續密度下，每個精確點的機率都為零。

寬度十分之一區間上的均勻變數，密度為十；整段區間的機率仍是一。因此，對密度取負對數可能得到負值。

微分熵（differential entropy）為

$$h(X) = -\int f_X(x) \log_2 f_X(x)\, dx.$$

h 是底數二的微分熵，積分在密度定義處進行。此式假設積分以適當意義存在。它可能負、無窮，有些分布沒有良好定義的有限熵。

與離散熵不同，微分熵本身不是精確表示連續值所需的最少位元數。精確無限精度重建，是另一個通常無界的要求。

符號刻意以小寫 h 表示微分熵、大寫 H 表示離散熵。適當條件下，它們有相似代數關係，解釋與符號限制卻不同。

量化連結到實際符號

真實儀器記錄的是格子，不是無限精確的點。格子越小，描述格子索引需要越多位元。

把純量密度量化成寬度 $\Delta > 0$ 的格子。密度足夠規則且量化夠細時，

$$H(X_\Delta) \approx h(X) + \log_2 \frac{1}{\Delta}.$$

X_Δ 是離散格子索引，Δ 使用與 x 相同的座標單位。這是細格子漸近關係，受規則性與尾端條件限制。

格子機率約等於密度乘格寬，因此負對數機率約等於負對數密度減去對數格寬。平均後就得到此式。

額外項不是小修正。解析度增加時，它會發散。有限微分熵不會讓任意精確值變得便宜。

粗解析度、尖銳不連續或厚尾下，近似可能不好。真實量化器也可能飽和、格子不均勻，或範圍有限。這些細節會改變離散分布，實際編碼實驗應納入。

對感測器，解析度與物理雜訊都重要。儲存數值精度超過量測精度，不一定創造新場景資訊，可能只是多了表示不確定振幅的數字。

程式：座標單位與量化熵

獨立實驗使用單位均勻變數，在不同解析度計算經驗格子索引熵，並對照縮放後解析微分熵變化。

```
import numpy as np

rng = np.random.default_rng(36)
x = rng.uniform(0, 1, size=100000)
def discrete_entropy(values):
    _, counts = np.unique(values, return_counts=True)
```

```
    p = counts / counts.sum()
    return -np.sum(p * np.log2(p))
for bins in [10, 100, 1000]:
    indices = np.floor(x * bins).astype(int)
    estimated = discrete_entropy(indices)
    theoretical = np.log2(bins)
    print("Bins / estimated index entropy / exact:", bins, estimated, theoretical)
    assert abs(estimated - theoretical) < 0.03
scale = 0.1
h_x, h_scaled = 0.0, np.log2(scale)
print("Differential entropy before / after scaling:", h_x, h_scaled)
assert h_scaled < 0
```

負的縮放熵，不是離散訊息的負不確定性；它反映每單位座標的密度。有限格子估計仍非負，而且罕見格子變多時，抽樣偏差會增加。

座標改變會改變微分熵

同樣長度用毫米而非公尺表示，會把數值分布拉伸，微分熵也跟著變。

對非零純量 a 與平移 b，

$$h(aX + b) = h(X) + \log_2 |a|.$$

a 縮放座標，b 平移。平移不改變密度體積，縮放則乘上 $|a|$。對作用於 d 維向量的可逆矩陣 A，

$$h(AX + b) = h(X) + \log_2 |\det A|.$$

A 是非奇異 d 乘 d 矩陣，b 是平移向量，$\det A$ 是行列式。兩側都需要良好定義的熵。

光滑一對一非線性座標映射 T，有相應 Jacobian 修正：

$$h(T(X)) = h(X) + \mathbb{E} \log_2 |\det J_T(X)|.$$

J_T 是 T 的導數矩陣，在隨機 X 處評估；期望平均對數體積變化。恆等式需要適當可微性、可逆性與可積分性。

這解釋潛在密度為何依賴表示座標。可逆轉換可以改變微分熵，而不丟棄資訊。只以原始微分熵比較特徵空間，可能把座標膨脹誤認成資訊品質。

從密度推導尺度修正

這個修正可以直接理解，不必死背。令 $V = aX + b$ 且 $a \neq 0$。座標改變保留機率，因此

$$f_V(v) = \frac{1}{|a|} f_X\left(\frac{v - b}{a}\right).$$

V 是轉換後變數，v 是座標，f_V 是密度，a, b 是尺度與平移。座標區間擴張 $|a|$ 倍，密度就縮小相同比例。因此負對數密度增加常數 $\log_2 |a|$；依未變的事件機率取平均，就得到熵修正。

取寬度 0.1 區間上的均勻來源，微分熵為 $\log_2(0.1) = -3.321928$。分成十個寬度 0.01 的等格，每格機率十分之一，離散熵為 $\log_2(10) = 3.321928$ 位元。密度固定且格子與支撐對齊，因此此處細格關係精確成立。

所有座標乘 100 以改變單位後，區間寬十，微分熵變成 $\log_2(10) = 3.321928$，格寬則變成一。同樣十種結果仍需要 3.321928 位元。縮放使密度熵與格寬修正朝相反方向改變，沒有創造更多可辨事件。只改格子數會改變觀測，改座標單位則不會。

相對熵與互資訊仍有意義

以比值比較密度，會抵消共同座標單位。兩變數關係不應因任一變數換成可逆座標而消失。

對密度 f, g，

$$D_{\mathrm{KL}}(f\|g) = \int f(x)\log_2\frac{f(x)}{g(x)}\,dx, \qquad I(X;Y) = D_{\mathrm{KL}}(f_{XY}\|f_X f_Y).$$

f_{XY} 是聯合密度，$f_X f_Y$ 是獨立乘積參考。散度非負，支撐或可積分性要求時可為無窮。

分別對 X, Y 做可逆座標轉換，密度比與積分中的 Jacobian 因子會抵消，所以互資訊不變。非可逆投影或量化不適用；它們可能丟棄資訊，受到資料處理關係控制。

所有熵有限且相容時，

$$I(X;Y) = h(X) + h(Y) - h(X,Y) = h(Y) - h(Y \mid X).$$

$h(X,Y)$ 是聯合微分熵，$h(Y \mid X)$ 是平均條件微分熵。條件微分熵可能負，但互資訊仍非負。

若熵差會相減無窮，密度比定義更安全。真正連續變數的無雜訊確定性恆等關係，可能有無窮互資訊與奇異聯合法則；一般全維聯合密度公式需要小心。

對相關係數 ρ 的聯合高斯純量對，

$$I(X;Y) = -\frac{1}{2}\log_2(1-\rho^2).$$

ρ 是相關係數，$|\rho| < 1$ 讓共變異數非奇異。只有聯合高斯假設下，相關係數才足以代入此式；一般相依不能只用一個共變異數數值概括。

固定共變異數下，高斯分布有最大熵

在指定共變異數的分布中，高斯以微分熵意義最廣泛地散布不確定性。這是受限制陳述，不是每個來源都高斯的宣稱。

共變異數 Σ 的非奇異 d 維高斯具有

$$h(G) = \frac{1}{2}\log_2\left((2\pi e)^d \det\Sigma\right).$$

G 是高斯向量，d 是維度，Σ 是正定共變異數，e 是自然對數底數。平均值不影響熵。

將另一個具有相同均值與共變異數的密度 f，與高斯密度 g 比較。高斯對數密度是常數加二次形式。因前兩階動差相同，它在 f 下的期望等於在 g 下的期望。因此

$$D_{\mathrm{KL}}(f\|g) = h(G) - h(X) \geq 0.$$

X 具有密度 f，等式假設需要的量有限。它證明最大熵性質，且等號對應高斯分布 [1,2]。

一維固定變異數 σ^2，最大熵為 $\frac{1}{2}\log_2(2\pi e\sigma^2)$。只指定支撐範圍，則有不同極值分布：該有界支撐上的均勻密度。限制決定最大熵分布。

這個證明不只對通訊有用。它讓模型選擇透明：從均值與共變異數採用高斯，在最大熵表示中沒有加入額外形狀限制。但它不會驗證擬合、排除多峰，也不讓高斯概似自動對離群值穩健。

程式：高斯資訊與可逆縮放

完整例子由合成高斯對估計共變異數、計算高斯模型資訊，並驗證可逆縮放改變熵，卻保留資訊公式。

```
import numpy as np

rng = np.random.default_rng(37)
rho = 0.7
covariance = np.array([[1, rho], [rho, 1]])
samples = rng.multivariate_normal([0, 0], covariance, size=100000)
estimated_cov = np.cov(samples, rowvar=False)
def gaussian_entropy(cov):
    sign, logdet = np.linalg.slogdet(np.atleast_2d(cov))
    assert sign > 0
    d = np.atleast_2d(cov).shape[0]
    return 0.5 * (d * np.log(2*np.pi*np.e) + logdet) / np.log(2)
def gaussian_information(cov):
    return (gaussian_entropy([[cov[0, 0]]]) +
            gaussian_entropy([[cov[1, 1]]]) - gaussian_entropy(cov))
estimated = gaussian_information(estimated_cov)
exact = -0.5 * np.log2(1-rho**2)
transform = np.diag([100, 0.2])
scaled_cov = transform @ estimated_cov @ transform.T
assert abs(gaussian_information(scaled_cov) - estimated) < 1e-12
assert abs(gaussian_entropy(scaled_cov) - gaussian_entropy(estimated_cov)
           - np.log2(abs(np.linalg.det(transform)))) < 1e-12
assert abs(estimated - exact) < 0.02
print("Estimated Gaussian-model MI / exact MI:", estimated, exact)
print("Entropy shift:", gaussian_entropy(scaled_cov) - gaussian_entropy(estimated_cov))
```

這是高斯參數計算，不是通用互資訊估計器。非高斯資料集代入共變異數，回答的是高斯模型問題，可能漏掉大量非線性相依。

推導實數高斯通道容量

訊號功率增加可區分輸出的散布；雜訊則讓它們模糊。精確高斯模型下，容量取決於兩者比例。

每次使用傳一個實數振幅：

$$Y = X + Z, \qquad Z \sim \mathcal{N}(0, N), \qquad Z \perp X, \qquad \mathbb{E}[X^2] \le P.$$

X 是輸入，Y 是輸出，Z 是獨立零均值高斯雜訊，$N > 0$ 是雜訊變異數，$P \ge 0$ 是平均輸入功率，$\perp$ 表示獨立。各次使用的雜訊獨立且同分布。

給定 X，只剩平移雜訊，所以 $h(Y \mid X) = h(Z)$。輸出變異數至多 $P + N$。高斯最大熵因此給

$$I(X;Y) \le \frac{1}{2}\log_2(2\pi e(P+N)) - \frac{1}{2}\log_2(2\pi eN) = \frac{1}{2}\log_2\left(1 + \frac{P}{N}\right).$$

P/N 是訊雜比。獨立高斯輸入 $X \sim \mathcal{N}(0, P)$ 讓輸出高斯，達到此單字母上界。相應編碼定理建立

$$C(P) = \frac{1}{2}\log_2\left(1 + \frac{P}{N}\right)$$

在標準平均功率模型下，每次實數通道使用的位元容量 [1,3]。最大化輸入均值為零，因確定性均值消耗功率，卻不增加可區分變動。

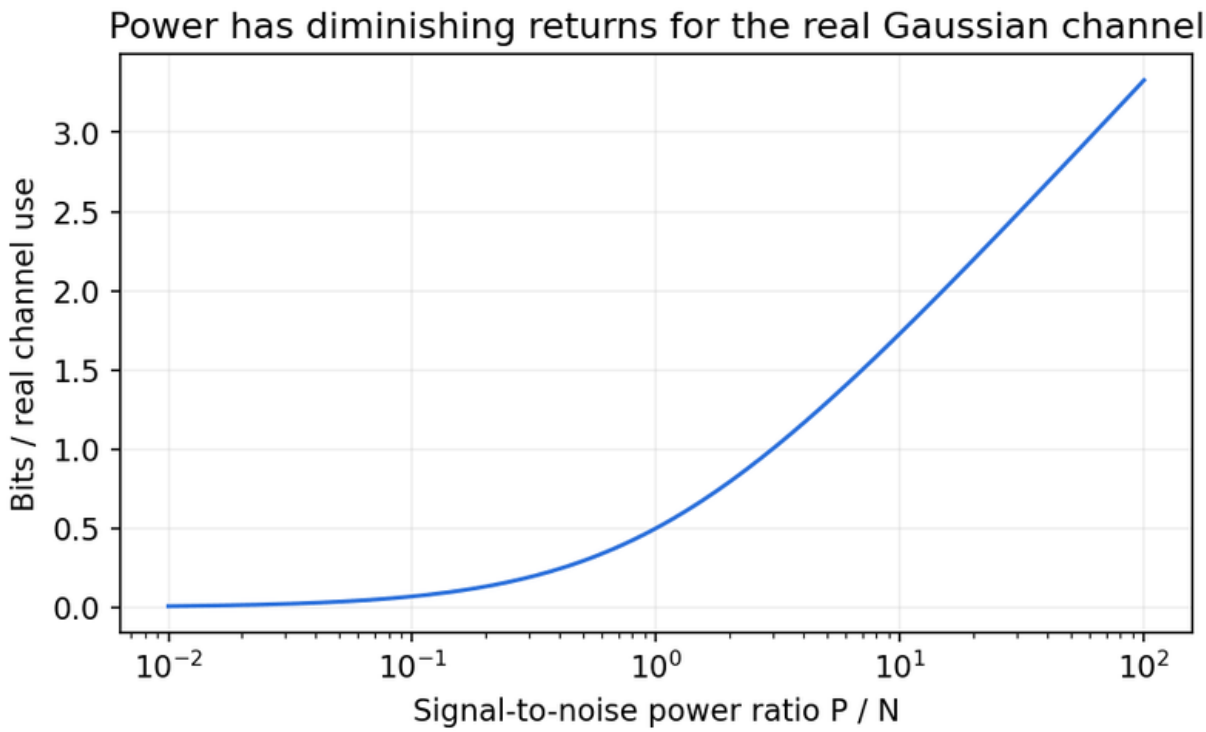


圖 *1* — 每次使用一個實數振幅、獨立高斯雜訊與平均功率限制。二分之一因子屬於此實數慣例。

平均功率限制不會替每個振幅設定嚴格最大值。加入峰值振幅限制，會改變最佳輸入分布，通常也改變容量。有限星座則定義更受限的訊號問題。

類似的複數公式常沒有二分之一，因一次複數使用含兩個實數維度，配合相應功率與雜訊慣例。未定義一次使用就比較公式，可能創造假的兩倍改善。

幾何觀點與限制

多次使用時，雜訊讓觀測形成發送碼字周圍的雲。好的碼字距離夠遠，讓雲通常可區分。

對 n 個獨立高斯雜訊樣本，平方範數集中於 nN 附近：

$$\frac{1}{n}\sum_{t=1}^{n} Z_t^2 \to N.$$

Z_t 是第 t 次雜訊，箭頭表示適當大數法則收斂。功率限制碼字的輸入半徑尺度約 $\sqrt{nP}$，輸出尺度則較大，涉及 $P+N$。

比較高維輸出體積與雜訊雲體積，暗示碼字數尺度為 $(1+P/N)^{n/2}$；正規化二進位對數就是容量式。這是幾何解釋，不是高斯雜訊支撐有界的宣稱。

雜訊有正機率離開任意有限球。可靠通訊要求非典型觀測足夠罕見，不要求所有可能結果的雜訊雲完全不重疊。這區分了誤差趨零容量與零錯誤通訊。

幾何也說明單次使用圖形為何可能誤導。長區塊允許很多座標合作，整體範數集中；短區塊星座則有自己的距離與解碼取捨。

平行通道與注水法

多個獨立通道雜訊不同時，把功率放在資訊邊際增加最多的地方。很吵的通道可能完全不分配功率。

對 m 個雜訊變異數 $N_i > 0$ 的平行實數高斯通道，在總功率 P_{total} 下分配 $P_i \ge 0$：

$$\max_{\sum_i P_i \le P_{\text{total}}} \sum_{i=1}^{m} \frac{1}{2}\log_2\left(1+\frac{P_i}{N_i}\right).$$

i 索引平行維度，m 是數量，目標是所有維度聯合使用一次的位元。獨立高斯輸入達到各分量速率。

作用中功率的導數為 $1/[2\ln 2(N_i+P_i)]$。共同預算下令導數相等，得到

$$P_i = (\mu - N_i)_+, \qquad \sum_i (\mu - N_i)_+ = P_{\text{total}}.$$

μ 是水位，$(u)_+ = \max(u,0)$。符號表示在雜訊地板上加水：作用中通道達到共同訊號加雜訊水位。

此結果來自凹最佳化的最佳性條件。雜訊地板高於水位的通道分配為零；若給正功率，邊際回報比作用中的低雜訊通道差。

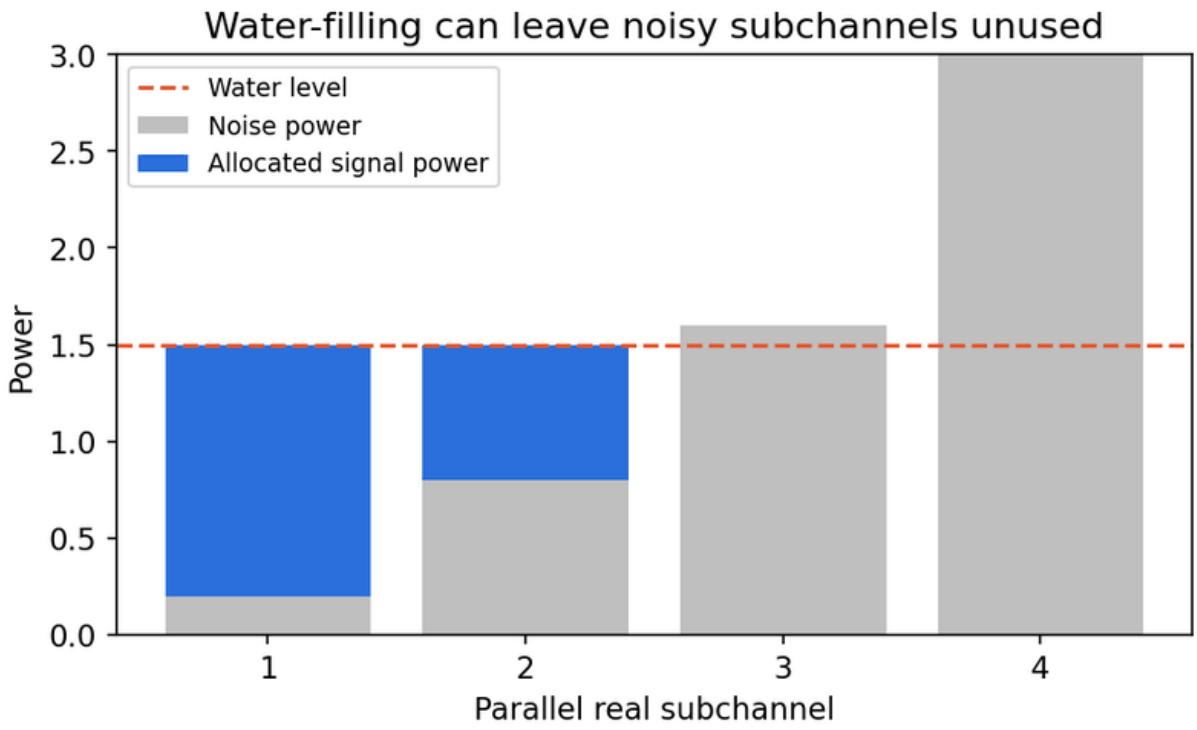


圖 *2* — 四個獨立實數通道的總訊號功率為 *2.0*。注水法是指定模型下的解析資源配置。

手算圖中的功率配置

雜訊地板為 0.2、0.8、1.6、3.0，總訊號功率預算二。先假設只啟用第一通道，總水位會是 $0.2 + 2 = 2.2$，高於第二地板，因此讓第二通道完全不用不可能最佳。

改假設啟用前兩通道，功率總和為 $(\mu - 0.2) + (\mu - 0.8) = 2$，得到 $\mu = 1.5$。它高於兩個作用中地板、低於下一地板 1.6，因此作用集合與假設一致。功率配置為 1.3、0.7、零、零。

聯合使用速率為 $\frac{1}{2}\log_2(7.5) + \frac{1}{2}\log_2(1.875) = 1.906891$ 位元。每通道平均分配 0.5，約得到 1.561252 位元。兩種設計都使用總功率二，改善來自配置位置，不是增加預算。

第三通道在水位超過 1.6 時啟用，先需要預算 $(1.6 - 0.2) + (1.6 - 0.8) = 2.2$。第四通道在水位超過三時啟用，閾值為 $(3 - 0.2) + (3 - 0.8) + (3 - 1.6) = 6.4$。這些閾值解釋容量與功率曲線中作用集合如何改變。

共同訊號加雜訊水位只適用作用中通道。地板高於水位者維持零訊號功率。這個互補條件說明：若強迫四通道全部有相同總水位，就會得到不可能的負功率。

程式：求水位並比較平均配置

完整例子以二分搜尋求水位，驗證預算與作用中最佳性條件，再比較平均功率速率。

```
import numpy as np

noise = np.array([0.2, 0.8, 1.6, 3.0])
budget = 2.0
low, high = noise.min(), noise.max() + budget
for _ in range(100):
    level = (low + high) / 2
    if np.maximum(level-noise, 0).sum() > budget:
        high = level
    else:
        low = level
level = (low + high) / 2
power = np.maximum(level-noise, 0)
equal = np.full(len(noise), budget / len(noise))
rate = lambda allocation: 0.5*np.log2(1+allocation/noise).sum()
assert abs(power.sum()-budget) < 1e-12
assert np.allclose(noise[power > 0]+power[power > 0], level)
assert rate(power) >= rate(equal)
print("Water level / power allocation:", level, power)
print("Water-filled / equal-allocation joint rate:", rate(power), rate(equal))
```

目標計算每個維度同時使用一次。除以維度數，會得到不同的每維平均；兩者都要先指定使用速率，才能轉成每秒位元。

有色高斯雜訊下，在相容線性輸入與功率假設中，共變異數特徵基底可以建立平行雜訊模式。有通道增益時，有效雜訊地板也依賴增益。零增益不能透過反轉變成有用通道。

向量通道連結幾何與共變異數

多個量測可以對多個輸入座標帶來重疊資訊。矩陣描述混合，共變異數描述雜訊雲與所選訊號散布。

實數線性通道為

$$Y = AX + Z, \qquad Z \sim \mathcal{N}(0, K_Z), \qquad X \sim \mathcal{N}(0, K_X), \qquad Z \perp X.$$

X 是 d 維輸入，Y 是 m 維輸出，A 是 m 乘 d 通道矩陣，K_X 是輸入共變異數，K_Z 是正定雜訊共變異數。輸入矩陣可半正定，允許不使用部分方向。

輸出共變異數為 $AK_XA^{\top} + K_Z$。相減高斯輸出與條件雜訊熵，得到

$$I(X;Y) = \frac{1}{2}\log_2 \frac{\det(AK_XA^{\top} + K_Z)}{\det K_Z}.$$

$A^{\top}$ 是矩陣轉置，行列式使用輸出維度共變異數。即使訊號低秩，正雜訊共變異數也讓分母與輸出共變異數非奇異。

平均總功率慣例下，$\operatorname{tr} K_X \le P$ 限制輸入座標變異數總和。在半正定 K_X 上最大化上式，就是相應高斯向量通道問題 [1]。跡是對角元素總和，P 是每次向量使用總功率。

有用方向同時依賴 A 與 K_Z。很大的原始座標變異數，可能在通道強烈衰減的方向；相關雜訊也可能讓某方向比邊際變異數暗示的更少資訊。相容轉換問題的對角化，揭露能使用相同邊際回報原則的平行模式。

這也說明應區分前處理與物理通道改變。可逆接收器座標轉換，同時改變訊號矩陣與雜訊共變異數，行列式比保持不變。投影會丟棄輸出方向，可能降低資訊；不可逆發送器映射則改變能產生哪些輸入訊號。

數值計算應用穩定分解求對數行列式，不要先算巨大行列式再取對數。行列式可能溢位，對數卻適中。解讀答案前，先檢查正定性與矩陣維度。

條件熵與確定性表示

表示在數值上可以連續，卻只含有限可能狀態。數學分布取決於如何生成，不取決於陣列用浮點數儲存。

微分熵定義假設相對連續座標體積有密度。浮點數儲存的離散標籤仍是離散；有限資料集經確定性映射得到的向量，即使座標為實數，經驗支撐仍有限。

高維特徵空間中的低維流形，相對全維體積具有奇異分布。擬合全維密度隱含加入雜訊或平滑模型。得到的熵屬於該模型，可能強烈依賴尺度。

同樣地，在確定性連續關係加入微小獨立雜訊，會讓密度計算可行，但雜訊大小成為資訊問題的一部分。當大小趨近零，互資訊可能趨無窮。未報告平滑慣例就給有限值，會隱藏此依賴。

比較神經表示時，這些觀察很有用。與離散任務標籤相關的資訊可保持有限，精確確定性連續輸入與表示的資訊卻可能奇異。壓縮或預測目標必須指定哪些變數、雜訊機制與量化器定義速率。

實務做法是先說明觀測模型，再估計資訊：離散格子、明確含雜訊通道、參數密度或固定解析度編碼程序。它們各有正當用途，也回答不同問題。

加入雜訊趨零，資訊為什麼可能發散？

令 $X \sim \mathcal{N}(0,1)$，表示為 $V = X + \varepsilon$，其中獨立雜訊 $\varepsilon \sim \mathcal{N}(0,\tau^2)$ 的變異數 τ^2 為正。這是訊號功率一的高斯通道，因此 $I(X;V) = \frac{1}{2}\log_2(1 + 1/\tau^2)$。$\tau^2 = 1$ 時資訊為半位元，$\tau^2 = 0.01$ 時約 3.329106 位元。雜訊變異數趨零，資訊無界增加。

雜訊恰好零時，$V = X$ 是精確連續副本，點對落在二維聯合空間的一條線上，沒有一般二維聯合密度。因此不能在該端點直接相減邊際與聯合微分熵。有限解析度分格則給出有限離散資訊問題。雜訊或解析度慣例，決定神經表示估計究竟回答哪個問題。

建模與估計：公式證明什麼

高斯公式證明的是高斯問題，不會把每個共變異數估計都變成可靠物理通訊極限。

純量容量推導使用獨立加性高斯雜訊、平均二階動差預算，以及無界實數輸入字母。裁切、量化、訊號相依雜訊或改變中的雜訊分布，可能違反一項以上假設。

有限輸出量化器的資訊，不能超過可能標籤數的對數。量化前算出的實數容量，是另一觀測模型的上界參考，不保證量化接收器能達到。

非高斯來源使用高斯熵近似，在必要條件下給動差上界，而非量測熵值。均值與共變異數吻合，不代表尾端或相依吻合。

維度相對觀測太高時，有限樣本共變異數可能奇異，對數行列式公式可能失效。正則化共變異數會建立修改後統計模型；正則化值影響資訊，應明確說明。

評估時應分開物理雜訊模型、座標慣例、估計參數與觀察到的解碼器表現。合成測試可以驗證代數與實作，假設則須由獨立收集資料驗證。

現代觀點

Butakov 等人的 NeurIPS 2024 論文 [4]，研究透過 normalizing-flow 轉換估計互資訊。它提供利用可逆密度模型協助估計的現代例子，但仍須評估估計器，不能把座標熵解讀成內在資訊。

當代語意通訊文獻的綜述觀點 [5] 區分接收器建模、生成式重建與學得聯合編碼。對連續通道，我們由此推論：應明確回報功率正規化、實數或複數使用慣例、接收器量化與通道失配。

重建影像很好看，仍可能對某個原始細節只有少量忠實資訊。任務目標可能刻意丟棄該細節；這是率失真選擇，不是另一種微分熵慣例。

開放問題包括高維可靠資訊估計、分布偏移下的概似模型評估、兩端一致處理有限精度，以及讓學得系統資源計算符合物理通道。可逆性與高斯假設簡化數學分析，卻不能代替這些檢查。

下一章問：容許部分重建誤差時，來源需要多少資訊？高斯容量處理如何運送資訊，高斯率失真則處理來源需求多少資訊。

重點整理

- 密度是每單位座標的機率，不是點機率。
- 微分熵可負，會隨座標縮放改變。
- 細量化加入與解析度相關的離散描述成本。
- 相對熵與互資訊在可逆座標轉換下保留。
- 高斯最大熵來自固定動差限制。
- 實數高斯容量含明確平均功率預算與二分之一。
- 注水法讓作用中平行通道邊際回報相等。
- 參數資訊公式與經驗解碼可靠性回答不同問題。

練習

1. 求寬度 0.01 的均勻變數微分熵，說明負值為何不允許負長度碼。

提示

用區間寬度對數，另外處理格子編碼。

2. 把單位均勻來源量化成 16、64、256 個等格，求精確離散熵並連到微分熵。

提示
每格等機率。

3. 驗證聯合高斯對分別非零縮放的互資訊不變性。若一變數取整成單一位元，會改變什麼？

提示
可逆尺度的行列式抵消，取整不可逆。

4. 比較訊雜比 0.1、10 時，功率倍增的容量增益。說明絕對速率增加與相對功率增加為何不同。

提示
相減倍增前後容量。

5. 增加注水預算直到第三、第四通道作用，從雜訊地板算出啟用閾值。

提示
把水位設為下一雜訊地板。

6. 說明只有兩輸出標籤的裁切接收器違反哪些假設，並給與訊號功率無關的資訊上界。

提示
輸出熵至多一位元。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 8–9. 出版社書頁與目錄。
2. T. Weissman, “Continuous Random Variables,” Stanford EE276 Lecture 8 notes, 2024. 第一手講義。
3. T. Weissman, “AWGN Channel,” Stanford EE276 Lecture 9 notes, 2024. 第一手講義。
4. I. D. Butakov, A. D. Tolmachev, S. V. Malanchuk, A. M. Neopryatnaya and A. A. Frolov, “Mutual Information Estimation via Normalizing Flows,” *NeurIPS*, 2024. 出版論文。
5. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, “A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding,” *IEEE Communications Surveys & Tutorials*, 2025. 出版資訊與作者版本。

IT 06・率失真與有損表示

推導二元與高斯率失真極限，計算有限字母測試通道，並以明確失真評估量化與學得表示。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/rate-distortion/

你將學到

- 稱重建可接受之前，先指定失真函數。
- 區分操作速率與資訊論測試通道。
- 推導 Bernoulli ／ Hamming 與高斯／平方誤差率失真函數。
- 解釋覆蓋可達性與逆定理的資訊計算。
- 以交替更新計算有限字母率失真點。
- 把實際量化與學習目標連到誠實的速率、品質量測。

先看全貌

無損編碼要求精確原始資料；有損編碼要求有用替代品。後者必須先給「有用」數學意義：哪些差異重要、成本多大，以及限制指平均誤差，還是罕見的大失敗。

> **用最簡單的話說**
>
> 你只能用幾個詞向朋友描述一幅畫，無法提到每個小點。若目標是認出動物，形狀比背景細點重要；若目標是精確重畫，省略同樣細節就是錯誤。

本章依 Cover 與 Thomas 第 10 章 [1]，以及 Stanford 的率失真講義 [2,3]。核心結果對假想重建法則最小化互資訊。最佳化定義極限，本身不建立有限檔案格式，也不保證神經解碼器行為。

這直接連到視覺：像素重建、機器可讀特徵與合理生成影像，保留不同性質。比較速率、失真以前，應明確說明目標與資源慣例。

先選失真，再選編碼器

誤差量指定接收器容忍哪些來源差異。沒有任何通用失真分數能涵蓋所有任務。

令 X 為來源符號，$\hat{X}$ 為可能屬於不同字母集合的重建。非負失真函數為 $d(x,\hat{x})$。長度 n 區塊的失真為

$$d_n(x^n,\hat{x}^n)=\frac{1}{n}\sum_{t=1}^{n} d(x_t,\hat{x}_t).$$

t 索引符號，n 是區塊長度，d_n 是每符號平均失真。常見限制是 $\mathbb{E}[d_n(X^n, \widehat{X}^n)] \le D$；$D$ 是允許平均值，期望依指定來源與編碼。

Hamming 失真在符號正確時零，其他時一。實數振幅的平方誤差為 $(x - \hat{x})^2$。前者計不符次數，後者更重視大數值偏差，回答不同問題。

平方誤差依賴單位。來源與重建乘以 a，失真乘以 a^2。率失真曲線應寫明來源正規化、維度平均與失真單位。

一般加性模型替每符號計成本再平均。結構化任務可能需要區塊準則，例如保留關係、罕見特徵或決策。仍可研究，但不會自動等同單字母問題。

平均失真不會限制每個重建的最壞誤差。小平均可能伴隨罕見大誤差。需要尾端機率、最大誤差或各群組保證時，應另列條件，不要從平均值推讀。

重建字母集合也是定義的一部分。限制為小碼本、整數像素或特定模型族，會縮小可行集合，無限制理論最佳值可能無法達到。反過來，允許輸出任務標籤而非影像，會改變字母集合與失真問題。公平比較應先說明選擇，再展示速率，讓讀者判斷系統是否保留同一物件。

操作與資訊論率失真

實際編碼發送有限描述之一；測試通道描述來源與重建可能的統計關係。定理連結它們的漸近極限。

區塊編碼器把 X^n 映到 M 個索引之一，解碼器把索引映到 $\widehat{X}^n$。固定速率為

$$R_n = \frac{1}{n} \log_2 M.$$

M 是可發送描述數，R_n 是每來源符號位元。操作函數 $R(D)$ 是指定失真下，漸近可達速率的下確界。

獨立無記憶來源與適當加性失真設定下，率失真定理給

$$R(D) = \min_{p(\hat{x}|x):\, \mathbb{E}[d(X,\widehat{X})] \le D} I(X; \widehat{X}).$$

來源法則 $p(x)$ 固定，$p(\hat{x} \mid x)$ 遍歷正規化重建法則。最小值符號適用最佳解存在的設定；更一般連續情況可能需要下確界與正則條件 [1,2]。

此重建法則常稱**測試通道（test channel）**。與通訊容量不同，它不最佳化來源輸入。容量在給定物理通道下最大化傳輸資訊，率失真在重建限制下最小化所需資訊。

隨機測試通道是描述所需聯合分布的數學物件。操作定理以區塊碼逼近資訊速率。逐符號獨立抽取含雜訊重建，本身不會說明接收器需要多少位元才能重現抽樣。

基本形狀與零速率重建

容許更多誤差，不會需要更多位元。某個失真以上，接收器可以固定輸出同一重建，完全不需訊息。

定義

$$D_0 = \min_{\hat{x}} \mathbb{E}[d(X, \hat{x})].$$

D_0 是最佳常數重建的平均失真，$\hat{x}$ 不依觀察來源選擇。$D \geq D_0$ 時，零速率可行。

標準設定下 $R(D)$ 非負、非遞增且凸。非遞增是因更大的失真容許值包含先前所有可行編碼。

凸性有分時解釋。長區塊的 α 比例使用第一種碼，其餘用第二種：

$$R(\alpha D_1 + (1-\alpha)D_2) \leq \alpha R(D_1) + (1-\alpha)R(D_2).$$

$\alpha \in [0,1]$ 是比例，D_1, D_2 是各失真，右側是平均速率。預定區塊邊界可以避免逐符號發送選擇。

零失真時，有限來源若只有完全相同才零成本，就回到一般熵極限。若數種不同來源值都同樣正確，零失真可能需要少於來源熵。真正連續高斯來源的平方誤差零失真速率則發散。

這些條件很重要。沒有指定失真零集合或字母集合，就說「零失真等於熵」，可能抹掉任務表示能壓縮的原因。

Hamming 誤差下的二元來源

公平二元來源要每符號一位元才能精確保留。容許受控的不符比例，會降低必要速率。

對 $X \sim \text{Bernoulli}(p)$、$0 < p \leq 1/2$ 與 Hamming 失真，

$$R(D) = \begin{cases} h_2(p) - h_2(D), & 0 \leq D \leq p, \\ 0, & D \geq p. \end{cases}$$

p 是一的機率，D 是不符機率，h_2 是二元熵。零速率重建永遠為零，因此錯誤占來源符號比例 p。

為求下界，令 $E = X \oplus \widehat{X}$ 是不符指示，$\oplus$ 表二元互斥或。則

$$I(X; \widehat{X}) = H(X) - H(E \mid \widehat{X}) \geq h_2(p) - h_2(D).$$

E 為一的機率至多 $D \leq p \leq 1/2$，二元熵在此範圍遞增。條件化不能增加離散熵。

等號以**反向**建構達到：選 $\widehat{X}$ 與獨立 $E \sim \text{Bernoulli}(D)$，再令 $X = \widehat{X} \oplus E$。重建機率滿足

$$\Pr(\widehat{X} = 1) = \frac{p - D}{1 - 2D}.$$

$D < p \leq 1/2$ 避開公平來源的奇異端點；端點由連續性或直接常數重建處理。建構符合指定來源法則，並讓不符與重建獨立。

公平來源在 $D = 1/2$ 前為 $1 - h_2(D)$。有偏來源若直接以前向機率 D 翻轉 X，一般不會達到相同最佳值。獨立條件的方向是推導一部分。

建立最佳二元聯合表

把機率逐項寫出，反向建構就更清楚。取來源機率 $p = 0.2$、允許不符 $D = 0.1$。若 $q = \Pr(\widehat{X} = 1)$，重建與不符獨立，則

$$p = q(1-D) + (1-q)D = D + q(1-2D).$$

p 是要求的來源一機率，q 是重建一機率，D 是獨立反向翻轉機率。解得 $q = (0.2-0.1)/(1-0.2) = 0.125$。重建機率乘獨立誤差機率，得到：

聯合機率	$\widehat{X}=0$	$\widehat{X}=1$	來源邊際
$X=0$	0.7875	0.0125	0.8
$X=1$	0.0875	0.1125	0.2
重建邊際	0.875	0.125	1.0

非對角項總和為 0.1，恰好符合失真限制。每個重建欄除以總和後，來源與重建不同的機率都是 0.1。因此 $H(X \mid \widehat{X}) = h_2(0.1)$，必要資訊為每來源符號 $h_2(0.2) - h_2(0.1) = 0.252933$ 位元。

等價的**前向**條件通道不對稱。給定 $X=0$，輸出一的機率是 $0.0125/0.8 = 0.015625$；給定 $X=1$，輸出零的機率是 $0.0875/0.2 = 0.4375$。罕見一的錯誤大、常見零的錯誤小，合計達到指定平均誤差，並符合最佳後驗不確定性。

相較之下，以 0.1 機率獨立前向翻轉來源，平均失真相同，但重建一的機率為 0.26。資訊變成 $h_2(0.26) - h_2(0.1) = 0.357751$ 位元，大於最佳值。「百分之十雜訊」不足以定義最佳測試通道；方向與獨立關係也重要。

```
import numpy as np

def entropy(p):
    p = np.asarray(p, dtype=float)
    positive = p > 0
    return -np.sum(p[positive] * np.log2(p[positive]))

source_one, D = 0.2, 0.1
q = (source_one-D)/(1-2*D)
joint = np.array([[(1-q)*(1-D), q*D],
                  [(1-q)*D, q*(1-D)]])
px, pr = joint.sum(axis=1), joint.sum(axis=0)
information = entropy(px)+entropy(pr)-entropy(joint)
assert np.allclose(px, [0.8, 0.2])
assert np.isclose(joint[0, 1]+joint[1, 0], D)
assert np.allclose([joint[1, 0]/pr[0], joint[0, 1]/pr[1]], D)
exact = entropy(px)-entropy([D, 1-D])
assert np.isclose(information, exact)
print("Backward joint table:", joint)
print("Optimal information:", information)
```

計算驗證的是統計最佳值，仍需要長區塊碼，才能把該資訊速率轉成傳送的重建索引。

平方誤差下的高斯來源

較分散的高斯來源，需要更多位元才能有相同絕對精度。要求更小誤差，速率以對數增加。

對 $X \sim \mathcal{N}(0, \sigma^2)$ 與 $d(x, \hat{x}) = (x - \hat{x})^2$，

$$R(D) = \max\left\{0, \frac{1}{2}\log_2 \frac{\sigma^2}{D}\right\}, \qquad D > 0.$$

σ^2 是來源變異數，D 是相同平方單位的均方誤差。$D \geq \sigma^2$ 時，零重建不用位元。$D \to 0$ 時，速率無界增加 [1,3]。

令 $E = X - \widehat{X}$ 是重建誤差。相容有限微分熵下，

$$\begin{aligned} I(X; \widehat{X}) &= h(X) - h(E \mid \widehat{X}) \\ &\geq h(X) - h(E) \geq \frac{1}{2}\log_2 \frac{\sigma^2}{D}. \end{aligned}$$

互資訊良好定義時，條件化降低微分熵；高斯最大熵以二階動差限制誤差熵。此式是非零速率範圍的下界。

資訊論最小值的可達建構使用獨立 $\widehat{X} \sim \mathcal{N}(0, \sigma^2 - D)$ 與 $E \sim \mathcal{N}(0, D)$，並令 $X = \widehat{X} + E$。再次是反向測試通道：誤差與重建獨立，不是與來源獨立。

等價失真速率關係為 $D(R) = \sigma^2 2^{-2R}$，描述漸近最佳區塊編碼。簡單純量量化器在其速率下，一般位於極限上方。

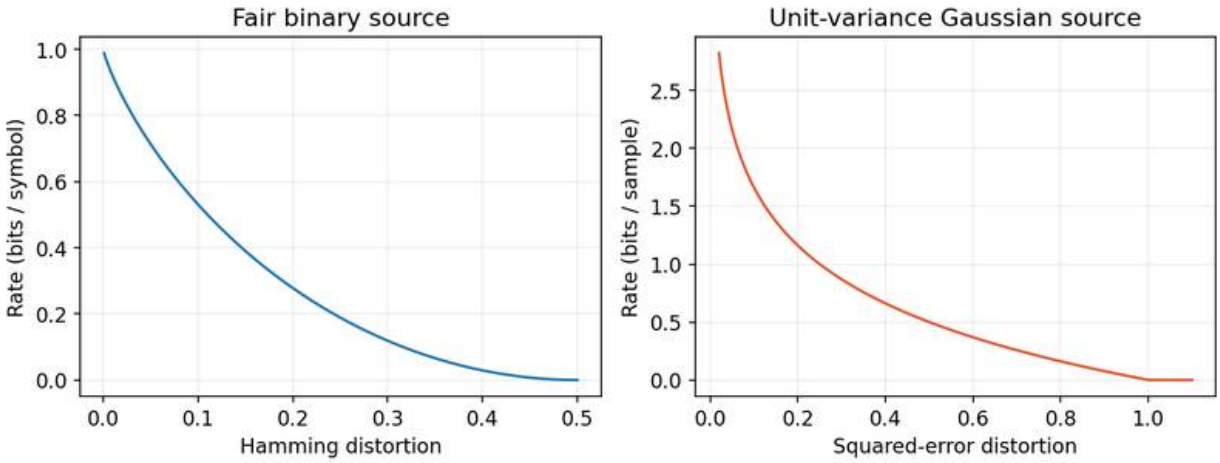


圖 *1* — 具有不同失真意義的解析例子。橫軸數值不能當成共同品質分數比較。

程式：測試高斯反向建構

完整合成實驗驗證來源變異數、誤差變異數與資訊公式。它沒有序列化區塊碼，只測最佳化聯合分布。

```
import numpy as np

rng = np.random.default_rng(38)
variance, distortion = 1.0, 0.2
```

```
reconstruction = rng.normal(0, np.sqrt(variance-distortion), size=100000)
error = rng.normal(0, np.sqrt(distortion), size=100000)
source = reconstruction + error
rho = np.corrcoef(source, reconstruction)[0, 1]
information = -0.5 * np.log2(1-rho**2)
exact = 0.5 * np.log2(variance/distortion)
print("Source variance / error MSE:", source.var(), np.mean(error**2))
print("Error-reconstruction correlation:", np.corrcoef(error, reconstruction)[0, 1])
print("Gaussian-model information / exact rate:", information, exact)
assert abs(source.var()-variance) < 0.02
assert abs(np.mean(error**2)-distortion) < 0.01
assert abs(information-exact) < 0.02
```

獨立性在取樣時就建立；有限相關只是檢查。任意非高斯變數的經驗相關很小，不能單獨證明獨立。

為何覆蓋不同於通道打包

有損來源碼需要足夠重建代表，在允許誤差內覆蓋可能來源區塊。通道碼則分隔傳輸代表，抵抗雜訊。

固定可行來源重建聯合分布，從重建邊際抽許多碼字。對典型來源區塊，隨機重建符合聯合關係的機率，指數尺度為 $2^{-nI(X;\widehat{X})}$。

代表數超過約 $2^{nI(X;\widehat{X})}$ 並帶小速率餘裕，就很可能找到匹配。編碼器選索引，解碼器回傳共享重建；聯合典型性把所選點對連到目標失真。

這是覆蓋論證。精確證明需要典型性定義與來源、失真假設；指數尺度解釋速率，不是有限碼表現公式。

逆定理使用傳輸索引 J：

$$nR_n \geq H(J) \geq I(X^n;\widehat{X}^n) \geq \sum_{t=1}^{n} R(D_t) \geq nR\left(\frac{1}{n}\sum_t D_t\right).$$

J 遍歷 M 個描述，$R_n = \log_2 M/n$，$D_t = \mathbb{E}[d(X_t,\widehat{X}_t)]$，來源無記憶。第一資訊界使用重建由索引決定；逐符號步驟用來源獨立與資訊論定義；最後用凸性。

編碼傳達的來源資訊，不能超過索引所含資訊。這說明解碼器再精細的先驗，也不能以零來源速率恢復任意遺失細節；它可以依指定失真選有用或可能的重建。

計算有限測試通道

離散字母下，交替更新重建邊際與來源條件重建法則。懲罰參數會沿率失真前緣移動。

最小化自然單位目標 $I_{\text{nats}}(X;\widehat{X}) + \beta\mathbb{E}[d]$，$\beta \geq 0$。給目前重建邊際 $q_{\hat{x}}$，

$$V(\hat{x}\mid x) = \frac{q_{\hat{x}}\exp[-\beta d(x,\hat{x})]}{\sum_z q_z \exp[-\beta d(x,z)]}, \qquad q_{\hat{x}}^{\text{new}} = \sum_x p_x V(\hat{x}\mid x).$$

p_x 是固定來源法則，V 是測試通道，z 索引重建，β 以自然單位衡量失真懲罰。位元速率將自然單位資訊除以 $\ln 2$。

教科書 [1] 討論 Blahut–Arimoto 交替計算。正初始邊際涵蓋全部候選重建符號。解出後應計算實際失真與互資訊，不要把 β 本身稱為失真。

指數測試通道更新從哪裡來？

交替規則來自受限制最小化，不是因為某公式看起來像機率。暫時固定正的重建邊際 q_j，對來源狀態 i 最小化列貢獻

$$\sum_j V_{ij}[\ln(V_{ij}/q_j) + \beta d_{ij}], \qquad \sum_j V_{ij} = 1.$$

V_{ij} 是給定來源 i 的重建 j 條件機率，q_j 是固定參照邊際，d_{ij} 是失真，β 是自然單位懲罰。正來源權重 p_i 乘此列，但不改變最小化的條件分布。

加入列正規化乘子 η_i，對 V_{ij} 微分，駐點方程為 $\ln(V_{ij}/q_j) + 1 + \beta d_{ij} + \eta_i = 0$。取指數使 V_{ij} 正比於 $q_j \exp(-\beta d_{ij})$，列正規化再提供更新式分母。

接著固定 V，其誘導邊際 $q_j^{\text{new}} = \sum_i p_i V_{ij}$ 在正規化 q 中最小化交叉熵部分。指定有限字母設定下，交替兩步降低懲罰目標。幾何也很清楚：β 大時各列集中於低失真重建，邊際因子則偏向母體常使用的重建。

公平二元來源與 Hamming 失真下，對稱性保持 q 均勻，非對角機率因此是 $D = 1/(1 + e^{\beta})$，速率為 $1 - h_2(D)$。程式檢查的正是此解析關係。它直接解釋懲罰，而不把懲罰值當成實際失真。

程式：對照精確二元曲線

實作檢查重建邊際收斂，以獨立解析公式驗證點。容許誤差是數值更新容許值，不是前章容量上下界間隙。

```
import numpy as np
from scipy.special import logsumexp

def entropy(p):
    p = np.asarray(p)
    return -np.sum(p[p > 0] * np.log2(p[p > 0]))
def rd_point(p, distortion_matrix, beta):
    p = np.asarray(p, dtype=float)
    d = np.asarray(distortion_matrix, dtype=float)
    q = np.full(d.shape[1], 1/d.shape[1])
    for _ in range(50000):
        scores = np.log(q)[None, :] - beta*d
        V = np.exp(scores-logsumexp(scores, axis=1, keepdims=True))
        updated = p @ V
        if np.max(abs(updated-q)) < 1e-13:
            q = updated
            break
```

```
        q = updated
    else:
        raise RuntimeError("Rate-distortion updates did not converge")
    joint = p[:, None]*V
    positive = joint > 0
    ratio = np.divide(V, q[None, :], out=np.ones_like(V), where=positive)
    rate = np.sum(joint[positive]*np.log2(ratio[positive]))
    distortion = np.sum(joint*d)
    return rate, distortion, V

d = np.array([[0, 1], [1, 0]])
for beta in [0.0, 1.0, 2.0, 4.0]:
    rate, D, V = rd_point([0.5, 0.5], d, beta)
    exact = 1-entropy([D, 1-D])
    assert abs(rate-exact) < 1e-10
    print("Penalty / D / rate / exact:", beta, D, rate, exact)
rate, D, V = rd_point([0.8, 0.2], d, 4.0)
exact = entropy([0.8, 0.2])-entropy([D, 1-D])
assert D < 0.2 and abs(rate-exact) < 1e-9
print("Biased-source D / rate / exact:", D, rate, exact)
```

任意失真矩陣的小邊際更新差，是有用收斂檢查，但不是嚴格全域數值憑證。應看目標改變、支撐與獨立特殊例子。很大懲罰可能造成機率下溢，需要更仔細實作。

純量量化是具體可達設計

把來源取整至有限解析度格點，再無損描述格點索引。取整造成失真，機率編碼降低索引速率。

步距 Δ 下使用

$$K = \operatorname{round}(X/\Delta), \qquad \widehat{X} = \Delta K, \qquad D_\Delta = \mathbb{E}[(X - \widehat{X})^2].$$

K 是離散索引，$\widehat{X}$ 是格點重建，D_Δ 是平方誤差失真。熵編碼索引的速率由索引分布決定，再加實際編碼成本。

密度夠平滑、步距夠小時，常用近似 $D_\Delta \approx \Delta^2/12$，把量化誤差視為格內近似均勻。粗步距、裁切附近或結構化集中數值，不一定符合。

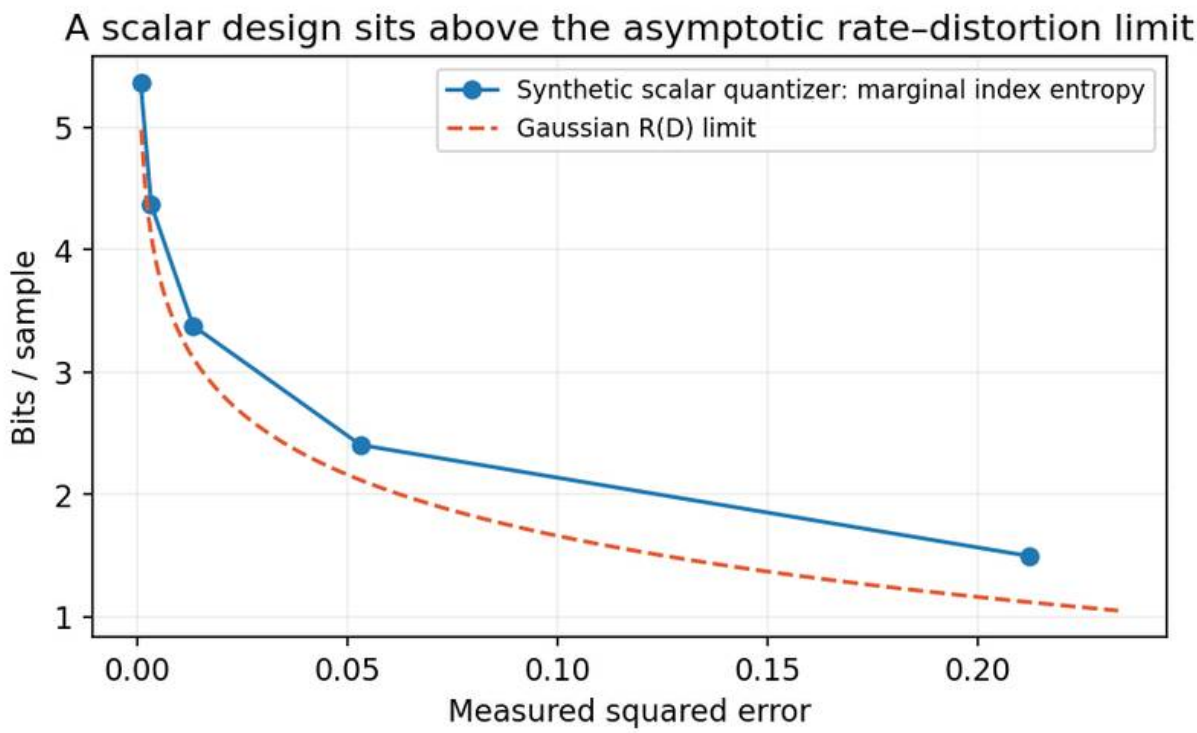


圖 *2* — *20,000* 個合成單位高斯樣本。索引熵是理想邊際編碼統計，沒有宣稱實際位元流長度。

```
import numpy as np

rng = np.random.default_rng(26)
source = rng.normal(size=20000)
for step in [0.1, 0.2, 0.4, 0.8, 1.6]:
    index = np.rint(source/step).astype(int)
    reconstruction = step*index
    _, counts = np.unique(index, return_counts=True)
    p = counts/counts.sum()
    rate_estimate = -np.sum(p*np.log2(p))
    mse = np.mean((source-reconstruction)**2)
    gaussian_limit = max(0, 0.5*np.log2(1/mse))
    print("Step / index entropy / MSE / reference rate:",
          step, rate_estimate, mse, gaussian_limit)
```

經驗分布來自同一批量化樣本，所以是描述統計。比較學得熵模型時，應另用訓練觀測擬合，在固定保留資料評分，並以明確慣例傳輸或共享參數。

索引熵不是實際壓縮檔大小。編碼器還須指定索引支撐、超界處理、重建步距、訊息長度與熵編碼終止。有限經驗估計可能在理論關係附近波動，不會推翻母體定理。

平行高斯來源使用反向注水

有限描述預算應更仔細保留高變異來源方向。低變異方向的整體變動若已在誤差容許值內，就可以捨棄。

對變異數 σ_i^2 的獨立高斯座標，總平方誤差預算為 D_{total}：

$$R(D_{\text{total}}) = \min_{\sum_i D_i \le D_{\text{total}}} \sum_i \max\left\{0, \frac{1}{2}\log_2 \frac{\sigma_i^2}{D_i}\right\}.$$

i 索引座標，D_i 是各自均方誤差，R 是完整來源向量的位元。正速率範圍中，微分目標使最佳作用中失真相等：

$$D_i = \min(\theta, \sigma_i^2), \qquad \sum_i \min(\theta, \sigma_i^2) = D_{\text{total}}.$$

θ 是反向水位，在預算小於總來源變異數時選擇。變異數低於水位的座標重建為均值，付出全部變異數但不付速率。

與通道注水的相似很有啟發，但角色不同。通道配置在雜訊地板上花訊號功率，最大化運送資訊；來源配置允許誤差到來源變異數，最小化所需描述資訊。

相關高斯向量的正交共變異數特徵基底，會產生獨立座標且保留總平方誤差，配置因此作用於特徵值。非線性任務失真或非高斯分布，不一定能簡化成相同計算。

這也說明低維重建的好處與限制。捨棄低變異座標，對總平方誤差便宜，但該座標仍可能決定特定標籤。任務相關與變異數是不同準則。

反向注水配置例

取三個獨立高斯座標，變異數為 4、1、0.25，每向量允許總平方誤差一。若每座標都給三分之一誤差，第三座標的預算會浪費：直接重建為零，已經只有 0.25 誤差。

其餘兩座標的最佳共同誤差水位滿足 $\theta + \theta + 0.25 = 1$，得到 $\theta = 0.375$。失真為 0.375、0.375、0.25，總和一。必要速率為

$$R = \frac{1}{2}\log_2 \frac{4}{0.375} + \frac{1}{2}\log_2 \frac{1}{0.375} + 0 \approx 2.415037.$$

R 是每個三座標來源向量的位元數。前兩座標需要正描述速率，第三重建為均值，速率零。此失真預算是每向量總和，不是每座標平均；將預算除以三，才是同一操作點的另一種慣例。

共同誤差水位來自對作用中座標速率微分：每增加一單位失真，速率降低的幅度為 $1/(2\ln 2\, D_i)$。讓邊際節省相等，就讓作用中 D_i 相等。當 D_i 到達來源變異數，已經沒有速率可省，該座標離開作用集合。

小變異座標仍可能編碼重要標籤。反向注水對此高斯平方誤差問題最佳，但標籤保留需要另一失真。這個例子把最佳化條件與接收者目標直接連起來。

比較操作點時，不要改變問題

公平率失真比較固定來源資料、品質準則與資源假設。否則看似更好的曲線，可能只是在解更容易問題。

經驗操作點是完整程序下的量測速率與失真。應說明速率為固定長度、平均變長、理想負對數機率，還是實際序列化位元。

本章理論曲線使用無記憶母體來源與漸近長碼。有限實驗有抽樣不確定性、有限區塊、受限模型族與實作成本。把全部叫同一間隙，會掩蓋原因。

不同編碼設定應使用同一保留觀測。量化器、機率模型與學得轉換使用訓練資料擬合；驗證資料選懲罰或操作點；最後測試集估計已選程序表現。

母體異質時，應回報有意義群組的失真。常見例子平均可能掩蓋某子群很差的重建。這不推翻平均失真定理，而是指出該平均未充分表達應用。

也應說明解碼隨機性。相同索引下，確定性與隨機生成解碼器可能有不同誤差分布。重建同一測試輸入多次可以揭露變動；看到原圖後才挑最佳隨機重建，加入真實接收器可能沒有的選擇步驟。

最後，共享旁資訊須可比。帶任務資料庫、預訓練先驗或傳送調適資訊的解碼器，與沒有這些資訊的程序不同。共享資源有用，但可用性與成本慣例應伴隨前緣宣稱。

表示、任務與學得速率懲罰

表示可以捨棄無關細節，保留任務。只有指定任務並在目標母體評估，這才有幫助。

學得編碼器可能產生量化潛在變數 Z，具有熵模型 q，解碼器回傳 $\widehat{X}$。常見訓練目標為

$$\mathcal{L} = \mathbb{E}[-\log_2 q(Z)] + \lambda \mathbb{E}[d(X, \widehat{X})].$$

第一項是位元單位理想編碼成本代理，$\lambda \geq 0$ 平衡指定失真，期望使用訓練分布。此目標不會自動等於母體率失真函數。

潛在字母集合、熵模型失配、有限編碼器、旁資料流與模型成本，影響實際速率。架構限制可達重建；最佳化可能找到局部或模型受限取捨，而非資訊論最小值。

若接收器需要標籤 Y，適當準則可評估從 Z 推得決策，而非重建全部 X。這是不同來源失真問題。保留某標籤的特徵，可能捨棄後續不相關任務需要的資訊。

感知準則可能獎勵合理紋理，即使紋理與來源不同；忠實重建準則可能懲罰同一改變。沒有通用正確目標，文字、圖形與評估都要保留區別。

多種失真可以一起回報，不需合成沒有解釋的單一分數。例如分開數值忠實度、明確任務誤差與分布真實性準則。

現代觀點

Yang、Mandt 與 Theis 的 2023 年專論 [4]，區分無損熵編碼、有損轉換設計、量化、感知目標與任務導向編碼。分類能定位系統哪些部分改變速率、哪些改變失真、哪些改變評估目標。

Huang 與 Wu 的 2024 年綜述 [5]，明確對照學得影像壓縮的人類與機器使用者。共同結論是：表示的用途依賴接收者；一個重建分數不能證明保留所有下游分析。

Hofer 與 Böhme 的 2024 年預印本 [6]，建立神經影像錯誤壓縮分類，鼓勵檢查哪些內容改變，而不假設平均數值誤差已完整描述重建行為。此處依已核對狀態引用為預印本。

由率失真理論，我們推論應評估**前緣**，而非單一有利操作點：固定測試母體上的量測位元流大小，對明確失真。來源正規化、模型共享假設與解碼資源預算應一致。

開放問題包括不可逆壓縮後的任務轉移、罕見細節失真量、分布偏移，以及生成先驗的公平資源計算。更長區塊或更精細解碼器可以改善特定取捨，卻可能增加延遲或依賴更強共享假設。

下一章把資訊延伸到編碼以外：散度描述統計證據，最大熵在限制下選分布，對數成長則連結預測與決策。

重點整理

- 失真定義有損重建可以改變什麼。
- 率失真固定來源，最佳化重建法則。
- 標準前緣非遞增且凸。
- 二元與高斯最佳解使用仔細定義的反向測試通道。
- 覆蓋與資訊計算連結測試通道和區塊碼。
- 交替更新需要明確對數與懲罰單位。
- 量化索引熵不同於實際序列化速率。
- 學習或感知目標，須依指定接收者與失真評估。

練習

1. 對機率 0.2 的 Bernoulli 來源，求零速率失真，以及失真 0、0.05、0.1 的速率。

提示

非零速率範圍用 $h_2(p) - h_2(D)$。

2. 建立 $p = 0.2$、$D = 0.1$ 的反向二元聯合分布，比較直接以前向機率 0.1 翻轉來源。

提示

讓不符與重建獨立。

3. 變異數 4 的高斯來源，求平方誤差 1、0.25 所需速率，並解釋改變座標單位後的結果。

提示

速率依賴變異數失真比。

4. 把有限測試通道程式延伸至三符號與非對稱失真矩陣，分開回報懲罰、實際速率與實際失真。

提示

檢查每列正規化，重建聯合分布。

5. 在純量量化加入裁切，分別比較中央與尾端誤差，不只回報平均。

提示

分開追蹤飽和例子。

6. 替小二元影像設計兩種失真：精確像素保留，以及保留是否存在任何前景像素。說明零失真速率為何不同。

提示

任務辨認影像等價類。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapter 10. 出版社書頁與目錄。
2. T. Weissman, “Lossy Compression & Rate Distortion Theory,” Stanford EE276 Lecture 13 notes, 2024. 第一手講義。
3. T. Weissman, “Lossy Compression & Rate Distortion Theory Continued,” Stanford EE276 Lecture 14 notes, 2024. 第一手講義。
4. Y. Yang, S. Mandt and L. Theis, “An Introduction to Neural Data Compression,” *Foundations and Trends in Computer Graphics and Vision*, 15(2), 113–200, 2023. 出版專論與作者版本。
5. C.-H. Huang and J.-L. Wu, “Unveiling the Future of Human and Machine Coding: A Survey of End-to-End Learned Image Compression,” *Entropy*, 26(5), 357, 2024. 出版綜述。
6. N. Hofer and R. Böhme, “A Taxonomy of Miscompressions: Preparing Image Forensics for Neural Compression,” arXiv preprint, 2024. 第一手預印本。

IT 07・統計證據、最大熵與成長

透過明確統計模型，連結概似比、罕見經驗分布、Fisher 資訊、最大熵與對數成長。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/statistics-decision-growth/

你將學到

- 把概似比解讀為證據，而非後驗機率。
- 以經驗型態解釋散度控制的罕見事件速率。
- 區分假說檢定誤差取捨與參數估計。
- 推導 Fisher 資訊與正則不偏估計下界。
- 從明確限制建立最大熵分布。
- 把對數最佳配置、通用混合連到編碼悔損。

先看全貌

資訊量不只用於儲存與通訊。它們也量化觀測如何區分競爭解釋、分布對參數變化多敏感，以及不良預測決策在重複試驗中多昂貴。

> **用最簡單的話說**
>
> 兩個袋子有不同顏色比例。看過許多珠子後，你可以問哪個袋子更能解釋它們。同樣機率也能幫你描述珠子字串、估計比例，或選重複遊戲策略。每個問題用觀測的方式不同。

書籍對照為 Cover 與 Thomas 第 11 章統計、第 12 章最大熵、第 6 章賭局與編碼，以及第 16 章投資組合理論 [1]。Stanford 型態方法講義 [2] 提供易讀的有限字母推導。

本章使用完全指定機率與報酬的數學決策遊戲，目的在揭露資訊恆等式，不從歷史資料推論實際回報。每節都區分來源法則、選定模型、觀測樣本與決策準則。

概似比比較解釋

某模型給樣本較大機率時，樣本便較支持該模型。重複獨立觀測會相加對數證據。

對支撐相容、完全指定的分布 P, Q，定義

$$\Lambda(x^n) = \log \frac{P(x^n)}{Q(x^n)} = \sum_{t=1}^{n} \log \frac{P(x_t)}{Q(x_t)}.$$

x^n 是長度 n 觀測樣本，最後等式假設各模型內的觀測獨立。log 可自然或底數二，但須一致。正值在概似比較中偏向 P。

P 下，期望二進位對數比為 $nD_{\mathrm{KL}}(P\|Q)$；Q 下為 $-nD_{\mathrm{KL}}(Q\|P)$。方向不同，是因期望使用不同法則。

概似比不是後驗機率。計算後驗勝算，需要乘先驗勝算；選擇行動，需要納入錯誤決策損失。零閾值適合某些對稱先驗與損失，不適合所有檢定。

簡單假說下，Neyman–Pearson 概似比原則描述指定偽警報限制的最強力檢定 [1]。「簡單」是分布固定，不是帶未知擬合參數的模型族。

若用同一樣本估計參數，就是另一套程序。模型選擇、干擾參數與閾值校準，都可能改變有限誤差。把擬合分類器分數當證據前，先用完全指定小模型理解。

檢定誤差形成取捨

更難宣布對立假說，可以減少偽警報，卻可能漏掉更多真對立情況。報告不應省略任一誤差。

設虛無分布 P_0、對立分布 P_1。拒絕區域 A 給

$$\alpha = P_0(X^n \in A), \qquad \beta = P_1(X^n \notin A).$$

α 是第一型或偽警報機率，β 是第二型或漏判對立機率，A 是宣布對立的集合。

Bernoulli 機率 $p_0 < p_1$ 時，對立相對虛無的對數概似比，是一的計數遞增函數。因此，計數閾值實作概似比檢定。

離散計數可能讓確定性閾值無法精確達到任意偽警報目標。邊界隨機化可以數學上達到中間操作點；部署規則應說明是否隨機化。

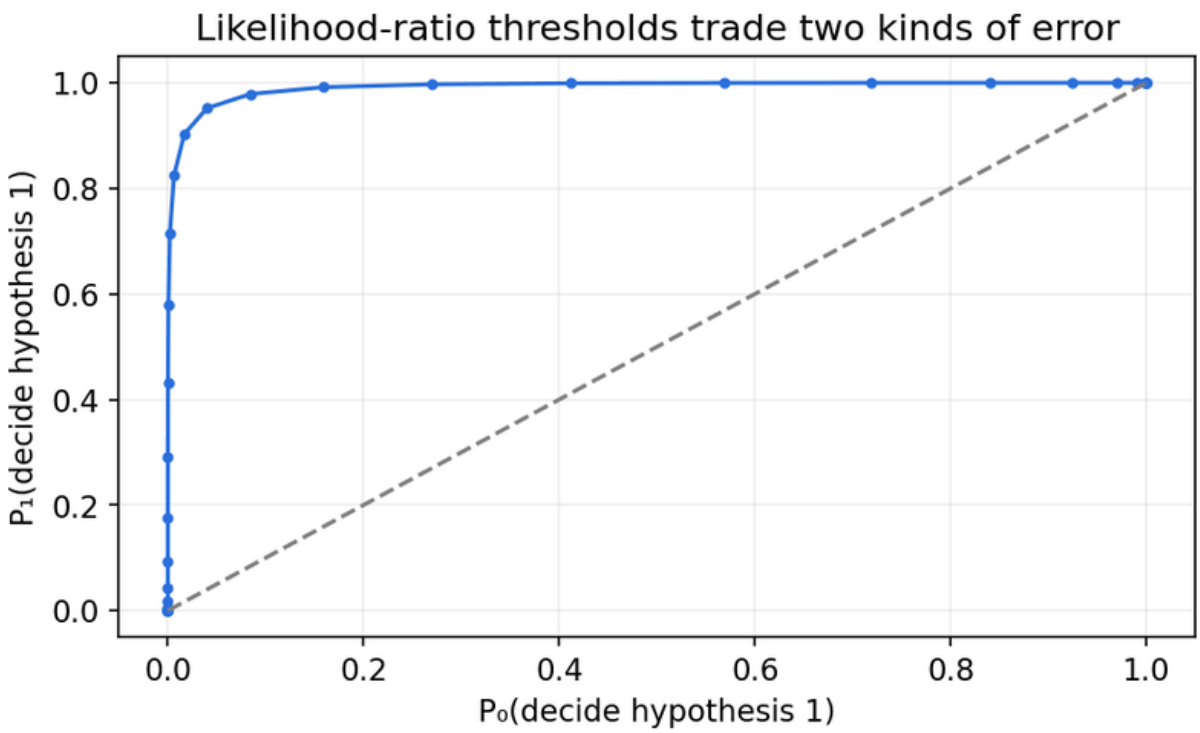


圖 *1* — 精確二項機率。偵測機率等於一減第二型誤差；每點對應明確計數閾值。

程式：精確誤差與 Monte Carlo 檢查

完整例子選擇首個符合 5% 偽警報限制的計數閾值，計算漏判機率，再分別驗證兩個法則。

```
import numpy as np
from scipy.stats import binom

n, p0, p1 = 30, 0.3, 0.6
thresholds = np.arange(n+2)
false_alarm = binom.sf(thresholds-1, n, p0)
detection = binom.sf(thresholds-1, n, p1)
threshold = thresholds[np.flatnonzero(false_alarm <= 0.05)[0]]
alpha = binom.sf(threshold-1, n, p0)
beta = binom.cdf(threshold-1, n, p1)
rng = np.random.default_rng(40)
observed_alpha = np.mean(rng.binomial(n, p0, 100000) >= threshold)
observed_beta = np.mean(rng.binomial(n, p1, 100000) < threshold)
print("Threshold / alpha / beta:", threshold, alpha, beta)
print("Simulation alpha / beta:", observed_alpha, observed_beta)
assert abs(observed_alpha-alpha) < 0.005
assert abs(observed_beta-beta) < 0.005
```

閾值使用已知分布，不在測試集搜尋。學習應用應在驗證資料調閾值，再以獨立測試觀測估計誤

差。資料群組與時間相依可能改變獨立性假設。

評估集中對立比例，影響正確宣布頻率，卻不改變固定檢定與法則的兩種條件誤差。即使偽警報機率小，對立極罕見時，仍可能有很多錯誤宣布。解讀時需結合條件誤差與目標先驗盛行率。這是條件機率直接結果，不是概似比原則失敗。應說明回報量以虛無、對立，還是檢定輸出為條件。

計算例：證據不等於信心

例子的虛無機率 0.3、對立機率 0.6，長度 30 區塊含 k 個一時，對立相對虛無的對數概似比為

$$\Lambda_k = k \ln \frac{0.6}{0.3} + (30 - k) \ln \frac{0.4}{0.7}.$$

k 是觀測計數，Λ_k 使用自然對數，兩個分數比較競爭法則下的一與零機率。18 個一時，$\Lambda_{18} \approx 5.761260$，概似比約 317.748375，證據明顯偏向對立。

若對立的先驗機率為 0.01，先驗勝算為 0.01/0.99。後驗勝算約 $317.748375 \times 0.01/0.99 = 3.209580$，對應後驗機率約 0.762447。概似比不是 99.7% 信心值，先驗會改變後驗解讀。

更實際的區別，是閾值檢定的全部陽性宣布。程式選閾值 14，偽警報約 0.040053，漏判約 0.048112。如果只有 1% 案例來自對立，陽性真正屬對立的機率為

$$\Pr(H_1 \mid \text{positive}) = \frac{\pi(1-\beta)}{\pi(1-\beta) + (1-\pi)\alpha} \approx 0.193588.$$

$\pi = 0.01$ 是對立先驗盛行率，α 是偽警報機率，β 是漏判機率。符號 H_1 表示案例由對立產生的事件。即使兩種條件誤差都低於 5%，此母體多數陽性仍是偽警報。

結論來自計算兩族群預期產生的陽性數，沒有改變概似比最佳性或量測條件誤差。決策報告應說明條件事件，讓讀者區分個別樣本後驗與所有陽性的平均組成。

```
import numpy as np
from scipy.stats import binom

n, p0, p1, prior = 30, 0.3, 0.6, 0.01
count = 18
log_ratio = count*np.log(p1/p0)+(n-count)*np.log((1-p1)/(1-p0))
posterior = 1/(1+((1-prior)/prior)*np.exp(-log_ratio))
threshold = 14
alpha = binom.sf(threshold-1, n, p0)
beta = binom.cdf(threshold-1, n, p1)
positive_precision = prior*(1-beta)/(prior*(1-beta)+(1-prior)*alpha)
assert alpha <= 0.05 and binom.sf(threshold-2, n, p0) > 0.05
print("Likelihood ratio / sample posterior:", np.exp(log_ratio), posterior)
print("Probability alternative given positive:", positive_precision)
```

經驗型態揭露散度為何控制罕見事件

許多字串具有同一直方圖。數量以熵指數成長，單一來源機率以交叉熵指數縮小，差就是散度。

大小 k 的有限字母中，長度 n 序列的型態 $\hat{p}$ 是經驗符號分布。型態類 $T(\hat{p})$ 包含所有相同計數序列。獨立來源法則 q 下，

$$q(x^n) = 2^{-n[H(\hat{p})+D_{\mathrm{KL}}(\hat{p}\|q)]}.$$

x^n 具有型態 $\hat{p}$，$H(\hat{p})$ 是經驗熵，散度以位元計。將相同對數機率按計數分組，即得公式。

類別大小的保守多項式界為

$$(n+1)^{-k}2^{nH(\hat{p})} \le |T(\hat{p})| \le 2^{nH(\hat{p})}.$$

$|T|$ 是序列數，n,k 是區塊長度與字母大小。乘單一序列機率得到

$$(n+1)^{-k}2^{-nD_{\mathrm{KL}}(\hat{p}\|q)} \le q(T(\hat{p})) \le 2^{-nD_{\mathrm{KL}}(\hat{p}\|q)}.$$

$q(T)$ 是型態類總機率。有限長度多項式因子重要，但會從正規化對數指數中消失 [1,2]。

方向值得強調：第一引數是異常經驗型態，第二是實際生成來源。這不同於在固定真來源下平均的編碼失配計算。

經驗分布集合的主要指數行為，由允許型態中最小散度控制。Sanov 定理在有限字母與集合正則條件下，形式化此大偏差陳述 [1]。

程式：精確二元型態機率與指數

獨立例子比較二項機率與指數界，使用對數避免下溢，並檢查有限多項式間隙。

```
import numpy as np
from scipy.stats import binom

q, target = 0.3, 0.6
for n in [30, 100, 300, 1000]:
    count = int(round(n*target))
    empirical = count/n
    divergence = (empirical*np.log2(empirical/q) +
                  (1-empirical)*np.log2((1-empirical)/(1-q)))
    negative_log_probability = -binom.logpmf(count, n, q)/np.log(2)
    exponent = n*divergence
    assert exponent <= negative_log_probability + 1e-10
    assert negative_log_probability <= exponent + 2*np.log2(n+1) + 1e-10
    print("Length / observed normalized exponent / divergence:",
          n, negative_log_probability/n, divergence)
```

罕見精確型態，不同於單邊尾端或典型鄰域。尾端指數來自對事件全部計數最佳化，不是直接代入最極端計數。

從單一型態到型態集合

由經驗分布集合 A 定義的事件，至多有 $(n+1)^k$ 種型態。加總上界得到

$$q(\hat{p} \in \mathcal{A}) \le (n+1)^k 2^{-n \inf_{v \in \mathcal{A}_n} D_{\mathrm{KL}}(v\|q)}.$$

$\mathcal{A}_n$ 是 $\mathcal{A}$ 中長度 n 可實現型態，v 是候選型態，下確界選 q 下最不意外的型態。空事件機率零，另行處理。

罕見事件最可能部分，常在最接近來源的邊界，而非最戲劇化極端。二元來源機率 0.3，要求頻率至少 0.6 的尾端，主要由接近 0.6 而非一控制。

有限型態格點重要：所需邊界可能在可達分數間。精確二項總和處理離散性；漸近散度圖像解釋趨勢，同時保留取整與多項式因子。

這可幫助解讀表面分布偏移。異常直方圖可作為反對來源法則的證據，卻不能辨認原因。相依、選擇偏差、量測改變與不同來源，都可能改變頻率。

非對稱與對稱假說指數

不同方式分配兩種檢定誤差，會導向不同資訊極限。散度只是其中一種，不是所有分類問題的答案。

假設虛無 P、對立 Q 下觀測獨立，第一型誤差保留在固定非平凡容許值內。最佳第二型指數趨近

$$-\frac{1}{n}\ln \beta_n \to D_{\mathrm{nats}}(P\|Q).$$

β_n 是對立下誤接受虛無的機率，D_{nats} 使用自然對數。這是適當來源與支撐條件下的經典 Stein 漸近設定 [1]。

方向依接受區域：典型虛無觀測必須仍被接受，對立看起來像虛無的機率，具有 P 到 Q 散度指數。交換假說會反轉散度，也可能改變指數。

固定非零先驗與對稱 Bayesian 誤差目標下，指數改為 Chernoff 資訊：

$$C_{\mathrm{Ch}}(P,Q) = -\min_{0\le s\le 1} \ln \sum_x P(x)^s Q(x)^{1-s}.$$

s 在兩法則間內插，x 遍歷有限字母。雖然加總項看似非對稱，整個量是對稱的。

指數描述大獨立樣本，不是短樣本精確誤差。前面的二項實驗直接計算有限情況。前置因子與決策邊界重要時，把漸近指數當精確機率可能不準。

複合假說、相依、改變中的母體與不等損失，都需要各自分析。資訊極限可解釋哪些來源差異有助區分，仍留下實際校準與抽樣設計問題。

推論中的充分性與資料處理

某摘要對參數充分，表示它在指定模型族中保留樣本的全部參數證據。

對模型族 $p_\theta(x^n)$，若給定統計量 $T(X^n)$ 的樣本條件法則不依賴 θ，就稱充分。分解準則為

$$p_\theta(x^n) = g_\theta(T(x^n))h(x^n).$$

θ 是參數，T 是統計量，g_θ 是依參數的因子，h 是與參數無關因子，須有相應模型族與支配假設 [1]。

獨立 Bernoulli 試驗的一計數，對未知機率充分；順序在此族不含額外參數證據。Markov 族則可能需要順序，單一一計數一般不再充分。

隨機參數 Θ 有指定先驗時，充分性給出 $I(\Theta; X^n) = I(\Theta; T(X^n))$ 所需條件關係。任意摘要只有資料處理上界。

「在模型族中」很重要。某假設模型的充分統計量，可能丟棄模型錯誤證據。平均值能估位置，卻隱藏多峰、相依或離群模式。

這連到學得表示：保留某目標資訊，不同於保留原資料全部診斷性質。充分性宣稱應伴隨任務與來源族。

Fisher 資訊衡量局部參數敏感度

Fisher 資訊衡量參數微移時，對數機率改變多強。它是模型局部性質，不同於隨機變數互資訊。

對純量參數 θ 與正則密度或質量函數 p_θ，定義分數與資訊：

$$s_\theta(X) = \frac{\partial}{\partial \theta} \ln p_\theta(X), \qquad \mathcal{J}(\theta) = \mathbb{E}_\theta[s_\theta(X)^2].$$

s_θ 是分數（score），$\mathcal{J}$ 是每觀測 Fisher 資訊，期望在參數化模型下。正則條件下，分數期望零。

n 個獨立觀測的資訊相加成 $n\mathcal{J}(\theta)$。正則不偏估計器 $\hat{\theta}$ 的 Cramér–Rao 不等式為

$$\operatorname{Var}_\theta(\hat{\theta}) \geq \frac{1}{n\mathcal{J}(\theta)}.$$

變異數描述固定參數的重複樣本。此界需要可微性、微分與積分交換，以及適當支撐、不偏假設 [1]。

Bernoulli 機率 $\theta \in (0,1)$ 的 $\mathcal{J}(\theta) = 1/[\theta(1-\theta)]$。樣本均值不偏，變異數 $\theta(1-\theta)/n$，達到該界。

資訊會隨參數座標改變。把機率重參數化為對數勝算，會以參數導數平方縮放資訊。統計族不變，只是參數誤差單位改變。

支撐依參數的非正則模型或有偏估計器，需要不同界。經驗變異數小，若均值偏離參數，也不能單獨證明準確。

推導 Bernoulli 資訊與變異數界

單一 Bernoulli 觀測 $x \in \{0,1\}$ 的對數概似為 $x \ln\theta + (1-x)\ln(1-\theta)$。微分得到

$$s_\theta(x) = \frac{x}{\theta} - \frac{1-x}{1-\theta} = \frac{x-\theta}{\theta(1-\theta)}.$$

$\theta \in (0,1)$ 是一的機率，s_θ 是分數。其兩種值為 $1/\theta$、$-1/(1-\theta)$，按機率 θ、$1-\theta$ 平均平方，得到 $\mathcal{J}(\theta) = 1/[\theta(1-\theta)]$。

為何倒數限制估計變異數？令 T 是依 n 個獨立觀測建立的正則不偏估計器，S_n 是分數總和。在正則假設下，對不偏性 $\mathbb{E}_\theta T = \theta$ 微分：

$$1 = \frac{d}{d\theta}\mathbb{E}_\theta T = \mathbb{E}_\theta[(T-\theta)S_n].$$

T 是觀測樣本的固定函數，$S_n = \sum_{i=1}^{n} s_\theta(X_i)$ 是總分數，且 $\mathbb{E}_\theta S_n = 0$。右側是估計器與分數的共變異數。共變異數平方不超過兩個變異數乘積；再利用 $\mathrm{Var}_\theta(S_n) = n\mathcal{J}(\theta)$，就得到變異數界。

$\theta = 0.3$、$n = 100$ 時，每觀測資訊約 4.761905，變異數下界 0.0021。樣本均值達到下界，因為中心化估計誤差與總分數成正比。獨立樣本數增加四倍，變異數除以四，標準差除以二。分數變異數能相加是因獨立；反覆測量同一相關試驗，不足以支持此縮放。

Fisher 資訊是局部散度曲率

適當二次可微正則條件下，鄰近參數分布滿足

$$D_{\mathrm{nats}}(p_\theta \| p_{\theta+\delta}) = \frac{1}{2}\mathcal{J}(\theta)\delta^2 + o(\delta^2).$$

δ 是小參數位移，$\mathcal{J}(\theta)$ 是自然對數分數資訊，$o(\delta^2)$ 的餘項除以 δ^2 趨零。底數二散度的首項要除以 $\ln 2$。

一階導數消失，因分數期望零；正則模型的二階導數給 Fisher 資訊。這連結局部估計敏感度與檢定、編碼使用的全域散度。

它是局部近似。大參數移動可能涉及不同曲率、支撐改變或多峰。把一個 Fisher 值當整個族的精確距離，會丟棄差異。

最大熵讓限制明確

若只知道少數期望，就從符合期望的分布選最大熵。答案依指定限制與允許支撐。

有限字母的特徵 $\phi_j(x)$ 要求 $\mathbb{E}[\phi_j(X)] = m_j$。帶正規化的最大熵給指數族形式

$$p_\lambda(x) = \frac{\exp(\sum_j \lambda_j \phi_j(x))}{Z(\lambda)}, \qquad Z(\lambda) = \sum_x \exp(\sum_j \lambda_j \phi_j(x)).$$

λ_j 是符合動差限制的乘子，Z 正規化質量函數。正號只是乘子符號慣例；存在性與內部支撐條件重要。

微分 $\ln Z$ 給受限制特徵期望。只知道均值、沒有支撐，不會決定熵最佳值。有界區間固定支撐無其他限制，給均勻密度；實數固定均值與變異數給高斯；非負支撐固定均值給指數密度 [1]。

最大熵是透明建模原則，不是來源真的屬於此族的證據。未量測相依與高階動差仍可能存在。應記錄限制、擬合乘子，再用獨立資料測預測。

指數形式也解釋與散度的關係。候選分布若符合最大熵模型的特徵期望，平均模型對數機率只依賴共享動差。同一對數底數下，候選到模型的相對熵，等於模型熵減候選熵。

這再次是限制把交叉熵項變成固定量。前章高斯最大熵證明就是連續動差例子。共同代數說明最大熵、指數族與資訊投影為何反覆一起出現。

推導與解讀最大熵分布

最佳化先使用自然對數，只會縮放熵與乘子。把正規化與特徵限制加到熵目標：

$$\mathcal{F}(p) = -\sum_x p_x \ln p_x + \alpha\left(\sum_x p_x - 1\right) + \sum_j \lambda_j \left(\sum_x p_x \phi_j(x) - m_j\right).$$

p_x 是候選機率，α 是正規化乘子，λ_j 是特徵乘子，$\phi_j(x)$ 是特徵 j，m_j 是要求期望。內部最佳點對各機率微分，得到 $-\ln p_x - 1 + \alpha + \sum_j \lambda_j \phi_j(x) = 0$；取指數再正規化，就得到前述指數形式。

狀態 0、1、2、平均 0.7 只有一個特徵 $\phi(x) = x$，相對權重為 $1, e^{\lambda}, e^{2\lambda}$。均勻分布平均一，較低目標平均因此需要負乘子。程式找到 $\lambda \approx -0.466121$，機率約 0.494780、0.310440、0.194780。

模型在符合平均的同時，將機率散布於全部允許狀態。點質量或兩狀態混合也可能符合某些動差，但此處熵較低。可行邊界上，部分機率可能為零，有限內部乘子不一定足夠。支撐、動差與存在條件是數學問題的一部分，不是選好喜歡密度後才補上的選擇。

程式：解有限最大熵模型並驗證 Fisher 變異數

完整例子求指定均值的三狀態分布，再比較 Bernoulli 估計器變異數與精確不偏界。

```
import numpy as np
from scipy.optimize import brentq
from scipy.special import softmax

states = np.array([0.0, 1.0, 2.0])
target_mean = 0.7
probability = lambda lam: softmax(lam*states)
lam = brentq(lambda value: probability(value) @ states-target_mean, -20, 20)
p = probability(lam)
assert abs(p @ states-target_mean) < 1e-12
print("Maximum-entropy multiplier / probabilities:", lam, p)

rng = np.random.default_rng(41)
theta, n = 0.3, 100
estimates = rng.binomial(n, theta, size=100000)/n
fisher = 1/(theta*(1-theta))
bound = 1/(n*fisher)
print("Empirical estimator variance / exact bound:", estimates.var(), bound)
assert abs(estimates.var()-bound) < 0.0001
```

最大熵計算精確使用列出的支撐；允許其他狀態會改變最佳化。變異數計算檢查合成正則例子，不是所有估計器的通用下界。

對數成長把失配轉成決策成本

重複乘法遊戲中，策略錯誤會累積。指定公平報酬模型下，最大期望對數成長使最佳配置符合結果機率。

結果 i 以機率 p_i 發生。配置比例 $q_i > 0$ 到其條件報酬，$\sum_i q_i = 1$。公平參考法則 $r_i > 0$ 在結果 i 發生時付乘數 $1/r_i$。期望二進位對數成長為

$$g(p, q; r) = \sum_i p_i \log_2 \frac{q_i}{r_i} = D_{\mathrm{KL}}(p\|r) - D_{\mathrm{KL}}(p\|q).$$

p 是結果法則，q 是配置，r 是報酬倒數機率向量，g 是每次試驗對數乘數。恆等式使用正規化法則與指定條件報酬遊戲 [1]。

最佳解在來源支撐為 $q = p$。失配策略精確損失 $D(p\|q)$ 的期望對數成長。兩結果皆乘數二時，r 公平，最佳值為 $1 - h_2(p)$。

這是預測與成長的 Kelly 連結，不表示每個投資組合都有公平條件報酬，或機率已知。加入現金、交易成本、槓桿限制或不同報酬，會改變最佳化。

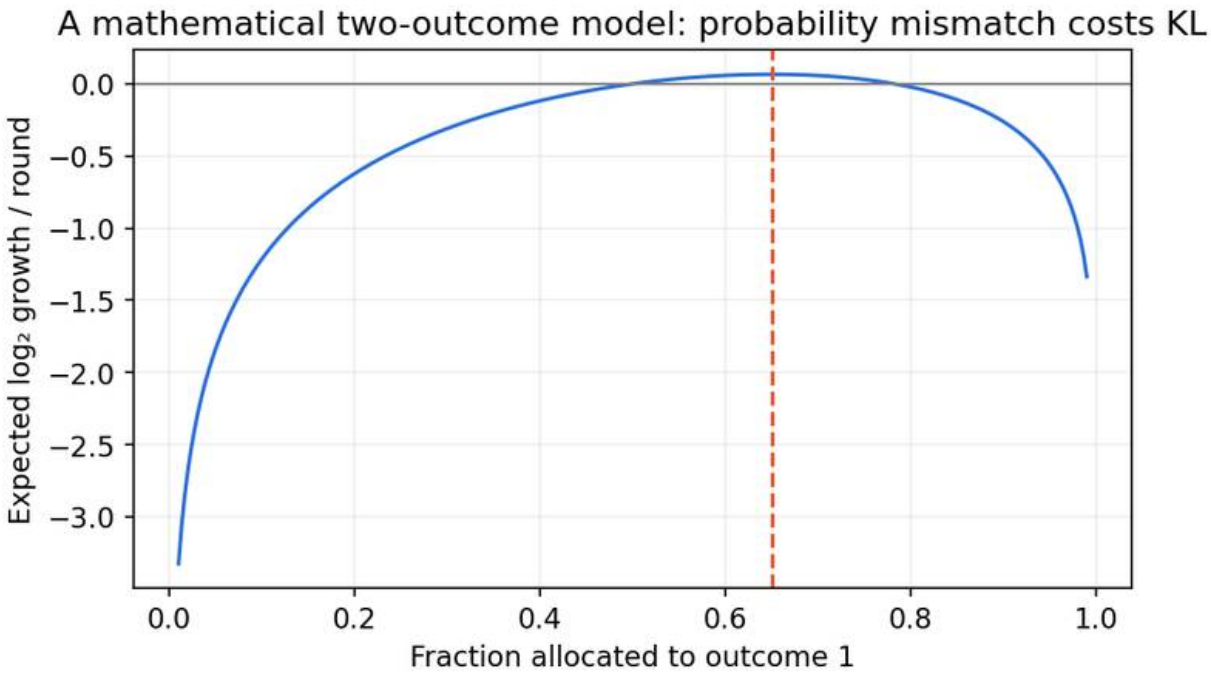


圖 2 — 解析雙結果報酬模型。相對最佳值的下降，就是機率模型散度。

每次結果前可觀察旁資訊 Y 時，條件最佳化配置。相對固定最佳配置的改善為

$$\mathbb{E}_Y[g(p_{X|Y}, p_{X|Y}; r)] - g(p_X, p_X; r) = I(X; Y).$$

X 是結果，Y 是旁資訊，參考報酬法則 r 不變。在此特定對數成長遊戲，互資訊有操作價值。

通用混合連結投資組合與編碼

不只選一個固定配置，而是混合許多固定策略的財富。過去結果決定各策略之後貢獻多少。

正報酬向量 R_t 與固定單純形配置 b 的累積財富為

$$W_n(b) = \prod_{t=1}^{n} b^\top R_t, \qquad W_n^{\mathrm{mix}} = \int W_n(b)\,\pi(db).$$

t 索引試驗，b 非負且總和一，R_t 含各分量相對報酬，π 是固定配置的先驗測度。混合有等價逐步配置，依過去財富加權策略。

Cover 與 Thomas 第 16 章在指定條件下，發展通用投資組合與長期比較最佳固定配置的想法 [1]。基準是事後固定配置，不是無限制、知道未來的交易者。

雙結果公平條件遊戲中，機率參數均勻先驗，給與 beta 混合相同的預測機制。t 次後，一的逐步機率為 $(N_1 + 1)/(t + 2)$。這是整數計數先驗，不同於無損編碼的半計數預測器。

程式：混合成長與最佳固定配置

完整例子以對數計財富，用 beta 積分求均勻混合。比較已知來源策略、回顧最佳固定策略與因果混合。

```
import numpy as np
from scipy.special import betaln

rng = np.random.default_rng(42)
p, n = 0.65, 2000
outcomes = rng.binomial(1, p, size=n)
ones, zeros = int(outcomes.sum()), n-int(outcomes.sum())
fitted = ones/n
oracle_log_wealth = n*np.log(2)+ones*np.log(p)+zeros*np.log(1-p)
best_log_wealth = n*np.log(2)+ones*np.log(fitted)+zeros*np.log(1-fitted)
mixture_log_wealth = n*np.log(2)+betaln(ones+1, zeros+1)
sequential_log_wealth, previous_ones = 0.0, 0
for t, outcome in enumerate(outcomes):
    q = (previous_ones+1)/(t+2)
    sequential_log_wealth += np.log(2*(q if outcome else 1-q))
    previous_ones += int(outcome)
assert abs(sequential_log_wealth-mixture_log_wealth) < 1e-9
assert mixture_log_wealth <= best_log_wealth
print("Source / best fixed / mixture growth bits per trial:",
      oracle_log_wealth/n/np.log(2), best_log_wealth/n/np.log(2),
      mixture_log_wealth/n/np.log(2))
print("Mixture regret to best fixed, bits:",
      (best_log_wealth-mixture_log_wealth)/np.log(2))
```

這是受控合成報酬實驗。最佳固定參數用整段序列，作為基準；混合則因果更新。對數財富減最佳基準，與負對數機率悔損有相同代數，但方向相反。

現代觀點

Hellström、Durisi、Guedj 與 Raginsky 的 2025 年出版專論 [3]，統一資訊論與 PAC-Bayesian 泛化界。分類區分期望、機率與條件互資訊表示，清楚指出統計保證需要指定隨機物件與評估準則。

El Gamal 的 2026 年擬議學習章 [4]，以教科書式說明連結散度恆等式與統計學習。它是草稿補充；本章把現代觀點搭配已出版第二版，不把擬議章當成完成新版。

我們推論，同一量只有指定模型後，才能具有不同操作意義。散度可以是編碼懲罰、異常型態指數、證據速率或對數成長損失。方向、對數底數與極限設定可能各異。

學習推論的開放問題包括大參數空間可靠資訊估計、相依或偏移資料保證，以及能對實際模型選擇程序提供資訊的界。資訊項無窮或難估時，形式不等式可能變得無用。

實際工作應搭配獨立有限實驗：精確小機率驗證實作、保留資料概似評估擬合、決策誤差評估用途。單一熵數值不能代替三者。

最後一章結合多來源、多接收者、較強不等式，並以小型分散式資訊專題連到現代學習。

重點整理

- 概似證據、後驗勝算與決策閾值不同。
- 檢定同時回報偽警報與漏判機率。
- 型態機率有散度指數，須明確引數方向。
- 假說指數依所選誤差設定。
- 充分性相對於模型族與目標參數。
- Fisher 資訊給局部正則估計極限。
- 最大熵依賴支撐與指定限制。
- 對數成長失配、通用混合悔損共享編碼恆等式。

練習

1. 反轉二項例子的虛無與對立，找 5% 偽警報限制閾值，比較漏判誤差。

提示

拒絕區域變低計數。

2. 按經驗計數分組符號，推導型態機率恆等式，說明散度引數順序。

提示

計數乘來源對數機率。

3. 比較 Bernoulli 機率 0.3、0.6 的雙向 Stein 散度，說明非對稱結果不違反對稱 Chernoff 指數。

提示

先固定接受區域，再選散度。

4. 將 Bernoulli 機率重參數化為對數勝算，推導 Fisher 資訊並解釋誤差單位。

提示

乘機率導數平方。

5. 求狀態 0、1、2、3、均值 1.2 的最大熵分布，再加入二階動差限制，描述額外乘子。

提示

第二特徵是狀態平方。

6. 公平條件遊戲中，來源機率 0.65，求配置 0.5、0.65、0.8 的期望成長，驗證散度損失，並說明哪種策略知道未來。

提示

沒有策略知道未來；事後基準只在觀察後擬合常數。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 6, 11–12 and 16. 出版社書頁與目錄。
2. T. Weissman, “Method of Types,” Stanford EE276 Lecture 15 notes, 2024. 第一手講義。
3. F. Hellström, G. Durisi, B. Guedj and M. Raginsky, “Generalization Bounds: Perspectives from Information Theory and PAC-Bayes,” *Foundations and Trends in Machine Learning*, 18(1), 1–223, 2025. 出版社資訊，作者版本。
4. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. 草稿與出版狀態。

IT 08・網路、資訊不等式與學習

分析分散式來源與多使用者，連結進階不等式和學習，並執行有限分散式解碼專題。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/networks-information-learning/

你將學到

- 描述分散式來源與多重存取通道的速率區域。
- 解釋廣播、中繼與一般網路為何需要不同論證。
- 使用 log-sum、Pinsker、熵功率與條件資訊不等式。
- 區分表示資訊與訓練樣本資訊。
- 執行小型分散式同位檢查解碼器，明確回報有限誤差。
- 把完整書籍章節地圖連到現代學習，不過度宣稱極限。

先看全貌

一個發送者與接收者會給單一容量，多個來源或接收者則給相容速率區域。提高某速率可能降低另一個；資訊也可能只有合在一起才有用，各分量單看卻沒有用。

> **用最簡單的話說**
>
> 兩位朋友各知道一條線索。單看任一線索可能毫無幫助，合起來卻解開謎題。若兩人同時說話，還得共享同一吵雜房間。我們必須追蹤哪些線索共享、誰聽到什麼，以及每人能說多少。

最後一章依 Cover 與 Thomas 第 15 章網路與第 17 章不等式 [1]。其他十五章已由前面內容與 IT 01 閱讀地圖涵蓋。最後以有限合成實驗連結來源相依、編碼、解碼與任務資訊。

課程使用已出版第二版，並以當代綜述與明確標示的 2026 年擬議章補充。

分散式來源編碼：相關性節省速率

兩編碼器可以各自描述相關觀測，讓聯合解碼器利用關係，不必每個都獨立發送全部位元。

獨立點對 (X_t, Y_t) 依固定有限聯合法則。第一編碼器只看 X^n，第二只看 Y^n，共同解碼器重建兩區塊。Slepian–Wolf 漸近近乎無損區域為

$$R_X \geq H(X \mid Y), \qquad R_Y \geq H(Y \mid X), \qquad R_X + R_Y \geq H(X, Y).$$

R_X, R_Y 是每對來源樣本由各編碼器發送的位元，熵使用指定聯合來源。邊界不等式描述閉包；編碼以適當速率餘裕操作，使區塊誤差趨零 [1]。

個別界假設另一來源可以在解碼時幫忙；總和界計全部聯合不確定性。沒有假設編碼器看見另一方目前區塊；允許此觀測會定義不同問題。

公平 X、$Y = X \oplus E$、獨立 Bernoulli 雜訊機率 e 下，兩邊際熵都一，聯合熵 $1 + h_2(e)$。忽略相依的分別編碼，漸近需每對兩位元；聯合解碼可利用較小聯合不確定性。

一個角點把 Y 以完整邊際速率傳送，並把 X 壓到接近 $H(X \mid Y)$。解碼器已有重建 Y 作旁資訊，啟發後面的同位檢查症候實驗。

隨機分箱提供存在論證：把來源區塊放入箱，再用另一已解碼來源，找發送箱中可能匹配區塊。箱索引比完整來源索引短，相關性解開歧義。

這不保證每個有限點對精確重建。罕見或模糊區塊可能失敗。真正有限無損程序，若要求每個允許實現都恢復，需額外逃脫機制或其他處理。

隨機分箱為什麼有三個速率限制？

各編碼器把每個來源區塊隨機指派到箱，傳送箱索引。解碼器在兩個指定箱中尋找聯合典型點對。競爭點對可能有三種錯誤：只有 X 錯、只有 Y 錯，或兩者都錯。

競爭點對	典型候選數尺度	每候選碰撞機率
X 錯、Y 正確	$2^{nH(X\mid Y)}$	2^{-nR_X}
X 正確、Y 錯	$2^{nH(Y\mid X)}$	2^{-nR_Y}
兩者都錯	$2^{nH(X,Y)}$	$2^{-n(R_X+R_Y)}$

n 是配對區塊長度，R_X, R_Y 是傳送箱索引速率，熵指數依固定聯合來源。尺度省略小典型性容許誤差與有限前置因子。獨立箱指派給碰撞因子，聯合典型性則限制需要考慮的候選數。

候選數乘碰撞機率，再用聯集界限。適當餘裕下，三項全部趨零的條件，正是兩個別速率超過條件熵、總和超過聯合熵。真正點對未通過典型性測試的機率也趨零。平均所有指派後，便證明某固定共享分箱規則具有小區塊錯誤 [1]。

證明解釋相關性的好處與限制。解碼旁資訊把 X 候選數從完整來源目錄降成條件目錄，卻不消除區分剩餘候選的需求。總和限制則處理兩個別界限無法單獨排除的同時歧義。

公平位元、翻轉機率 0.1 時，條件熵約 0.468996，聯合熵每對 1.468996 位元。角點約 $(0.468996, 1)$、$(1, 0.468996)$，對稱點的各編碼器速率是 0.734498。各給 0.5 雖通過個別界限，卻違反總和界限。這些是極限邊界值，實際漸近構造會保留餘裕，不保證短區塊恰在邊界可行。

多重存取：獨立訊息共享輸出

兩發送者都可對同一接收器通訊。接收器觀測不只限制每人速率，也限制資訊總和。

有限無記憶通道 $W(y \mid x_1, x_2)$、獨立訊息與固定乘積輸入法則的區域包括

$$R_1 \le I(X_1; Y \mid X_2), \qquad R_2 \le I(X_2; Y \mid X_1), \qquad R_1 + R_2 \le I(X_1, X_2; Y).$$

X_1, X_2 是發送者輸入，Y 是共享輸出，R_1, R_2 是每次共同通道使用位元。對乘積輸入法則取聯集與分時，得到一般雙使用者多重存取容量區域 [1]。

個別界的條件化表示知道另一輸入時，接收器如何區分該使用者訊息；總和界處理聯合解碼。各自最大化個別式，不會取消共享總和限制。

分時引入共同預定變數 T，給定 T 時輸入為乘積。各界也條件於 T，把操作點凸化。共享排程不代表發送者取得任意依訊息的合作能力。

獨立實數高斯訊號、功率 P_1, P_2、加性雜訊變異數 N 下，

$$R_1 \le \tfrac{1}{2}\log_2(1 + P_1/N),$$
$$R_2 \le \tfrac{1}{2}\log_2(1 + P_2/N),$$
$$R_1 + R_2 \le \tfrac{1}{2}\log_2(1 + (P_1 + P_2)/N).$$

一次使用含兩個同時實數輸入，訊號有獨立訊息與標準平均功率限制。二分之一保留實數慣例。

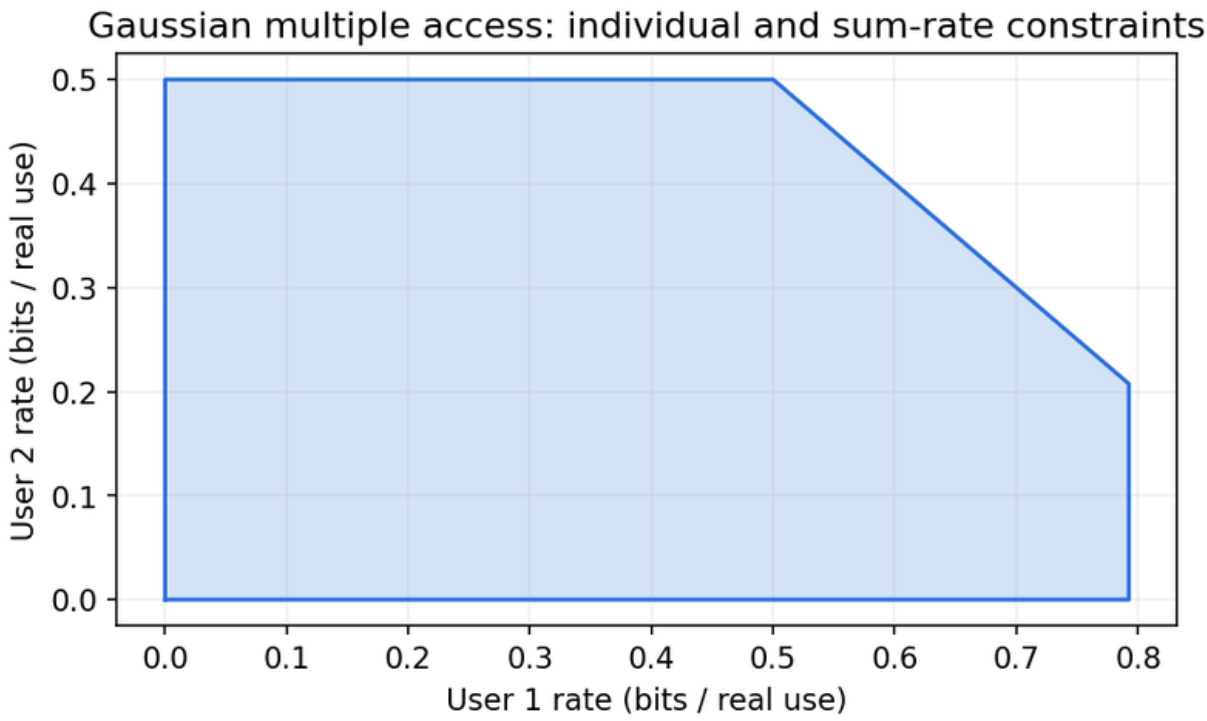


圖 *1* — P_1=2、P_2=1、N=1。總速率限制使兩人的個別最大值無法同時達到。

逐次解碼可達角點：先把另一訊息當雜訊，解一個訊息，再移除已解碼貢獻。順序改變誰取得較有利的條件速率。聯合解碼與分時描述更廣區域。

手算兩種高斯解碼順序

圖中功率 $P_1 = 2$、$P_2 = 1$，雜訊變異數 $N = 1$。使用者一的個別界限約每次 0.792481 位元，使用者二為 0.5，共享總和界限一。點 $(0.75, 0.4)$ 通過兩個別界，卻違反總和；$(0.6, 0.35)$ 則通過三者。

先解使用者一時，暫時把使用者二的獨立高斯訊號當成變異數一的額外雜訊，因此第一人的速率

限制為 $\frac{1}{2}\log_2(1+2/(1+1)) = 0.5$。成功解碼並相減後，第二人只剩原雜訊，可達 0.5，得到角點 $(0.5, 0.5)$。

先解第二人時，其初始速率為 $\frac{1}{2}\log_2(1+1/(1+2)) \approx 0.207519$。移除後，第一人可達 $\frac{1}{2}\log_2(1+2/1) \approx 0.792481$，得到角點 $(0.792481, 0.207519)$。兩角點總和都是一，但解碼順序改變分配。

相減論證假設第一訊息解碼正確。第一個錯誤決策可能傳播至第二個，因此有限接收器必須評估聯合訊息錯誤，也評估個別錯誤。漸近角點可指引設計，不應掩蓋此實作問題。

廣播與中繼改變資訊模式

一發送者、兩接收者，不是兩發送者、一接收者的反向問題。先聽再傳的中繼，又是另一種模式。

物理退化廣播通道的 $X \to Y_1 \to Y_2$，讓接收器一在指定統計意義較強。疊加建構使用 $U \to X \to Y_1 \to Y_2$ 的輔助變數 U：

$$R_2 \le I(U; Y_2), \qquad R_1 \le I(X; Y_1 \mid U).$$

R_2 是較弱接收器私人訊息速率，R_1 是較強接收器私人訊息速率，U 是兩者可解的層。對適當 $p(u,x)$ 取聯集，給退化通道容量區域 [1]。

Markov 順序是假設接收觀測的關係，不只是依樣本準確度排序。一般廣播通道不一定退化，不能繼承此精確區域。

無記憶中繼的來源輸入 X_s、中繼輸入 X_r、中繼觀測 Y_r、目的端觀測 Y_d，有 cut-set 上界

$$C_{\text{relay}} \le \max_{p(x_s,x_r)} \min\{I(X_s; Y_r, Y_d \mid X_r), I(X_s, X_r; Y_d)\}.$$

第一切割把來源與中繼加目的端分開；第二把來源加中繼與目的端分開。上界輸入聯合法則允許統計合作，作為外部鬆弛。

Cut-set 界不會自動可達。中繼可以解訊息、描述觀測或採其他因果策略，各有條件。許多一般網路容量仍未解。

重點是先畫實際資訊模式，再用公式：獨立訊息、相關來源、旁觀測、因果中繼與接收需求，都是不同假設。單一點對點容量不能概括全部。

聯合用途可能不見於邊際資訊

兩個各自沒有資訊的特徵，可能一起決定目標。一次只排名一特徵，可能把兩者都丟掉。

令 X_1, X_2 是獨立公平位元，$Y = X_1 \oplus X_2$。則

$$I(X_1; Y) = I(X_2; Y) = 0, \qquad I(X_1, X_2; Y) = 1, \qquad I(X_1; Y \mid X_2) = 1.$$

Y 是二元互斥或目標。任一邊際輸入仍讓目標公平，知道兩者就精確決定；給定另一輸入，第一輸入顯示完整相關性。

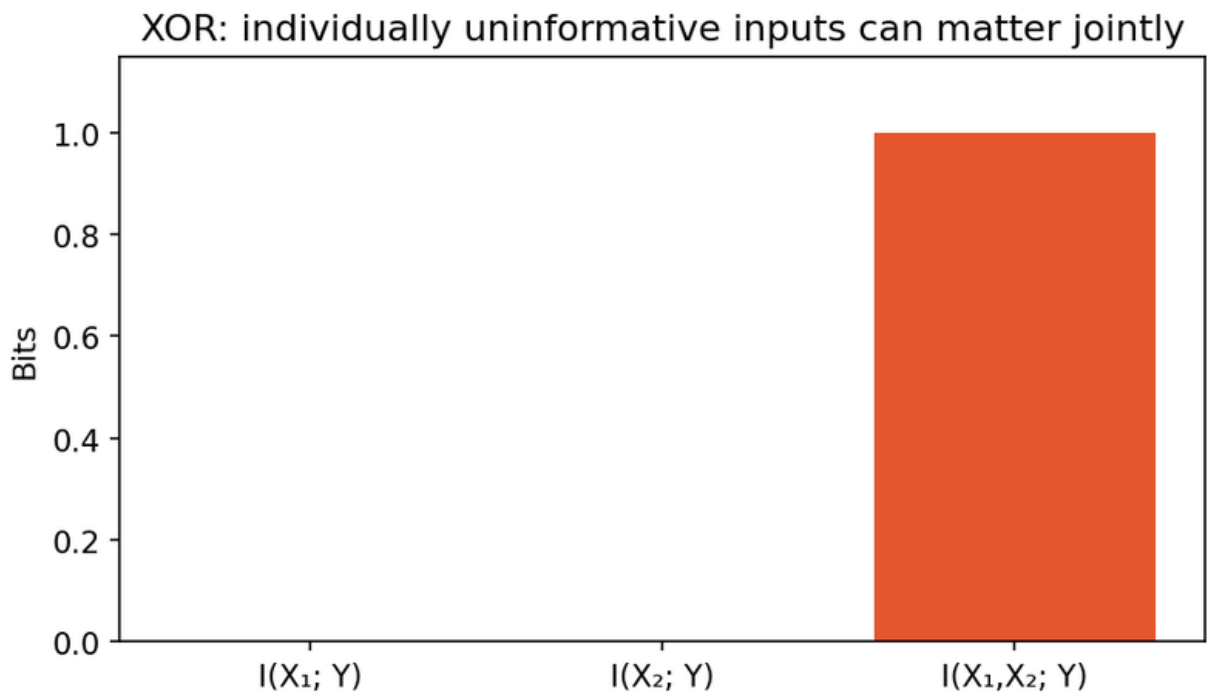


圖 2 — 精確合成聯合法則。示範聯合用途，沒有援引特定冗餘與協同分解。

「協同」可非正式描述只在合併後出現的資訊，但一般量化冗餘與協同分解，需要額外定義。三個互資訊值本身，不會定義任意變數的通用分解。

這對多模態學習重要。單看弱的模態可能在上下文有用；兩個強邊際特徵也可能大量重疊。應分析聯合法則與接收任務，不把邊際相關等同唯一貢獻。

計算函數是另一個分散式問題

假設各編碼器觀察獨立公平區塊 X_1^n, X_2^n，解碼器只需逐座標互斥或。輸出每對有一位元熵，很容易以為每對總共一位元就夠。

這忽略誰看輸入。額外把第二原始區塊給解碼器：若能從兩傳送索引求互斥或，就能恢復第一區塊。因輸入獨立，第二編碼器索引沒有提供額外第一區塊資訊。

因此，本例第一編碼器要使誤差趨零，就必須傳約每座標一位元；對稱地，第二也一樣。所需函數熵雖一，分散式總需求仍可能兩位元。

若中央編碼器看到兩輸入，就可直接用每對一位元傳互斥或。目標不變，觀測模式不同。這具體說明任務導向速率宣稱應附網路圖。

本例也區分兩種「聯合資訊」。接收者需要聯合計算，發送者卻沒有聯合觀測。不能只因某中央特徵算好後足夠，就假設學習系統已取得它。

實際應用可能有相關輸入、互動、共同隨機性或其他旁資訊，都會改變問題。未推導相應限制前，不能套用獨立非互動例子的速率結論。

較強不等式延伸工具

資訊不等式常比較合併結果、改變分布或加入獨立雜訊後還剩什麼。單位與假設決定意義。

非負 a_i、正 b_i 的 log-sum 不等式為

$$\sum_i a_i \ln \frac{a_i}{b_i} \geq A \ln \frac{A}{B}, \qquad A = \sum_i a_i, \quad B = \sum_i b_i.$$

i 索引元素，A, B 是總和，零項用標準連續慣例。全零 A 時兩側零。此式支援散度凸性與聚合資料處理形式 [1]。

在分割每格內使用，可證合併結果不增加相對熵。精細量測能區分的分布差異，粗摘要可能已看不到。

有限機率向量 p, q 的 Pinsker 不等式為

$$\mathrm{TV}(p, q) = \frac{1}{2} \sum_i |p_i - q_i| \leq \sqrt{\frac{1}{2} D_{\mathrm{nats}}(p \| q)}.$$

總變差限制事件機率差，散度使用自然對數。若以位元回報，根號內散度要乘 $\ln 2$。小散度可控制事件機率差，大散度則可能讓界無用。

離散熵也具次模性：

$$H(X_A) + H(X_B) \geq H(X_{A \cup B}) + H(X_{A \cap B}).$$

A, B 是變數索引集合，X_A 是相應變數群，空群熵零。差是非重疊部分給定交集的條件互資訊，因此非負。

這是子集熵關係，不表示每個任務相關函數都次模。XOR 提醒我們，另一特徵加入後，特徵資訊可以增加。

熵功率與高斯雜訊

微分熵能轉成類似變異數的量。獨立連續訊號相加後，熵功率不小於原本總和。

d 維向量有有限自然單位微分熵時，定義熵功率

$$\mathcal{N}(X) = \frac{1}{2\pi e} \exp\left(\frac{2h_{\mathrm{nats}}(X)}{d} \right).$$

d 是維度，h_{nats} 使用自然對數。純量高斯熵功率等於變異數；等向高斯向量則等於共同座標變異數。

相同維度獨立向量的熵功率不等式為

$$\mathcal{N}(X + Y) \geq \mathcal{N}(X) + \mathcal{N}(Y).$$

X, Y 須滿足相應密度與有限性假設。標準非退化設定下，獨立高斯向量且共變異數成比例時等號成立 [1]。

de Bruijn 恆等式連結雜訊平滑與 Fisher 資訊：

$$\frac{d}{dt} h_{\text{nats}}(X + \sqrt{t}\,G) = \frac{1}{2} J_{\text{loc}}(X + \sqrt{t}\,G).$$

G 是獨立標準高斯向量，$t > 0$ 是每座標雜訊變異數，$J_{\text{loc}}(V) = \mathbb{E}\|\nabla \ln f_V(V)\|^2$ 是位置 Fisher 資訊。需要平滑性與可積分性。

此 Fisher 是與位置估計相關的密度梯度量，不是前章任意參數 Fisher。明確定義可避免混淆同名物件。

加雜訊可以增加觀測熵，卻減少原始資訊，沒有矛盾：熵功率描述輸出散布，資料處理描述保留關係。現代含雜訊表示模型應區分兩者。

表示資訊與樣本資訊

特徵可保留目標，限制原始細節；訓練模型也可揭露對特定樣本依賴多少。這是不同資訊問題。

$Y \to X \to Z$ 的資訊瓶頸目標為

$$\min_{p(z|x)} I(X;Z) - \beta I(Z;Y).$$

X 是觀測，Y 是選定相關目標，Z 是可能隨機的表示，$\beta \geq 0$ 平衡壓縮與目標資訊。Goldfeld 與 Polyanskiy 的出版教學文 [2] 發展理論與學習連結。

目標選擇相對任務的統計表示，不證明每個神經網路自動符合相同最佳化，也不證明表示保留所有未來目標。

瓶頸如何移除干擾、保留標籤？

有限例子讓表示目標具體。令標籤 Y 與干擾位元 N 是獨立公平位元，觀測 $X = (Y, N)$ 包含兩者。比較完整表示 $Z = X$、只含標籤的 $Z = Y$，以及常數表示。

表示	$I(X;Z)$	$I(Z;Y)$	目標 $I(X;Z) - \beta I(Z;Y)$
兩個位元	2	1	$2 - \beta$
只含標籤	1	1	$1 - \beta$
常數	0	0	0

資訊單位都是位元，相關權重 β 無單位。只含標籤的表示保留此標籤全部需求，比完整表示少花一位元。$\beta < 1$ 時常數勝出，$\beta > 1$ 時保留標籤勝出；$\beta = 1$ 時標籤與常數平手。

結論不限於這三個例子。資訊鏈式法則給出

$$I(X;Z) - \beta I(Z;Y) = (1 - \beta) I(Y;Z) + I(N;Z \mid Y).$$

$X = (Y, N)$，N 是獨立干擾位元，Z 是任意由 X 產生的表示。條件項非負，因此 $\beta \leq 1$ 時全域下界零，由常數達到。$\beta > 1$ 時，利用 $I(Y;Z) \leq H(Y) = 1$ 得下界 $1 - \beta$，由 $Z = Y$ 達到。保留任務的最佳解來自鏈式法則與明確來源，不是「壓縮一定改善學習」的非正式主張。

若未來目標改成 $Y' = Y \oplus N$，舊的純標籤表示對此新公平目標互資訊零，完整觀測卻能決定它。不可逆任務壓縮即使對原相關準則最佳，也會限制未來任務轉移。

符號也區分本節兩種資訊問題：此處討論單一觀測與表示，後面的學習界限則討論模型 W 與全部訓練樣本 S。減少 Z 中干擾資訊，不直接證明 $I(W;S)$ 較小，也不直接建立數值泛化保證。

學習時，令 $S = (Z_1, \dots, Z_n)$ 是獨立訓練樣本，W 是學得模型。母體與經驗損失為

$$L(W) = \mathbb{E}_{Z\sim P}[\ell(W,Z)], \qquad \hat{L}_S(W) = \frac{1}{n}\sum_{i=1}^{n} \ell(W,Z_i).$$

P 是母體法則，新 Z 與訓練獨立，ℓ 是所選損失。相關資訊可為 $I(W;S)$，而非特徵資訊 $I(X;Z)$。

每個固定模型損失在 P 下都是 σ 次高斯時，標準期望界形式為

$$\left|\mathbb{E}[L(W) - \hat{L}_S(W)]\right| \le \sqrt{\frac{2\sigma^2 I_{\text{nats}}(W;S)}{n}}.$$

期望平均樣本與學習器隨機性，σ 是共同集中參數，資訊使用自然對數。$[0,1]$ 有界損失可用 $\sigma = 1/2$ [3]。

這是期望界，不自動成為某訓練網路的高機率憑證。確定性連續學習器可能有無窮樣本資訊，使界無用。條件資訊與其他精細框架處理不同隨機化、抽樣慣例 [3]。

程式：精確有限學習界

完整例子只依多數標籤學常數二元分類器。輸出是確定性位元，所以 $I(W;S) = H(W)$，全部期望都可對二項計數加總。

```
import numpy as np
from scipy.stats import binom

p = 0.65
for n in [5, 20, 100]:
    counts = np.arange(n+1)
    mass = binom.pmf(counts, n, p)
    prediction = counts >= n/2
    training_loss = np.where(prediction, (n-counts)/n, counts/n)
    population_loss = np.where(prediction, 1-p, p)
    gap = mass @ (population_loss-training_loss)
    probability_one = mass[prediction].sum()
    probabilities = np.array([probability_one, 1-probability_one])
    positive = probabilities > 0
    information_nats = -np.sum(probabilities[positive]*np.log(probabilities[positive]))
    bound = np.sqrt(information_nats/(2*n))
    assert abs(gap) <= bound + 1e-12
    print("Length / expected gap / information bound:", n, gap, bound)
```

計算納入模型與樣本的選擇相依。把所選分類器當成與訓練標籤獨立，會抹掉正在分析的泛化間隙。

專題：同位檢查分箱與旁資訊解碼

編碼器替二元區塊發送短同位檢查標籤。解碼器結合標籤與相關區塊，選最可能原始資料。

二元區塊 $x \in \{0,1\}^n$、具有 r 個獨立列的公開二元矩陣 H，傳送症候

$$s = Hx \pmod 2.$$

H 為 r 乘 n，s 含 r 個同位檢查位元，乘法用二元體加法與乘法。每個症候辨認一箱候選區塊。

解碼器取得 $y = x \oplus e$，獨立翻轉機率低於二分之一，再從症候相容候選選與 y Hamming 距離最小者。指定法則下，這是最大條件概似。

編碼器從不看 y，矩陣與區塊長度共享，解碼器取得 y 作旁資訊。負載速率為 r/n，不含傳送或重建旁資訊程序。

因此，實驗對應來源編碼角點，不是免費壓縮兩觀測。若第二區塊也須傳送，其長度應計入總速率。共享旁資訊是有來源的假設，不是不明解碼優勢。

程式：窮舉有限解碼與整區塊誤差

完整 CPU 例子採十位元區塊，讓窮舉可行。它在二元體檢查矩陣秩，不使用實數數值秩。

```
import numpy as np

rng = np.random.default_rng(43)
n, trials, flip = 10, 3000, 0.1
patterns = ((np.arange(2**n)[:, None] >> np.arange(n)) & 1).astype(np.uint8)
source_indices = rng.integers(2**n, size=trials)
source = patterns[source_indices]
side = source ^ rng.binomial(1, flip, size=source.shape).astype(np.uint8)

def binary_rank(matrix):
    work = matrix.copy()
    rank = 0
    for column in range(work.shape[1]):
        pivots = np.flatnonzero(work[rank:, column])
        if len(pivots) == 0:
            continue
        pivot = rank + pivots[0]
        work[[rank, pivot]] = work[[pivot, rank]]
        for row in range(work.shape[0]):
            if row != rank and work[row, column]:
                work[row] ^= work[rank]
        rank += 1
        if rank == work.shape[0]:
            break
    return rank

for parity_bits in [4, 6, 8]:
    while True:
        H = rng.integers(2, size=(parity_bits, n), dtype=np.uint8)
        if binary_rank(H) == parity_bits:
```

```
            break
    weights = 2**np.arange(parity_bits)
    syndromes = ((patterns @ H.T) % 2) @ weights
    bins = [np.flatnonzero(syndromes == s) for s in range(2**parity_bits)]
    assert all(len(group) == 2**(n-parity_bits) for group in bins)
    decoded = np.empty(trials, dtype=int)
    for t, (index, observation) in enumerate(zip(source_indices, side)):
        candidates = bins[syndromes[index]]
        distances = np.count_nonzero(patterns[candidates] != observation, axis=1)
        decoded[t] = candidates[np.argmin(distances)]
    block_error = np.mean(decoded != source_indices)
    print("Parity bits / payload rate / full-block error:",
          parity_bits, parity_bits/n, block_error)
conditional_entropy = -flip*np.log2(flip)-(1-flip)*np.log2(1-flip)
print("Asymptotic side-information reference:", conditional_entropy)
```

這是有限示範，不是達容量建構。不同隨機矩陣有不同誤差；增加同位位元，不保證每個獨立抽樣矩陣都單調改善。參考熵是長區塊極限，不是十位元精確預測。

平手依候選順序確定處理。雜訊可能把旁區塊移得更近錯誤但症候相容的來源。要求每次含雜訊試驗都往返完全一致，是錯誤測試：程序刻意回報有限訊息誤差。

真正無損檔案程序應提供逃脫或互動修正，明確計傳送成本。此處學習目標是看旁資訊如何縮小箱，以及有限歧義如何保留。

現代觀點

Hellström 等人的 2025 年出版專論 [3]，透過期望、機率與條件資訊表示，整理資訊論與 PAC-Bayesian 泛化。分類幫助區分平均選擇相依界，與單一部署預測器保證。

Nguyen 等人的 2025 年語意通訊綜述 [4]，區分接收者意圖建模、生成解碼與學得聯合編碼。網路觀點把接收者知識與任務目標放中心，應納入數學資訊模式。

El Gamal 的 2026 年擬議章 [5]，把教科書討論延伸至統計學習，草稿狀態保持明確。現代推論與生成模型可以用相同散度工具，卻另有計算與評估問題。

開放問題包括一般多使用者區域、短區塊分散式碼、真實相依與通道偏移、罕見內容任務準則，以及高維學習系統有用資訊界。新架構不會只因產生合理樣本就解決它們。

全課程推論是：解讀資訊宣稱前，先問五件事：涉及哪些隨機變數；誰觀察各變數；假設哪些來源與通道法則；需要什麼重建或決策；以及如何量測速率、誤差與資源成本。

資訊檢核可以把問題變成簡短實驗紀錄：

選擇	應記錄內容
來源	母體、分組與相依假設
觀測	各編碼器、接收器分別可用的變數
描述	字母集合、實際位元流、共享參數與旁資料流
通道	條件法則、記憶、功率與使用慣例
目標	精確訊息、有損重建或指定任務

選擇	應記錄內容
評估	固定測試集、誤差定義、速率與執行時間

即使沒有緊的定理，紀錄仍有用。它避免數字在訓練、分析與部署間悄悄變義，也讓負面結果提供資訊：不佳速率可能來自機率模型、重建限制或有限解碼器，而非同一底層極限。

課程後的閱讀與專題路徑

可以三次重讀完整書籍。先用熵、編碼、通道與失真例子鞏固第 1–5、7–10 章；再把第 6、11–14 章連到預測、證據、最大熵、通用碼與描述複雜度；最後讀第 15–17 章，把網路假設與不等式放在每個定理旁。

小專題可選明確聯合法則，或獨立收集資料集，配一個接收者目標。實作基準編碼器與解碼器，回報實際速率與決策誤差，再一次只改一元件。機率模型、量化器與通道碼回答不同實驗問題。

自迴歸視覺章 連結逐步機率分解與現代模型；表示章 發展任務特徵；語言模型章 連結條件預測與 token 序列。

資訊定義具體時，連結最強。Tokenization 改變字母與單位，學得潛在變數改變觀測，生成重建改變失真問題。比較數值資訊前，應先說明選擇。

重點整理

- 網路產生相容速率區域，不是通用純量速率。
- 分散式解碼利用相關性，不賦予未觀察輸入。
- 多重存取個別速率仍受共享總和限制。
- 廣播與中繼公式依特定資訊模式。
- 聯合任務資訊可能不見於個別特徵資訊。
- 進階不等式需要明確單位、支撐與獨立假設。
- 表示資訊與樣本相依泛化是不同物件。
- 有限解碼、實際速率與誠實誤差，完成理論圖像。

練習

1. 公平 X、$Y = X \oplus E$、翻轉機率 0.2 下，求 Slepian–Wolf 角點與聯合熵，說明各編碼器觀察什麼。

   提示

   條件熵用 $h_2(0.2)$。

2. 求圖參數的兩個高斯多重存取角點，驗證總和限制並說明逐次解碼順序。

提示

一人達個別最大值，另一填總和界。

3. 說明一般廣播通道為何不能繼承物理退化區域，指出缺少的條件關係。

提示

檢查 $X \to Y_1 \to Y_2$。

4. 對 $(0.6, 0.4)$、$(0.5, 0.5)$ 驗證 Pinsker 界，一致使用自然與二進位散度。

提示

以 $\ln 2$ 把位元轉 nats。

5. 把同位碼區塊長度改 12，比較數種同位位元數，保留窮舉成本，回報區塊誤差而不只位元誤差。

提示

候選表有 2^n 列。

6. XOR 目標下，比較只保留 X_1 與保留點對，再說明確定性連續表示的原輸入資訊為何可無窮，而離散任務資訊仍有界。

提示

區分目標熵與精確連續關係。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 15 and 17. 出版社書頁與目錄。
2. Z. Goldfeld and Y. Polyanskiy, “The Information Bottleneck Problem and Its Applications in Machine Learning,” *IEEE Journal on Selected Areas in Information Theory*, 1(1), 2020, DOI: 10.1109/JSAIT.2020.2991561. 機構最終手稿紀錄。
3. F. Hellström, G. Durisi, B. Guedj and M. Raginsky, “Generalization Bounds: Perspectives from Information Theory and PAC-Bayes,” *Foundations and Trends in Machine Learning*, 18(1), 1–223, 2025. 出版社資訊，作者版本。
4. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, “A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding,” *IEEE Communications Surveys & Tutorials*, 2025. 出版資訊與作者版本。
5. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. 草稿與出版狀態。

篇 III

部落格文章

介紹 Pixels → Perception

透過淺白說明、程式碼與互動實驗，學習影像處理、機器學習及其背後觀念的中英雙語教學網站。

網站版本: https://blog.shyandram.dev/zh-tw/blog/introducing-pixels-perception/

一張影像，起點是一個由數字組成的陣列。要理解這些數字代表什麼，就會碰到訊號、模式、學習與不確定性等問題。為什麼一個小小的運算，就能改變影像的樣貌？模型如何學會辨認物件？在相信它的輸出之前，又該檢查哪些事情？

想和大家介紹 Pixels → Perception，一個提供英文與繁體中文內容的教學網站，試著把這些問題串成有脈絡的學習路徑。網站涵蓋影像處理、圖形識別、機器學習、深度學習、電腦視覺、自然語言處理、最佳化與資訊理論，希望讓讀者能在直覺說明、背後的數學，以及可以親自觀察的實驗之間來回對照。

這個網站想處理的一個落差，是「看懂公式」與「理解公式用起來會發生什麼事」之間的距離。定義在紙上可能很清楚，但當你改變參數、觀察圖表，或遇到一個失敗的例子時，才更容易看見它的假設。教學的安排，就是希望把這些關係說清楚。

學習區分成八條主題路線：

- **基礎**：介紹網站的使用方式、建議學習路徑，以及表示與操作影像的基本觀念。
- **影像處理**：把影像視為訊號，從強度與空間運算，逐步延伸到復原、壓縮與分割。
- **自然語言處理**：探討如何表示語言、建立處理文字的系統，並連結語言學觀念與學習方法。
- **圖形識別**：從特徵、統計決策與分類出發，理解辨認模式時涉及的不確定性。
- **機器學習**：涵蓋數學基礎、神經網路、表徵學習、生成模型與評估。
- **視覺深度學習**：把相關觀念放進影像分類、物件偵測、復原、增強與追蹤等具體任務。
- **最佳化**：練習如何建立問題、表達限制條件，並選擇求解方式。
- **資訊理論**：研究不確定性、壓縮與通訊，從另一個角度理解其他路線中反覆出現的觀念。

這些主題其實有許多共通之處。分析視覺模型時使用的數學工具，也能幫助我們理解語言模型；關於影像品質的問題，可能需要回到成像過程的假設；訓練上的疑問，也可能在釐清最佳化目標之後變得明確。把這些路線放在同一個網站裡，是希望讀者既能掌握各主題的架構，也能沿著彼此的連結繼續探索。

教學採用一致的閱讀節奏：先說明學習目標，用淺白的方式建立全貌，再逐步介紹定義、公式與例子。程式碼與圖表提供可以實際檢查的對象；後面的段落則連結研究文獻、討論限制，並以重點整理與練習收尾。當某個說明預設了你還不熟悉的知識時，可以透過先備知識連結回頭補足。

例如，互動實驗裡有一個影像濾波示範。你可以修改小型權重矩陣中的數值，直接觀察輸出如何改變。這讓學習形成一個循環：先預測、調整權重、觀察結果，再回到文字說明。其他實驗也涵蓋不同主題路線中的觀念。操作與計算都在瀏覽器中進行，不必先設定程式執行環境，就能開始嘗試。

程式碼範例則讓讀者進一步查看運算本身。你可以修改範例、比較輸出，或透過練習確認自己的理解。部分教學也附有可執行的筆記本連結。這些小型範例著重於說明運作方式，並不代表已重

現完整研究系統的規模或效能。

英文與繁體中文版本希望提供相同的學習體驗。繁體中文內容採用臺灣常用的術語，頁首的語言切換則連結到對應頁面。當你習慣用其中一種語言理解概念，卻又需要熟悉另一種語言的專業用語時，就能交互對照。

網站也在有系統的教學之外，保留了部落格。課程提供依序學習一個主題的路徑，部落格則容納介紹、筆記與課程順序以外的討論。如果只是想找回某個觀念，也可以使用搜尋，不必從整條主題路線的開頭重新讀起。

關於網站的製作方式，也想清楚說明：許多內容初稿與程式碼是在 AI 協助下建立的。主題與課程架構由人決定，人工審閱仍是必要的工作。**目前的內容仍在審閱中**。說明讀起來流暢，仍可能有錯誤；範例能夠執行，也可能省略重要假設。當細節會影響你的理解或實作時，請搭配文中引用的原始資料查證，並把小型示範當作繼續探究的起點。

如果剛開始接觸這些主題，可以從入門指南與影像就是陣列開始。如果已經有想釐清的問題，可以直接瀏覽主題路線，或先打開互動實驗，再沿著連結閱讀相關教學。你可以循序讀下去、回頭補一段數學，也可以只花一點時間弄懂一個觀念。

邀請大家挑一個問題，先猜猜結果，再動手試一個例子。希望 Pixels → Perception 能幫助你把紙上看似抽象的概念，變成自己能夠說明、驗證與運用的知識。

歡迎造訪 Pixels → Perception 繁體中文版或英文版。

歡迎來到 Pixels → Perception

這個網站涵蓋哪些內容、教學如何安排，以及如何執行程式碼。

網站版本: https://blog.shyandram.dev/zh-tw/blog/welcome/

這是一個可以依序閱讀、也可以隨時跳著看的電腦視覺課程。我們從「影像就是一堆數字」開始，一路介紹到現代的深度網路。

內容安排

學習區分成八個單元：基礎、影像處理、自然語言處理、圖形識別、機器學習、電腦視覺深度學習、最佳化與資訊理論。每篇教學都會列出先備知識，讓你知道該先讀哪一篇。

執行程式碼

部分教學提供對應的 Jupyter 筆記本，其他教學則包含可直接執行的程式碼區塊。若有筆記本，點選頁面上方的**在 Colab 開啟**，就能直接在瀏覽器中執行。

這篇文章所在的部落格，則用來放課程以外的內容：論文筆記、工具評測與較長的文章。